

Unity®

PERFORMANCE TUNING TIPS 2026

Written By
CyberAgent Game & Entertainment Division



CyberAgent

Unityパフォーマンスチューニングティップス2026

株式会社サイバーエージェント | ゲーム・エンターテイメント事業部 著

第1章

Unity Managed Code Stripping 實踐指南

向井 祐一郎 Mukai Yuichiro

株式會社 Applibot

yucchiy　X @yucchiy_　blog.yucchiy.com

作為縮減應用程式大小的手段，本文介紹 Unity 中可在建置時移除未使用程式碼的 Managed Code Stripping 功能。此功能只需一項設定即可□用，使用簡便，但加強縮減程度後，可能出現「在編輯器上能運作，只有建置後的應用程式中特定功能損壞」的問題。本章介紹 Managed Code Stripping 的機制與使用方法、程式碼被過度縮減的典型模式及其對策等□容。

驗證環境如下。

- Unity 6000.3.19f1
- Addressables 4.0.1
- macOS Sequoia 15.7.3

先□結論，Managed Code Stripping 不是將設定改為 High 就可以了事的最佳化。

必須搭配明確指定靜態分析看不見的相依性，並持續驗證建置後應用程式的大小與行為，才能安全地使用此功能。

1.1 什麼是 Managed Code Stripping

1.1.1 功能概要

Managed Code Stripping 是 Unity 的標準功能，會在建置時偵測未使用的受控程式碼（C# 程式碼），並將其從建置後的應用程式中移除。

移除場景與□本都未使用的類別與方法，即可縮減應用程式的程式碼大小。

縮減對象不僅限於使用者程式碼，也包含 UnityEngine 的模組、.NET 的類別庫，以及專案所含套件與外掛的組件，因此連自己難以修改的程式庫□部的未使用程式碼也能縮減。

尤其在 IL2CPP 建置中，縮減受控程式碼後，IL2CPP 後續□生的 C++ 程式碼也會減少，因此也有助於縮小應用程式大小與縮短建置時間。

□外，名稱相似的設定 Strip Engine Code 是以 Unity 引擎的原生程式碼為對象的縮減設定，與以受控程式碼為對象的 Managed Code Stripping 是不同的功能。

Managed Stripping Level

要多積極地縮減程式碼，由 Managed Stripping Level 設定控制。

可在 Edit > Project Settings > Player > Other Settings > Optimization 的「Managed Stripping Level」中設定，可選擇下列等級。

表 1.1: Managed Stripping Level 的選項

Level	概要
Disabled	不執行 Stripping。僅可在 Mono 建置中選擇
Minimal	僅縮減 UnityEngine 與 .NET 類別庫，不移除使用者程式碼。為 IL2CPP 建置的預設□
Low	使用者程式碼也納入分析對象，但採保守的縮減
Medium	以所有組件為對象，在大小與安全性之間取得平衡進行縮減
High	以最積極的方式縮減所有組件

Mono 建置的預設□為 Disabled，因此若不做任何設定，就不會執行 Stripping。

□一方面，IL2CPP 建置的預設□為 Minimal，且無法選擇 Disabled，因此無論程度高低，必定會執行 Stripping。

此外，Unity 6.3 的文件已預告 Low 將於未來廢止，不建議使用。

參考：Managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping.html` / Configure managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-configure.html`

1.1.2 基本運作原理

Managed Code Stripping 是由 Unity 自行開發的工具 **UnityLinker** 所實現。

UnityLinker 在應用程式的建置管線中，於 C# 程式碼編譯完成後執行，從已編譯的組件（DLL）中移除未使用的程式碼。

在 IL2CPP 建置中，經過此 Stripping 的組件會成為後續 C++ 程式碼轉換的輸入。

UnityLinker 以下列三個階段判定未使用的程式碼。

1. 決定分析起點的程式碼（**root**）
2. 從 root 出發，遞迴追蹤被靜態參照的型別與成員，並標記為使用中（**mark**）
3. 將最後仍未被標記的程式碼視為無法到達，並從組件中移除

被視為 root 的代表性項目如下。

- 被建置所含的場景或資□參照的型別
- 繼承 MonoBehaviour 或 ScriptableObject 的型別
- 附有 [RuntimeInitializeOnLoadMethod] 等屬性的方法
- 以 link.xml 或 [Preserve] 屬性明確指定保留的程式碼（於後述對策中□明）

此處重要的是，是否保留程式碼並非取決於「執行時是否實際使用」，而是取決於「建置時的靜態分析能否從 root 到達」。

靜態分析是指不執行程式碼，僅機械式地追蹤已編譯程式碼中出現的參照的分析。

例如以 new 建立型別或直接呼叫方法時，被參照的型別與成員資訊會原樣保留在編譯後的程式碼中，因此 UnityLinker 能□追蹤。

□一方面，以程式碼上不出現參照的形式被呼叫的程式碼，無論執行時是否使用，都會被視為無法到達。

因此，只有靜態分析看不見的參照的程式碼，即使執行時需要，也會被判定為未使用而遭移除。

□外，mark 是以成員為單位而非以型別為單位進行，因此即使型別保留下來，未被使用的方法仍會被個別移除。

實際上□些程式碼會被移除，將於下一節搭配具體程式碼與測量結果確認。

以 Mono.Cecil 確認□些□容被移除

本章在後續各節中會反覆確認「□些型別與方法保留在建置中」。

確認時使用的是分析 .NET 組件的程式庫 **Mono.Cecil**。

讀取建置輸出所含的組件（macOS 用 Mono 建置時為「應用程式名稱.app/Contents/Resources/Data/Managed/Assembly-CSharp.dll」），並將型別與方法的清單寫出到檔案。

▼清單 1.1　型別與方法清單的輸出

```
// 讀取組件，寫出型別與方法（含 .ctor）的清單
using var assembly = AssemblyDefinition.ReadAssembly(dllPath);
var lines = assembly.MainModule.Types
    .Where(type => type.Name != "<Module>")
    .SelectMany(type => new[] { type.FullName }
        .Concat(type.Methods.Select(method => $"  {method.Name}")));
File.WriteAllLines(outputPath, lines);
```

方法清單也包含建構函式（.ctor），因此也可用於確認後述「型別仍在但只有建構函式消失」的症狀。

Mono.Cecil 可透過 Unity Package Manager 安裝使用。在編輯器用的 Assembly Definition 中□用 overrideReferences，並將 Mono.Cecil.dll 加入 precompiledReferences，即可從編輯器程式碼參照。

取得兩個不同 Level 建置的清單並比較差異，即可確定□些型別消失了、□些方法以成員為單位被移除。

建置後，Stripping 後的組件有時會作為中間□物留在 Library 下。

在本次環境中，Library/Bee/artifacts/MacStandalonePlayerBuildProgram/ManagedStripped/ 中留有與建置輸出□容相同的 DLL，不需展開建置成品即可以此作為分析對象。

官方文件並未明確記載 Stripping 後的組件會儲存於此路徑，因此找不到時，請分析建置輸出端。

後續各節中寫到「確認了型別與方法」之處，基本上都是以此方法確認。比較 Stripping 前後的 DLL，還可以確認縮減的細項。

參考：Unity linker marking rules `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-marking-rules.html`

1.1.3 為什麼要縮減程式碼

使用 Managed Code Stripping 的主要目的是縮減應用程式大小。

次要效果是，未使用的程式碼不會載入記憶體，因此也可期待降低執行階段的記憶體使用量。

為了確認效果程度，建置一個只放置一個結束處理□本的簡單場景，並比較 Managed Stripping Level 的 Minimal 與 High。

測量結果如下（表 1.2）。

表 1.2: 簡單場景中 Minimal 與 High 的比較

指標	Minimal	High	縮減
受控組件的總大小	7,702,016 bytes	7,188,480 bytes	約 6.7%
組件（DLL）數量	52	51	1 個
應用程式整體大小	89,069,307 bytes	88,548,299 bytes	約 0.6%

即使是小型的範例專案，受控程式碼也減少了 6.7%。

從細項來看，完全未被使用的 System.Runtime.Serialization.dll 被整個移除，mscorlib.dll 減少約 272KB，包含許多未被場景參照的程式碼的 Assembly-CSharp.dll 也減少了 22.5%。

受控程式碼的縮減量，取決於專案所參照的程式庫數量，以及其中實際未被使用的程式碼比例。

大型專案傾向於導入許多程式庫，而程式庫中實際用不到的功能往往也很多，因此較容易受惠於程式碼縮減。

在 IL2CPP 建置中，被縮減的受控程式碼部分也不再需要轉換與編譯為 C++，因此效果也會擴及原生二進位檔的大小與建置時間。

1.2 套用 Managed Code Stripping 時的注意事項

如前一節所見，Managed Code Stripping 的判斷依賴建置時的靜態分析。

因此，只有靜態分析看不見的參照的程式碼，即使執行時需要，也會被移除。

此問題之所以棘手，在於症狀只會出現在建置後的應用程式中。

在編輯器中執行，或在未□用 Stripping 的預設 Mono 建置中都能正常運作，因此開發期間不易察覺，往往在提高 Stripping Level 後，才以特定功能損壞的形式浮現。

本節將介紹程式碼被過度裁剪的兩種典型模式。

一是透過反射呼叫的程式碼，□一是未被 Unity 進入點參照的程式碼。

1.2.1 透過反射呼叫的程式碼消失

以具體程式碼確認前一節所□明的「靜態分析能否從 root 到達」的判斷。

準備三個被參照方式不同的類別（清單 1.2）。

▼清單 1.2　三個被參照方式不同的類別

```
// 以 new 直接建立
public class DirectlyUsedService
{
    public string Describe() => "DirectlyUsedService is alive";
}

// 僅以 typeof 參照型別
public class TypeofOnlyService
{
    public string Describe() => "TypeofOnlyService is alive";
}

// 僅透過反射建立
public class ReflectionOnlyService
{
    public string Describe() => "ReflectionOnlyService is alive";
}
```

從放置於場景的 MonoBehaviour，如下使用這三者（清單 1.3）。

▼清單 1.3　使用三個類別的 MonoBehaviour

```
public class StrippingProbe : MonoBehaviour
{
    private static readonly string ReflectionTypeName = "ReflectionOnlyService";
    private static readonly string TypeofTypeName = "TypeofOnlyService";
    private static readonly string TypeofMethodName = "Describe";

    private void Start()
    {
        // 靜態參照：被參照的型別與方法資訊會保留在編譯後的程式碼中
        var direct = new DirectlyUsedService();
        Debug.Log($"direct: {direct.Describe()}");

        // 靜態參照僅限型別：typeof 會參照型別，但沒有對成員的靜態參照
        Debug.Log($"typeof: type is referenced: {typeof(TypeofOnlyService).Name}");

        // 為確認成員是否保留，以靜態分析看不見的路徑重新取得型別
        var typeofType = Type.GetType(TypeofTypeName);
        var describeMethod = typeofType?.GetMethod(TypeofMethodName);
        if (describeMethod == null)
        {
            Debug.Log($"typeof: type {typeofType?.Name} exists but Describe is stripped
    ");
        }
        else
        {
            Debug.Log($"typeof: {describeMethod.Invoke(Activator.CreateInstance(
```

```
    typeofType), null)}");
        }

        // 靜態分析看不見的參照：透過反射建立
        var type = Type.GetType(ReflectionTypeName);
        if (type == null)
        {
            Debug.Log($"reflection: Type.GetType(\"{ReflectionTypeName}\") returned
    null");
        }
        else
        {
            var instance = Activator.CreateInstance(type);
            Debug.Log($"reflection: {type.GetMethod("Describe")?.Invoke(instance, null)
    }");
        }
    }
}
```

將此範例的 Managed Stripping Level 改為 Minimal 與 High 進行建置並執行，結果如下。

```
# Minimal
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: TypeofOnlyService is alive
reflection: ReflectionOnlyService is alive

# High
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: type TypeofOnlyService exists but Describe is stripped
reflection: Type.GetType("ReflectionOnlyService") returned null
```

在 High 中，typeof 的情形雖可取得型別，但 GetMethod 回傳 null；反射的情形則是 Type.GetType 本身回傳 null。

彙整後，行為如下（表 1.3）。

表 1.3: Minimal 與 High 的保留情形

對象	Minimal	High
StrippingProbe	保留	保留
DirectlyUsedService（以 new 參照）	保留	型別與方法皆保留

表 1.3: Minimal 與 High 的保留情形

對象	Minimal	High
TypeofOnlyService（僅以 typeof 參照型別）	保留	型別保留，但 Describe 被移除
ReflectionOnlyService（僅反射）	保留	整個型別被移除

Minimal 不會移除使用者程式碼，因此三者都原樣保留。

在 High 中，被參照方式的差異會直接反映在結果上。

DirectlyUsedService 以 new 參照型別，且直接呼叫 Describe，因此型別與方法都保留。

以 typeof 參照的 TypeofOnlyService，型別本身保留，但沒有任何地方呼叫的 Describe 則以成員為單位被移除。

而完全沒有靜態參照的 ReflectionOnlyService，則整個型別被移除。

此外，本實驗中型別名稱與方法名稱的字串全都經由 static readonly 欄位讀取，這是刻意的設計。

UnityLinker 理解部分典型的反射用法，已確認直接傳入 Type.GetType("ReflectionOnlyService") 這樣的常數字串時，即使在 High 中型別也會被保留。

□外，對以 typeof 取得的 Type 直接呼叫 GetMethod 時，即使方法名稱不是常數，該型別的 public 方法也會被一併保留。

□一方面，UnityLinker 不會追蹤經由欄位傳入的字串。

以讀取欄位的方式，重現型別名稱或成員名稱來自設定檔、序列化資料、伺服器回應等建置時無法確定的路徑的實際情況。

若將欄位設為 const，會以常數字串嵌入使用處，因此與直接傳入常數的情形處理方式相同。

DI 容器的型別解析損壞

到此為止的範例為了□明而簡化，但相同結構的問題會在實際的程式庫中發生。代表性的例子是 DI 容器。

許多 DI 容器會預先登錄介面與實作型別的對應，並在解析時以反射呼叫建構函式來建立執行個體。

以模仿此行為的最小容器來確認。與實際的 DI 容器一樣，採用以 Register 登錄、以 Resolve 解析的介面（清單 1.4）。

▼清單 1.4　最小的 DI 容器

```
public sealed class MiniContainer
{
    private readonly Dictionary<Type, Type> _registrations = new Dictionary<Type, Type
    >();

    // 以型別引數登錄實作型別。TImplementation 相當於 typeof 的靜態參照
    public void Register<TInterface, TImplementation>() where TImplementation :
    TInterface
    {
        _registrations[typeof(TInterface)] = typeof(TImplementation);
    }

    // 以反射將已登錄的實作型別執行個體化並回傳
    public TInterface Resolve<TInterface>()
    {
        var implementationType = _registrations[typeof(TInterface)];
        return (TInterface)Activator.CreateInstance(implementationType);
    }
}
```

所登錄的實作型別假設僅經由容器使用，沒有任何地方靜態參照（清單 1.5）。

▼清單 1.5　所登錄的介面與實作

```
public interface INotificationService
{
    string Notify();
}

// 僅經由容器使用，沒有任何地方靜態參照的實作
public sealed class NotificationService : INotificationService
```

```
{
    public string Notify() => "NotificationService.Notify succeeded";
}
```

從放置於場景的 MonoBehaviour 執行登錄與解析（清單 1.6）。

▼清單 1.6　執行登錄與解析的 MonoBehaviour

```
public sealed class DiContainerProbe : MonoBehaviour
{
    private void Start()
    {
        try
        {
            var container = new MiniContainer();
            container.Register<INotificationService, NotificationService>();
            Debug.Log($"[DiContainerProbe] explicit: {container.Resolve<
    INotificationService>().Notify()}");
        }
        catch (Exception exception)
        {
            Debug.Log($"[DiContainerProbe] explicit failed: {exception.GetType().Name}:
     {exception.Message}");
        }
    }
}
```

以與先前相同的條件建置，結果如下。

```
# Minimal
[DiContainerProbe] explicit: NotificationService.Notify succeeded

# High
[DiContainerProbe] explicit failed: MissingMethodException: Default constructor not
    found for type NotificationService
```

在 High 中，Register<INotificationService, NotificationService>() 的型別引數成為靜態參照，因此 NotificationService 型別本身會保留。

此外，經由介面被靜態呼叫的 Notify 也會保留。

然而，僅由反射呼叫的預設建構函式沒有靜態參照，因此以成員為單位被移除。

結果 Activator.CreateInstance 找不到建構函式，並以 MissingMethodException 導致

解析失敗。

這就是「型別已登錄，卻只有在建置後的應用程式中解析失敗」這類問題的原因。

實際的 DI 容器結構也相同。

只要建構函式的解析依賴反射，即使是明確登錄的型別，隱含的預設建構函式也會成為移除的候選。

若是從組件掃描實作型別並自動登錄的方式，沒有靜態參照的實作型別會整個被移除，導致登錄本身失敗。

1.2.2 未被 Unity 進入點參照的程式碼消失

□一個模式是，場景與□本都不參照，僅透過 AssetBundle 等外部□容才需要的程式碼。

如前一節所見，root 的來源是建置所含的場景與資□。

□外建置為 AssetBundle 的□容不包含在應用程式本體的建置中，因此該□容所參照的型別不會被視為 root。

因此，只有□容端需要的型別，若應用程式的程式碼中沒有靜態參照，無論執行時是否載入，都會被判定為未使用。

以 Timeline 資□重現此模式。

將 Timeline 資□僅放入 AssetBundle，應用程式端不靜態參照 Unity.Timeline，而是以資□名稱載入並由 PlayableDirector 播放（清單 1.7）。

▼清單 1.7　從 AssetBundle 播放 Timeline 資□的 MonoBehaviour

```
public class TimelineAssetBundleProbe : MonoBehaviour
{
    private const string BundleDirectoryName = "TimelineBundles";
    private const string BundleName = "timeline.bundle";
    private const string TimelineAssetName = "TimelineOnly";
```

```
    private void Start()
    {
        var bundlePath = Path.Combine(Application.streamingAssetsPath,
    BundleDirectoryName, BundleName);
        var bundle = AssetBundle.LoadFromFile(bundlePath);
        if (bundle == null)
        {
            Debug.Log("timeline bundle: failed to load asset bundle");
            return;
        }

        var asset = bundle.LoadAsset(TimelineAssetName);
        if (asset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} failed to load");
            bundle.Unload(true);
            return;
        }

        Debug.Log($"timeline bundle: loaded {asset.GetType().AssemblyQualifiedName}");
        var playableAsset = asset as PlayableAsset;
        if (playableAsset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} is not playable");
            bundle.Unload(true);
            return;
        }

        var director = gameObject.AddComponent<PlayableDirector>();
        director.playableAsset = playableAsset;
        director.Play();
        Debug.Log($"timeline bundle: play state={director.state}, graph valid={director
    .playableGraph.IsValid()}");
        bundle.Unload(false);
    }
}
```

建立 AssetBundle 後，先從專案中刪除原本的 Timeline 資□再建置應用程式，藉此模擬與應用程式分開更新□容的方式。

將 Managed Stripping Level 改為 Minimal 與 High 進行建置。兩個等級的結果相同。

```
# Minimal / High 皆如此
The referenced script (UnityEngine.Timeline.TimelineAsset) on this Behaviour is missing
    !
```

```
timeline bundle: asset TimelineOnly failed to load
```

□外，確認建置後應用程式的 Managed 資料夾，兩種建置中都不存在 Unity.Timeline.dll 本身。

需要注意的是，Minimal 與 High 的結果並無不同。

此模式的對策不是調整 Stripping Level，而是明確指定需要保留的型別或組件。

其方法將於下一節□明，使用 Addressables 時的機制則於進階篇□明。

參考：How code stripping affects content https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-content.html / Timeline 資□消失的案例（Qiita, 2021） https://qiita.com/KyoheiOkawa/items/0cf9cc09679eababec75

1.3 程式碼被過度移除問題的對策

1.3.1 基本對策

防止必要程式碼被移除最基本的手段，是明確告知 UnityLinker「此程式碼會被使用」。

Unity 為此提供了 link.xml 檔案與 [Preserve] 屬性。

以任一方式指定的程式碼都會被視為分析的 root，即使沒有靜態參照也不會被移除。

link.xml

link.xml 是列舉想要保留的組件、型別與成員的 XML 檔案。

將其放在專案的 Assets 下，UnityLinker 會在建置時讀取。

前一節範例中整個型別被移除的 ReflectionOnlyService，可用以下 link.xml 保留（清單 1.8）。

▼清單 1.8　保留 ReflectionOnlyService 的 link.xml

```
<linker>
  <assembly fullname="Assembly-CSharp">
    <type fullname="ReflectionOnlyService" preserve="all" />
  </assembly>
</linker>
```

將此 link.xml 放在 Assets 直下並以 High 重新建置後，執行結果變成如下。

```
# （無High link.）xml
reflection: Type.GetType("ReflectionOnlyService") returned null

# （有High link.）xml
reflection: ReflectionOnlyService is alive
```

確認建置所含的組件，也可以確認 ReflectionOnlyService 連同 Describe 一併被保留。

link.xml 可在程式碼之外指定保留對象，因此也能用於自己無法編輯的 DLL 或套件中的型別。

保留範圍可以組件為單位、型別為單位，或方法與欄位等成員為單位指定。

☒外，以 preserve="all" 保留整個組件的寫法雖然簡便，但會抵銷該組件的 Stripping 效果。

可用於問題的釐清，但作為長久對策，最好能縮小到必要的型別與成員。

[Preserve] 屬性

若是自己可以編輯的程式碼，可將 UnityEngine.Scripting 命名空間的 [Preserve] 屬性直接附加在對象上。

需要注意的是，[Preserve] 的保留範圍會因附加位置而異。

以附加在型別上的類別與僅附加在方法上的類別這兩個來確認此差異（清單 1.9、清單 1.10）。

▼清單 1.9　在型別上附加 Preserve 的情形

```
using UnityEngine.Scripting;
```

```
// 在型別上附加 [Preserve]。方法不附加]
[Preserve]
public sealed class TypePreservedService
{
    public string Describe() => "TypePreservedService.Describe is alive";
}
```

▼清單 1.10　僅在方法上附加 Preserve 的情形

```
using UnityEngine.Scripting;

// 型別不附加，僅在方法上附加 [Preserve]
public sealed class MethodPreservedService
{
    [Preserve]
    public string Describe() => "MethodPreservedService.Describe is alive";
}
```

以不具靜態參照的形式，透過反射建立並呼叫這兩者，在 High 中的結果如下（表 1.4）。

表 1.4: Preserve 的附加位置與 High 中的保留範圍

對象	型別	預設建構函式	Describe
TypePreservedService	保留	保留	被移除
MethodPreservedService	保留	被移除	保留

附加在型別上的 [Preserve] 會保留型別與預設建構函式，但不會保留未附加屬性的 Describe。

附加在方法上的 [Preserve] 會保留該型別與方法，但這次預設建構函式未被保留，Activator.CreateInstance 的建立會以 MissingMethodException 失敗。

若要讓透過反射建立並呼叫的流程能運作，必須在型別與方法兩者都附加 [Preserve]。

遇到「明明附加了 [Preserve] 卻無法運作」時，首先應檢視屬性的附加位置與保留範圍是否對應。

參考：Preserving code using annotations `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-preserving.html`

1.3.2 進階篇

活用 Preserve 以外的屬性

[Preserve] 是以「保留此元素」的方式逐點指定，而 UnityEngine.Scripting 命名空間中還有以模式宣告保留的屬性，例如「保留此介面的所有實作」。

在框架中常見的「以反射尋找並呼叫使用者實作的型別」這類結構，可以不必逐一以 [Preserve] 指定實作型別，而是在介面或基底類別上附加屬性來因應。

- RequireImplementorsAttribute
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireImplementorsAttribute.html`：附加在介面上時，保留實作該介面的型別
- RequireDerivedAttribute
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireDerivedAttribute.html`：附加在類別上時，保留繼承該型別的型別
- RequiredMemberAttribute
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredMemberAttribute.html`：附加在成員上時，若其型別被保留，則該成員也一併保留
- RequiredInterfaceAttribute
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredInterfaceAttribute.html`：附加在型別上時，保留所指定介面的實作資訊

以 [RequireImplementors] 為代表確認其行為。

在介面上附加屬性，實作則沒有任何地方靜態參照（清單 1.11）。

▼清單 1.11　附加 RequireImplementors 的介面與實作

```
// 附加在介面上時，實作型別會成為保留對象
[RequireImplementors]
public interface IRequiredContract
{
}
```

```
// 沒有任何地方靜態參照的實作
public sealed class RequiredContractImpl : IRequiredContract
{
}
```

以 High 建置後，RequiredContractImpl 連同預設建構函式一併被保留。

沒有靜態參照的實作型別，僅憑介面端的宣告即可保留。

對於附加 [RequireDerived] 的基底類別的衍生型別，也同樣確認了保留。

以 IUnityLinkerProcessor 在建置時□生 link.xml

若要保留的程式碼已確定，只要在 link.xml 中指定對象的類別或成員即可。

□一方面，在每個建置目標要保留的型別不同，或所需型別因建置所含□容而改變等情況下，持續以手動更新 link.xml 並不實際。

使用 **IUnityLinkerProcessor** 可在建置時動態□生 link.xml 並傳給 UnityLinker。

IUnityLinkerProcessor 是編輯器用的介面，將實作類別放在編輯器組件中，就會在建置時自動被呼叫。

GenerateAdditionalLinkXmlFile 所回傳路徑的 link.xml, 會被加入 UnityLinker 的輸入。

最小的實作如下（清單 1.12）。

▼清單 1.12　□生 link.xml 的最小 IUnityLinkerProcessor 實作

```
using System.IO;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEditor.UnityLinker;
using UnityEngine;

public sealed class PreserveListLinkerProcessor : IUnityLinkerProcessor
{
    private const string LinkXmlContent = @"<linker>
  <assembly fullname=""Assembly-CSharp"">
    <type fullname=""GeneratedPreserveService"" preserve=""all"" />
```

```
  </assembly>
</linker>
";

    public int callbackOrder => 0;

    // 所回傳路徑的 link.xml 會被加入 UnityLinker 的輸入
    public string GenerateAdditionalLinkXmlFile(BuildReport report,
    UnityLinkerBuildPipelineData data)
    {
        var path = Path.Combine(Path.GetTempPath(), "managed-stripping-sample-generated
    -link.xml");
        File.WriteAllText(path, LinkXmlContent);
        Debug.Log("[PreserveListLinkerProcessor] generated 1 assembly / 1 type.");
        return path;
    }
}
```

此範例為了□明而固定了□生□容，但實際上會將此部分替換為從建置時的輸入組裝的處理，例如讀取設定資□或分析□容的相依性。

準備沒有靜態參照的 GeneratedPreserveService 並以 High 建置，即使 Assets 下沒有放置 link.xml，也確認了型別、建構函式與方法都被保留。

```
[GeneratedLinkXmlProbe] type=present
[GeneratedLinkXmlProbe] ctor=present, method=GeneratedPreserveService.Describe is alive
```

參考：IUnityLinkerProcessor https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Build.IUnityLinkerProcessor.html

Addressables □生的 link.xml

針對前一節所見「只有□容端需要的型別消失」的問題，Addressables 以自動□生 link.xml 作為□容建置的成品來因應。

確認實際會□生什麼。□容採最小構成，建立一個僅顯示 Cube 網格的 prefab（在 GameObject 上附加 MeshFilter 與 MeshRenderer），並登錄到 Addressables 的群組。

執行□容建置後，會□生 link.xml。

在 Addressables 4.0.1 中，輸出位置為 Library/com.unity.addressables/aa/

<platform>/AddressablesLink/link.xml。

本次□生的□容如下（清單 1.13）。

▼清單 1.13　Addressables □生的 link.xml

```
<linker>
  <assembly fullname="Unity.Addressables, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.AddressableAssets.Addressables" preserve="all" />
  </assembly>
  <assembly fullname="Unity.ResourceManager, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    AssetBundleProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    BundledAssetProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    IAssetBundleResource" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.InstanceProvider"
    preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.SceneProvider"
    preserve="all" />
  </assembly>
  <assembly fullname="UnityEngine.CoreModule, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null">
    <type fullname="UnityEngine.GameObject" preserve="all" />
    <type fullname="UnityEngine.MeshFilter" preserve="all" />
    <type fullname="UnityEngine.MeshRenderer" preserve="all" />
    <type fullname="UnityEngine.Transform" preserve="all" />
  </assembly>
</linker>
```

□容分為兩種。

一是 Addressables 執行階段（Unity.Addressables 與 Unity.ResourceManager）的 provider 型別，依群組構成而需要。

□一是所登錄□容參照的型別。本次構成 prefab 的 GameObject、Transform、MeshFilter、MeshRenderer 都直接列為保留對象。

可知 Addressables 會分析□容端的相依性，即使應用程式的程式碼沒有參照的型別，也會納入保留對象。

參考：Addressables Build Artifacts `https://docs.unity3d.com/ja/Packages/com.unity.addressables@1.20/manual/BuildArtifacts.html`（link.xml 的□明在 1.20 的手冊中）

以 Source Generator 的靜態呼叫取代反射的程式碼呼叫

到此為止的對策，都是以指定保留的方式，防止必要的程式碼被移除。

相對地，也有以 **Source Generator** 的程式碼□生，來取代□生靜態分析看不見的相依性的反射本身的方法。

Source Generator 是 C# 編譯器在編譯時分析原始碼並□生額外原始碼的機制。

若在編譯時就完成型別的探索與□生，相依性從一開始就以靜態參照的形式表達，就不再需要保留指定。

例如，考慮接收命令名稱字串並執行對應處理常式（handler）的分派處理。

處理常式以屬性宣告命令名稱。若在執行時以反射建立，結構上就是從組件中尋找附有屬性的型別並建立（清單 1.14）。

▼清單 1.14　以屬性宣告命令名稱的處理常式

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class CommandHandlerAttribute : Attribute
{
    public CommandHandlerAttribute(string command)
    {
        Command = command;
    }

    public string Command { get; }
}

public interface ICommandHandler
{
    string Execute();
}

// 處理常式沒有任何地方靜態參照
[CommandHandler("greet")]
public sealed class GreetCommandHandler : ICommandHandler
```

```
{
    public string Execute() => "GreetCommandHandler.Execute succeeded";
}

[CommandHandler("farewell")]
public sealed class FarewellCommandHandler : ICommandHandler
{
    public string Execute() => "FarewellCommandHandler.Execute succeeded";
}
```

兩個處理常式都沒有任何地方靜態參照，因此若維持執行時掃描的作法並以 High 建置，如前一節所見，整個型別都會被移除。

對於此命令名稱到處理常式的解析，改以 Source Generator 實現。

準備一個□生器，在編譯時收集附有 [CommandHandler] 的類別，並□生由命令字串□找具體型別的分派器。

□生器的工作分為兩個階段，前半（ForAttributeWithMetadataName）是從附有屬性的類別收集命令名稱與型別名稱，後半（GenerateDispatcher）是將收集到的配對填入範本。

以下為□生器的完整程式碼（清單 1.15）。

▼清單 1.15　□生分派器的 Source Generator

```
#nullable enable
using System.Collections.Immutable;
using System.Linq;
using System.Text;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.Text;
namespace StrippingSample.SourceGenerators
{
    [Generator]
    public sealed class CommandDispatcherGenerator : IIncrementalGenerator
    {
        public void Initialize(IncrementalGeneratorInitializationContext context)
        {
            // 從附有 [CommandHandler] 的類別收集命令名稱與型別名稱的配對
            var handlers = context.SyntaxProvider
                .ForAttributeWithMetadataName(
```

```
                "CommandHandlerAttribute",
                static (_, _) => true,
                static (syntaxContext, _) => (
                    Command: (string)syntaxContext.Attributes[0].
ConstructorArguments[0].Value!,
                    TypeName: ((INamedTypeSymbol)syntaxContext.TargetSymbol)
                        .ToDisplayString(SymbolDisplayFormat.FullyQualifiedFormat))
)
            .Collect();
        // 從收集到的配對⌧生分派器
        context.RegisterSourceOutput(handlers, static (sourceContext,
commandHandlers) =>
        {
            if (!commandHandlers.IsDefaultOrEmpty)
            {
                sourceContext.AddSource(
                    "CommandDispatcher.g.cs",
                    SourceText.From(GenerateDispatcher(commandHandlers), Encoding.
UTF8));
            }
        });
    }
    private static string GenerateDispatcher(
        ImmutableArray<(string Command, string TypeName)> handlers)
    {
        // 為使輸出具確定性，先依命令名稱排序再填入
        var cases = string.Join("\n", handlers
            .Distinct()
            .OrderBy(static handler => handler.Command, System.StringComparer.
Ordinal)
            .Select(static handler => $$"""
                        case {{SymbolDisplay.FormatLiteral(handler.Command, true)
}}:
                            return new {{handler.TypeName}}().Execute();
            """));
        return $$"""
            // <auto-generated/>
            public static class CommandDispatcher
            {
                public static string Execute(string command)
                {
                    switch (command)
                    {
            {{cases}}
                        default:
                            throw new global::System.ArgumentException($"Unknown
command: {command}");
                    }
```

```
                    }
                }
                """;
        }
    }
}
```

依命令名稱排序，是為了從相同輸入□生相同程式碼，使輸出不會在每次建置時變動。

此□生器會在編譯時□生如下的程式碼（清單 1.16）。

▼清單 1.16　所□生的 CommandDispatcher

```
// <auto-generated/>
public static class CommandDispatcher
{
    public static string Execute(string command)
    {
        switch (command)
        {
            case "farewell":
                return new global::FarewellCommandHandler().Execute();
            case "greet":
                return new global::GreetCommandHandler().Execute();
            default:
                throw new global::System.ArgumentException($"Unknown command: {command}
    ");
        }
    }
}
```

原本在執行時由字串尋找型別的處理，變成了字串與具體型別的對應表。

建構（new）與 Execute 的呼叫都出現在□生的程式碼中，因此對型別、建構函式與方法的靜態參照在□生的程式碼□即可完成。

使用端只傳入命令字串，完全不參照處理常式的具體型別。

與反射的實驗相同，命令字串經由 static readonly 欄位傳入，維持為 UnityLinker 無法追蹤的輸入（清單 1.17）。

▼清單 1.17　所□生分派器的使用端

```
// 命令字串經由欄位傳入，作為 UnityLinker 無法追蹤的輸入
private static readonly string[] Commands = { "farewell", "greet" };

private static void RunGeneratedDispatcher()
{
    foreach (var command in Commands)
    {
        var result = CommandDispatcher.Execute(command);
        Debug.Log($"[SourceGeneratorProbe] dispatch command={command}, result={result}"
    );
    }
}
```

以 High 建置並執行後，確認兩個命令都成功執行。

```
[SourceGeneratorProbe] dispatch command=farewell, result=FarewellCommandHandler.Execute
     succeeded
[SourceGeneratorProbe] dispatch command=greet, result=GreetCommandHandler.Execute
    succeeded
```

字串本身仍是建置時無法追蹤的輸入，但由字串到型別的對應不再是執行時探索，而是□生程式碼的 switch，因此在 UnityLinker 看來，與手寫程式碼同樣是普通的靜態參照。

以 Mono.Cecil 確認，兩個處理常式的型別、建構函式與 Execute，以及 CommandDispatcher.Execute 也都被保留。

這種取代的優點不僅是對 Stripping 的耐受性。

- 相依性對 UnityLinker 可見，因此不再需要維護 link.xml 或屬性的保留指定
- 執行時的反射探索消失，可降低□動成本與執行時失敗的可能性
- 對象的收集在編譯時進行，因此可在編譯時診斷登錄遺漏或重複

實際的程式庫中，也有採用此類實作的案例。

在序列化器方面，MessagePack for C# 長期提供針對 IL2CPP 環境的程式碼□生，目前已轉為以 Source Generator 為基礎的□生。

在 DI 容器方面，VContainer 提供以 SourceGenerator □生注入程式碼、減少執行時反射的選項，以 Source Generator 解決了前一節所見「建構函式消失導致解析失敗」的症狀。

在 Unity 專案中導入 Source Generator 的步驟（將 netstandard2.0 的 DLL 附上 RoslynAnalyzer 標籤後放置的構成），請參☒ Unity 手冊。

參考：Create and use a source generator https://docs.unity3d.com/6000.3/Documentation/Manual/create-source-generator.html / MessagePack for C# https://github.com/MessagePack-CSharp/MessagePack-CSharp / MemoryPack https://github.com/Cysharp/MemoryPack / VContainer https://github.com/hadashiA/VContainer

1.4 總結

Managed Code Stripping 不是提高 Stripping Level 就可以了事的最佳化。

如本章所見，經由反射或外部☒容的相依性在靜態分析中看不見，即使是執行時需要的程式碼，也會從建置中消失。

而且症狀只會出現在建置後的應用程式中，因此必須將明確指定看不見的相依性，與持續在建置後的應用程式中驗證，搭配運作。

明確指定保留的手段，依對象與情況區分使用（表 1.5）。

表 1.5: 對策的區分使用

方法	適用情況
[Preserve]	保留自己可編輯的程式碼中的少數元素
link.xml	保留已確定的對象，包含無法編輯的 DLL
Require 系列屬性	在框架端宣告「保留實作或衍生型別」的契約
IUnityLinkerProcessor	保留對象依建置目標或☒容而改變
Source Generator	將反射的探索本身替換為靜態參照

此外，列出讓 Stripping 更容易生效的方針。

- 除了會造成問題的情形外，不過度使用 [Preserve] 或 link.xml
- 減少以反射探索與建立型別，並考慮改為明確登錄或以 Source Generator 轉為靜態參照
- 保留程式碼的指定，盡可能縮小到型別、成員的單位。整個組件的 preserve="all" 僅限用於問題釐清

- 對於僅由與應用程式分開建置的□容所使用的型別，其保留不靠手動，而是活用 Addressables 的 link.xml 或以 IUnityLinkerProcessor □生等機制
- 從開發期間就執行並驗證已提高 Stripping Level 的建置

第2章

將調優寶典做成 AI 程式設計代理的技能

中村光寿 Teruhisa Nakamura

株式會社 Applibot

所屬: 株式會社 Applibot

2.1 前言

像本書這樣的調優知識有一個結構性的弱點：只對讀過的人有效。各章雖然收錄了以實測為依據的見解，但這些見解能否在專案的程式碼審⍰中被實際指出，取決於審⍰者是否讀過相關章節，並且能在審⍰當下想起來。我們無法期待全體成員背下所有章節，進入營運階段後若人員更替，這些見解也會逐漸淡化。⍰一方面，在撰寫程式碼的現場，AI 程式設計代理（Claude Code、OpenAI Codex、GitHub Copilot 等）已經常駐。只要將其納入審⍰請求或 CI，就能對所有差異套用相同的標準。目前缺少的，只是關於 Unity 效能的系統化評估標準。因此，本章將介紹如何把調優寶典的知識轉換為「AI 代理可以執行的評估項目」，並封裝成不依賴特定工具的通用技能。作者實際將前作中已公開知識所對應的 77 條規則，與從本書（2026 年版）原稿中提取的 86 條規則整合，共計 163 條評估規則並製作成技能。本章以此案例為基礎，整理轉換的設計方針與運用上的注意事項。

2.2 Agent Skills 這個通用格式

2.2.1 SKILL.md 的規格

作為讓 AI 代理具備專業知識的方法，2025 年底公開了名為 Agent Skills[*1] 的開放規格。技能的實體是以 SKILL.md 這個 Markdown 檔案為中心的目錄，結構如下。

▼清單 2.1　技能目錄的結構

```
unity-perf-review/ ├──
 SKILL.md          # 中繼資料 + 執行步驟（必要） ├──
 references/       # 規則集等參考文件 ├──
 scripts/          # 可執行□本（選用） └──
 assets/           # 範本等（選用）
```

SKILL.md 的開頭會寫入稱為 frontmatter 的中繼資料。必要欄位只有 name 與 description 兩個。

▼清單 2.2　SKILL.md 的 frontmatter（description 為配合版面而縮短，metadata 等選用欄位予以省略）

```
---
name: unity-perf-review
description: >
  Unity 從效能的觀點審□專案的程式碼、資□與設定。
  將 C# □本、著色器、.、metaProjectSettings 與
  163 的規則比對，並輸出 Critical / Warning / Info 的指摘。
  Unity 在被要求進行效能審□或最佳化調□時使用。
license: MIT
---
```

重點在於 description 的寫法。代理會將使用者的請求□容與 description 比對，以自動□用技能，因此不只寫「做什麼」，連「何時使用」也一併寫出，能提高□用的準確度。

2.2.2 主要代理的支援情況

這個規格的價□在於多個代理可以讀取相同的格式。2026 年 8 月時間點的支援情況如表 2.1所示。

[*1] Agent Skills 規格 https://agentskills.io/specification

表 2.1: 主要程式設計代理的技能支援（2026 年 8 月時間點）

代理	支援	放置位置
Claude Code	原生支援 SKILL.md[*2]	claude/skills/ 或 ~/.claude/skills/
OpenAI Codex	支援 SKILL.md	agents/skills/ 或 ~/.agents/skills/
GitHub Copilot	支援 SKILL.md[*3]	github/skills/
Gemini CLI	支援 SKILL.md[*4]	gemini/skills/ 或 ~/.gemini/skills/

Codex / Copilot / Gemini CLI 也支援共通的 .agents/skills/ 放置方式。此外 Copilot 也會讀取專案的 .claude/skills/，因此只要放在 .claude/skills/ 一處，就能同時涵蓋 Claude Code 與 Copilot（Copilot 的個人用放置位置為 ~/.copilot/skills/ 或 ~/.agents/skills/）。支援情況與放置位置會隨時間改變，導入時請以各代理的官方文件確認第一手資訊。只要將調優知識轉成 SKILL.md 格式，就能對使用任何代理的成員發送相同的審□標準（規則與步驟）。Agent Skills 統一的是封裝格式與步驟，並不會因模型與工具權限的差異而使執行結果一致。即便如此，「審□標準放在儲存庫，執行交給代理」的分工仍然成立。

2.3 將見解轉換為「評估項目」

技能化的核心不是撰寫 SKILL.md，而是將各章的見解拆解到代理能機械式套用的粒度。筆者統一採用以下格式。

▼清單 2.3 規則格式（取自 ParticleSystem 章的擷取範例。grep_pattern 的排除條件與處理方式的細節為配合版面而縮短）

```
### US11 對 ParticleSystem.Simulate 指定較大的秒數
- 等級：（□用Warning Noise 模組時為 ）Critical
- 對象：程式碼審□ / 資□設定
- 偵測模式：對 Simulate 的第 1 個引數傳入較大的秒數。
  特別是與□用 NoiseModule 的 ParticleSystem 併用時
- grep_pattern: \.Simulate\(
- 原因：Simulate 會一次模擬到指定的時間。
  實測中，指定 20 秒會使 NoiseJob 被呼叫約 2000 次，
  並□生約 50ms 的尖峰
- 處理：對□用 Noise 的粒子避免使用 Simulate
```

這個格式包含 4 項設計判斷。

2.3.1 明示偵測是否可機械化

每條規則都附上 grep_pattern，作為可對 C# 程式碼與 Unity 的 YAML（.meta、Prefab、ProjectSettings）機械執行的搜尋規格。多數是一條正規表示式，但也包含需要跨多行比對，或偵測「應有的描述不存在」的規則。代理先以此規格進行初步偵測，只精讀命中的位置後再判斷。如此即使是大型儲存庫，也能以實際可行的成本進行審□。不勉強寫出無法用正規表示式表達的項目，同樣重要。例如「未登錄於 Addressables 的共用資□被多個 Bundle 參照」無法用 grep 取得。這類規則要明確標示 (手動確認)，並在處理欄寫上 Addressables Analyze 等確認手段。若謊稱可以偵測，代理就會大量□生誤判，或是默默略過無法偵測的規則。

2.3.2 將實測□留在原因欄

「原因：效能會惡化」這類規則缺乏□服力。因此將原稿中的實測□直接引用到原因欄。

- 未壓縮紋理：2048x2048 下 Compressed 為 2.7MB / Uncompressed 為 16.0MB（約 6 倍差距）
- 每個角色一個圖的手動 Playable：600 個角色為 8.396ms，對比整體一個圖為 4.500ms（1.87 倍差距）
- Simulate(20f) 與 Noise 併用：NoiseJob 約 2000 次呼叫、約 50ms 的尖峰

有了數□，代理的指摘就能直接成為「為何應該修正」的□明，也能成為接受審□的開發者回到原稿相關章節的路徑。□外，這些都是原稿所記載特定環境下的實測□。規則的原因欄引用代表□，測量條件的細節則參照出處章節。

2.3.3 以等級與覆寫機制□起運用

規則分為 Critical / Warning / Info 三個等級（Critical = 確定造成效能劣化的原因，確認適用條件成立後必須修正；Warning = 視條件而定可能成為問題的潛在風險；Info = 建議選擇性改善），並□外提供專案端可覆寫的機制（rules-overrides）。例如即使「使用 LINQ 為 Critical」，在初始化時（init 情境）也可降為 Warning，在 Editor 工具（tooling）中則為 Skip。調優規則本質上依情境而定。若沒有覆寫機制，就會出現「我們的專案有很多不適用的指摘」，導致整個技能被停用。標準共通、適用依專案而定，這種兩層結構是長久維持的訣竅。

2.3.4 無法靜態偵測的見解歸入診斷知識

原稿的見解中，有許多是光讀程式碼無法得知的□容。例如「削減了 CPU 處理但影格時間沒有改變，時間移轉到 Semaphore.WaitForSignal 等等待標記時，應懷疑渲染執行緒位於關鍵路徑，並在 Timeline 檢視中確認等待對象」，這類是關於 Profiler 解讀方式的見解。「僅憑等待標記無法斷定瓶頸」這項限制，也同樣作為見解保留下來。這些□容與評估規則分開，□存為「症狀 → 原因 → 處理」的表格檔案（表 2.2）。在筆者的儲存庫中，效能分析用的代理（Claude Code 用）只在診斷時才會載入。本章散布的技能將其作為參考資料一併收錄，當靜態審□無法判斷的疑慮要以「建議進行效能分析測量」回報時，可作為依據。

表 2.2: 診斷知識的範例

症狀	原因	處理
CPU 圖表呈現谷底與尖峰交錯的梳狀	忘記錯開抽稀的相位	以 index % interval 分散相位
MeshSkinning.CalcMatrices 偏大	骨骼 Transform 階層被實體化	套用 Optimize Game Objects
僅 iOS，沒有錯誤與 Shader.CreateGPUProgram，只出現 GpuProgramMetal.GetCachedPipeline	懷疑著色器在 AssetBundle 中重複打包	確認重複的實例並獨立成專用 Bundle
僅 iOS，預熱完成後 SubPass □的著色器仍進行 PSO 編譯	SubPass（Native Render Pass）□著色器的預熱失敗	重新檢視 SubPass，或在載入期間預先渲染

2.4 組裝成技能

2.4.1 分階段載入與檢□清單方式

若將 163 條規則全部寫入 SKILL.md 本文，會壓迫代理的上下文。Agent Skills 的設計思想是分階段載入（progressive disclosure）。

1. 代理隨時只看 frontmatter 的 name 與 description
2. 技能□用後，讀取 SKILL.md 本文（執行流程）
3. 只從 references/ 讀取需要的參考文件

初期讀取□些□容（是否包含路徑等）依代理的實作而異，但「本文與參考資料在□用前不會被讀取」這項原則是共通的。這裡要注意的是完整性。單純的做法是「依對象檔案

種類，整份載入各類別的規則檔案」，但此方式會讓代理在看程式碼之前，先讀取最多超過 2 萬 token 的規則（由字數概算，token 數依模型而異）。若讓「記住數十條規則並全部套用」依賴模型的注意力，部分規則就會被默默略過。因此，我們自動□生檢□清單，將全部規則壓縮為每條規則一行的「ID / 等級 / 適用條件 / grep_pattern / 規則名稱 / 詳細檔名」，並將審□分成兩個階段。

1. **初步掃描**：隨附□本 scan.py 對對象檔案機械執行檢□清單所有行的 grep_pattern，輸出命中清單（檔案:行）與 Coverage（沒有 python3 的環境則由代理直接執行所有行 grep 作為備援）
2. **二次精□**：只針對命中的規則，從詳細檔案讀取偵測條件、原因與處理，並排除誤判

無法用 grep 表達的部分規則，在檢□清單上明確標為 MANUAL，只在符合對象種類時才精讀。以「應有的描述不存在」作為指摘條件的規則則標上 ABSENT，在 grep 沒有命中時確認詳細□容（這是為了避免命中與否同指摘條件的對應被顛倒而明確標示）。將初步掃描做成□本，是因為讓 LLM 逐一執行數十條 grep 可能會漏讀，而且正規表示式的方言差異（PCRE 的先行斷言在 macOS 標準 grep 上無法運作等）會使結果依環境而異。如此一來，初步掃描的完整性便與模型的記憶力□鉤。指摘是否正確（二次精□中排除誤判）仍由代理判斷。事前只需讀取檢□清單（程式碼用約 16KB、資□用約 8KB。token 數依模型而異，合計概算約 1 萬 token 上下），詳細□容只讀取命中的部分。檢□清單由規則本體自動□生，因此新增規則時不需要手動同步。

2.4.2　依環境排除規則

規則中有些依賴 Unity 版本或組態。本技能收錄的 RenderGraph 相關規則（URP 用）以 Unity 6 以上為對象，LTO 類僅限 IL2CPP，iOS 特有的 PSO 問題與僅發行 Android 的作品無關。比對不適用的規則只會造成雜訊與額外工夫。因此為依賴環境的規則附上適用條件標籤（unity>=6000.3、urp、il2cpp、ios 等），並與由專案設定檔□生的環境設定檔比對，只排除「明確不滿足條件的規則」。標籤的比對由初步掃描□本以確定性的方式處理，無法偵測的項目（unknown）則歸向安全側（仍作為比對對象），被略過的規則 ID 會列於報告中以便稽核。□生步驟在導入步驟一節□明。

2.4.3　強制輸出規則 ID

輸出格式要求每項指摘都必須附上規則 ID。

▼清單 2.4　審□輸出的範例

```
### Critical
- Assets/Scripts/EffectPlayer.cs:42 -
  對□用 Noise 的粒子呼叫 Simulate(20f) [US11]
  - 依據：NoiseJob 被呼叫約 2000 次，□生約 50ms 的尖峰（實測）
  - 建議：移除 ，或將秒數縮到最小Noise
```

有了 ID，就能將指摘的依據追溯到規則檔案進行驗證，並與「代理只是□得像有那麼回事」區分開來。即使是誤判，也能明確知道該修正□條規則的偵測模式。

2.5 發佈與放置到各代理

技能的公開只要整個目錄共享即可成立，GitHub 儲存庫或 zip 皆可。使用者只要複製到表 2.1的放置位置，各代理就能發現該技能。運用上的重點是維持規則的單一來源。在筆者的儲存庫中，規則本體只放在 rules/ 目錄，發佈用封裝由□本組裝（使用者端的導入步驟見下一節）。

▼清單 2.5　發佈封裝的組裝（維護者用）

```
./package-skill.sh
cp -r dist/unity-perf-review /path/to/repo/.claude/skills/   # Claude Code
cp -r dist/unity-perf-review /path/to/repo/.agents/skills/   # Codex / Copilot / Gemini
     CLI
cp -r dist/unity-perf-review /path/to/repo/.github/skills/   # Copilot 專用放置（上
    面 2 行任一個都能涵蓋，故為選用）
```

若開始依代理分別管理規則，每次更新標準都會□生偏差。建議一開始就決定「來源只有一處，放置交給□本」。

2.6 導入自己的專案

以下是讓上述機制在讀者專案中運作的步驟。若已安裝 python3，複製與初始□生的時間約 10 分鐘。新增到儲存庫的只有技能目錄與環境設定檔（tuning-env.md），不會變更 Unity 的 Assets / ProjectSettings。

2.6.1 取得與放置技能

整套技能（SKILL.md、收錄 163 條規則的 references/、環境偵測／初步掃描／檢□清單重新□生的 3 個□本、導入步驟的 README）公開於本書儲存庫的 skills/unity-perf-

review/*5。只要取得後，複製到所用代理的技能目錄即可。□外，技能是包含「給代理的指示與可執行□本」的發佈物。來自不可信任發佈來源的技能，請先確認 SKILL.md 與 scripts/ 的□容再導入，並建議固定版本。

▼清單 2.6　取得與放置技能

```
git clone https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026
    .git
cd UnityPerformanceTuningTips2026/skills

# 依所用的代理，複製到其中之一
cp -r unity-perf-review /path/to/your-repo/.claude/skills/   # Claude Code
cp -r unity-perf-review /path/to/your-repo/.agents/skills/   # Codex / Copilot / Gemini
     （共通放置）
    CLI
cp -r unity-perf-review /path/to/your-repo/.github/skills/   # Copilot 專用放置（上
    面 2 行任一個都能涵蓋，故為選用）
```

放在儲存庫根目錄，就能將相同的審□標準發送給全體成員。若只是個人試用，放在主目錄側（~/.claude/skills/ 等）也能運作。

2.6.2　放置後的狀態

以 Claude Code 為例，完成放置與下一項的環境設定檔□生後，儲存庫會呈現以下狀態（清單 2.7）。

▼清單 2.7　放置後的儲存庫結構

```
your-repo/
|-- .claude/
|   \`-- skills/
|       \`-- unity-perf-review/
|           |-- SKILL.md                      # name/description + 執行流程
|           |-- README.md                     # 供人□讀的導入步驟
|           |-- references/
|           |   |-- checklist-code.md         # 初步掃描用（程式碼類）
|           |   |-- checklist-asset.md        # 初步掃描用（資□／設定類）
|           |   |-- checklist-profile.md      # 效能分析用（靜態審□中未使用的附屬資料）
|           |   |-- rules-csharp.md 等共 13 個檔案  # 詳細規則（共 163 條）
|           |   |-- diagnosis-knowledge.md    # 症狀原因處理的診斷集->->
|           |   \`-- rules-overrides.example.md
|           \`-- scripts/
```

*5 https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026/tree/main/skills/unity-perf-review

```
|                |-- detect-env.py              # ☒生環境設定檔
|                |-- scan.py                    # 機械執行初步掃描
|                \`-- gen-checklists.py          # 修正規則時重新☒生檢☒清單
|-- tuning-env.md                               # 環境設定檔（於下一項☒生）
\`--  （Unity 專案本體不變更）
```

3 份檢☒清單依對象別（程式碼 / 資☒ / 效能分析）區分，同一條規則可能列在多份中，因此行數合計與規則數 163 不一致。放置的時間點，原則上載入代理的只有 frontmatter 的 name 與 description（合計數百 token。如前所述，依實作可能加上路徑等）。請求審☒後，會依前述的分階段載入，依序只讀取必要的部分：SKILL.md 本文 → 對應對象的檢☒清單 → 命中規則的詳細☒容。通常不需要一次投入 163 條規則全文（約 125KB，概算 4 萬 token 多）。rules-overrides.example.md 維持原樣是無效的。要開始專案專用的調整時，將其重新命名為 rules-overrides.md 後編輯（詳見後述）。

2.6.3 ☒生環境設定檔

如前所述，163 條規則中包含依賴環境的規則。為了讓代理判斷排除不適用於專案的規則，需要☒生環境設定檔。

▼清單 2.8 ☒生環境設定檔（在 clone 下來的儲存庫的 skills/ 中執行）

```
python3 unity-perf-review/scripts/detect-env.py \
    /path/to/your-repo /path/to/your-repo/tuning-env.md
```

若在放置後執行，請將路徑替換為放置位置（.claude/skills/ 或 .agents/skills/ 等，所用代理的技能目錄）。即使是 Unity 專案不在儲存庫根目錄、而位於子目錄的 monorepo 組態，只要傳入儲存庫根目錄，就會遞迴搜尋設定檔並偵測。執行☒本需要 python3（沒有的環境可手動建立 tuning-env.md，或跳過此步驟以全規則比對方式運用。初步掃描的 scan.py 也使用 python3，但沒有時會以代理直接執行 grep 的備援方式運作）。☒本會從 ProjectVersion.txt、Packages/manifest.json、GraphicsSettings.asset、ProjectSettings.asset 推定 Unity 版本、渲染管線、Scripting Backend 與是否有 Addressables。這終究只是從設定檔推定的☒，確定的資訊以手動填寫到 tuning-env.md 為準。無法偵測的項目（依平台分別設定的 Scripting Backend 混用時等）會成為 unknown，並歸向安全側（仍作為比對對象）處理。請開☒輸出的 tuning-env.md 確認☒容，特別是發行平台與是否使用 SideFX VAT 請手動修正，並確認 render_pipeline 是否為預期的☒。

2.6.4　專案專用的調整（選用）

若有刻意偏離規則的部分，就在 rules-overrides.md 中記錄等級的覆寫，例如允許初始化時使用 LINQ、在 Editor 工具中停用日誌規則等調整。分工是：環境設定檔負責排除「在該環境中原理上無法適用」的規則，而 overrides 負責「專案因素的調整」。若要新增或修改規則本身，請編輯 references/ 的 rules-*.md，並以 python3 scripts/gen-checklists.py references/ 重新□生檢□清單。檢□清單是自動□生物，因此不直接編輯。只靠發佈物就能完成這個新增與重新□生的循環。

2.6.5　執行

只要對代理提出請求即可。通常會因請求文字與 description 一致而自動□用（□用由模型判斷，因此若要確實使用，請在請求文字中明寫技能名稱 unity-perf-review，或使用各代理的明確呼叫方式）。

▼清單 2.9　審□請求的範例

```
請從效能的觀點審□這份差異

Assets/Scripts/Battle/ 請用調優寶典的規則檢□此目錄下的□容
```

輸出最後附上的 Coverage 區段，可確認比對的規則數、因環境而略過的規則數（附 ID）與 grep 命中數。這些數量是初步掃描□本輸出的轉錄，不依賴代理的記憶（只有二次精□中採用與排除誤判的數量由代理彙總）。若覺得排除過多，請調整 tuning-env.md；若覺得指摘過多，請調整 rules-overrides.md。

2.7　運用上的注意事項

以下列出實際導入專案後得知的注意事項。**初次從差異或特定目錄開始，而非整個儲存庫。** 初步掃描本身即使對整個專案也能在短時間□結束，但在有運用歷史的程式碼庫中，命中的規則會達數十條的規模，二次精□（排除誤判）的負擔會集中在此。先在較窄的範圍□精□，將長期允許的項目寫入 overrides 後再擴大對象，指摘比較不會被雜訊淹沒而能□穩定落地。**靜態分析也會指出刻意的設定。** 例如刻意不壓縮的紋理，也會在 AS02（未設定紋理壓縮）命中。請不要照單全收偵測結果而一併修正，應逐件確認意圖後再修正。長期允許的項目附上理由記錄在 overrides 中，可減少之後的雜訊。overrides 的調整單位是整條規則或情境（in-game = 熱路徑 / init = 初始化 / tooling = Editor），僅針對

特定資□的例外，則在備註欄寫上路徑與理由，作為代理的判斷依據（專用的適用範圍指定在目前版本中尚未支援）。**實測□附有測量環境。** 規則所引用的數□，是在特定 Unity 版本與裝置上的實測。不能將「Pixel 8a 上的實測」外推到最新高階裝置的判斷。請在規則本身寫上對象版本，並將 Unity 或渲染管線的更新作為規則重新檢視的觸發點。**最後做判斷的是人。** 借用本書其他章節的話，就是「制定與重新檢視規約的是人，讓人遵守的是機器」。AI 代理是「讓人遵守的機器」的新實作，對規約＝評估項目是否妥當負責的仍然是人類。若無法認同代理的指摘，請將其視為應改善規則的偵測模式或原因欄的訊號。

2.8 結語

將調優寶典的見解規則化後，筆者深刻體會到：「同時寫有實測□與處理方式的原稿，幾乎可以直接成為機器可讀的評估項目」。反過來□，沒有依據數□的經驗法則，在規則化的階段就會暴露其薄弱之處。這也是對撰寫端的回饋。明記測量環境與數□的見解，不只送達書籍的讀者，也會透過 AI 代理送達所有的程式碼審□。本章介紹的技能與規則集，與其□是唯一的正解，不如□是起點。期待它隨著各專案的 overrides 逐步成長、新章節的見解陸續以規則加入，成為一份活的規約集。

[*2] Claude Code 的 Skills 文件 `https://code.claude.com/docs/en/skills`

[*3] GitHub Docs "About agent skills" `https://docs.github.com/en/copilot/concepts/agents/about-agent-skills`

[*4] Gemini CLI 的 Agent Skills 文件 `https://geminicli.com/docs/cli/using-agent-skills/`

第3章

前言

塩塚 勇気 Yuki Shiotsuka

QualiArts

標題：以 ResetControllerState 安全地進行 Animator 物件池化姓名：塩塚 勇気羅馬拼音：Yuki Shiotsuka所屬：QualiArts

本文探討 Unity 6.3 新增的 Animator.ResetControllerState。

本文將 ResetControllerState 整理為「可明確且簡單地實作物件池歸還時初始化的 API」，並根據在 Unity 6000.3.21f1 中確認的行為，□明安全的物件池歸還處理。

3.1 Animator 物件池化所□生的問題

ObjectPool 是藉由重複使用已建立的實例，來降低頻繁建立與銷毀之負載的常用手法。Unity 的 ObjectPool 也具備可在取得、歸還與銷毀的時機插入處理的機制[*1]。

為了物件池而停用 GameObject 時，再次□用後 Animator 會執行 Rebind，可能抵銷物件池帶來的效能收益。因此，Unity 官方的 Animator 類別□明建議停用元件，而非停用 GameObject[*2]。本文依循此建議，以下列物件池為前提：GameObject 保持□用狀態來保管，歸還時停用 Animator 與 Renderer，取得時再重新□用。

□一方面，既然物件池會重複使用實例，GameObject 與元件就可能帶著上次使用時的狀態回到池中。Animator 也是如此，停用後停止的只有求□，□部狀態會原樣保留。

[*1] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html

[*2] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html

若不做任何處理，下次使用時會沿用上次的 State 與各種 Parameter，因此可能從上次中途的 State 開始播放，或因殘留的 Parameter 而發生非預期的轉換。

ResetControllerState 是可將 AnimatorController 各 Layer 的 State 與 Parameter 一併還原為預設狀態的 API。發行□明中也將其作為方便對有動畫的角色進行物件池化的新增功能加以介紹[*3]。

3.2 以 ResetControllerState 重設的項目

若將 ResetControllerState 的參數 resetParameters 指定為 true，除了 State 之外，Parameter 也會還原為預設□。指定為 false 時，只有 State 會還原[*4]。將附有 Animator 的角色歸還到物件池時，通常使用 ResetControllerState(true)，一併還原 State 與 Parameter。無論是未使用、一般轉換、AnyState 轉換或 BlendTree 的用途，Float、Bool、Int 都會回到預設□，Trigger 則會被重設。此外，所有 Layer 的目前 State 都會還原為預設 State。

不過，各項目反映的時機並不相同。Parameter 的□在呼叫後立即成為預設□，但 GetCurrentAnimatorStateInfo 在 Animator 下次求□之前，仍會傳回重設前的 State。

AnimatorControllerPlayable 也提供了重設 Controller 的 State 與 Parameter 的 ResetControllerState 方法[*5]。

3.3 不會被 ResetControllerState 重設的項目

□一方面，以下項目僅靠 ResetControllerState 不會回到預設□。

- 以 SetLayerWeight 變更的 Layer Weight
- Animator.speed
- 在 StateMachineBehaviour 中實作的自訂執行階段變數

Layer Weight 與 Animator.speed 是與 Controller 的 State 及 Parameter 分開的屬性。

[*3] https://unity.com/releases/editor/whats-new/6000.3.0f1

[*4] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html

[*5] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

若不希望在歸還物件池時保留變更後的□，請明確地將其還原。StateMachineBehaviour 的自訂變數同樣需要個別初始化。

此外，Rigidbody、ParticleSystem、AudioSource 等其他元件的狀態，以及自訂邏輯的狀態，都在 AnimatorController 之外管理，因此不屬於 ResetControllerState 的處理對象。

3.4 實作範例

清單 3.1 是將歸還物件池時的初始化與停用整合在一起的範例。預期從 ObjectPool<T> 的 actionOnRelease 中呼叫。依專案規格不同，其中也包含不必要的處理。

▼清單 3.1　歸還至物件池時的 Animator 初始化範例

```
// 整合歸還物件池時所需的初始化與停用
public void ReturnToPool()
{
    // 還原 AnimatorController 各 Layer 的 State 與 Parameter
    animator.ResetControllerState(true);
    // defaultLayerWeight □外設定為重複使用時要還原的□
    animator.SetLayerWeight(layerIndex, defaultLayerWeight);
    // 還原播放速度
    animator.speed = 1f;
    // 僅在必要時執行。在歸還處理中完成重設後的第一次求□
    animator.Update(0f);
    // 還原 Animator 以外的自訂狀態
    ResetGameState();
    // GameObject 保持□用，停止求□與繪製後保管
    animator.enabled = false;
    characterRenderer.enabled = false;
}
```

3.5 總結

ObjectPool 並不是只減少建立與銷毀的次數就完成了。必須決定重複使用的 GameObject 在下次使用時應從□個狀態開始，並將還原到該狀態的處理一併納入設計。

若不停用 GameObject，而是停用 Animator 來保管，Animator 的□部狀態會原樣保留。使用 ResetControllerState(true) 即可將 AnimatorController 各 Layer 的 State，以及 Float、Bool、Int、Trigger 的 Parameter 一併還原為預設□。□一方面，Layer Weight、Animator.speed、StateMachineBehaviour 的自訂變數，以及其他元件與自訂邏輯的狀

態不會被重設。

將初始化對象區分為 AnimatorController 以外，歸還至物件池的處理職責就會變得明確。建議以 ResetControllerState 為核心，只補上專案特有的狀態來組成歸還處理。

3.6 參考資料

- Unity Scripting API - ObjectPool<T0>
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html
- Unity Scripting API - Animator
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html
- Unity 6000.3.0f1 Release Notes
  https://unity.com/releases/editor/whats-new/6000.3.0f1
- Unity Scripting API - Animator.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html
- Unity Scripting API - AnimatorControllerPlayable.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

第4章

Unity 3D 角色動畫的調優

斉藤 俊介 Shunsuke Saito

株式會社 Colorful Palette

- 姓名: 斉藤俊介 - Shunsuke Saito
- 所屬: 株式會社 Colorful Palette

4.1 前言

本章將從資□設定與實作等多個面向，介紹「Unity 中 3D 角色動畫功能的最佳化方法」。

4.1.1 測量環境

本章的實作與測量在以下環境中進行。

表 4.1: 測量環境

項目	□
Unity	6000.5.7f1
平台	macOS Standalone (StandaloneOSX)
Scripting Backend	IL2CPP / IL2CPP Compiler Configuration = Release / Code Generation = OptimizeSpeed
Graphics API	Metal / GPU Skinning = □用
Quality Level	PC / Skin Weights = Four Bones

4.1.2 使用的資□與其結構

測量所用的角色以 unity-chan 為基礎，重新匯入為 **Generic 骨架**。後述的 Optimize Game Objects 套用前後，規格如下。

表 4.2: 測量所用預置物的 Transform 數與頂點數

預置物	Transform 數	頂點數
Base	166	13,391
Optimized（Optimize Game Objects）	24	13,391

4.2 Unity Animation 的基本控制

首先整理在 Unity 中控制動畫的 3 種方法。

4.2.1 使用 Animator 元件控制

這是為 `Animator` 指派 `RuntimeAnimatorController` 的最常見方法。狀態機、轉換、混合樹、參數等機制一應俱全，可直接使用 Mecanim 的工作流程。

▼清單 4.1　以 Animator 元件控制

```
var animator = GetComponent<Animator>();
animator.runtimeAnimatorController = controller;   // 指派狀態機
animator.SetFloat("Blend", 0.5f);                  // 以參數控制
```

重點在於，Animator 在□部**建構並執行 Playable Graph**。狀態機的評估與剪輯的取樣，在□部都是作為圖的節點處理。接下來的 2 種方法，相當於自行建構這個□部圖。

4.2.2 以 Playable API 建構 Playable Graph，並以自動更新控制

這是自行建構 `PlayableGraph` 的方法。無須經過狀態機即可完全控制混合結構，並可將後述的 Animation Job（`AnimationScriptPlayable`）插入圖中的任意位置。

▼清單 4.2　以 Playable API 建構圖（自動更新）

```
var graph = PlayableGraph.Create("MyGraph");
var output = AnimationPlayableOutput.Create(graph, "Animation", animator);
```

```
var clipA = AnimationClipPlayable.Create(graph, waitClip);
var clipB = AnimationClipPlayable.Create(graph, walkClip);
var mixer = AnimationMixerPlayable.Create(graph, 2);
graph.Connect(clipA, 0, mixer, 0);
graph.Connect(clipB, 0, mixer, 1);
mixer.SetInputWeight(0, 0.5f);
mixer.SetInputWeight(1, 0.5f);

output.SetSourcePlayable(mixer);
graph.Play();
```

更新模式預設為 `DirectorUpdateMode.GameTime`。在此模式下，會在 Unity 的生命週期（位於 Update 與 LateUpdate 之間的 Animation 階段）中，與 Animator 元件在相同時機**自動評估**。呼叫端無須每個影格□外做任何事。

4.2.3 以 Playable API 建構 Playable Graph，並以手動更新控制

對同一個圖設定 `DirectorUpdateMode.Manual` 後，Unity 將完全不進行自動評估，只有在呼叫 `Evaluate()` 時才會評估該圖。

▼清單 4.3　手動更新時的圖評估

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    graph.Evaluate(Time.deltaTime);   // 只有呼叫時才會評估
}
```

由於可以完全控制評估的時機與頻率，因此「僅停止特定角色（圖）的更新」「每數個影格才更新一次」等控制可以直接實作。

3 種方法的關係整理如下。

表 4.3: 動畫控制 3 種方法的比較

方法	圖的建構	評估時機
Animator 元件	Unity（□部）	Unity 的 Animation 階段
Playable API + 自動更新	自行建構	Unity 的 Animation 階段
Playable API + 手動更新	自行建構	呼叫 `Evaluate()` 時

4.3 資□的設定

在介紹程式碼端的最佳化之前，先看看可在資□端設定中套用的最佳化。本節的主題是**消除骨骼的 Transform 階層**。有 2 種方法可以得到相同的結果。

4.3.1 Optimize Game Objects（匯入時）

在 FBX 匯入設定（Rig 分頁）中□用 `Optimize Game Objects` 後，執行時不會再建立骨骼的 GameObject / Transform，Animator 只持有□部骨架。以這次的角色而言，**Transform 從 166 個減少為 24 個**（根物件 + 23 個 Renderer）。

※由於骨骼不再以 GameObject 的形式存在，沿著階層存取骨骼的程式碼將無法運作。若只需要保留特定骨骼（例如武器的附加點），請以 `Extra Transforms to Expose` 個別保留。

4.3.2 Optimize Hierarchy（執行階段）

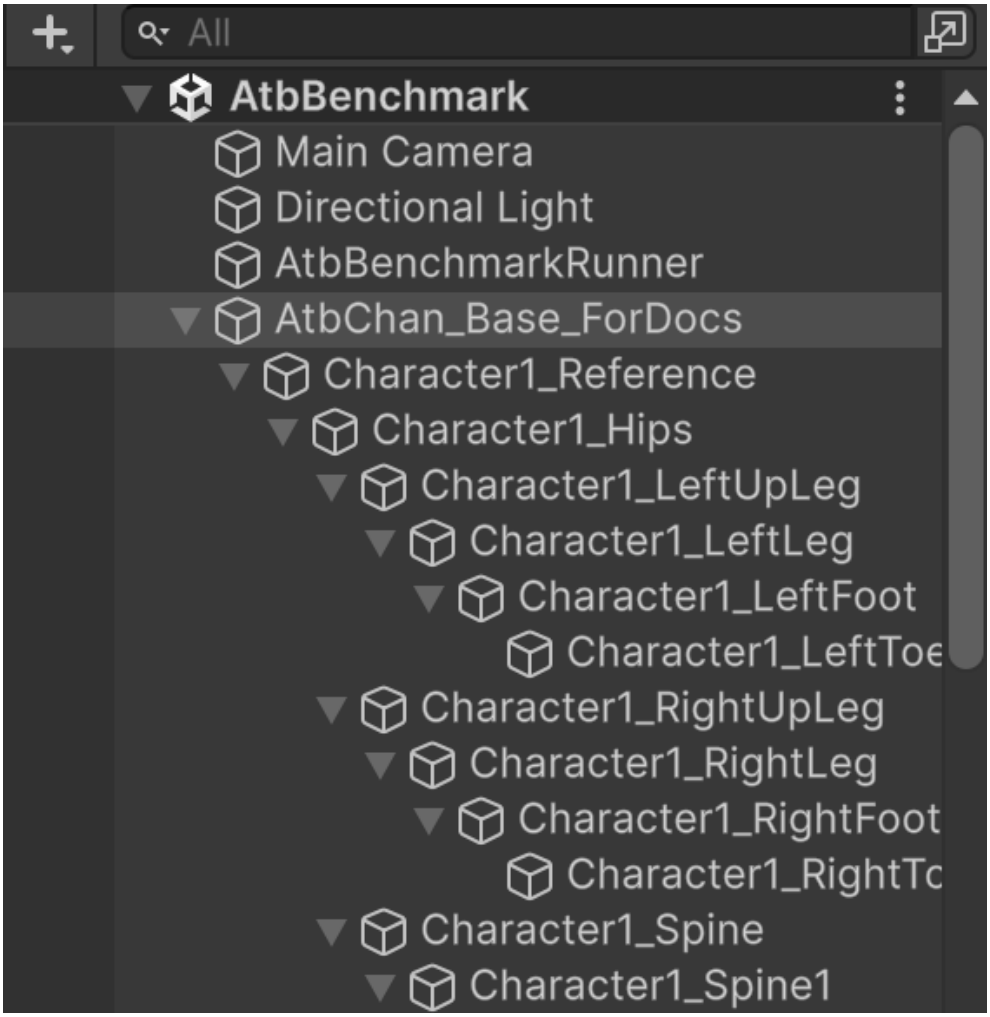
在執行時進行相同處理的 API 是 `AnimatorUtility.OptimizeTransformHierarchy`。

▼清單 4.4　執行階段的階層扁平化

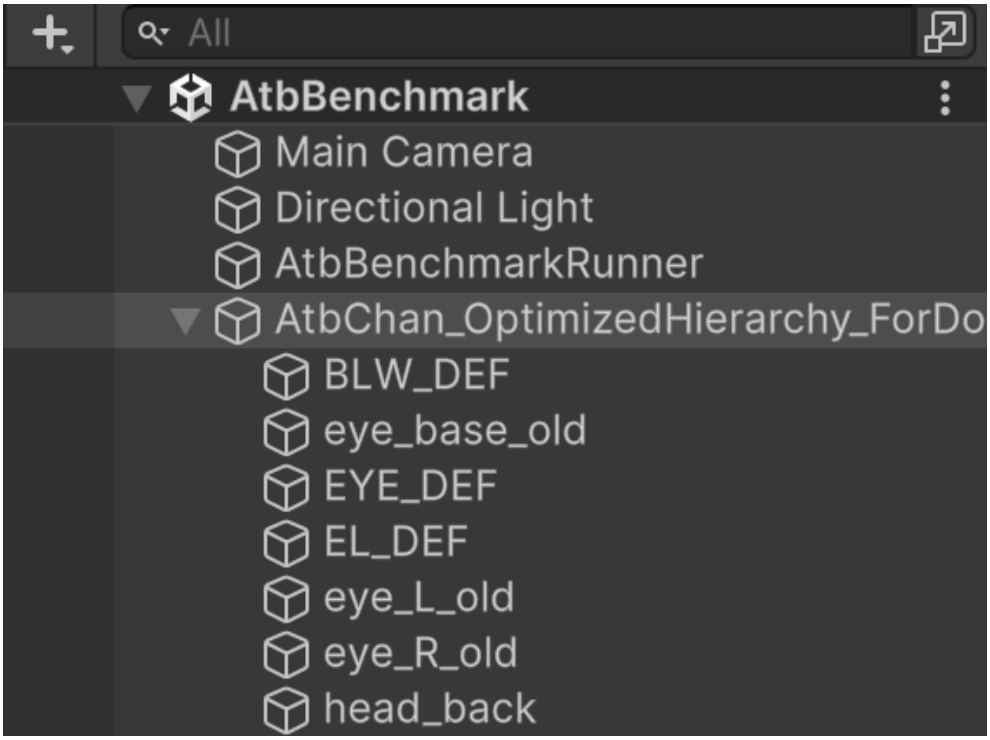
```
// go 是持有 （以及有效Animator ）的根物件。第Avatar 2 個引數是要公開的骨骼名稱陣列
AnimatorUtility.OptimizeTransformHierarchy(go, null);

// 也有反向轉換（在編輯器擴充或除錯時想暫時還原階層的情況）
AnimatorUtility.DeoptimizeTransformHierarchy(go);
```

即使是無法變更匯入設定的資□，在建立時執行也能得到相同的結構。實際確認後，對 Base 預置物（166 個 Transform）套用的結果為 **24 個 Transform**・`hasTransformHierarchy == false`，與匯入時套用 Optimize Game Objects 的預置物結構完全相同。



▲圖 4.1　最佳化前的 Hierarchy。`Character1_Hips` 底下接續 166 個骨骼 Transform 階層。



▲圖 4.2 套用 `AnimatorUtility.OptimizeTransformHierarchy` 後。骨骼階層消失，根物件正下方只剩 23 個 SkinnedMeshRenderer（24 個 Transform）。

4.3.3 實測: 階層扁平化的效果

繪製負載分 2 個等級進行測量。第 1 種是□用全部 Renderer 的配置（300 隻 × 23 個 Renderer = 6,900 個 Renderer），第 2 種是每個角色只□用頂點數最多的 1 個 SkinnedMeshRenderer、降低繪製負載的配置。影格時間為中位數。

表 4.4: 階層扁平化的效果（300 隻・全部 Renderer 與 600 隻・1 個 Renderer）

配置	變體	影格時間	`MeshSkinning.Update` (main)	`MeshSkinning.CalcMatrices` (all)	`Semaphore.WaitForSignal`
300 隻・全部 Renderer	Base	18.59 ms	6.23 ms	36.633 ms	5.33 ms

表 4.4: 階層扁平化的效果（300 隻・全部 Renderer 與 600 隻・1 個 Renderer）

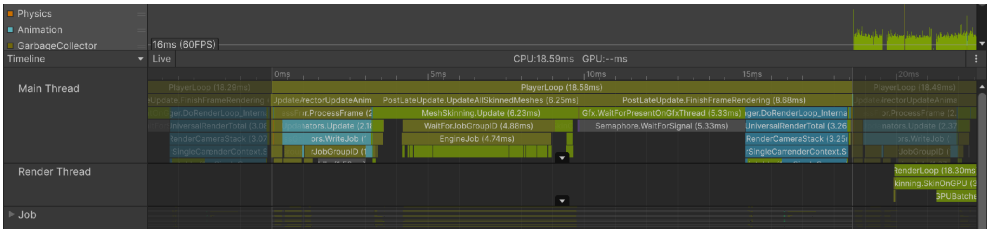
配置	變體	影格時間	`MeshSkinning.Update` (main)	`MeshSkinning.CalcMatrices` (all)	`Semaphore.WaitForSignal`
300 隻・全部 Renderer	Optimize Game Objects	18.497 ms (±0)	**1.537 ms** (−75.7 %)	**0 ms**	**10.255 ms**
300 隻・全部 Renderer	Optimize Hierarchy	18.307 ms (±0)	**1.504 ms** (−76.2 %)	**0 ms**	**9.977 ms**
600 隻・1 個 Renderer	Base	6.797 ms	0.735 ms	4.385 ms	5.741 ms
600 隻・1 個 Renderer	Optimize Game Objects	**5.991 ms (−11.9 %)**	0.207 ms	**0 ms**	4.901 ms
600 隻・1 個 Renderer	Optimize Hierarchy	**6.009 ms (−11.6 %)**	0.208 ms	**0 ms**	5.010 ms

CPU 端的處理在兩種配置下都以相同比例減少。主執行緒的蒙皮處理從 6.321 降至約 1.5 ms（全部 Renderer）、從 0.735 降至約 0.21 ms（1 個 Renderer），皆削減 7 成以上。至於蒙皮矩陣的計算（`MeshSkinning.CalcMatrices`），所有執行緒合計的 36.6 ms / 4.4 ms **完全消失**。

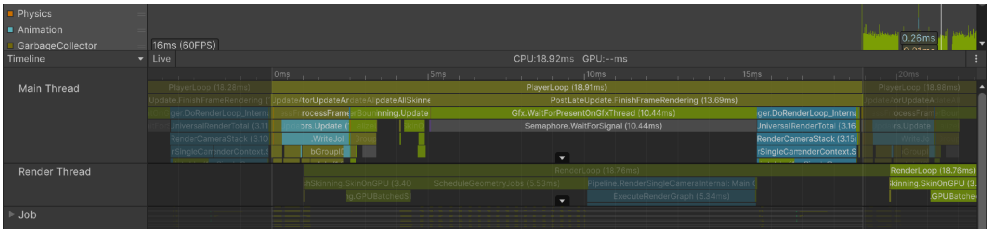
然而在「全部 Renderer 配置」中，渲染執行緒的處理成為影格時間的關鍵路徑，因此影格時間並未縮短。

在降低繪製負載的 1 個 Renderer 配置中，相同的削減直接體現為**約 12 % 的影格時間縮短**。300 隻時也是相同的趨勢，為 3.312 → 2.899 / 2.901 ms（− 12.5 % / − 12.4 %）。**匯入時的 Optimize Game Objects 與執行階段的 Optimize Hierarchy 之差未滿 0.1 %**，可見結構相同，效果也相同。

在 Profiler 的 Timeline 中也發生相同的情況。兩者皆為 300 隻・全部 Renderer、影格時間中位數附近的影格。



▲圖 4.3 Base（166 個 Transform）。在 18.59 ms 的影格時間中，主執行緒上 `MeshSkinning.Update` 佔 6.23 ms，`Gfx.WaitForPresentOnGfxThread` / `Semaphore.WaitForSignal` 為 5.33 ms。



▲圖 4.4 套用 `AnimatorUtility.OptimizeTransformHierarchy` 後。`MeshSkinning.Update` 的區塊縮小到看不清標籤的程度，□一方面 `Semaphore.WaitForSignal` 從 5.33 延長為 10.44 ms，`PlayerLoop` 為 18.58 → 18.91 ms，幾乎沒有變化。減少的 CPU 時間被等待時間取代。

4.3.4 實測: 停用全部 Renderer（僅動畫評估）

完全排除繪製與蒙皮，觀察對動畫評估本身的影響。為 300 隻。

表 4.5: 停用全部 Renderer 時的影格時間

變體	影格時間
Base	2.702 ms
Optimize Game Objects	2.687 ms (− 0.6 %)
Optimize Hierarchy	2.496 ms (− 7.6 %)

剪輯取樣與混合等評估處理無論有無 Transform 階層都會執行，因此可□對這部分幾乎沒有影響。由此可知，這項最佳化的目標主要是**蒙皮矩陣的計算與寫回 Transform**，而非動畫評估本身。

4.3.5 本章的結論

- 階層扁平化能削減**蒙皮矩陣計算**（`MeshSkinning.CalcMatrices`）
- 匯入時（Optimize Game Objects）與執行階段（`AnimatorUtility.OptimizeTransformHierarchy`）的效果相同
- 選擇⌧一種屬於營運問題: 自家資⌧以匯入設定處理即可，若因故無法變更匯入設定則使用執行階段 API
- **在製作階段就將模型本身的頂點數、骨骼數調整至適當範圍**為最優先事項

4.4 活用 Job Thread

4.4.1 Unity 的動畫平行化

Animator 元件與自動更新的 Playable Graph，由 Unity **統一排程所有圖，並分散到 Animation Job Thread**。

問題在於手動更新。`PlayableGraph.Evaluate()` **只會平行化該圖⌧部的評估處理**，不會跨圖平行化。因此，若以「1 個角色 = 1 個 Playable Graph」這種直覺的配置控制 N 隻角色，就會呼叫 N 次 `Evaluate()`，**N 個圖會序列處理**。

這是 Unity Playable Graph 無法避免的規格，但對策很簡單，就是**將 N 隻合併為 1 個圖**。

▼清單 4.5　將所有角色彙整為 1 個圖

```
// 在 1 個圖上掛載各角色的輸出與子樹
_sharedGraph = PlayableGraph.Create("Shared");
_sharedGraph.SetTimeUpdateMode(DirectorUpdateMode.Manual);

foreach (var character in characters)
{
    var output = AnimationPlayableOutput.Create(_sharedGraph, "Animation", character.
    Animator);
    output.SetSourcePlayable(BuildClipSource(_sharedGraph));   // 各角色的混合器
}

_sharedGraph.Play();

// 只更新 1 次
```

```
void Update() => _sharedGraph.Evaluate(Time.deltaTime);
```

4.4.2 實測 1: 控制方法 × 角色數

停用全部 Renderer，只取出動畫評估的成本。為影格時間的中位數。

表 4.6: 各控制方法與角色數的影格時間

控制方法	100 隻	300 隻	600 隻
Animator 元件	0.899 ms	2.413 ms	**5.004 ms**
Playable API + 自動更新	0.899 ms	2.498 ms	**5.093 ms**
Playable API + 手動更新（1 隻 1 個圖）	1.395 ms	3.925 ms	**8.396 ms**
Playable API + 手動更新（整體 1 個圖）	0.800 ms	2.115 ms	**4.500 ms**

Animator 與 Playable 自動更新的成本幾乎相同。「Playable API 比較輕量」的□法，至少在「簡單的動畫控制」條件下並不成立。Animator 的狀態機評估（`AnimatorControllerPlayable.PrepareFrame`）即使在 600 隻時，所有執行緒合計也只有 2.624 ms，其中主執行緒僅 0.161 ms，這項處理成為瓶頸的可能性極低。（不過，2 ms 算大還是算小，取決於遊戲本身。）

手動更新會因圖的結構而□生 1.87 倍的差異。即使同樣是 600 隻，依圖的彙整方式不同，可能比自動更新更快，也可能更慢。

4.4.3 實測 2: 執行緒的使用情形

600 隻，Renderer 已停用。這是從 Unity Profiler 的擷取資料依執行緒彙總的結果。

表 4.7: 圖粒度造成的執行緒使用差異（600 隻）

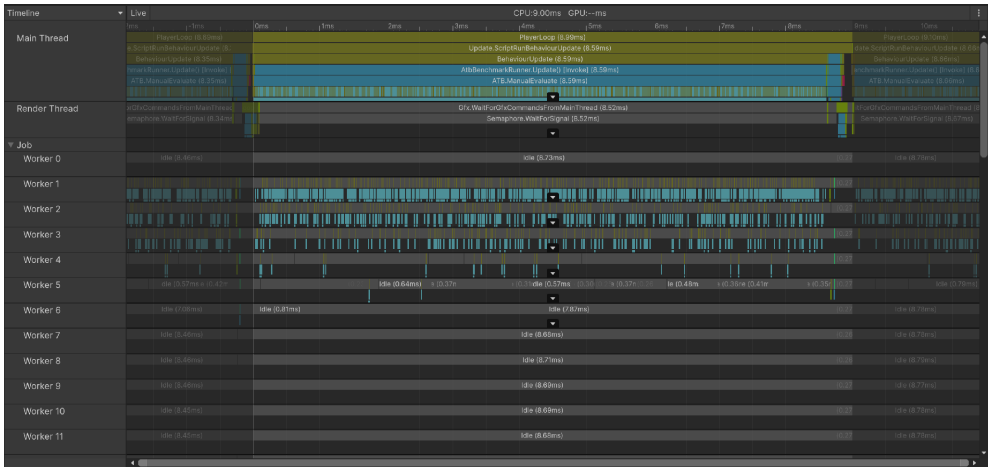
1 隻 1 個圖（600 個圖）	600 隻 1 個圖	
影格時間	**9.490 ms**	**4.928 ms**
主執行緒的 `Evaluate`	9.028 ms	4.556 ms
負責動畫處理的工作執行緒數	**5**	**15**
`Animator.ProcessGraph` 所有執行緒合計	0.920 ms	**19.034 ms**

表 4.7: 圖粒度造成的執行緒使用差異（600 隻）

1 隻 1 個圖（600 個圖）	600 隻 1 個圖	
每條工作執行緒的 `Animator.ProcessGraph`	0.32 / 0.15 / 0.03 / 0.02 / 0.002 ms	各約 **1.19 ms**（15 條幾乎均等）

1 隻 1 個圖時，動畫處理幾乎全部在主執行緒上。600 次 `Evaluate()` 只是依序執行，工作執行緒幾乎沒有運作。

600 隻 1 個圖時，15 條工作執行緒全部幾乎均等地執行 `Animator.ProcessGraph`。



▲圖 4.5　1 隻 1 個圖（600 個圖）的 Timeline。主執行緒的 `ATB.ManualEvaluate` 佔約 8.6 ms，大部分 Job 工作執行緒維持「Idle」。



▲圖 4.6 600 隻 1 個圖的 Timeline。主執行緒的 `ATB.ManualEvaluate` 減半至約 4.3 ms，所有工作執行緒同時執行各約 1.2 ms 的 `EngineJob`。

關於數☒的解讀，有 1 點補充。

此表的數☒取自連接 Unity Profiler 狀態下的擷取資料，因此比未連接 Profiler 的測量結果（4.500 ms / 8.396 ms）略重。章末的彙整表使用的是未連接 Profiler 的數☒。

4.4.4 本章的結論

表 4.8: 600 隻時的影格時間彙整

配置	600 隻的影格時間
1 隻 1 個圖 + 手動更新	8.396 ms
Animator（自動）	5.004 ms
600 隻 1 個圖 + 手動更新	**4.500 ms**

- Animator 與自動更新的 Playable Graph, 由 Unity 統一將所有圖分散到 Animation Job Thread
- 手動更新只能以圖為單位平行化
- 選擇手動更新時，務必將圖的粒度納入設計項目
- 改用 Playable API 本身幾乎沒有最佳化效果。使用 Playable API 的真正價☒在於「可插入 Animation Job」「可控制圖結構與更新時機」等**動畫控制的彈性**

4.5 Animation C# Jobs 的使用與 SIMD 支援

遊戲開發中必然會出現「想以程序化方式控制動畫」的需求，例如「想讓角色的□依照地面形狀移動以貼合地面」。

這種情況下可採用的手法之一是 Two-Bone IK。

實際實作 Two-Bone IK 這類程序化動畫控制時，有 3 個選項。

1. 以 `LateUpdate` 直接改寫 Transform
2. 將 `IAnimationJob` 插入 `AnimationScriptPlayable`
3. 在 2 的基礎上再以 Burst 編譯

這次著眼於這些方法的效能。

3 種都實作為進行**完全相同的運算**。解法被抽出為 1 個 static 方法，LateUpdate 版、Job 版、Burst 版共用該方法，因此差別只在於執行的位置。

4.5.1 什麼是 Animation C# Jobs

Animation C# Jobs 是透過 `AnimationScriptPlayable` 與 `IAnimationJob`，**將自訂處理插入動畫評估管線**的機制。Job 會在與剪輯評估、混合相同的流程中被呼叫，並透過 `AnimationStream` 與 `TransformStreamHandle` 存取骨骼。

與 `LateUpdate` 的差異在於程式碼執行的位置。`LateUpdate` 在主執行緒上，直接改寫動畫評估結束後的 Transform。處理會依角色數量序列堆疊在主執行緒上。Animation Job 在評估管線□部（Animation Job Thread）被呼叫，因此 Unity 可以和其他動畫評估一樣將其分散到工作執行緒。即使是相同的運算，是否成為主執行緒的負擔也會有所不同。

必須經由 `AnimationStream` 存取的限制是其代價。Job 只能看到事先□定好的控制代碼，無法存取任意的 `Transform` 或 `GameObject`。

4.5.2 實作

Job 本體只需以 `ProcessAnimation` 呼叫解法。

▼清單 4.6　Two-Bone IK 的 Animation Job（支援 Burst）

```
[BurstCompile]
public struct TwoBoneIkJobBurst : IAnimationJob
{
    public TransformStreamHandle Root;
    public TransformStreamHandle Mid;
    public TransformStreamHandle Tip;
    public NativeArray<float3> Target;   // 長度為 。每個影格由主執行緒寫入1
    public float Weight;
    public int Iterations;

    public void ProcessRootMotion(AnimationStream stream) { }

    [BurstCompile]
    public void ProcessAnimation(AnimationStream stream)
    {
        TwoBoneIkCore.SolveStreamIterations(
            ref stream, Root, Mid, Tip, Target[0], Weight, Iterations);
    }
}
```

骨骼的讀寫以 `TransformStreamHandle` 與 `AnimationStream` 成對進行。

▼清單 4.7　從 AnimationStream 讀寫骨骼

```
static void SolveStream(
    ref AnimationStream stream,
    TransformStreamHandle root, TransformStreamHandle mid, TransformStreamHandle tip,
    float3 target, float weight)
{
    var rootPosition = (float3)root.GetPosition(stream);
    var midPosition = (float3)mid.GetPosition(stream);
    var tipPosition = (float3)tip.GetPosition(stream);
    var rootRotation = (quaternion)root.GetRotation(stream);
    var midRotation = (quaternion)mid.GetRotation(stream);

    Solve(rootPosition, midPosition, tipPosition, target, rootRotation, midRotation,
    weight,
        out var newRootRotation, out var newMidRotation);

    mid.SetRotation(stream, newMidRotation);
    root.SetRotation(stream, newRootRotation);
}
```

控制代碼以 `Animator.BindStreamTransform` 建立，並將 Job 作為剪輯的後段插入圖中。

▼清單 4.8　將 Animation Job 插入圖中

```
var job = new TwoBoneIkJobBurst
{
    Root = animator.BindStreamTransform(rootTransform),
    Mid = animator.BindStreamTransform(midTransform),
    Tip = animator.BindStreamTransform(tipTransform),
    Target = ikTarget,   // NativeArray<float3>
    Weight = 1f,
    Iterations = iterations,
};

var ikPlayable = AnimationScriptPlayable.Create(graph, job, 1);
graph.Connect(clipSource, 0, ikPlayable, 0);   // clipSource = 剪輯端的混合器
ikPlayable.SetInputWeight(0, 1f);
output.SetSourcePlayable(ikPlayable);
```

IK 目標這類每個影格都會變動的輸入，放在 `NativeArray` 中，並由主執行緒寫入。

Burst 透過對類別加上 `BurstCompile` 屬性來套用。

Burst Compiler 的最佳化是否確實生效，可使用 Burst Inspector 等工具確認。

4.5.3　實測

300 隻，Renderer 已停用。將每隻的 IK 解算次數變更為 1 / 16 / 64。

表 4.9: 各 IK 解算次數的影格時間（300 隻・Renderer 已停用）

實作	1 次	16 次	64 次
無 IK（基準）	2.501 ms	2.601 ms	2.596 ms
LateUpdate	2.699 ms	4.400 ms	**9.094 ms**
Animation Job	2.411 ms	2.496 ms	**2.795 ms**
Animation Job + Burst	2.494 ms	2.496 ms	**2.792 ms**

CPU 時間的明細（64 次時）：

表 4.10: 各 IK 實作的 CPU 時間明細（解算 64 次）

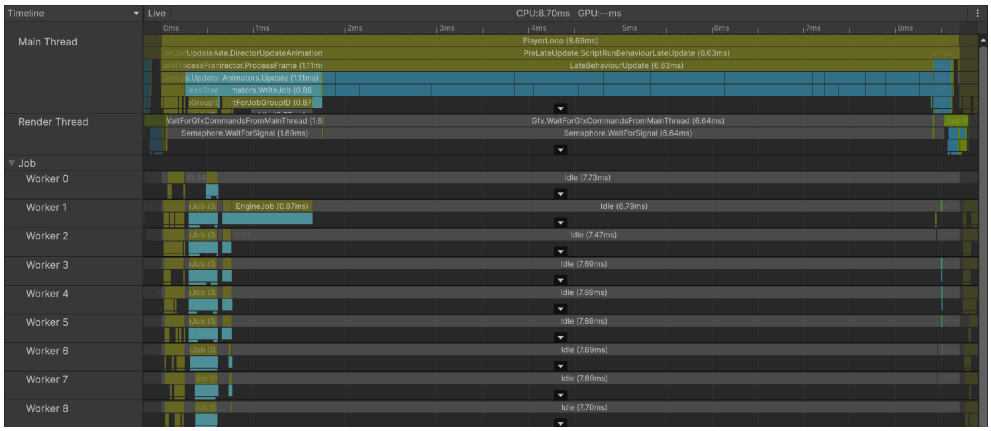
實作	IK 的 CPU 時間（所有執行緒）	其中主執行緒	對影格時間的增加量
LateUpdate	6.910 ms	**6.910 ms（100 %）**	+6.498 ms

表 4.10: 各 IK 實作的 CPU 時間明細（解算 64 次）

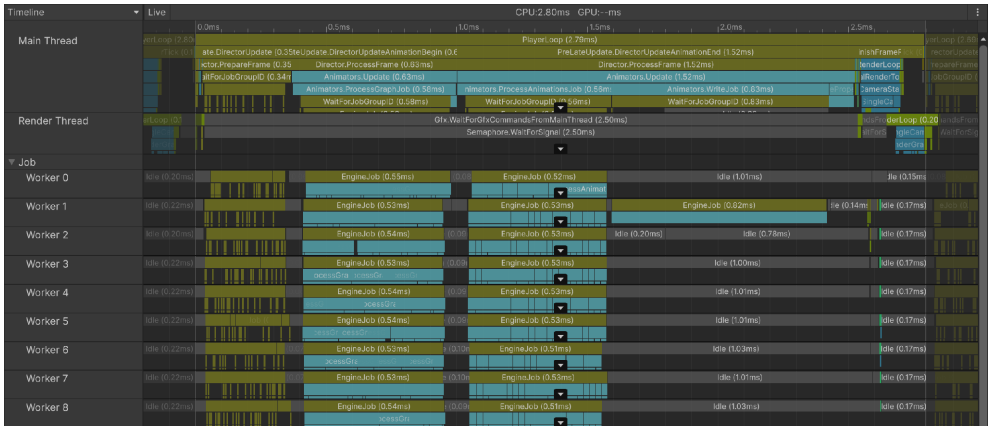
實作	IK 的 CPU 時間（所有執行緒）	其中主執行緒	對影格時間的增加量
Animation Job	7.790 ms	0.487 ms（6.3 %）	+0.199 ms
Animation Job + Burst	6.932 ms	0.436 ms（6.3 %）	+0.196 ms

4.5.4 從 Profiler Timeline 看執行緒的差異

相同的運算在□個執行緒上執行，可以在 Unity Editor 的 Profiler 中看到。以下是在 Profiler 視窗中開□解算 64 次・300 隻的擷取資料，並選取影格時間中位數附近的影格。由於是連接 Profiler 時的擷取資料，與上表的數□並不完全一致。



▲圖 4.7 LateUpdate 實作。在 8.698 ms 的影格時間中，`LateBehaviourUpdate` 佔 6.63 ms。15 條工作執行緒在影格的大半時間（約 6.8～7.7 ms）維持 `Idle`。



▲圖 4.8 Animation Job 實作。相同的運算作為 `AnimationScriptPlayable.ProcessAnimation` 分散到 15 條工作執行緒，每條各執行約 0.5 ms。影格時間為 2.796 ms。

在 LateUpdate 實作中，300 隻份的 `ATB.TwoBoneIk.LateUpdate`（合計 6.577 ms）全部位於主執行緒的 `LateBehaviourUpdate` 之中。此時工作執行緒負責的只有基礎動畫評估（`Animator.ProcessGraph` 每條 0.12～0.29 ms），完全沒有 IK 的計算。

在 Animation Job 實作中，`AnimationScriptPlayable.ProcessAnimation` 所有執行緒合計為 7.891 ms，其中主執行緒為 0.498 ms。其餘 7.393 ms 幾乎均等地分散到 15 條工作執行緒（每條 0.47～0.51 ms）。總 CPU 時間雖多於 LateUpdate 實作，但由於分散到 16 條執行緒，影格時間可從 8.698 降至 2.796 ms。

4.5.5 可以得知的事

針對此案例的最佳化，最有效的不是 Burst，而是轉移執行的執行緒。LateUpdate 實作會把 IK 的 CPU 時間全數由主執行緒負擔，因此在 64 次時影格時間會從 2.596 膨脹到 9.094 ms。將相同的運算移到 Animation Job 後，主執行緒的負擔幾乎消失，對影格時間的增加量□生**33 倍的差距**。

□一方面，**以總 CPU 時間而言，LateUpdate 版反而較少**。Animation Job 有每個 `AnimationScriptPlayable` 節點的固定成本，300 隻時約為 2.16 ms。即便如此，平行化的收益仍遠遠超過。

在接近實際營運的「每隻 1 次」條件下，3 種實作的差異幾乎在測量雜訊的範圍□。若

是約 300 隻的 Two-Bone IK，選擇□種實作都感受不到差異。

4.5.6 Burst 並非加上屬性就會生效

認為加上 `[BurstCompile]` 就會 SIMD 化而快上數倍，是錯誤的理解。這次的實測中，Burst 帶來的總 CPU 時間削減為 11.0 %。扣除節點固定成本，以「每次解算的邊際成本」來看，

- 無 Burst: (7.790 − 2.157) / 63 = **0.0894 ms**
- 有 Burst: (6.932 − 2.167) / 63 = **0.0756 ms**（− 15.4 %）

Burst 做的是透過 LLVM □生原生程式碼，是否會□生 SIMD 指令取決於所撰寫程式碼的形式。**若不實作能適當 SIMD 化的程式碼，就無法獲得大幅的效益。**

Animation C# Jobs 有 2 個使 SIMD 化變得困難的性質。

處理中頻繁出現 `AnimationStream` 的讀寫。每次解算都以 `GetPosition`・`GetRotation` 讀取骨骼，再以 `SetRotation` 寫回。由於每次迭代 Stream 的狀態都會改變，迭代之間仍有相依關係，無法將多次的處理合併為 1 個向量指令。此外，`IAnimationJob` 接收的 `AnimationStream` 只有 1 隻份，Job 中原本就沒有可「同時計算 4 隻」的資料。

大量使用超越函數。由餘弦定理求彎曲角度的 `acos`、`quaternion.AxisAngle` □部呼叫的三角函數、`math.length` 的平方根。這些都會變成跨 lane 的計算或純量的函式庫呼叫，阻礙向量化。

Burst 的 LLVM 也會**明確回報無法向量化**重複呼叫解法的迴圈（最佳化 remark `loop not vectorized: instruction return type cannot be vectorized`）。

4.5.7 充分發揮 Burst 效益的策略

若想充分發揮 Burst 的效益，可考慮使用 `C# Job System`，採用**不經由 `AnimationStream` 讀寫 Transform** 的配置。Transform 的讀寫可由 `TransformAccessArray` 與 `IJobParallelForTransform` 集中在 Job 端處理，也可以只讓 Animation Job 負責骨骼的收集與寫回，並將繁重的計算切出成彙整 N 隻份的批次 Job。這兩種做法都是**將 N 隻份的資料齊備於同一個 Job 中**，而非以角色為單位，因此滿足 SIMD 化的前提。

在此基礎上，批次 Job 的□容必須調整如下。

- 採用每個分量一個陣列的 SoA（Structure of Arrays），讓角色在記憶體上連續
- 將分支改為 `math.select` 或 `math.max`，使迴圈本體成為單一直線
- 將超越函數移出迴圈（以恆等式改寫、□表化、近似）
- 不使用 `float3`，而以 `float4` 逐 lane 撰寫「4 隻份」，不依賴自動向量化的判斷

代價是設計的複雜度。`IAnimationJob` 的最大優點是只需在圖中插入 1 個節點，就能在評估管線的正確時機存取骨骼的最新狀態，而這個策略等於捨棄了這項優點。本文並未實測此配置，因此不在此提出能加快多少。請將其視為一項提案: 若期望 Burst 帶來數倍的效果，就必須以這個方向的設計變更為前提。

4.5.8 本章的結論

- Animation Job 的價□在於**能活用 Animation Job Thread**
- 運算量較少時，`AnimationScriptPlayable` 的節點固定成本較大。直接以 Late Update 撰寫的總 CPU 時間較少
- `[BurstCompile]` 並非加上就會 SIMD 化。必須預先設計能套用 SIMD 的資料與處理

4.6 動畫 LOD 的實作

4.6.1 機制與有效的情境

雖然取決於遊戲規格，但基本上遠處的角色不需要每個影格都更新。若每 N 個影格只更新 1 次，該角色的平均負載就會降為 1/N。這就是「動畫 LOD」。這與網格 LOD 依距離分段控制繪製成本的概念相同，只是套用在**動畫的更新頻率**上。

在以下情境中特別有效。

- 畫面□有數百隻規模的角色，且大半是遠景或路人
- 遠處角色相當於 30 fps / 15 fps 的動作在體感上可以接受（鏡頭越遠，降頻越不易被察覺）
- 動畫評估在影格時間中佔有不可忽視的比例

反之，若套用在近距離、被鏡頭特寫的角色上，卡頓會立刻被察覺。實際營運時，較合理的做法應是依距離或螢幕佔有率切換降頻的階段。

4.6.2 簡單的實作: 自行呼叫 Animator.Update()

設為 animator.enabled = false 以停止 Unity 的自動更新，只在需要的影格明確呼叫 Animator.Update()。跳過的影格時間會累積後一併傳入。

此實作有個副作用，就是會失去前面□明過的 Animation Job Thread 平行性。

▼清單 4.9 自行呼叫 Animator.Update() 的降頻實作

```
animator.enabled = false;   // 初始化時。停止 Unity 的自動更新

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;   // Slot = index %
    interval

        character.Animator.Update(character.PendingDeltaTime);    // 以累積的時間明確更新
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

4.6.3 直接使用 Playable API 的實作範例

若為手動更新（DirectorUpdateMode.Manual）的 Playable Graph，只需降低 Evaluate() 的呼叫次數。

▼清單 4.10 降低手動更新圖的 Evaluate() 呼叫頻率的實作

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;

        character.Graph.Evaluate(character.PendingDeltaTime);
```

```
            character.PendingDeltaTime = 0f;
        }
        _frameIndex++;
    }
```

有 1 點要注意。在前一章推薦的「將所有角色彙整為 1 個圖」配置中，整個圖以 1 次 `Evaluate()` 評估，因此**無法表達以角色為單位的降頻**。如上方程式碼般採用 1 隻 1 個圖就能降頻，但相對地會失去彙整所帶來的平行化。

無論⊠種實作，重要的都是 `Slot`（相位）。若所有角色都在同一個影格更新，負載就會集中在那個影格。以 `index % interval` 錯開相位，就會變成每個影格更新 1/N 隻。

4.6.4 實測

600 隻，Renderer 已停用。首先是使用 Animator 的實作。

平均⊠表示 12 秒間的總負載，**中位數與 p95 的差距**表示影格時間的波動程度。

表 4.11: 降頻與錯開相位的影格時間統計（Animator・600 隻）

方式	平均	中位數	p95	p99	最大	標準差
每個影格更新（Animator 自動）	5.037 ms	5.000 ms	5.303 ms	5.707 ms	16.901 ms	0.433
每 2 個影格 1 次・不錯開相位	4.781 ms	0.648 ms	9.607 ms	9.816 ms	12.194 ms	4.544
每 2 個影格 1 次・錯開相位	4.831 ms	4.803 ms	5.099 ms	5.212 ms	8.804 ms	0.171
每 4 個影格 1 次・不錯開相位	2.613 ms	**0.382 ms**	**9.470 ms**	9.658 ms	11.171 ms	3.863
每 4 個影格 1 次・錯開相位	2.578 ms	**2.593 ms**	**2.707 ms**	2.908 ms	11.722 ms	0.185

Playable API（1 隻 1 個圖・手動更新）也是相同的趨勢。

表 4.12: 降頻與錯開相位的影格時間統計（Playable API・1 隻 1 個圖）

方式	平均	中位數	p95	標準差
每個影格更新	8.496 ms	8.493 ms	9.001 ms	0.337
每 4 個影格 1 次・不錯開相位	2.438 ms	0.369 ms	8.855 ms	3.570
每 4 個影格 1 次・錯開相位	2.486 ms	**2.495 ms**	**2.613 ms**	0.173

4.6.5 錯開相位不是「減少負載」，而是「使其平整」

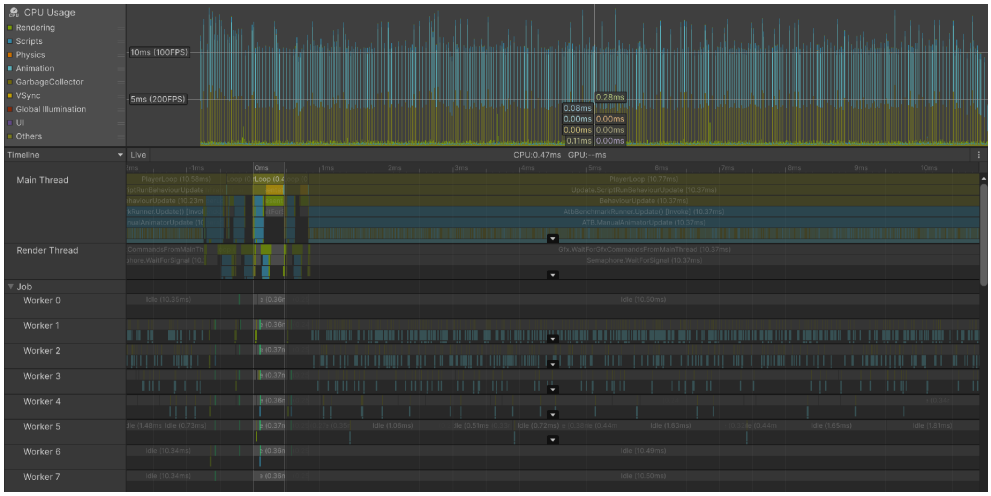
降頻為 1/4 時的平均□，有無錯開相位幾乎沒有差別，**總負載幾乎相同**。錯開相位並不會減少工作量。

□一方面，影格時間的分布出現了以下差異。

- 不錯開相位: 4 個影格中有 3 個影格什麼都不做，在 1 個影格中一次處理 600 隻份。中位數雖然下降，但 p95 與標準差會大幅上升
- 錯開相位: 每個影格處理 150 隻。中位數與 p95 幾乎一致，波動很小

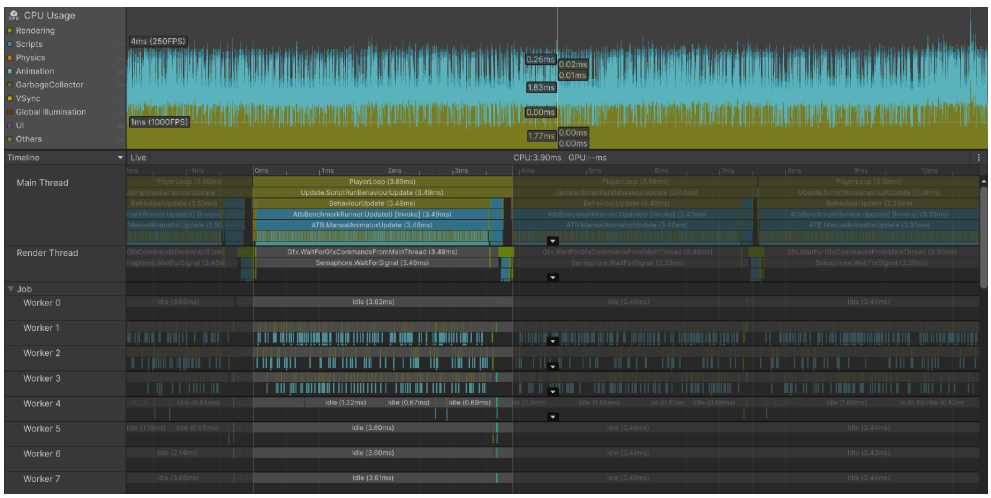
若只看中位數 0.382 ms 這個數字而高興地認為「降到 1/13 了」，實際上會做出每 4 個影格就出現一次 9.5 ms 尖峰的遊戲。**錯開相位是為了體感品質的措施**，標準差 0.185 比每個影格更新的 0.433 更穩定。

這種分布的差異會直接呈現在 Profiler 的 CPU 圖表上（兩者皆為連接 Profiler 時的擷取資料，因此比上表未連接時的數□偏重。此外，圖表的縱軸刻度為自動調整，所以 2 張圖不同）。



▲圖 4.9

每 4 個影格 1 次・不錯開相位。CPU 圖表呈現「3 個影格近乎 0 ms 的低谷 + 1 個影格約 10 ms 的尖峰」的梳狀。下方的 Timeline 中，在所選的空影格（0.47 ms）兩側，可以看到一次處理 600 隻份的 `ATB.ManualAnimatorUpdate`（10.37 ms）的影格。



▲圖 4.10

每 4 個影格 1 次・錯開相位。由於每個影格處理 150 隻，波形平坦（約 3.5 ms），尖峰消失。

4.6.6 單純的實作會失去平行化

每 2 個影格 1 次的降頻，幾乎沒有削減任何東西。雖然更新次數減半，平均影格時間卻只減少了 **4.1 %**。

原因可從 Profiler 標記的明細看出。

表 4.13: 單純降頻實作所失去的平行性

方式	每個影格更新的隻數	主執行緒的動畫更新時間	`Animator.ProcessGraph` 所有執行緒合計	其中主執行緒
每個影格更新（Unity 的批次更新）	600	`Animators.Update` 4.046 ms	20.104 ms	1.257 ms
每 2 個影格 1 次・錯開相位	300	`ATB.ManualAnimatorUpdate` 4.532 ms	0.549 ms	0.317 ms
每 4 個影格 1 次・錯開相位	150	`ATB.ManualAnimatorUpdate` 2.316 ms	0.278 ms	0.180 ms

每個影格更新時，工作量中留在主執行緒的只有 6 %，其餘分散在工作執行緒。□一方面，明確呼叫 `Animator.Update()` 時會**當場同步評估**，因此工作執行緒幾乎沒有被使用。Unity 批次更新原有的平行化，被拿去換了降頻。

換算成每隻的主執行緒成本，

- 每個影格更新: 4.046 ms / 600 隻 = **6.7 μs/隻**（平行）
- 每 2 個影格 1 次: 4.532 ms / 300 隻 = **15.1 μs/隻**（序列）
- 每 4 個影格 1 次: 2.316 ms / 150 隻 = **15.4 μs/隻**（序列）

一旦序列化，每隻的成本就變為約 2.3 倍，之後不論降頻倍率為何都維持固定。也就是□，**2 倍降頻所得到的 2 倍削減，幾乎被序列化造成的 2.3 倍惡化抵消**。實測的 4.1 % 不過是其殘差。

若維持單純的實作，要到 4 倍降頻（− **48.8 %**）才終於出現實質的效果。不過，這是可以避免的損失。

4.6.7 保持平行性並降頻

`Animator.Update()` 是同步的，這點無法避免。不過只要採用**「每個影格選出要參與 Unity 批次更新的個體」**的形式，就能在降頻的同時保持平行性。有 2 種實作方式，兩者都繼續使用 Animator（及其狀態機）。

方法 1: 切換 `Animator.enabled`

Unity 的動畫更新會收集當下⊠用的 Animator 彙整為 1 個批次，並分散到 Animation Job Thread。因此不必呼叫 `Animator.Update()`，只要**將該影格要更新的個體設為 `enabled = true`** 即可。評估由 Unity 進行，所以平行化得以保留。

▼清單 4.11　切換 Animator.enabled 的降頻實作

```
// 初始化時。以 speed 補回跳過的影格份
foreach (var character in _characters)
{
    var animator = character.Animator;
    animator.keepAnimatorStateOnDisable = true;    // 停用時不要倒回狀態（必要）
    animator.speed = interval;                     // 補正每 interval 個影格才更新 1 次的份
}

void Update()
{
    foreach (var character in _characters)
    {
        var scheduled = _frameIndex % interval == character.Slot;
        if (character.Animator.enabled != scheduled)
        {
            character.Animator.enabled = scheduled;    // 只需選出要參與 Unity 批次更新的
    個體
        }
    }
    _frameIndex++;
}
```

千萬不要忘記 `keepAnimatorStateOnDisable = true`。在預設⊠ `false` 下，每次停用 Animator 時控制器的狀態都會倒回，動畫就不會前進。

限制在於 `Animator.speed` 是**速率**。無法像 `Animator.Update(pendingDeltaTime)` 那樣「精確傳入累積的 Δt」，只會前進更新影格的 `Time.deltaTime × interval`。在影格時間有波動的環境中，前進幅度也會波動，但平均的播放速度可以維持。

方法 2: 依相位分割圖

前一章的結論是「將 N 隻合併為 1 個手動圖就能平行化」。將此與降頻組合。**建立與相位數量相同的圖，每個影格只對其中 1 個圖呼叫** `Evaluate()`。1 個圖中包含 N/interval 隻，因此在這 1 次 `Evaluate()` 之中平行化會生效。

只要將狀態機作為 `AnimatorControllerPlayable` 放進圖中，就會執行與指派給 Animator 元件時相同的評估。

▼清單 4.12　依相位分割圖的降頻實作

```
// 初始化時：每個相位 bucket 建立 1 個手動圖
for (var bucket = 0; bucket < interval; bucket++)
{
    var graph = PlayableGraph.Create($"Bucket_{bucket}");
    graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
    _bucketGraphs[bucket] = graph;
}

foreach (var character in _characters)
{
    var graph = _bucketGraphs[character.Slot];
    character.Animator.runtimeAnimatorController = null;   // 評估交由圖端處理
    var output = AnimationPlayableOutput.Create(graph, "Animation", character.Animator)
    ;
    output.SetSourcePlayable(AnimatorControllerPlayable.Create(graph, controller));
}

foreach (var graph in _bucketGraphs) graph.Play();

void Update()
{
    for (var bucket = 0; bucket < interval; bucket++) _pendingDeltaTime[bucket] += Time
    .deltaTime;

    var scheduled = _frameIndex % interval;
    _bucketGraphs[scheduled].Evaluate(_pendingDeltaTime[scheduled]);   // 每個影格只處
    理 1 個 bucket
    _pendingDeltaTime[scheduled] = 0f;
    _frameIndex++;
}
```

這種方式可以**將累積的 Δt 原樣傳給** `Evaluate()`，因此前進幅度精確。相對地，圖的數量會等於降頻階段數，若要動態變更 LOD 階段，就必須在圖之間移動角色（重新建立

輸出）。

實測: 3 種實作的比較

600 隻，Renderer 已停用。為平均影格時間，以及相對於每個影格更新（5.037 ms）的削減率。

表 4.14: 3 種降頻實作的影格時間比較（600 隻）

實作	1/2 降頻	1/4 降頻
明確呼叫 `Animator.Update()`（錯開相位）	4.831 ms（− 4.1 %）	2.578 ms（− 48.8 %）
切換 `Animator.enabled`	**3.350 ms（− 33.5 %）**	**1.835 ms（− 63.6 %）**
依相位分割的手動圖（`AnimatorControllerPlayable`）	**3.082 ms（− 38.8 %）**	**1.541 ms（− 69.4 %）**

單純的實作最不划算。尤其是 1/2 降頻的差距很大，− 4.1 % 變成了 − 33.5 % / − 38.8 %。前一節「2 倍降頻會被平行化的損失吃掉」的結論，**只要改變實作就不成立。**

平行性是否保持，可以用 Profiler 標記確認。

表 4.15: 3 種降頻實作的各執行緒明細

方式	`Animator.ProcessGraph` 所有執行緒合計	其中主執行緒	主執行緒端的更新時間
每個影格更新（600 隻/影格）	20.104 ms	1.257 ms	`Animators.Update` 4.046 ms
`Animator.Update()`・1/4（150 隻/影格）	0.278 ms	0.180 ms	`ATB.ManualAnimatorUpdate` 2.316 ms
`Animator.enabled`・1/4（150 隻/影格）	**4.424 ms**	0.288 ms	`Animators.Update` 1.104 ms + 切換 0.125 ms
依相位分割的手動圖・1/4（150 隻/影格）	**4.253 ms**	0.279 ms	`ATB.BucketEvaluate` 1.278 ms

在新的 2 種實作中，**`Animator.ProcessGraph` 的工作量恰為每個影格更新的 1/4**（每個影格只評估 150 隻，這是理所當然的），而且這些工作仍然分散在工作執行緒上。☒一方面，`Animator.Update()` 版幾乎沒有計入此標記，工作的本體位於主執行緒的 `ATB.ManualAnimatorUpdate` 之中。

以每隻的主執行緒成本並列比較，差異就很明顯。

- 每個影格更新: 4.046 ms / 600 隻 = **6.7 μs/隻**
- `Animator.enabled` 切換・1/4: 1.229 ms / 150 隻 = **8.2 μs/隻**
- 依相位分割的手動圖・1/4: 1.278 ms / 150 隻 = **8.5 μs/隻**
- `Animator.Update()`・1/4: 2.316 ms / 150 隻 = **15.4 μs/隻**

新的 2 種實作，每隻的成本幾乎維持與每個影格更新相同（差異是切換與 `Evaluate` 呼叫的額外開銷）。因此降頻倍率會直接趨近削減率。

也確認了播放速度得以維持。測量最後生成的角色（相位槽位最大的個體）指尖骨骼每秒移動的距離，每個影格更新為 0.509 m/s，`Animator.enabled` 切換為 0.496 m/s，依相位分割的手動圖為 0.517 m/s。在降頻使動作被線性近似所造成的誤差範圍內，兩者的速度都沒有改變。

關於分布有 1 點要注意。1/2 降頻的手動圖版 p99 為 9.102 ms、標準差為 1.086，尾部比 `enabled` 切換版（p99 4.100 ms、標準差 0.562）更重。在 1/4 時則相反，手動圖版較穩定（標準差 0.306 對 0.340）。行為會因圖的數量與每個圖的隻數而改變，因此若要採用，不只要看平均，也請看 p99。

選擇哪一種取決於需求。若想繼續直接使用 Animator 的狀態機與 Inspector 的工作流程，切換 `Animator.enabled` 壓倒性地簡便（只需數行，對既有程式碼幾乎沒有影響）。若需要精確的前進幅度，或插入 Animation Job 等圖端的控制，則選擇依相位分割的手動圖。**無論哪一種，都沒有理由選擇自行呼叫 `Animator.Update()` 的實作。**

4.6.8 本章的結論

- 動畫 LOD 應以「降低更新頻率 + 錯開相位」為一組來設計
- 錯開相位不會減少總負載（平均 2.613 對 2.578 ms）。它減少的是**影格時間的波動**（標準差 3.863 → 0.185），這是為了體感品質的措施
- **不要選擇自行呼叫 `Animator.Update()` 的實作。**由於會當場同步評估而失去 Job 平行化，每隻的主執行緒成本從 6.7 → 15.4 μs，變成約 2.3 倍
- 改為**每個影格選出要參與 Unity 批次更新的個體**。可採用切換 `Animator.enabled`（需以 `keepAnimatorStateOnDisable = true` 與 `Animator.speed` 補正），或依相位分割手動圖並掛載 `AnimatorControllerPlayable` 的形式。每隻成本可控制在 8.2〜8.5 μs，並保留平行化
- 改變實作後，**即使是 2 倍降頻也有效**: − 4.1 % 變成 − 33.5 % / − 38.8 %。4 倍時

為 − 63.6 % / − 69.4 %

- 以角色為單位的降頻無法用 1 個彙整圖表達。不過若「依相位數量分割圖」，就能兼顧平行化與更新頻率控制
- 不要只憑中位數判斷。總負載看平均，體感則看 p95、p99 與標準差

4.7 總結

各章的實測結果如下。

表 4.16: 各章實測結果彙整

章	對策	條件	結果
資□的設定	Optimize Game Objects	600 隻・1 個 Renderer	6.797 → 5.991 ms（− **11.9 %**）
資□的設定	Optimize Hierarchy（執行階段）	600 隻・1 個 Renderer	6.797 → 6.009 ms（− **11.6 %**）
資□的設定	上述 2 種方法	300 隻・全部 Renderer	18.205 → 18.3〜18.5 ms（無改善・受限於繪製）
Job Thread	將手動更新圖彙整為 1 個	600 隻	8.396 → 4.500 ms（− **46.4 %**）
Animation C# Jobs	將 Two-Bone IK 改為 Animation Job	300 隻・解算 64 次	9.094 → 2.795 ms（− **69.3 %**）
Animation C# Jobs	再以 Burst 編譯	300 隻・解算 64 次	2.795 → 2.792 ms（影格時間無差異）
動畫 LOD	1/4 降頻・明確呼叫 `Animator.Update()`	600 隻	平均 5.037 → 2.578 ms（− **48.8 %**）
動畫 LOD	1/4 降頻・依相位分割的手動圖	600 隻	平均 5.037 → 1.541 ms（− **69.4 %**）
動畫 LOD	1/2 降頻・明確呼叫 `Animator.Update()`	600 隻	平均 5.037 → 4.831 ms（− 4.1 %）
動畫 LOD	1/2 降頻・依相位分割的手動圖	600 隻	平均 5.037 → 3.082 ms（− **38.8 %**）

4.7.1 最終的最佳化方法

以下是依據本文實測所得、大量角色動畫最佳化的進行方式。

1. 最佳化資□設定

首先請將模型本身的頂點數與骨骼數規格訂定為適當的□。理想上，應先在作為基準的支援裝置上測量基準測試，訂定適當的模型規範。在此之上，再考慮在 Unity 端套用 Optimize Game Objects。在 3D 遊戲中，位於上游的模型製作，在量□已進展的專案後

期很難大幅修正，因此最好在專案初期進行。

2. 提高 Animation Job Thread 的處理平行性

Animator 與自動更新的 Playable Graph 由 Unity 自動平行化，但手動更新只能以圖為單位平行化。在設計角色控制的階段，就請將「能否有效活用 Animation Job Thread」納入考量。

3. 進行包含 Animation Job 套用 Burst 的最佳化

IK 等後處理，首先從主執行緒移到 Animation Job。

若需要更高的效能，請套用可 SIMD 化的資料與處理設計。

4. 採取動畫 LOD 等應用程式層面的做法

在用盡資□、引擎設定與程式碼最佳化之後，最後的做法是「根本不處理動畫」。

雖須與專案需求協商，但請在探索遊戲體驗上可接受的界線的同時，考慮抑制處理的機制。

4.8 註

4.8.1 Unity 官方文件

- Playable API
  https://docs.unity3d.com/Manual/Playables.html – Playable Graph 的建構與 DirectorUpdateMode
- AnimatorUtility.OptimizeTransformHierarchy
  https://docs.unity3d.com/ScriptReference/AnimatorUtility.OptimizeTransformHierarchy.html – 執行階段的階層扁平化
- Model Import Settings: Rig（Optimize Game Objects）
  https://docs.unity3d.com/Manual/FBXImporter-Rig.html
- IAnimationJob
  https://docs.unity3d.com/ScriptReference/Animations.IAnimationJob.html – 插入 AnimationScriptPlayable 的 Job
- AnimationScriptPlayable
  https://docs.unity3d.com/ScriptReference/Animations.AnimationScriptPlayable.html
- C# Job System
  https://docs.unity3d.com/Manual/JobSystem.html – 不經由 AnimationStream 撰寫平行處理的機制

4.8.2 使用的資

- Unity-chan 3D 模型資料
  https://unity-chan.com/ – © Unity Technologies Japan/UCL

第5章

營運階段的資□膨脹與以 Project Auditor 進行調□

財津 大睦 Zaitsu Hiromu

株式會社 Sumzap

X @RomGameclient

所屬：株式會社 Sumzap X 帳號 / 網站 / GitHub 帳號（選填）：@RomGameclient

在持續營運的專案中，應用程式的容量與記憶體使用量會逐漸增加。本章處理的是即使團隊已知對策，膨脹仍會再度發生的原因、阻止膨脹的方法，以及找出持續增加的資□的手段。個別資□的輕量化本身不在本章範圍□。

5.1　營運階段膨脹持續惡化的原因

膨脹是在日常營運中一點一點累積的。例如為季節活動新增的高解析度橫幅、限時角色的語音與 Cut-in、聯名企劃中收下的外部製作素材、公告結束後仍殘留的活動用資□。每一項都在可容許範圍□，但隨著版本不斷發布，累積到無法忽視的量。而且累積越多，重新檢視的成本就越高，整理的優先順序也隨之下降。原因與其□在技術，不如□在營運規則的鬆弛。營運階段會反覆發生以下 4 種情況。・規範過時：即使裝置與需求已改變，使規範與實際□節，也沒有人改寫規範。以「僅此一次」為由放行的例外不斷累積。・檢□變得鬆散：截止日接近時，交付素材的確認被省略，負責人交接時也交接得不完整。・臨時處置一直留著：為趕時間放入的未壓縮紋理或個別輸出的 Sprite 就這樣保留，之後沒有進行還原作業。・判斷基準沒有記錄：設定的依據沒有留在文件中，因此負責人更換後無法重現相同的基準。這些問題都無法僅靠導入工具來解決。

5.2 持續運用規範

決定要達成的指標、選定保證運作的裝置，並在負載最高的場景中驗證。這套規範本身的制定方法，已在前作《Unity 效能調優寶典》第 1 章中□明。在營運階段成為問題的，是規範訂定之後的事。規範一旦制定，就會開始與實際□節。因此，應預先決定好保證運作裝置的變更、Unity 或渲染管線的更新、測量方法的變更等時機，作為重新檢視的契機。尤其測量方法一改變，即使是同一個資□數□也會變動，因此測量方式也要與規範一併記錄。規範中的數□，應設計成在匯出、匯入、自動建置的其中一個環節執行檢□。要擋下或放行應依項目決定，即使放行，也請把實測□以建置日誌或報表的形式，留在全體團隊成員看得到的地方。只在本人畫面上顯示警告的做法會被略過不看。修正超出規範的資□，並非改好數□就結束。若要變更壓縮設定，必須以與正式環境相同的條件確認外觀是否在可容許範圍□；若要重新編排 Bundle，則須確認參照是否□落。如前作所警告，量□後的重做是包含從發包、品質檢□到除錯的整個工序。理想做法不是集中修正，而是每次發布都進行測量，並當場修正增加的部分。

5.3 膨脹的模式分類

若依素材種類整理對策，雖然能知道□些素材較重，卻無從得知為何變重。原因不除，同樣的膨脹就會再發生。因此跨越素材種類，依原因分為 6 類。

表 5.1: 膨脹的 6 種模式

模式	發生了什麼
壓縮設定錯誤	匯入設定維持未壓縮或預設□而被擱置，佔用了所需的數倍容量
解析度過高	未指定最大解析度就匯入，容量隨面積成比例增加
多餘的附帶資料	□用了不使用的輔助資料並一併打包，在記憶體中也持續保留
格式選擇失誤	選用了不符合用途或平台的格式，多佔用容量，或在執行時展開而消耗記憶體
彙整方式不一致	Sprite Atlas 或 Bundle 的彙整方式與實際使用單位不符，例如顯示 1 張 Sprite 就載入整個 Atlas，連不使用的□容也一併載入
Bundle 之間的重複	未登錄共用資□就加以參照，使其依參照來源 Bundle 的數量被重複打包

前 4 項可在資□ Import 時判定，但後 2 項取決於與其他資□的關係，因此在該時間點無法判定。這個差異，就是下一節將對策分為兩階段的原因。

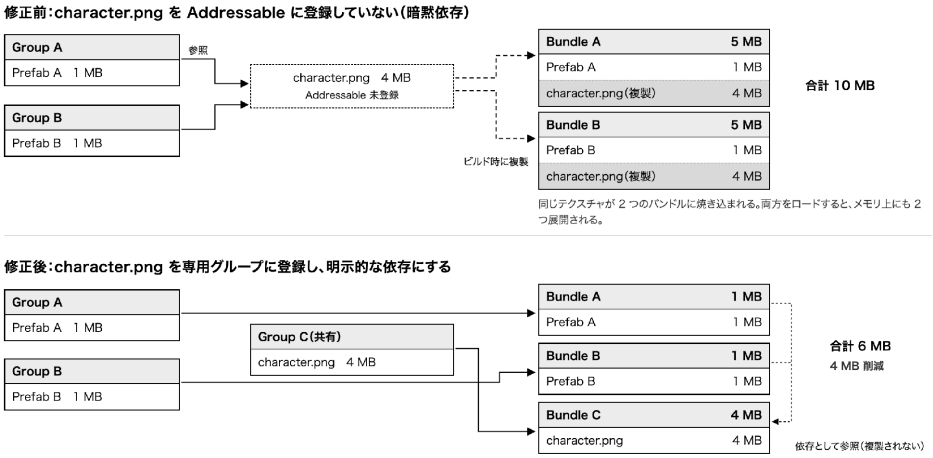
5.4 Import 時的自動套用與建置前的檢□

要阻止再度發生，須在資□ Import 時套用正確的設定，並在建置前找出漏網之魚。自動套用的手段有 2 種。若要依條件改變□，使用可在 Import 前後插入處理的 Asset Postprocessor。若只是固定各型別的初始□，使用 Preset 即可。不過，僅靠 Import 時的套用仍會有漏網之魚。連同 .meta 檔案一起帶入的資□會保留原有設定進來，而前一節的後 2 項在此階段也無法判定。因此在建置前或自動建置階段，檢□是否混入違反規範的資□。若將紋理的最大解析度、網格的多邊形數等限制規範寫成測試，編輯器與自動建置就能使用相同的基準。也有 AssetRegulationManager 這類針對此用途的 OSS 公開 [*1]。若不將這類檢□機制化，而只是把規範寫成文件並周知，就會原封不動地招致開頭所述的規範過時與判斷基準未留存的狀況。制定規則是人的工作，但要求遵守的部分請交給機器處理。

5.5 Bundle 之間重複的發生機制

在 6 種模式中，最難察覺的是 Bundle 之間的重複。原因不在個別資□的設定，而在資□之間的參照關係。Addressables 以群組為單位管理資□，並在建置時依群組彙整為 Bundle。此時，未登錄為 Addressable 的資□會被視為隱含依賴，並複製到各個參照它的 Bundle 中。

[*1] https://github.com/CyberAgentGameEntertainment/AssetRegulationManager/blob/main/README_JA.md



▲圖 5.1　未登錄的共用資□被複製的情況（上），與登錄至專用群組的情況（下）

圖中的範例裡，僅因 2 個 Prefab 參照一張 4MB 的紋理，Bundle 合計就變成 10MB。若將此紋理登錄至專用群組成為明確的依賴，複製即可消除，合計降至 6MB。參照來源增加多少就會被複製多少份，同時載入時在記憶體中也會展開同樣的量，因此越放著不管影響越大。此外，實際的 Bundle 會被壓縮，所以數□只是□明機制的參考□。這類重複難以靠目視追蹤，因此以工具偵測。使用 Addressables 時可使用 Addressables Analyze [*2]（2.0 系列中曾暫時移除，於 2.3.1 恢復）。也有自動解決所偵測到重複的功能，但這會改寫群組組成，因此套用後請確認建置結果。

5.6　調□已膨脹的資□

雖然規範與檢□能阻止新的膨脹，但已經累積的資□必須□行找出。資□的效能分析有多種用途各異的工具。

表 5.2: 用於調□容量與記憶體的工具

工具	可得知的資訊
Build Report / Editor Log	建置大小的分類細項
Addressables Analyze	偵測並修正 Addressables 的重複 Bundle 依賴
Project Auditor	整個專案的匯入設定與靜態組成問題

[*2] https://docs.unity3d.com/Packages/com.unity.addressables@latest

表 5.2: 用於調□容量與記憶體的工具

工具	可得知的資訊
Memory Profiler	執行時的記憶體使用量與物件的參照關係

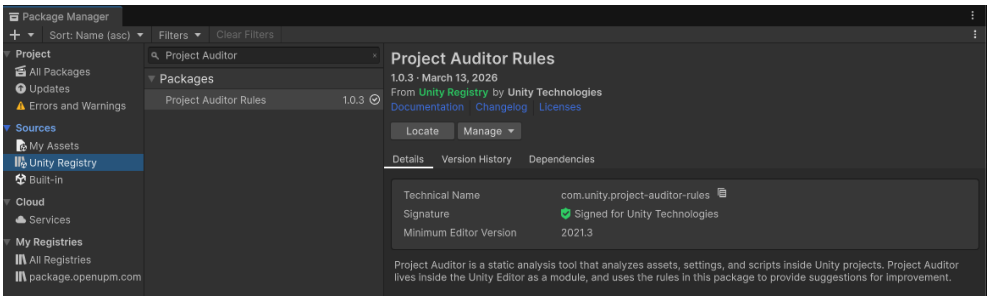
其中 Memory Profiler 在前作的第 3 章「效能分析工具」已有□明。本章介紹前作未涵蓋的 Project Auditor。

5.7 以 Project Auditor 找出問題

Project Auditor 是對專案□的資□、程式碼與設定進行靜態分析，並將有問題的部分以清單輸出的工具。即使尚未建立規範，也能掌握可能成為問題的項目。它長期是實驗性套件，於 2025 年 2 月發布 1.0 成為正式版。以下的畫面與項目，是以 Unity 6.4 與 Rules 套件 1.0.3 確認的□容。

5.7.1 導入方法依 Unity 版本而異

Unity 6.4 之前，需從 Package Manager 新增 com.unity.project-auditor [*3]。6.4 起已□建於 Editor，無須新增 [*4]，但由於判定所用的規則已分離為 com.unity.project-auditor-rules，若不導入該套件就無法分析（圖 5.2）。



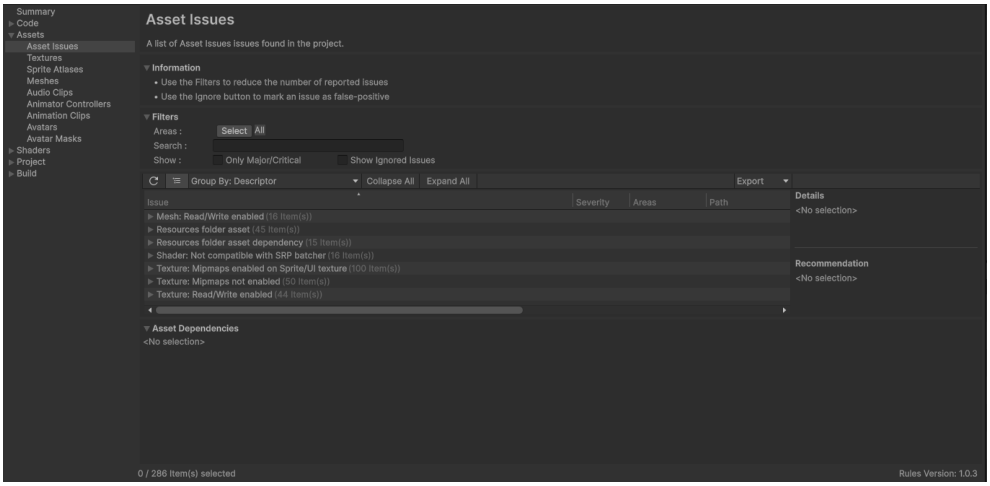
▲圖 5.2　Rules 套件。分析本體以 Editor 模組的形式存在

由於規則已與本體分離，不必升級 Editor 版本，即可只更新規則。

[*3] https://docs.unity3d.com/Packages/com.unity.project-auditor@1.0/manual/index.html
[*4] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/project-auditor-introduction.html

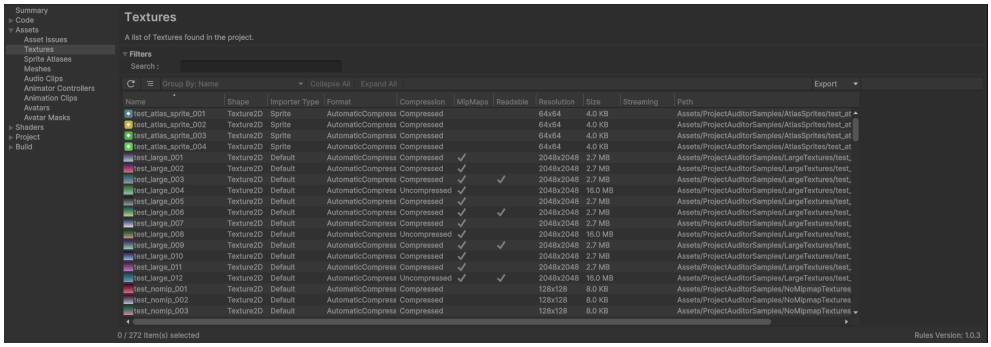
5.7.2 Assets 檢視的使用方式

開啟 Window > Analysis > Project Auditor 並執行分析後，結果會分為 Summary、Code、Assets、Shaders、Project、Build 檢視（檢視可能隨 Unity 版本增加）。調查容量與記憶體時使用的是 Assets 檢視，其分為 Asset Issues、Textures、Sprite Atlases 等 9 個。最先開啟的是 Asset Issues（圖 5.3）。偵測到的問題依種類彙整，並顯示各找到幾筆。



▲圖 5.3　Asset Issues 檢視。於驗證用專案中的偵測範例

展開種類後會顯示個別資產，選取後右側會出現詳細資訊與建議的處理方式。掌握整體概況後，再以 Textures 等個別檢視確看其細項（圖 5.4）。



▲圖 5.4　Textures 檢視。設定與結果的大小並排顯示

這個檢視的優點是設定與結果並列在同一列。即使同為 2048×2048 的紋理，Compressed 的列佔用 2.7MB，Uncompressed 的列佔用 16.0MB。無須開□ Inspector，就能看出僅 1 處設定之差就造成近 6 倍的差距。將 Size 欄依降冪排序，從上方依序確認 Compression、Resolution、Format，即可找出壓縮設定錯誤、解析度過高與格式選擇失誤。

5.7.3　減少偵測結果

在營運中的專案上執行分析時，偵測結果可能超過數千筆。在這種狀態下從上方逐一處理並不會有進展。請以前面所列 圖 5.3 中的 Filters 縮小顯示範圍。勾選 Only Major/Critical 則只會留下重要度高的項目，以 Areas 選擇對象領域，則可只顯示與容量相關的指摘。偵測到的問題會附上重要度。文件中□明為 Major、Moderate、Minor、Ignored 4 種，但依定義也存在 Critical。Ignored 並非重要度的等級，而是指被指定為忽略的狀態。處理應從 Major 依序進行，若偵測到 Critical 則以其為最優先。由於靜態分析也會指出有意為之的設定，逐筆確認後再修正較為安全。對於判斷為誤判的指摘，請加上 Ignore。Ignore 是記錄於開□中報表的狀態，因此不會寫入設定檔，重新分析後即會解除。偵測臨界□之類的判定參數則儲存於 ProjectSettings/ProjectAuditorSettings.asset，因此提交後即可讓全員一致。若要持續抑制指摘，請在自動建置端的□本中，對 AnalysisParams.WithAdditionalDiagnosticRules 傳入停用規則。

5.7.4 整合至自動建置

Project Auditor 也能從命令列執行 [*5]。若將其整合至自動建置，並在每次發布時保留偵測件數，就能追蹤是在□個版本增加的。Preferences 中有在建置後將偵測件數輸出至日誌的設定，以及將偵測視為錯誤的設定。在自行撰寫失敗判定之前，請先確認這些設定是否足□。□一方面，由於未提供比較 2 份已儲存報表的 API，要取得與前次的差異須自行處理。此外，也有建置後的自動分析無法完成而停滯，或分析中耗盡記憶體等回報 [*6]。請設定逾時，避免分析失敗導致整個建置停止。

5.8 結語

膨脹對策與其□是讓個別資□變輕的技術，更偏重於建立讓團隊阻止再發的機制。營運過程中規範逐漸鬆弛，是任何專案都會發生的事。正因如此，將人的時間花在決定與重新檢視基準上，並把要求遵守的部分交給機器，最終才是最能長久持續的做法。Project Auditor 雖是剛成為正式版的工具，但作為尚未建立規範的專案掌握現況的入口，已足□實用。請先執行一次分析，從觀察自己專案的傾向開始。

[*5] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/run-from-command-line.html

[*6] https://github.com/Unity-Technologies/ProjectAuditor/issues

第6章

UI 系統的取捨 － IMGUI / uGUI / UI Toolkit

財津 大睦 Zaitsu Hiromu

株式會社 Sumzap

X @RomGameclient

所屬：株式會社 Sumzap X 帳號 / 網站 / GitHub 帳號（選填）：@RomGameclient

前作《Unity 效能調優寶典》[*1]介紹了拆分 Canvas、調整 Raycast Target 等應對 uGUI 負載的方法。本章再往前一步，討論 UI 系統的選擇本身，因為選擇□一套系統，決定了 UI 的負載結構與可採取的對策類型。Unity 有 IMGUI、uGUI、UI Toolkit 三種 UI 系統。本章依序介紹這三者，整理它們目前在執行階段與編輯器擴充中的定位。本章的描述以 Unity 6.5 時點的資訊為依據。

6.1 IMGUI

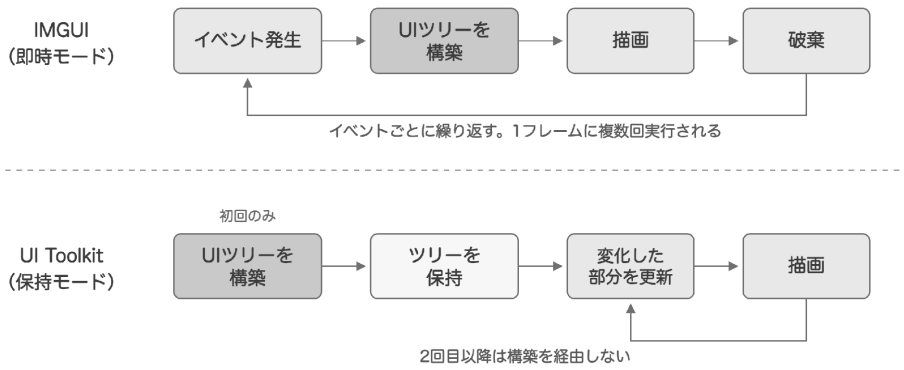
執行階段的 IMGUI 已不在遊戲□ UI 的選項之中。官方手冊明確指出，它並非用於玩家操作的一般遊戲□ UI，並列舉遊戲□的除錯顯示與工具、自訂 Inspector、編輯器擴充作為預期用途。在除錯覆蓋層與執行階段日誌檢視器等用途中，目前仍在使用。□一方面，在編輯器擴充中，IMGUI 在官方立場上仍然現役。官方的 UI 系統比較頁面 [*2] 將 IMGUI 列為替代手段，並指出其優點有兩項：能不受限制地存取編輯器的擴充功能，以及是能在畫面上快速繪製 UI 的輕量 API。官方的 IMGUI 遷移指南 [*3] 中，明確列出了 UI Toolkit 中

[*1] https://cyberagentgameentertainment.github.io/UnityPerformanceTuningBible/ja/preface.html

[*2] https://docs.unity3d.com/6000.5/Documentation/Manual/UI-system-compare.html （2026 年 8 月 6 日參照）

[*3] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-IMGUI-migration.html

沒有對應項目的 API。實務上影響較大的是 InspectorTitlebar()、Toolbar()、IntPopup()、LinkButton() 等。這些都是讓編輯器視窗的外觀貼近 Unity 本體的 API，在需要的場合，目前仍必須使用 IMGUI。不過，作為編輯器擴充使用時，有一些應該知道的負載特性。IMGUI 是每次都在 OnGUI() 中撰寫 UI 的即時模式系統，不保留 UI 樹的資訊，每次事件都會重複建構與銷毀。與保留模式的 UI Toolkit 的差異如 圖 6.1 所示。OnGUI() 在 1 個影格中會執行多次，每次都容易發生堆積配置。在持續重新繪製的編輯器視窗中，這會成為 GC 的負載。



▲圖 6.1　即時模式與保留模式的繪製週期

6.2　uGUI

這是以 Canvas 與 GameObject 為基礎的系統。每個元素都是 GameObject，並具有 Transform 階層。它僅限執行階段，無法用於編輯器擴充。在執行階段，它事實上是標準。優點有三項。・可直接在 Scene 檢視中配置，並當場預覽・可用 Animator 與 Timeline 讓 UI 動起來・Asset Store 中有大量以 uGUI 為前提的套件就負載而言，元素發生變更的 Canvas 會重建網格，且 Transform 階層的更新與 Raycast Target 的掃描成本會隨元素數量增加。前作介紹的 Canvas 拆分與 Raycast Target 調整，都是針對這種結構所□生之負載的代表性對策。

6.3 UI Toolkit

這是一次建構 VisualElement 樹後，之後只更新有變化部分的保留模式系統。邏輯以 C#、UI 結構以 UXML、樣式以 USS 分開撰寫。執行階段與編輯器擴充皆可使用，在編輯器擴充中是官方的首選推薦。不過，官方的□法是「若要製作複雜的編輯器工具」。若只是並排幾個欄位的工具，不引入 UXML 與 USS，只用 C# 組建也沒有問題。在負載方面，與 uGUI 的差異就直接成為對策的差異。與變更一個元素就會重建整個 Canvas 網格的 uGUI 不同，UI Toolkit 只更新有變化的部分，因此不需要相當於 Canvas 分割的對策。不過，元素數量增加時，版面計算的成本同樣會增加，因此仍需要減少元素的做法。在 Unity 6 系列中，事件分派的加速、幾何□生的 Job 化、版面計算快取的改善等效能改進也持續進行。

6.3.1 與 uGUI 相比不足之處

截至 Unity 6.5，無法做到的事項有以下這些。這些都是 Unity 列為開發中的項目 [*4]，但若現在要判斷是否採用，就必須以無法做到為前提來考量。・以 Animation 視窗處理 VisualElement 關鍵影格動畫的功能・在 Scene 檢視中編輯 UI・從 MonoBehaviour □本參照以 UXML 定義的 VisualElement 的機制・支援自訂著色器的頂點資料生態系也依然薄弱。Asset Store 的資□多以 GameObject 階層為前提，無法直接使用。TextMeshPro 也無法在 UI Toolkit 中使用，UI Toolkit 使用專用的文字系統（6.5 起成為預設的 Advanced Text Generator）。DOTween 也新增了 VisualElement 用的捷徑，但對象為 Unity 6.3 以後 [*5]。從 uGUI 遷移時，錨點與樞軸沒有直接的對應項目。UI Toolkit 的版面是在父元素中依序流入並排列子元素的方式。與逐元素指定位置與大小的 uGUI 相比，組建方式本身就不同。

6.4 官方推薦的演變

官方的 UI 系統比較頁面依手冊版本分別公開，比對□讀就會發現執行階段 UI 的推薦理由已經對調 [*6]。其明細如 表 6.1 所示。

[*4] https://discussions.unity.com/t/ui-toolkit-development-status-and-next-milestones-november-2025/1698009

[*5] https://dotween.demigiant.com/download.php

[*6] https://docs.unity3d.com/6000.0/Documentation/Manual/UI-system-compare.html （2026 年 8 月 6 日參照）

表 6.1: 執行階段 UI 推薦理由的變化

需求、用途	Unity 2022.3～6.1	Unity 6.5
處理大量 UI	UI Toolkit	UI Toolkit
需要面向美術與設計師的製作工作流程	UI Toolkit	UI Toolkit
無紋理的 UI 繪製	UI Toolkit	UI Toolkit
配置於 3D 空間並受光照的 UI	uGUI	UI Toolkit
以自訂著色器與材質表現	uGUI	UI Toolkit
易於從 MonoBehaviour 參照	uGUI	uGUI

從 2022.3 到 6.1 一貫位於 uGUI 一側的兩項 [*7]，隨著世界空間 UI 在 6.2、自訂著色器在 6.3 針對 URP 實作，轉移到了 UI Toolkit 一側。剩下的「易於從 MonoBehaviour 參照」，參照以 UXML 定義之元素的機制也已列入開發中的項目。不過，比較頁面的結構本身在 Unity 6.5 中也沒有改變。執行階段 UI 仍是以 uGUI 為預設介紹，並將 UI Toolkit 作為其替代方案並列。對調的只有推薦的理由，以□一個為預設並未改變。同一頁面上依用途列出的清單，標題在 2022.3 至 6.2 為「Recommendation（推薦）」，6.3 以後則改為「Often used for（常用於）」。Unity 自己也不再稱之為推薦。

6.5 併用 UI 系統時的限制

在一個專案中使用多個 UI 系統本身是可行的，但組合後會出現一些問題。以下分為執行階段與編輯器擴充來看。

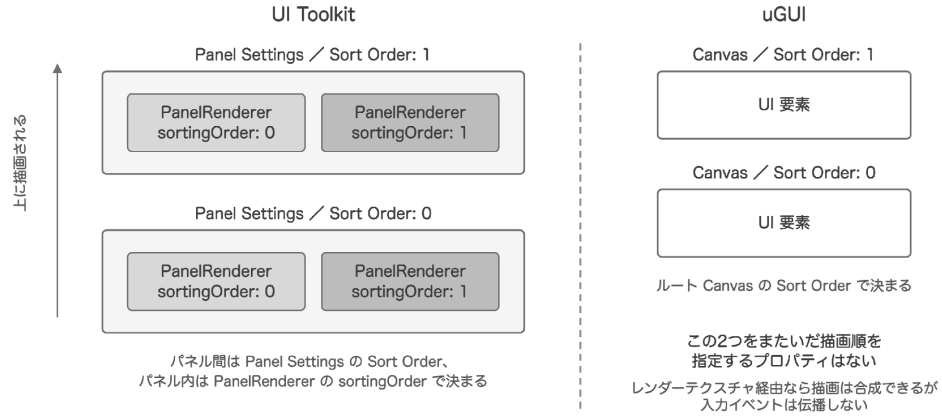
6.5.1 執行階段

uGUI 與 UI Toolkit 可以在同一專案中使用，但無法將其中一方嵌入□一方之中。導覽與樣式的不一致，官方也列為限制。繪製順序只能在各自的系統□控制（圖 6.2）。要用 UI Toolkit 在畫面上顯示 UI，需將 Panel Renderer 元件附加到 GameObject 上。這個角色在 Unity 6.4 以前由 UIDocument 擔任，6.5 起被取代 [*8]。若使用 LTS 的 6.3，則為 UIDocument。此 Panel Renderer 是排序的單位，面板之間以 Panel Settings 的 Sort Order，同一面板□的 Panel Renderer 之間則以 PanelRenderer.sortingOrder 決定順序。uGUI 以根 Canvas 的 Sort Order 排序。沒有可跨越這兩者指定順序的屬性。

[*7] https://docs.unity3d.com/2022.3/Documentation/Manual/UI-system-compare.html（2026 年 8 月 6 日參照）

[*8] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-create-ui-document-component.html

從 uGUI 遷移的指南 [*9] 也列出以 Render Texture 合成繪製結果的做法，但明確指出輸入事件不會傳達。由於無法用於需要操作的 UI，請避免將 uGUI 與 UI Toolkit 重疊的設計。

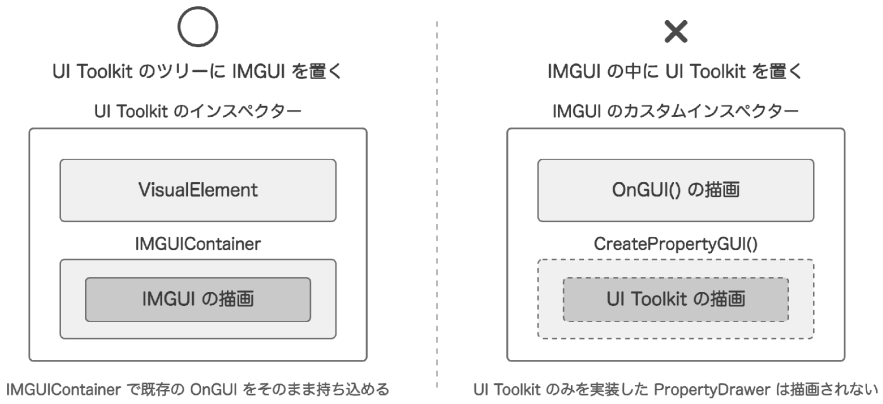


▲圖 6.2　UI Toolkit 與 uGUI 的繪製順序決定方式

6.5.2　編輯器擴充

在編輯器擴充中，兩個系統可以巢狀嵌套的方向僅限單向（圖 6.3）。可用 IMGUI Container 在 UI Toolkit 的樹中放置 IMGUI 的繪製，但反過來，在 IMGUI 的繪製中放置 UI Toolkit 的樹則無法做到。這成為問題的情況是自製 Inspector 顯示時。Inspector 的自訂分為負責整個元件的自訂 Inspector，以及逐欄位替換外觀的 PropertyDrawer 兩層，□側的 PropertyDrawer 無法選擇自己是從□一個系統的 Inspector 被呼叫。僅以 UI Toolkit 實作的 PropertyDrawer，若從以 IMGUI 撰寫的自訂 Inspector 或 IMGUI 的 PropertyDrawer 中被呼叫，該欄位的□欄就會被替換為「No GUI Implemented」（未實作 GUI）的顯示。Unity 2022.2 以後 Inspector 預設為 UI Toolkit，因此多數情況下不會發生，但在仍保留 IMGUI 自訂 Inspector 或 PropertyDrawer 的專案中，PropertyDrawer 端也必須準備 IMGUI 版的繪製。

[*9] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-Transitioning-From-UGUI.html



▲圖 6.3　PropertyDrawer 能否巢狀嵌套

6.6　結語

調□後印象深刻的是，官方的推薦理由在 Unity 6.5 中幾乎已轉向 UI Toolkit 一側。執行階段仍以 uGUI 為標準，沒有理由遷移已在運作的□容，但在新開發中避開 UI Toolkit 的理由確實減少了。猶豫時，可依對 Animator 與 Timeline 演出以及 Asset Store 資□的依賴程度來判斷。至於編輯器擴充，複雜的工具使用 UI Toolkit，需要貼近 Unity 本體外觀時使用 IMGUI，這樣的劃分目前應該足以應付。UI 系統的選擇，是決定是否需要採取前作所介紹對策的最上游調優。

第7章

Unity 6 Render Graph 圖塊記憶體最佳化

清原 隆行 Kiyohara Takayuki

株式會社 CyberAgent SGE 核心技術本部

X @nezumimusume

所屬：株式會社 CyberAgent SGE 核心技術本部X 帳號：nezumimusume

7.0.1 前言

本章□明如何利用 Unity 6 的 Render Graph，運用行動 GPU（圖塊式渲染架構）的圖塊記憶體，以削減 GPU 時間、記憶體頻寬與記憶體使用量。在行動遊戲開發中，記憶體頻寬往往比 GPU 的運算效能更早成為瓶頸。此外，記憶體頻寬的消耗與耗電量和發熱直接相關，因此不僅影響影格率，也會影響裝置的發熱與電池消耗。本章以下列四個主題進行□明，每個主題都有同名的章節。

1. 圖塊記憶體的載入/儲存
2. Render Pass/Subpass
3. 影格緩衝區擷取（Frame Buffer Fetch）
4. 無記憶體（memoryless）

7.1 圖塊式渲染的基礎

本節□明行動 GPU 為何具備圖塊記憶體。

7.1.1 行動 GPU 與記憶體頻寬

現代 GPU 的運算速度與記憶體讀寫速度之間的差距日益擴大，記憶體存取已成為主要瓶頸。GPU 核心的運算效能（FLOPS）隨世代大幅成長，記憶體頻寬的成長卻追趕不上。PC 的獨立 GPU 透過搭載 GDDR 這類圖形專用的高頻寬記憶體來緩解此問題，但行動 SoC 無法採用這種做法。

項目	PC（獨立 GPU）	行動 SoC
圖形記憶體	專用（GDDR6 等，數百 GB/s）	無（LPDDR 由 CPU/GPU/ISP 等共用）
頻寬規模	數百 GB/s～1 TB/s	數十 GB/s（且為共用）
電力限制	數百 W 等級的電源與散熱	數 W，僅被動散熱

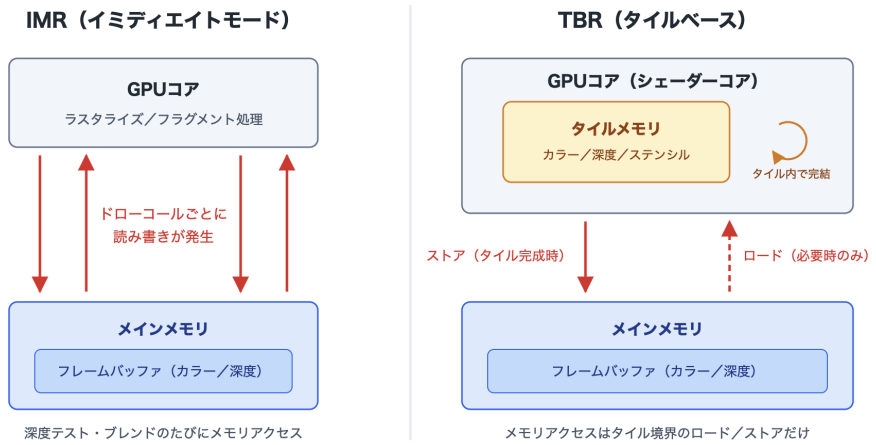
行動 GPU 沒有專用的圖形記憶體，與 CPU 共用同一組 LPDDR 記憶體。而且共用對象不只有 CPU，還包括相機的 ISP 與機器學習加速器等 SoC 上的所有區塊。頻寬本身有限，還要彼此爭用。此外，存取主記憶體（DRAM）是 SoC 上耗電特別大的操作。使用的頻寬越多，耗電量越增加並產生發熱，CPU/GPU 的時脈會因熱節流（thermal throttling）而下降，影格率隨之降低。也就是說，在行動裝置上會發生下列流程：

```
記憶體頻寬消耗
  → 耗電量增加 → 發熱 → 節流 → 效能下降
```

若遊戲最初幾分鐘能以 60FPS 執行，遊玩一段時間後影格率卻下降，原因往往就是這種頻寬與發熱。為了在這樣的限制下完成繪製而誕生的，就是圖塊式渲染（TBR: Tile-Based Rendering）架構的 GPU。

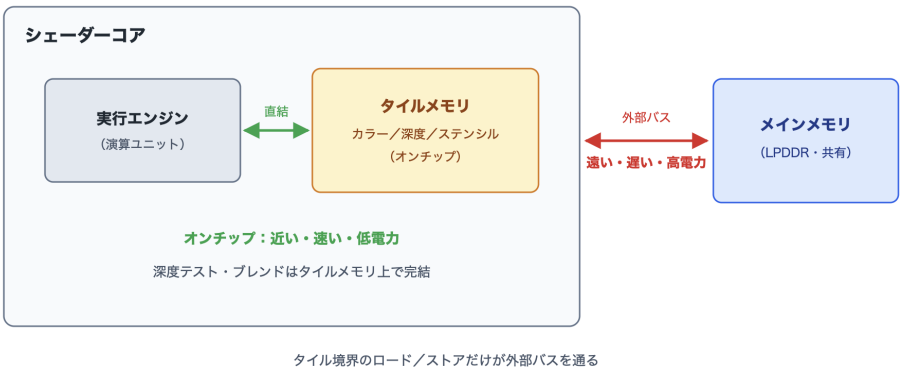
7.1.2 IMR 與 TBR

為了理解 TBR，先與 PC GPU 的主流架構，即立即模式渲染（IMR: Immediate Mode Rendering）做比較。IMR 對由繪製呼叫（draw call）送入的圖元當場進行光柵化、執行片段著色器，並將結果寫入記憶體上的渲染目標。每次深度測試或混合都會對渲染目標進行讀寫，PC 的 GPU 以高頻寬的圖形記憶體與大容量快取承受這些存取。一方面，TBR 將畫面分割成小圖塊，逐一完成每個圖塊的渲染。每個著色器核心搭載稱為圖塊記憶體的小型晶片記憶體，圖塊的繪製（顏色、深度、模板的讀寫與混合）全部在此圖塊記憶體上進行。原則上，存取主記憶體只發生兩次：圖塊開始繪製時（載入）與結束時（儲存）。



▲圖 7.1　IMR 與 TBR 的比較

下圖是 TBR GPU 中著色器核心的概念圖，著色器核心直接連接圖塊記憶體。由於位於晶片⊠且實體距離近，存取圖塊記憶體比存取主記憶體快上數個數量級，耗電量也低得多。實際的 GPU 也能確認這種結構，例如 ARM Mali-G71 官方資料中的著色器核心設計圖，就顯示了圖塊記憶體直接連接著色器核心的情形。



▲圖 7.2　著色器核心與圖塊記憶體（概念圖）

如此一來，在 IMR 中記憶體存取最頻繁的深度測試與混合，在 TBR 中於圖塊記憶體

□即可完成。

7.1.3 TBR 的兩個階段

TBR 的 GPU 將一個影格的繪製大致分成兩個階段執行。

分塊階段（幾何階段）

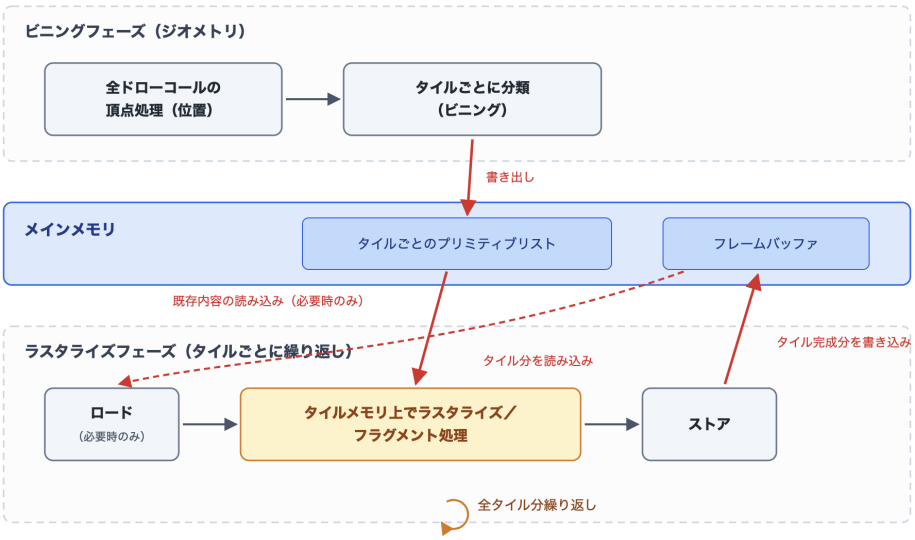
首先，對影格□所有繪製呼叫執行頂點著色器（位置計算的部分），將每個圖元依其涉及的圖塊進行分類（binning）。分類結果會以「各圖塊的圖元清單」形式寫出到主記憶體。需要注意的是，分塊階段本身也會消耗主記憶體的頻寬。頂點資料越多，圖元清單的讀寫就越多。這就是 TBR 的 GPU 在高多邊形場景中容易處於劣勢的原因。

光柵化階段（渲染階段）

接著逐一處理圖塊。對每個圖塊依下列步驟進行繪製：

1. 必要時，將渲染目標的既有□容載入圖塊記憶體
2. 對涉及該圖塊的圖元進行光柵化並執行片段著色器（讀寫全部在圖塊記憶體上進行）
3. 將完成的圖塊□容儲存到主記憶體

其中步驟 1 與 3 就是本章的主題，即載入/儲存。詳情將在下一節「圖塊記憶體的載入/儲存」□明，TBR 的成本結構是：相較於圖塊本身的繪製，圖塊邊界上與主記憶體之間的資料交換更為昂貴。



▲圖 7.3　分塊階段與光柵化階段

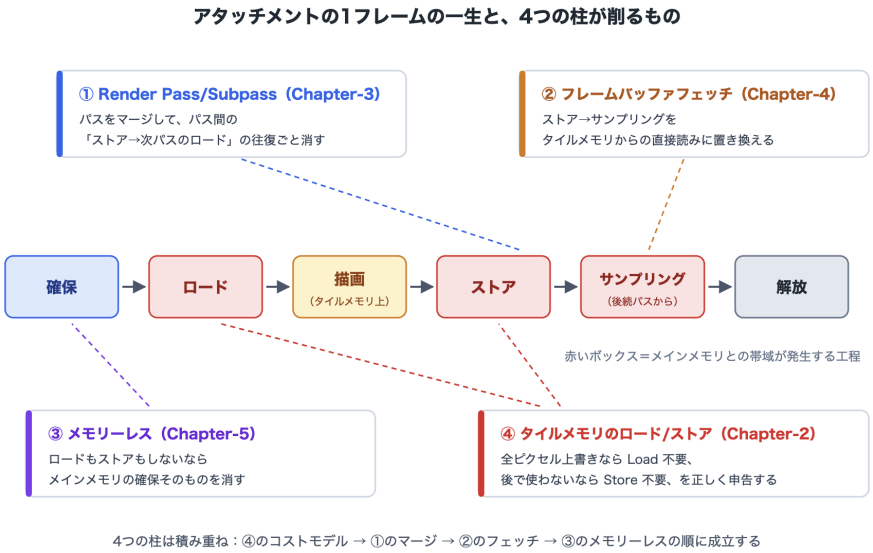
各廠商的名稱與特性

TBR 的基本結構是共通的，但各 GPU 廠商的名稱與實作細節不同。本章之後以圖塊記憶體／分塊作為總稱。

廠商（GPU）	階段/機制的名稱	備註
Qualcomm Adreno	Binning / GMEM	將稱為 GMEM 的晶片□記憶體分割使用。具有依繪製□容切換為類 IMR 運作的模式
ARM Mali	Tiler / Tile Memory	圖塊基本為 16×16 像素
Imagination（PowerVR/IMG）	TBDR	以硬體進行隱藏面移除（HSR）的 TBDR。Pixel 10 的 IMG DXT 也屬於此系譜
Apple GPU	Tile Memory	PowerVR 系譜的 TBDR。可從 Metal 明確操作圖塊記憶體

7.1.4 本章處理的四個主題

至此我們已了解圖塊邊界的載入/儲存會□生成本。那麼，該如何削減這項成本呢？本章處理的四個主題，都是針對此問題的手段。



▲圖 7.4　附件的生命週期與四個主題的關係

試著考慮某個渲染目標（附件）在一個影格□的生命週期。

```
配置
 → 載入 → 繪製 → 儲存 →（在後續 Pass 中作為紋理）取樣 → 釋放
```

依照要削減生命週期中的□個部分，四個主題可整理如下。

主題	削減對象	想法	章節
載入/儲存	不必要的載入與不必要的儲存	明確指出「若會覆寫全部像素就不需載入」「之後不會使用就不需儲存」	「圖塊記憶體的載入/儲存」

主題	削減對象	想法	章節
Render Pass/Subpass	Pass 與 Pass 之間的儲存→載入往返	將多個 Pass 整合（合併）為一個 Render Pass，使中間結果留在圖塊記憶體	「Render Pass / Subpass」
影格緩衝區擷取	儲存→取樣的往返	直接從圖塊記憶體讀取前一個子通道的結果	「影格緩衝區擷取」
無記憶體	主記憶體配置本身	若既不載入也不儲存，主記憶體上就不需要實體	「無記憶體」

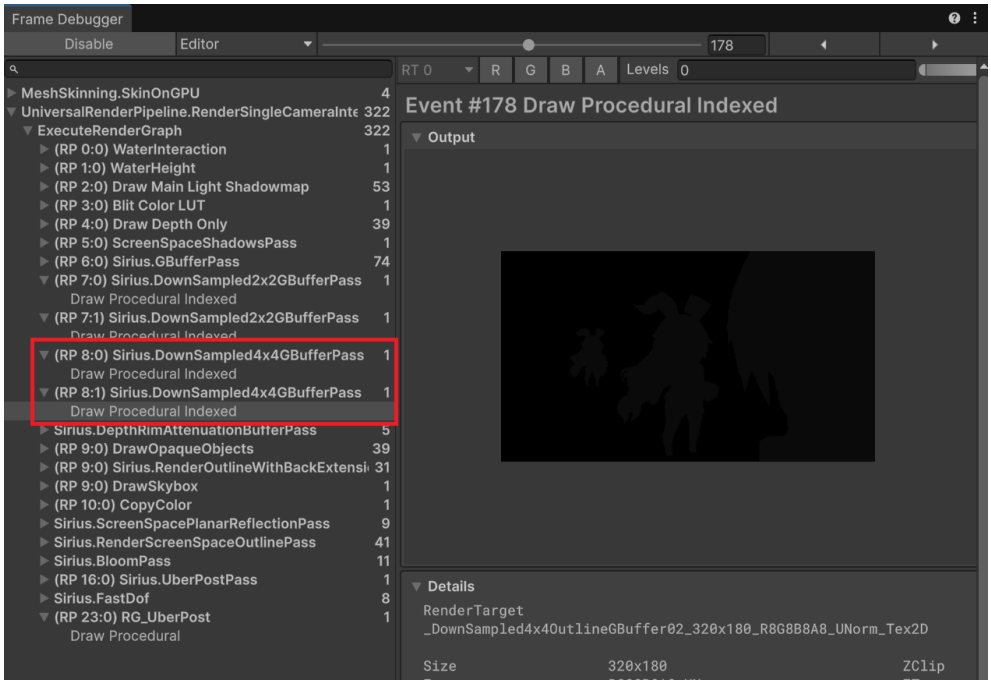
這四者並非獨立的技巧。Pass 先被合併才能使用影格緩衝區擷取，載入與儲存都消失後無記憶體才得以成立。本章的章節結構也依循這種相依關係：成本模型（載入/儲存）→機制（Subpass）→ 活用（擷取）→ 收尾（無記憶體）。此外，Render Graph 是先蒐集各 Pass 的資源使用資訊（讀取什麼、寫入什麼），再制定渲染執行計畫的系統。因此，它能自動進行 Pass 合併、決定載入/儲存動作、無記憶體化等最佳化。本章所□明的，可以□就是如何將資源的使用方式正確告知 Render Graph，使這些最佳化得以運作。

7.1.5 驗證工具的使用方式

本章反覆採用「先以工具確認管線狀態，再進行測量」的流程。主要使用下列三種工具。

7.1.6 Frame Debugger

Unity 的 Frame Debugger（Window > Analysis > Frame Debugger）是確認 Pass 結構的基本工具。本章特別關注事件名稱的（`RP n:m`）標記。若如 `RP 4:0`、`RP 4:1` 這樣，相同的 Render Pass 編號（n）後接續排列子通道編號（m），就表示這些 Pass 已合併為一個 Native Render Pass。此外，Pass 合併（Native Render Pass）只在 Vulkan/Metal 等環境中運作。若編輯器的圖形 API 為 D3D 等，則無法觀察到合併，因此在編輯器中確認時，請將 Graphics API 改為 Vulkan，或將 Frame Debugger 附加到實機建置上。



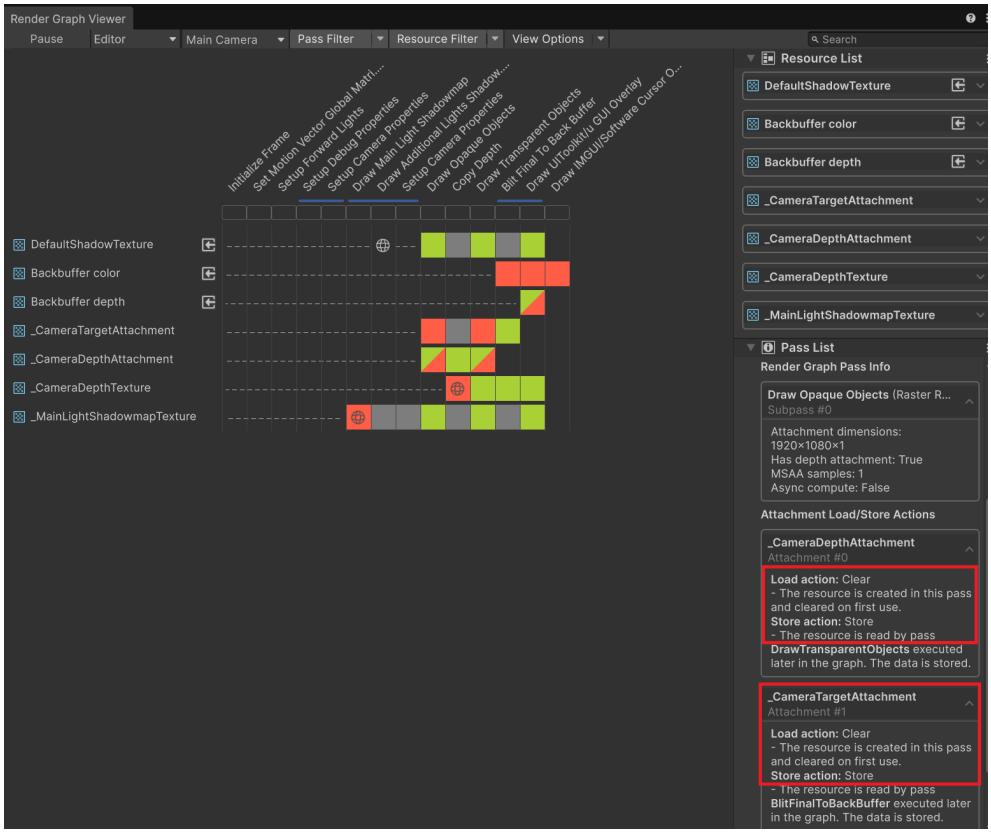
▲圖 7.5　Frame Debugger 中的合併狀態

7.1.7　Render Graph Viewer

Render Graph Viewer（Window > Analysis > Render Graph Viewer）是將 Render Graph 所建構的執行計畫本身視覺化的工具。

- Pass 清單，以及各 Pass 所存取資源的時間軸（□個資源在何時建立、何時銷毀）
- 顯示各附件的 Load Action / Store Action 以及 Memoryless 狀態。本章將使用此顯示來確認最佳化的結果
- 顯示 Pass 是否已合併
- 顯示未能合併的原因（Pass Break Reasoning）

第 4 項的 Pass Break Reasoning，是在「應該合併的 Pass 卻沒有合併」時最先確認的資訊。在「Render Pass / Subpass」一節之後會反覆使用。



▲圖 7.6　在 Render Graph Viewer 中確認合併狀態與載入/儲存動作

7.1.8　RenderDoc

以 RenderDoc 擷取 Vulkan 後，可在 API 層級確認各 Render Pass 的附件所指定的 `loadOp`/`storeOp`。想進一步深入 Render Graph Viewer 的顯示，或想比對 Unity 的顯示與傳給驅動程式的⌧時使用。

7.2　圖塊記憶體的載入/儲存

本節⌧明作為本章整體基礎的載入/儲存動作。

7.2.1 何謂載入/儲存動作

TBR 的 GPU 在圖塊開始繪製時，會將渲染目標的既有□容讀入圖塊記憶體（載入），並在繪製結束時將圖塊記憶體的□容寫出到主記憶體（儲存）。但載入與儲存並非總是必要。圖形 API 提供了可針對各附件指定「開始時如何處理」「結束時如何處理」的動作。

載入動作

動作	行為	頻寬成本
Load	將主記憶體的既有□容讀入圖塊記憶體	高（相當於解析度 × 格式的讀取量）
Clear	以指定顏色清除圖塊記憶體	近乎零（僅圖塊記憶體□的操作）
DontCare	不做任何事（圖塊記憶體的初始□不確定）	零

儲存動作

動作	行為	頻寬成本
Store	將圖塊記憶體的□容寫出到主記憶體	高（相當於解析度 × 格式的寫入量）
Resolve	僅寫出 MSAA 取樣解析（resolve）後的結果	中（僅解析後的 1 個取樣）
DontCare	不寫出任何□容（□容被捨棄）	零

也就是□，消耗頻寬的只有 Load 與 Store（以及 Resolve）。反過來□，以 Clear/DontCare 開始並以 DontCare 結束的附件，完全不會與主記憶體交換資料。這種狀態是後述無記憶體的前提條件。要最佳化載入/儲存，需針對各附件確認下列兩點。

1. 開始時，是否需要先前的□容？ 若 Pass 會覆寫全部像素，則不需要（DontCare）。從清除開始時為 Clear，只有在疊加繪製於先前的繪製結果上時才使用 Load。
2. 結束時，之後是否還會使用這些□容？ 若後續 Pass 或下一影格會使用則為 Store，若在此 Render Pass □即用完則為 DontCare。

7.2.2 頻寬成本的估算

一次載入/儲存的成本可用簡單的乘法概算。

```
一次載入（或儲存）的資料量
 = 水平解析度 × 垂直解析度 × 每像素位元組數
```

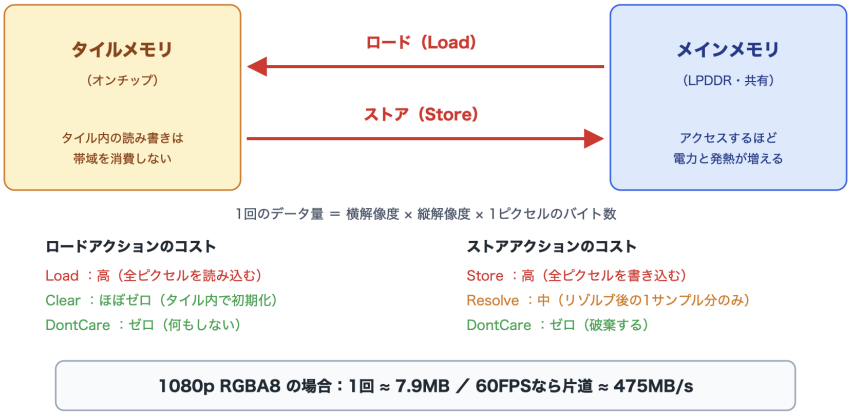
以下為 1080p（1920×1080）下代表性格式的數□。

格式	每像素	一次載入/儲存	60FPS 時的頻寬
R8G8B8A8 (RGBA8)	4 位元組	約 7.9MB	約 475MB/s
B10G11R11_Float	4 位元組	約 7.9MB	約 475MB/s
R16G16B16A16_Float	8 位元組	約 15.9MB	約 950MB/s
D24S8 / D32（深度）	4 位元組	約 7.9MB	約 475MB/s

例如僅是「儲存相機顏色（RGBA8），並在下一個 Pass 載入」，一次往返就約 16MB，以 60FPS 計算會消耗約 950MB/s 的頻寬。考慮到行動 SoC 的實效頻寬只有數十 GB/s，而且還要與 CPU 爭用，這個量不容忽視。此外也請注意以下幾點。

- MSAA 會有倍率。若不 Resolve 而直接 Store 4x MSAA 的顏色緩衝區，資料量會變成 4 倍（MSAA 與儲存的關係會在「無記憶體」一節□明）。
- 載入與儲存兩者都要計算。每跨越一個 Pass，就會成對發生「前一個 Pass 的 Store ＋ 下一個 Pass 的 Load」。
- 若在後續 Pass 中對已儲存的結果取樣，（未命中快取的部分）還會再□生讀取。

建議針對開發中遊戲的渲染管線，調□一次一個影格□有多少 MB 往返於圖塊記憶體與主記憶體之間。只要在 Render Graph Viewer 中確認各 Pass 的附件與動作，再套用上表進行乘法計算，就能看出該從□個 Pass 開始削減的優先順序。



▲圖 7.7　圖塊記憶體 ⇔ 主記憶體的載入/儲存與頻寬成本

7.2.3　以 AccessFlags 宣告資源使用方式

那麼，在 Render Graph 中要如何指定這些載入/儲存動作呢？實際上，Render Graph 並不直接指定載入/儲存動作。新增 Pass 時，開發者向建構器（builder）宣告資源的使用方式，Render Graph 再依這些資訊決定動作。此時用於宣告的就是 `AccessFlags`。

▼清單 7.1　以 AccessFlags 宣告資源

```
public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Example", out var
    passData))
    {
        // 宣告輸出附件。
        // AccessFlags.WriteAll = 此 Pass 會寫入全部像素（不需要先前的□容）
        builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
        ...
    }
}
```

`AccessFlags` 的主要□與意義如下。

AccessFlags	意義	對動作的影響
Read	讀取先前的□容	載入動作變成 Load
Write	寫入（可能只寫入部分像素）	需要保留先前的□容，載入可能變成 Load
ReadWrite	讀取與寫入兩者	載入變成 Load
Discard	可捨棄先前的□容	可將載入設為 DontCare
WriteAll	寫入全部像素（Write + Discard）	可將載入設為 DontCare

其中特別需要注意 Write 與 WriteAll 的差異。單純的 `Write` 是「可能只寫入部分像素」的宣告，因此 Render Graph 為了保留未被寫入的像素□容，不得不將載入動作設為 Load。對於全螢幕 Blit 這類必定覆寫全部像素的 Pass，請指定 `WriteAll`。以 1080p 的 RGBA8 為例，光是這個指定的差異，就決定了每影格是否會□生約 8MB 的載入。□一方面，決定儲存動作的是該資源之後是否會在後續 Pass 中使用。這並非由開發者指定，而是由 Render Graph 檢視整張圖後判斷。下一項將□明其規則。

7.2.4 Render Graph 決定動作的規則

Render Graph 在依所有已記錄 Pass 的資源使用資訊建構 Native Render Pass 時，會決定各附件的載入/儲存動作。判斷規則整理如下。

載入動作的決定

條件	決定的動作
以 AccessFlags.WriteAll（或 Discard）宣告	DontCare
資源的首次使用，且要求清除	Clear
有對先前□容的讀取/部分寫入	Load

儲存動作的決定

條件	決定的動作
後續任何 Pass 都不使用	DontCare
被後續 Pass 以 UseTexture（作為紋理使用）	Store
僅被後續 Pass 作為 SetInputAttachment（輸入附件）使用，且已合併於同一個 Native Render Pass □	DontCare（在圖塊記憶體□傳遞）

條件	決定的動作
被設定為全域紋理	Store（因為 Render Graph 看不到後續的使用，所以保守地儲存）
MSAA 且有解析目標	Resolve

從這些規則可導出兩項實務上的指引。

- 將設為全域紋理視為最後手段。 以 `SetGlobalTextureAfterPass` 等方式設為全域的紋理，Render Graph 無法追蹤其使用狀況，因此一律會被儲存，也會被排除在無記憶體化的對象之外。若抱著先設成全域再□的寫法，最佳化就會失效。
- `UseTexture` 與 `SetInputAttachment` 在頻寬方面完全不同。同樣是「讀取前一個 Pass 的結果」，前者會□生儲存→取樣的往返，後者則可能在圖塊記憶體□完成（詳見「影格緩衝區擷取」一節）。

匯入紋理的處理

若以 `renderGraph.ImportTexture` 匯入 `RTHandle` 等由應用程式端管理的紋理，Render Graph 並不知道該紋理在圖之外如何被使用。因此預設會保守處理（即使是首次使用也會載入，最後一次使用後也會儲存）。匯入時傳入 `ImportResourceParams`，即可放寬這種保守的行為。

▼清單 7.2　匯入時的參數設定

```
var importParams = new ImportResourceParams
{
    clearOnFirstUse = true,   // 首次使用時清除（可將載入設為 ） Clear
    discardOnLastUse = true,  // 最後一次使用後可銷毀（可將儲存設為 ） DontCare
};
var handle = renderGraph.ImportTexture(_rtHandle, importParams);
```

若匯入的紋理是「每影格重新建立，不需要先前的□容」，請設定 `clearOnFirstUse`/`discardOnLastUse`。□外，若是 Render Graph 管理的紋理（以 `renderGraph.CreateTexture` 建立者），這些都會自動判斷。除了確實需要自行管理生命週期的紋理之外，基本上應交由 Render Graph 管理。

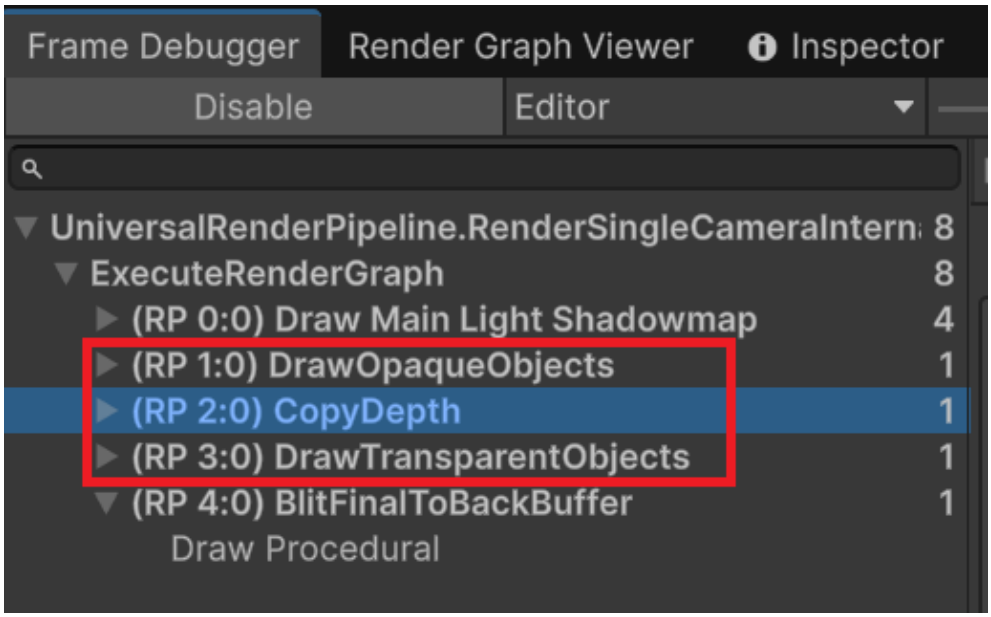
解讀載入/儲存動作

了解規則後，來實際解讀一個影格。在 Render Graph Viewer 中□看 URP 標準前向渲染的影格，可觀察到例如下列的結構。

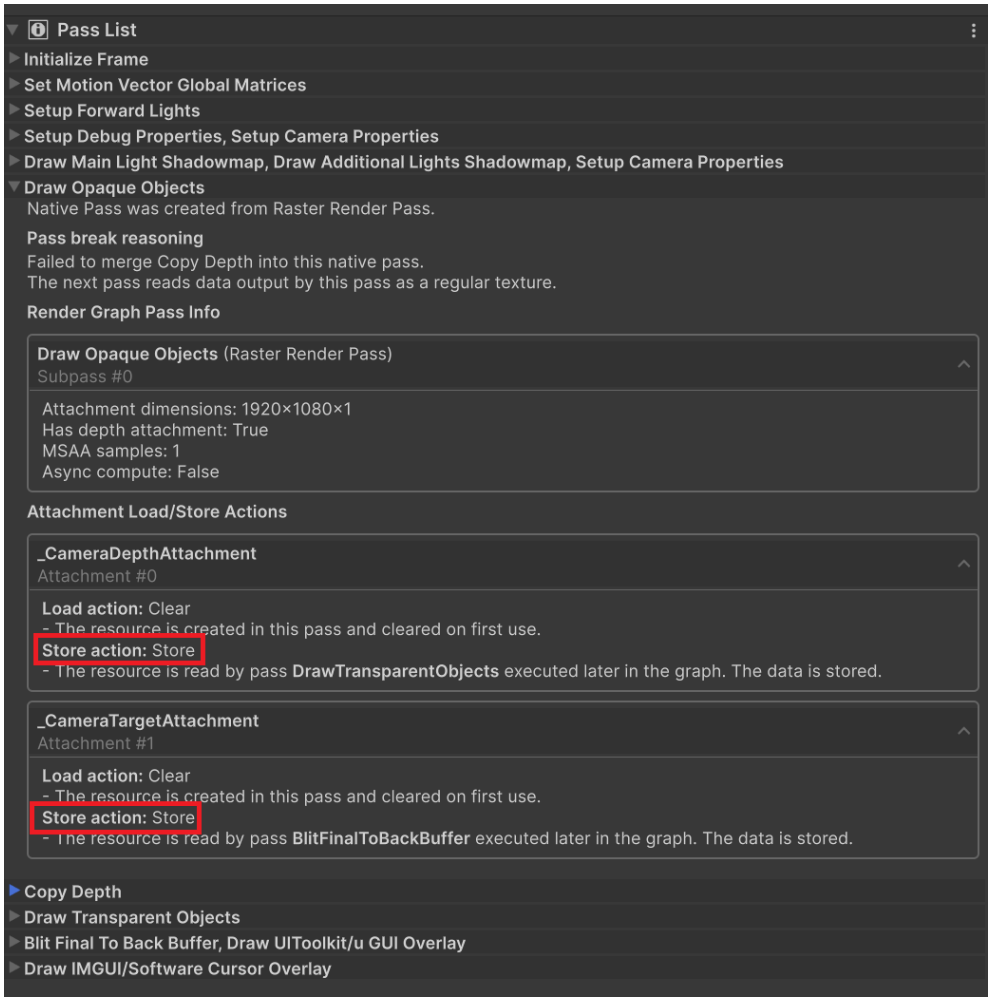
Pass	附件	Load	Store
MainLightShadow	陰影貼圖（深度）	Clear	Store
DrawOpaqueObjects	相機顏色	Clear	（視後續而定）
相機深度	Clear	（視後續而定）	
DrawTransparentObjects	相機顏色	（合併時為延續）	Store
FinalBlit	後緩衝區	DontCare	Store

解讀的重點如下。

- 陰影貼圖是「繪製後再取樣」的對象，因此只能是 Store。這是無法削減的儲存。
- 若不透明→半透明已合併為一個 Native Render Pass，則兩者之間不會發生相機顏色的儲存/載入。反之，若中間夾入了其他東西而使合併中斷，該處就會□生往返。
- 也請留意相機深度的儲存動作。開□ Depth Texture 後，相機深度就會被 Store。著色器或後處理並未要求 `_CameraDepthTexture`，卻只有設定被開□，這是實際專案中常見的浪費。



▲圖 7.8　Copy Depth 使 Draw Opaque Objects 與 Draw Transparent Objects 被分斷



▲圖 7.9　因為被分斷，所以在 Draw Opaque Objects 結束時 Store

像這樣逐一確認各附件的 Store 是否有正當理由（後續的使用），就是圖塊記憶體最佳化的基本流程。

7.2.5　以 AccessFlags 削減的範例

最後，以實例確認光是 `AccessFlags` 的指定就能改變動作。針對進行全螢幕 Blit 的 Pass，比較將輸出附件宣告為 `Write` 與 `WriteAll` 的情形。

▼清單 7.3　AccessFlags 的 Write 與 WriteAll 比較

```
// 模式 ：（不宣告寫入全部像素）AWrite
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.Write);

// 模式 ：（宣告寫入全部像素）BWriteAll
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
```

兩者的繪製結果相同。但在 Render Graph Viewer 中確認時，模式 A 的載入動作為 Load，模式 B 則為 DontCare。由於是全螢幕 Blit，載入的□容會被全部像素覆寫而□棄。也就是□，模式 A 的載入是浪費的。

7.3　Render Pass / Subpass

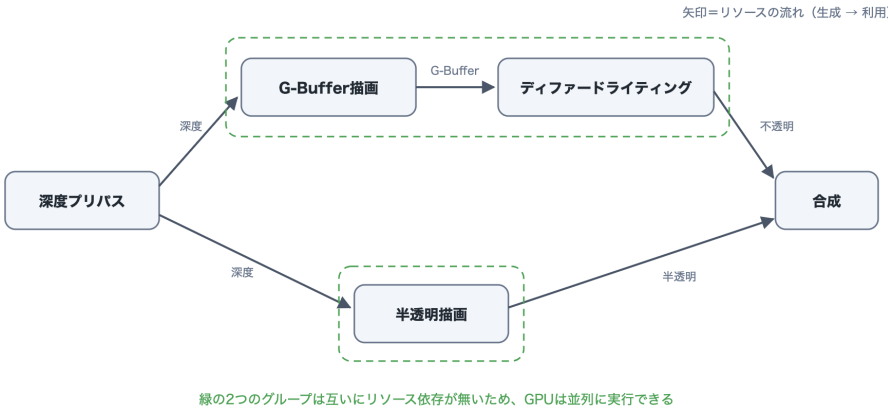
至此我們已了解 Pass 與 Pass 之間的儲存→載入往返會□生成本。本節□明用來消除這項成本的機制，即 Render Pass/Subpass，以及 Render Graph 的 Pass 合併。

7.3.1　資源相依性與 DAG

完成遊戲畫面的一個影格的流程稱為渲染管線，構成該管線的每一項處理稱為渲染 Pass。每個 Pass 都有「讀取□些資源、寫入□些資源」的資源相依性。例如包含延遲渲染（deferred rendering）的典型幾何繪製，具有下列相依關係。

1. 以深度預先 Pass（depth pre-pass）建立深度緩衝區
2. 將不透明物件繪製到 G-Buffer（使用深度緩衝區）
3. 對不透明緩衝區進行延遲光照（使用 G-Buffer 與深度緩衝區）
4. 將半透明物件繪製到半透明緩衝區（使用深度緩衝區）
5. 合成不透明緩衝區與半透明緩衝區並繪製到影格緩衝區

若以 Pass 為節點、資源相依性為邊來繪製，這條管線就成為有向無環圖（DAG: Directed Acyclic Graph）。第 4 個 Pass 並未使用第 2、3 個 Pass 的輸出，因此只要將相依性傳達給 GPU，就能平行執行。



▲圖 7.10　渲染 Pass 的 DAG 與平行化

Render Graph 這個名稱中的「Graph」指的就是這個 DAG。Render Graph 是蒐集各 Pass 的資源相依性來建構 DAG，並據此制定執行計畫的系統。

7.3.2　Vulkan 的 RenderPass/Subpass

作為將資源相依性傳達給 GPU 的機制，Vulkan 有 RenderPass/Subpass 這個概念。Render Graph 最終建構的就是這個 Vulkan 的 RenderPass，因此在此稍微談一下 API 層級的□容。Vulkan 的 RenderPass 由下列要素構成。

- 附件：宣告 RenderPass 中使用的渲染目標。每個附件需指定 `loadOp`（載入動作）與 `storeOp`（儲存動作）。前面「圖塊記憶體的載入/儲存」一節所見動作的真面目就是這個。
- 子通道：RenderPass □繪製的單位。各子通道宣告輸出附件（顏色/深度）與輸入附件。
- 子通道相依性：宣告子通道之間的執行順序與記憶體相依關係。

在此有兩項需要掌握的性質。

1. 載入/儲存只在 RenderPass 的邊界發生，子通道之間不會發生。也就是□，只要能把多個處理組成一個 RenderPass 的子通道，其間的中間結果就會留在圖塊記憶體。
2. 使用輸入附件，可以僅限於相同像素位置讀取前一個子通道的結果。這就是下一

節要□明的影格緩衝區擷取在 API 上的樣貌。

若以 RenderPass/Subpass 表示前述的幾何繪製，則如下所示。

- 幾何繪製（RenderPass）
  - 深度預先 Pass（Subpass）— 輸出：深度
  - G-Buffer 繪製（Subpass）— 輸入：深度 / 輸出：G-Buffer
  - 延遲光照（Subpass）— 輸入：G-Buffer / 輸出：不透明緩衝區
  - 半透明繪製（Subpass）— 輸入：深度 / 輸出：半透明緩衝區
  - 合成（Subpass）— 輸入：不透明、半透明緩衝區 / 輸出：影格緩衝區

採用這種結構，深度、G-Buffer、不透明緩衝區、半透明緩衝區全都是子通道之間的傳遞，跨越 RenderPass 邊界的只有最終輸出的影格緩衝區。原本可能發生 5 次儲存的結構，變成只有 1 次儲存。順帶一提，Metal 也有同等的機制。Metal 的 Render Pass 具有載入/儲存動作，並且可以從片段函式直接讀取圖塊記憶體上的顏色附件（以及圖塊著色器）。雖然沒有像 Vulkan 那樣明確的 Subpass 宣告，但 Render Graph 所做的最佳化在 Metal 上同樣有效。

7.3.3 Unity 的 Native Render Pass

那麼，Render Graph 如何處理這個 Vulkan 的 RenderPass/Subpass 呢？ 首先要掌握的是，以 `AddRasterRenderPass` 新增的 Pass 並不會直接成為子通道。以 `RecordRenderGraph` 新增的 Pass 終究只是 Render Graph 的邏輯 Pass。Render Graph 在記錄完影格□所有 Pass 之後，會在編譯階段進行下列處理。

1. 由資源相依性建構 DAG，並剔除不需要的 Pass（剔除，culling）
2. 將相鄰 Pass 中滿足條件者合併為一個 Native Render Pass（相當於 Vulkan 的 RenderPass），並使每個被合併的 Pass 成為子通道
3. 決定各附件的載入/儲存動作（參見前一節的「Render Graph 決定動作的規則」）
4. 將可行的資源無記憶體化（「無記憶體」一節）

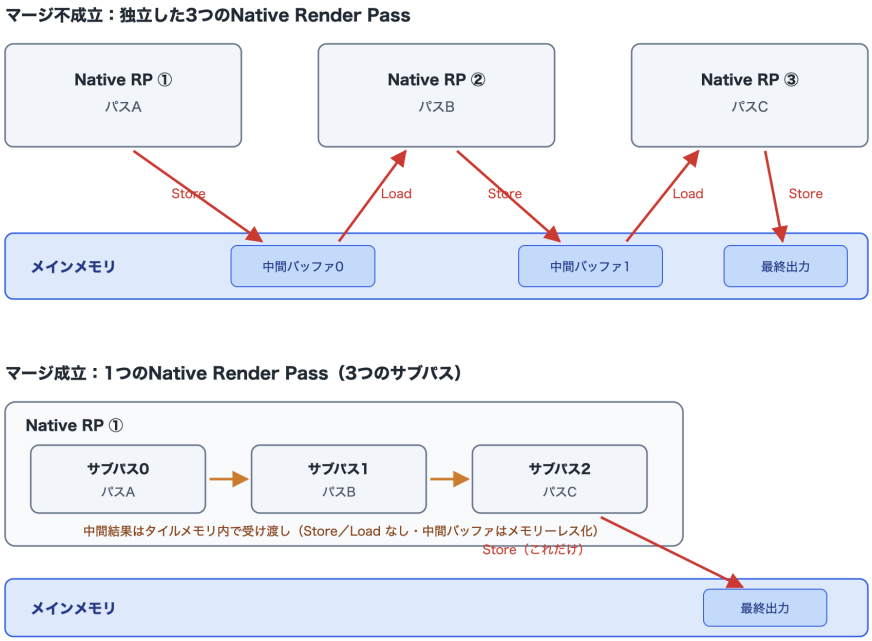
也就是□，開發者的工作不是手動組合子通道，而是將 Pass 設計成可合併的形式，並正確宣告資源的使用方式。合併是否成功，可透過 Frame Debugger 的（`RP n:m`）標記與 Render Graph Viewer 確認。

7.3.4 Pass 合併的條件

以下整理 Render Graph 合併 Pass 的條件。

1. 兩者都是 Raster Render Pass。 `AddComputePass` 與 `AddUnsafePass` 不會被合併。若 Raster Pass 的序列之間夾有 Compute Pass，Native Render Pass 就會在該處中斷。
2. Pass 被連續記錄。 合併是在圖上相鄰的 Pass 之間進行。
3. 輸出附件的解析度一致。 若改變中間緩衝區的解析度（降低取樣），在該邊界就無法合併。
4. MSAA 取樣數一致。
5. 後面的 Pass 沒有將前面 Pass 的輸出作為紋理使用。 以 `UseTexture` 使用會要求儲存→取樣，因此會妨礙合併。以 `SetInputAttachment`（輸入附件）使用則不會妨礙合併。輸入附件與紋理是不同的東西。
6. 附件的合計數量不超過上限（8 張）。 合併後的 Native Render Pass 所使用的附件總數有上限。
7. 沒有設為全域紋理等□離 Render Graph 追蹤的使用。

只要有一項條件未滿足，該 Pass 就會成為獨立的 Native Render Pass。未能合併的原因會顯示在 Render Graph Viewer 的 Pass Break Reasoning 中，因此遇到未合併的情況時，請先確認該處。



▲圖 7.11　合併前後的 Native Render Pass 結構比較

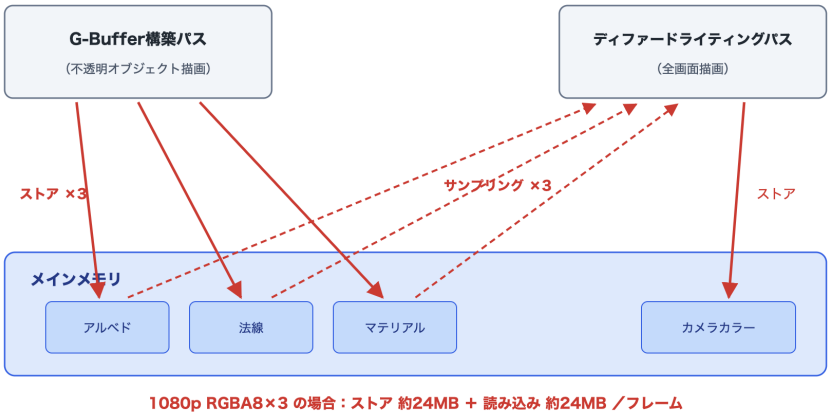
7.4　影格緩衝區擷取

我們已了解滿足條件即可將 Pass 合併為一個 Native Render Pass。本節⌧明在已合併的子通道之間，直接從圖塊記憶體讀取前一個子通道結果的技術，即影格緩衝區擷取（Frame Buffer Fetch，以下簡稱 FBF）。

7.4.1　何謂影格緩衝區擷取

以建立 G-Buffer 並進行光照的延遲渲染為例。若採用簡單的實作，處理如下。

1. 建立 G-Buffer（將反照率、法線、材質資訊寫入圖塊記憶體）
2. 將反照率、法線、材質資訊儲存到主記憶體
3. 將已儲存的 G-Buffer 作為紋理取樣，進行延遲光照



▲圖 7.12　簡單延遲渲染的資料流

這裡會□生一個疑問。光照 Pass 想讀取的是「目前正在處理的像素」的 G-Buffer □，而該□剛剛才由前一個 Pass 寫入同一圖塊的圖塊記憶體。能否不經過主記憶體往返，直接從圖塊記憶體讀取呢？ 使這成為可能的就是 FBF。使用 FBF 時，處理如下。

1. 建立 G-Buffer（寫入圖塊記憶體）
2. 直接從圖塊記憶體讀取 G-Buffer，進行延遲光照



▲圖 7.13　使用影格緩衝區擷取的延遲渲染資料流

3 張 G-Buffer 的儲存，以及為了取樣而進行的讀取都不再需要。若是 1080p 的 RGBA8 × 3 張，光是儲存就每影格削減約 24MB，在 60FPS 下削減約 1.4GB/s 的頻寬。在 API 層級，FBF 是以前面「Vulkan 的 RenderPass/Subpass」一節提到的 Vulkan 輸入附件（Metal 中為直接讀取顏色附件）來實現。因此 FBF 只能用於同一個 Render Pass 內的子通道之間。Pass 已被合併是 FBF 的前提條件。

7.4.2　理解限制

FBF 並非紋理取樣的替代品，具有下列限制。

1. 只能讀取與處理中片段相同的像素位置。 由於圖塊記憶體中只有「目前正在處理的圖塊」的內容，原理上無法進行偏移取樣（讀取周邊像素）。
2. 無法使用過濾與 mipmap。 只能載入單一的圖。
3. 輸入側與輸出側的解析度必須一致（這也是合併條件）。

我們來看限制 1 發揮作用的實例。考慮「在單色/復古色調合成的結果上，疊加扭曲 UV 的雜訊加工」這項處理。雜訊加工的著色器會如下偏移 UV 進行取樣。

▼清單 7.4　雜訊加工著色器

```
// 使用雜訊扭曲 UV
float2 distortedUV = input.texcoord + noise * 0.02;
// 以扭曲後的 UV 對紋理取樣。
```

```
// 由於載入的位置與片段的位置不同，因此無法使用影格緩衝區擷取。
half4 color = SAMPLE_TEXTURE2D_X(_BlitTexture, sampler_LinearClamp, distortedUV);
```

因為想讀取的位置與處理中片段的位置不同，所以此 Pass 無法使用 FBF。必須將合成結果以 `UseTexture` 宣告，合成 Pass 與雜訊 Pass 之間會□生儲存→取樣的往返。因此，在設計特效的階段先確認「此 Pass 是否只讀取相同的像素？」，就能及早判斷是否可使用 FBF。只需相同像素的處理（色調映射、色彩分級、合成、延遲光照）是 FBF 的候選，而讀取周邊的處理（模糊、扭曲、Bloom）則屬於紋理使用。

7.4.3 著色器 API 的□部

要從 Unity 的著色器使用 FBF，需利用 SRP Core 提供的巨集。

▼清單 7.5 影格緩衝區擷取的使用方式

```
// 宣告第 0 號輸入附件
FRAMEBUFFER_INPUT_X_HALF(0);

half4 frag(Varyings input) : SV_Target
{
    // 從第 0 號輸入附件載入處理中片段位置的□
    half4 color = LOAD_FRAMEBUFFER_X_INPUT(0, input.positionCS.xy);
    ...
}
```

巨集依精度與用途提供多種變體。

宣告巨集	載入巨集	用途
FRAMEBUFFE R_INPUT_HALF(n)	LOAD_FRAMEBUFFE R_INPUT(n, pos)	half 精度的顏色
FRAMEBUFFE R_INPUT_FLOAT(n)	LOAD_FRAMEBUFFE R_INPUT(n, pos)	float 精度
FRAMEBUFFE R_INPUT_X_HALF(n)	LOAD_FRAMEBUFFE R_X_INPUT(n, pos)	支援 XR（單通道執行個體化，single-pass instancing）的版本

帶有 `_X` 的變體與 `TEXTURE2D_X` 系列巨集一樣，支援 XR 的渲染目標陣列。在可能支援 XR 的專案中，統一使用帶有 `_X` 的版本較為安全。了解這些巨集在各平台上會展開成什麼，有助於理解其行為與 fallback。

平台	展開結果	實體
Vulkan	subpassInput + subpassLoad	Render Pass 的輸入附件
Metal	以 [[color(n)]] 屬性讀取顏色附件	直接讀取圖塊記憶體
OpenGL ES	GL_EXT_shader_framebuffer_fetch 等擴充（僅限支援的 GPU）	不支援擴充時使用下方的 fallback
不支援的環境	一般紋理（_UnityFBInput0 等）+ 載入指令	不會成為 FBF，而是一般的紋理讀取

需要注意的是最後一列。巨集在不支援的環境中也會 fallback 成可運作的形式，但這時沒有頻寬削減的效果。也就是□，即使能運作，也不代表 FBF 已經生效。是否生效，請以 Render Graph Viewer 的動作顯示（輸入側緩衝區的儲存是否為 DontCare）與實機的頻寬測量來確認。

7.4.4 SetInputAttachment 與 UseTexture 的差異

在 C# 端，要以 FBF 讀取的資源以 `SetInputAttachment` 宣告。

▼清單 7.6　宣告輸入附件

```
using (var builder = renderGraph.AddRasterRenderPass<PassData>("Lighting", out var
    passData))
{
    builder.SetRenderAttachment(cameraColor, 0, AccessFlags.Write);
    // 對應著色器的 FRAMEBUFFER_INPUT_X_HALF(0)
    builder.SetInputAttachment(gBufferAlbedo, 0);
    // 對應 FRAMEBUFFER_INPUT_X_HALF(1)
    builder.SetInputAttachment(gBufferNormal, 1);
    ...
}
```

`SetInputAttachment` 第 2 個引數的索引，對應著色器端 `FRAMEBUFFER_INPUT_X_HALF(n)` 的 `n`。在此以表格整理它與從「圖塊記憶體的載入/儲存」一節起反覆出現的 `UseTexture` 之間的差異。

項目	UseTexture	SetInputAttachment
可讀取的位置	任意 UV（可偏移、過濾、使用 mip）	僅限處理中的片段位置
與前一個 Pass 的合併	妨礙	不妨礙

項目	UseTexture	SetInputAttachment
前一個 Pass 輸出的儲存	強制為 Store	合併成立時可設為 DontCare
頻寬成本	儲存＋取樣讀取	零（在圖塊記憶體□完成）
著色器端的讀取方式	SAMPLE_TEXTURE2D 系列	LOAD_FRAMEBUFFER 系列

7.4.5 深度的擷取

不只是顏色，深度也位於圖塊記憶體上。若能擷取深度，軟粒子、貼花（decal）、霧效等「參照場景深度的處理」，也有可能消除 `_CameraDepthTexture` 的儲存→取樣往返。SRP Core 提供了處理深度用輸入附件的一組巨集，但將深度附件作為輸入使用，是容易受平台與 URP 版本限制的領域。在得到驗證結果之前，本章建議採用確實可運作的下列替代方案。替代方案是將深度寫入顏色附件再進行擷取。在建構 G-Buffer 的子通道中，將線性化的深度□寫入顏色附件的 1 個通道（R16_Float 等），後續子通道再以 FBF 讀取。雖然附件會多一張，但只要被無記憶體化，實際記憶體的成本為零，並且能確實消除深度紋理的儲存往返。

7.4.6 平台差異與陷□

最後，整理使用 FBF 時應確認的平台差異。

環境	FBF 的使用	備註
Vulkan (Adreno)	○ 輸入附件	廣泛支援。驅動程式品質因裝置而異
Vulkan (Mali)	○ 輸入附件	廣泛支援
Vulkan (IMG / Pixel 10)	○ 輸入附件	TBDR 系。需注意驅動程式特有的行為差異（最後一節的「GPU 廠商差異」）
Metal (Apple)	○ 讀取顏色附件	穩定運作
OpenGL ES	△ 依賴擴充	僅限支援 GL_EXT_shader_framebuffer_fetch 的 GPU。不支援者使用 fallback
編輯器 (D3D 等)	× fallback 運作	以一般紋理運作。可確認動作，但無法測量效果

特別需要注意的有下列三點。

1. 在編輯器中看不到效果。 因為 fallback 也能正確繪製，所以請以「必須在實機上測量才能看出差異」為前提來安排工作流程。

2. 併用 MSAA 時容易觸及合併條件（取樣數一致）。 □用 MSAA 的相機顏色，與未使用 MSAA 建立的中間緩衝區無法合併。中間緩衝區的 TextureDesc 請如 `GetTextureDesc(cameraColor)` 這樣從原本的 Desc 複製，並沿用取樣數。
3. 一個子通道可讀取的輸入附件數量有上限。 附件合計 8 張的限制（前一節的「Pass 合併的條件」）也包含輸入側。設計 G-Buffer 時，請以通道打包（channel packing）節省張數。

7.5 無記憶體（memoryless）

至此，中間緩衝區的載入與儲存都已能消除。在此想思考一件事：既不載入也不儲存的緩衝區，主記憶體上是否還需要它的實體？

7.5.1 何謂無記憶體

其實並不需要。僅在圖塊記憶體上使用並在圖塊記憶體上銷毀的資源，根本不需要配置主記憶體。這種狀態稱為無記憶體。Vulkan/Metal 有一種模式，可將資源建立為「只存在於圖塊記憶體（晶片□記憶體），不具備主記憶體的後備儲存（backing store）」（Vulkan 中為 `LAZILY_ALLOCATED` 記憶體，Metal 中為 `MTLStorageMode.memoryless`）。被指定為無記憶體的資源具有下列性質。

- 不消耗主記憶體（應用程式的記憶體佔用量減少）
- 載入動作無法指定 Load，儲存動作無法指定 Store（因為沒有可供讀寫的實體）

也就是□，無記憶體是前面各節消除載入/儲存之後的結果。載入/儲存消失就能無記憶體化，若無法無記憶體化，就表示某處仍殘留載入或儲存。

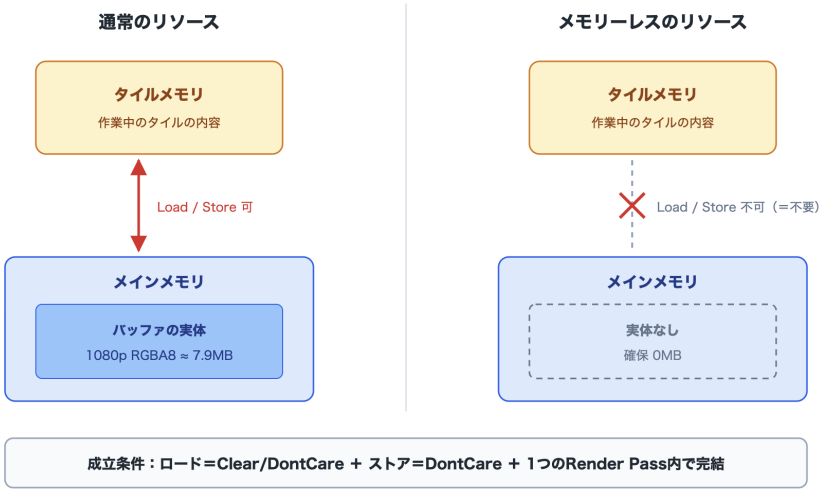
7.5.2 成立條件

以前面使用的用語來□，某個資源能□無記憶體化的條件如下。

1. 載入動作為 Clear 或 DontCare（不從主記憶體讀取先前的□容。即「以 Access Flags 宣告資源使用方式」一項所見的 WriteAll/Discard 宣告，或首次使用時的 Clear）
2. 儲存動作為 DontCare（後續 Pass 不會將其作為紋理使用，也不會延續到下一影格。參見「Render Graph 決定動作的規則」一項）
3. 使用在一個 Native Render Pass □完結（可使用子通道之間的傳遞，即輸入附件/FBF。「Render Pass / Subpass」一節的合併，以及「影格緩衝區擷取」一節

的 FBF）

如各位所見，這三項條件正是前面各節所□明的□容本身。



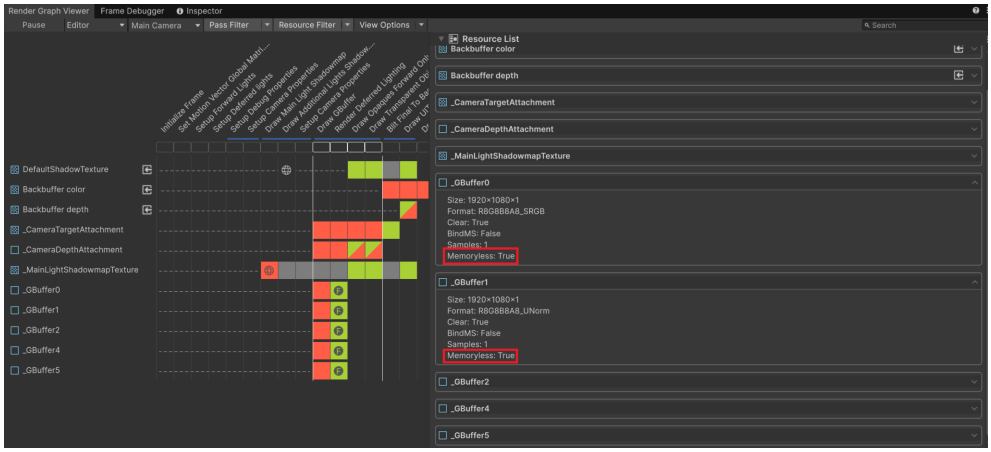
▲圖 7.14　無記憶體化前後的記憶體配置比較

自動無記憶體化與明確指定

在 Unity 的 Render Graph 中，無記憶體透過兩種途徑實現。

自動無記憶體化

Render Graph 管理的紋理（以 `renderGraph.CreateTexture` 建立）若滿足「成立條件」一項的三項條件，Render Graph 會自動將其無記憶體化。開發者不需要指定任何事項。是否已無記憶體化，可在 Render Graph Viewer 的 Resource List 中確認。



▲圖 7.15　Render Graph Viewer 中的 Memoryless 顯示

明確指定

也可以透過 `TextureDesc` 的 `memoryless` 欄位明確指定。

▼清單 7.7　明確指定無記憶體

```
var desc = renderGraph.GetTextureDesc(universalResourceData.cameraColor);
desc.memoryless = RenderTextureMemoryless.Color; // 指定無記憶體
desc.name = "Intermediate Texture";
var handle = renderGraph.CreateTexture(desc);
```

明確指定的優點是能把設計意圖留在程式碼中。若不小心在後續 Pass 中對已指定無記憶體的資源執行 `UseTexture`，該時點就會暴露出結構上的矛盾。若交由自動處理，當重構時某處混入了 `UseTexture`，就無法察覺無記憶體已失效（＝記憶體使用量與頻寬增加了）。若設計意圖是讓此資源在圖塊記憶體☒完結，建議以明確指定的方式留在程式碼中。

7.5.3　削減量的量化

無記憶體化的削減量可以計算得出。以下為 1080p 下的代表例。

資源配置	計算	削減量
RGBA8 中間緩衝區 1 張	1920×1080×4B	約 7.9MB
RGBA16F 中間緩衝區 1 張	1920×1080×8B	約 15.9MB

資源配置	計算	削減量
G-Buffer 3 張（RGBA8）＋深度	7.9MB×4	約 31.7MB
4x MSAA RGBA8 顏色	1920×1080×4B×4	約 31.7MB
4x MSAA 深度（D32）	1920×1080×4B×4	約 31.7MB

如表所示，削減量大的是 MSAA 與 G-Buffer。尤其 4x MSAA 的顏色與深度合計超過 60MB，因此這些是否消失，對應用程式整體的記憶體預算影響相當大。□外請注意，無記憶體所削減的是「記憶體使用量」，頻寬的削減則來自前面各節所進行的載入/儲存削減。兩者是不同的指標（無記憶體是載入/儲存削減的結果，因此在實務上會一併達成）。

7.5.4 MSAA 渲染目標的無記憶體化

無記憶體效果最大的是 MSAA。我們從機制開始看。在 4x MSAA 中，圖塊記憶體上每個像素保有 4 個取樣的顏色與深度。但最終顯示在畫面上的，只有將 4 個取樣平均（解析，resolve）後的 1 個取樣。在此儲存動作的選擇就會□生影響。

- 選擇 Store 時，4 個取樣的資料會原樣寫出到主記憶體（頻寬 4 倍、記憶體 4 倍）。
- 選擇 Resolve 時，解析會在圖塊記憶體上進行，只寫出解析後的 1 個取樣。

在 TBR 的 GPU 上，解析可在圖塊記憶體上以幾乎零成本完成。而選擇 Resolve 時，4 個取樣的 MSAA 緩衝區本身不會被儲存，因此符合無記憶體的條件。也就是□，正確配置的 MSAA 會是下列結構：

- MSAA 顏色緩衝區（4 個取樣） → 無記憶體（不配置主記憶體、不儲存）
- 解析目標緩衝區（1 個取樣） → 一般的儲存
- MSAA 深度緩衝區（4 個取樣） → 無記憶體＋不儲存（後續不使用深度時）

這就是 TBR 的 GPU 上 MSAA 被認為（相較於 IMR）成本較低的原因。在 URP 中，只要□用 Universal Render Pipeline 資□的 MSAA 設定，Render Graph 就會自動組成這種結構。應確認的是它是否被破壞。請在 Render Graph Viewer 中確認 MSAA 顏色緩衝區的儲存動作是否為 Resolve，以及 Memoryless 是否為 yes。典型的破壞情形，是後續 Pass 要求將 MSAA 緩衝區本身作為紋理（自訂 Pass 中的錯誤宣告、Opaque Texture 的設定錯誤等）。

7.5.5 GPU 廠商差異

本章所處理的技術都建立在標準的 Vulkan/Metal 功能之上，在主要廠商上皆可運作。在此基礎上，了解各廠商的特性差異，有助於解讀測量結果與處理問題。

廠商	架構特性	注意事項
Qualcomm Adreno	以 GMEM 為基礎的分塊。依繪製□容可能切換為直接模式（類 IMR 運作）	切換為直接模式後，以圖塊記憶體為前提的最佳化效果會改變。請以效能分析工具確認處於□種模式
ARM Mali	以 16×16 圖塊為基本的 TBR	外部頻寬計數器完備，是本章頻寬測量的基準廠商
Imagination (IMG)	具備硬體隱藏面移除（HSR）的 TBDR。Pixel 10 的 IMG DXT 屬於此類	由於 HSR，不透明過度繪製（overdraw）本來就少，最佳化奏效的地方與其他廠商不同。由於驅動程式較新，建議驗證行為差異
Apple	PowerVR 系譜的 TBDR。可在 Metal 中明確操作圖塊記憶體	FBF/無記憶體穩定運作。先在 iOS 上確認效果，再進行 Android 的驗證也是有效的做法

7.5.6 結語

最後再次整理本章處理的四個主題。使用 Render Graph 時，開發者要做的有兩件事：正確宣告資源的使用方式，以及以 Frame Debugger 與 Render Graph Viewer 確認結果。這些工作雖然樸實，但累積起來會體現為裝置發熱與電池消耗的差異。希望本章能對各位專案的最佳化有所幫助。

7.5.7 參考文獻

- ARM, "Arm GPU Best Practices Developer Guide", `https://developer.arm.com/documentation/101897/latest/`
- Qualcomm, "Adreno GPU Best Practices", `https://docs.qualcomm.com/doc/80-78185-2/topic/mobile_best_practices.html`
- Apple, "Metal Best Practices Guide", `https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/`

- Unity, "Render graph system" (URP Documentation), https://docs.unity3d.com/Manual/urp/render-graph.html
- 《學園偶像大師》在行動裝置上也完整呈現「光照與反射」！【CEDEC2024】, https://gamemakers.jp/article/2025_01_29_90729/
- 在行動裝置上高速執行 Deferred Rendering 的方法, CEDiL, https://cedil.cesa.or.jp/cedil_sessions/view/2457
- 從 Unity 6 開始熟練運用 RenderGraph — 基本功能篇, https://blog.sge-coretech.com/entry/2024/06/04/171757
- 從 Unity 6 開始熟練運用 RenderGraph！ 應用實作篇 1「建構 RenderPass」, https://blog.sge-coretech.com/entry/2024/09/11/105032
- 著色器最佳化入門 第 2 回「何謂圖塊式渲染的 GPU？」, https://blog.sge-coretech.com/entry/2025/01/27/145242

第8章

活用 Renderer Shader User Value 的圖形最佳化

瀨戶 徳 Noboru Seto

株式會社 CyberAgent SGE 核心技術本部

ruccho

8.1 前言

Unity 6.3 新增了名為 Renderer Shader User Value (RSUV) 的功能。RSUV 是可針對每個 Renderer 元件設定的 32 位元整數，著色器也能參照。在 Unity 6000.3.0f1 之後，MeshRenderer 與 SkinnedMeshRenderer 可使用 RSUV；在 Unity 6000.5.0f1、6000.4.4f1、6000.3.13f1 之後，SpriteRenderer[*1]、SpriteShapeRenderer、Tilemap Renderer 也支援。用法很簡單，只要以 Renderer.SetShaderUserValue() 對渲染器設定 RSUV，並在著色器中使用 unity_RendererUserValue 參照即可。

```
uint value = /* ... */;
meshRenderer.SetShaderUserValue(value);
value = meshRenderer.GetShaderUserValue();
```

```
uint value = unity_RendererUserValue;
```

與此範例相同的事情，也可以使用材質參數達成。但使用 RSUV 時，**特別是在大量繪製的情境下，與材質相比有可能改善記憶體配置量與影格率**。本章說明 RSUV 登場的背景與有效的使用方式，並介紹效能測量結果。

[*1] 取得與設定 SpriteRenderer 的 RSUV 需要 2D Module。

8.1.1 Unity 的大量繪製功能與 RSUV 登場的背景

首先回顧 RSUV 之所以必要的背景。在 Unity 中同時顯示大量物件時的效能，向來是令人頭痛的問題。大量繪製的效能問題主要有三點：「切換材質時的 CPU 負載」「繪製呼叫（DrawCall）增加」「複製材質造成的記憶體配置」。

切換材質時的 CPU 負載

繪製多個具有不同材質的物件時，每次繪製影格都必須針對每個物件蒐集材質資料並傳送到 GPU，此時的 CPU 處理成本很高。SRP Batcher 解決了這個問題，它將材質資料常駐於 GPU 記憶體上，只要著色器相同，就能以相對低的成本繪製具有不同材質的多個物件。

DrawCall 增加

GPU 執行個體化（GPU instancing）是以一次 DrawCall 繪製多個具有相同著色器、相同網格、相同材質的物件的手法。它的適用條件比 SRP Batcher 嚴格，但在繪製大量物件時能發揮超越 SRP Batcher 的效果。隨著 Unity 6 推出 GPU Resident Drawer，即使是使用一般 MeshRenderer 的繪製，只要滿足條件就會自動進行 GPU 執行個體化。

複製材質造成的記憶體配置

最後的問題是，在使用 SRP Batcher 或 GPU 執行個體化的同時，還想讓材質有所變化的情境。Unity 很早就有 MaterialPropertyBlock 這項功能，可在不複製材質執行個體的情況下僅覆寫部分參數，但它與 SRP Batcher 和 GPU Resident Drawer 不相容。RSUV 解決了這個問題，能在使用 SRP Batcher 與 GPU Resident Drawer 的同時，**不經由材質**，將每個物件不同的 32 位元參數傳給著色器。

RSUV 是同時解決上述三個問題所需的最後一塊拼圖。它能以 SRP Batcher 抑制切換材質時的成本，以 GPU Resident Drawer 進行有效率的 GPU 執行個體化，同時為每個執行個體進行有變化的繪製。即使在 GPU Resident Drawer 的自動執行個體化無法生效的情況下，也能避免複製材質執行個體，進而抑制記憶體消耗。

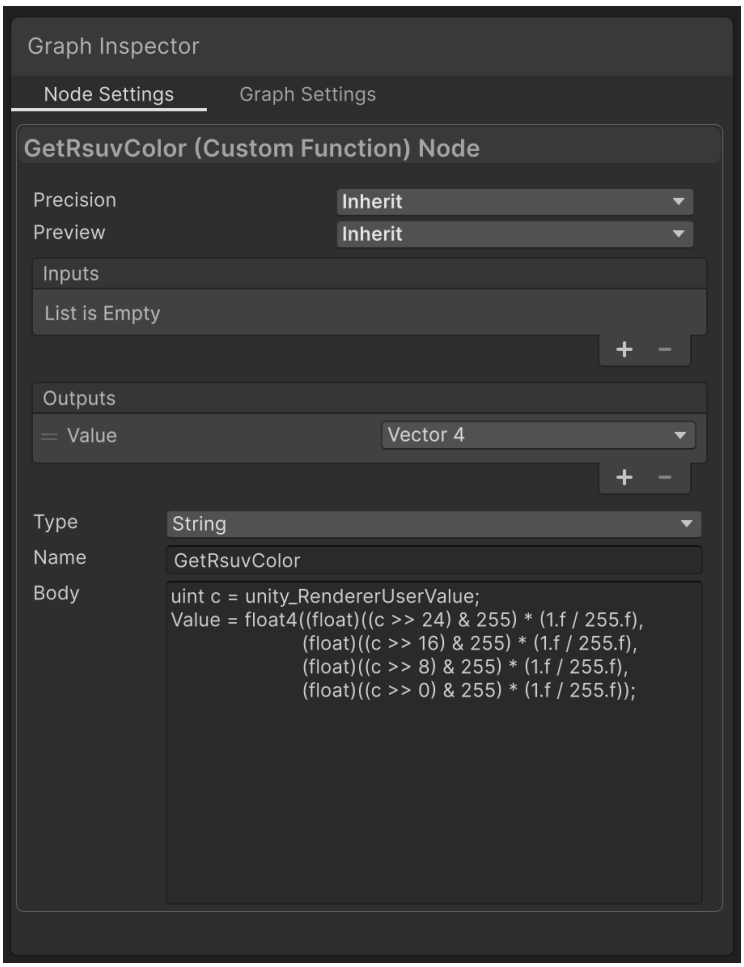
8.2 簡單的使用方式

來實際看看 RSUV 的使用方式。RSUV 在 C# / HLSL 中以 32 位元整數（uint）表示，因此例如要傳遞 32 位元的顏色（RGBA 的 4 個通道各 8 位元），需如下以位元運算進行

打包。

```
Color32 color = /* ... */;
var colorEncoded = (uint)(color.r << 24 | color.g << 16 | color.b << 8 | color.a);
meshRenderer.SetShaderUserValue(colorEncoded);
```

接著是著色器端的實作，這裡假設使用 ShaderGraph。由於 ShaderGraph 沒有取得 RSUV 的專用節點，因此使用 Custom Function 節點，如下設定（圖 8.1）並輸入程式碼。



▲圖 8.1　節點的設定

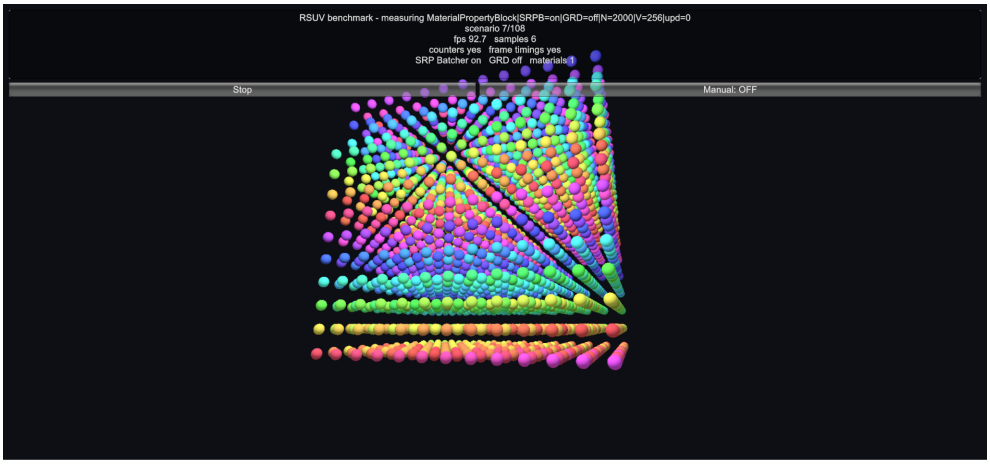
```
uint rsuv = unity_RendererUserValue;
Value = float4((float)((rsuv >> 24) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 16) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 8) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 0) & 255) * (1.f / 255.f));
```

連接節點，確認 RSUV 已被接收。

8.3 效能測量

為了確認 RSUV 在大量繪製的情境下有效，我們以下列條件測量了記憶體配置量與影格率。

- Unity 6000.3.15f1
- URP 17.3
- iPhone 12
- 場景：10000 個 Sphere
- □用 SRP Batcher
- 比較每影格參數有無變化
- 比較□用與停用 GPU Resident Drawer
- 以三種方法為每個 Sphere 設定不同的著色器參數
  - 每個 Renderer 的材質：建立 10000 個 Material 並指派
  - RSUV：以 1 個 Material 為每個 Renderer 設定 RSUV
  - MaterialPropertyBlock：1 個 Material 與 10000 個 MaterialPropertyBlock（不在 SRP Batcher / GPU Resident Drawer 的對象□）


RSUV benchmark - measuring MaterialPropertyBlock|SRPB=on|GRD=off|N=2000|V=256|upd=0
scenario 7/108
fps 92.7 samples 6
counters yes frame timings yes
SRP Batcher on GRD off materials 1
Stop
Manual: OFF

▲圖 8.2　基準測試畫面

首先是各參數設定手法的記憶體配置量比較（表 8.1）。

表 8.1: 記憶體增加量

條件	記憶體增加量
每個 Renderer 的材質	15.79 MB
MaterialPropertyBlock	3.32 MB
RSUV	1.66 KB

由於材質不會被複製，記憶體配置量理所當然地大幅減少。接著看影格率。首先是每影格參數不變的情況（表 8.2）。

表 8.2: 每影格參數不變時的影格率 (FPS)

條件	無 GPU Resident Drawer	有 GPU Resident Drawer
每個 Renderer 的材質	33.8	38.8
RSUV	49.9	59.8
MaterialPropertyBlock	28.9	28.9

由此可知，RSUV 的效能高於每個 Renderer 的材質，且併用 GPU Resident Drawer 時效果更高。僅使用 RSUV 就能削減材質切換的成本，再加上可使用 GPU Resident Drawer 進行 GPU 執行個體化，便能減輕渲染執行緒上發出繪製命令的負載。接著看每影格參數會變化的情況（表 8.3）。

表 8.3: 每影格參數會變化時的影格率 (FPS)

條件	無 GPU Resident Drawer	有 GPU Resident Drawer
每個 Renderer 的材質	11.1	8.1
RSUV	48.0	60.0
MaterialPropertyBlock	26.1	25.2

與沒有更新參數時相比，每個 Renderer 的材質效能大幅下降，而 RSUV 幾乎沒有劣化。變更 1 個材質參數就需要將整個材質重新上傳到 GPU，而 RSUV 是作為每影格、每個 Renderer 一律傳送到 GPU 的緩衝區的一部分被傳送[*2]，因此更新 RSUV 所額外產生的成本得以維持在較低水準。有趣的是，大量更新材質參數與 GPU Resident Drawer 的相容性不佳，效能反而比停用 GPU Resident Drawer 時更低。如上所述，在「參數更新」與「繪製命令發出」兩方面，RSUV 都明顯具有優勢。尤其在 RSUV 使 GPU 執行

[*2] 僅限未進行執行個體化的情況。進行執行個體化時，會與其他執行個體參數一起透過 GraphicsBuffer 傳遞，且只有在整體參數被變更、GraphicsBuffer 變為 dirty 的時機才會上傳。

個體化成為可能的情況下（例如存在大量著色器或網格相同的物件時），RSUV 的效果會大幅提高。

8.4 傳遞超過 32 位元的資料

RSUV 雖然有助於提升效能，但只能傳遞 32 位元以□的□，這是它的一大弱點。以下介紹克服此限制的技巧。要以 RSUV 傳遞無法容納於 32 位元的資料，有效的做法是將目標資料儲存在□一個 GraphicsBuffer，並將該 GraphicsBuffer 的索引傳給 RSUV。由於這個做法在上傳 GraphicsBuffer 時會□生額外成本，對於 GraphicsBuffer 的更新頻率與大小，可能需要一定程度的留意。

```
[StructLayout(LayoutKind.Sequential)]
struct VariantGpuData
{
    public Vector4 baseColor;
    public Vector4 surface;
}
private static readonly int VariantBufferId = Shader.PropertyToID("_RsuvVariantBuffer")
    ;

// 上傳並指派 GraphicsBuffer
var buffer =  new GraphicsBuffer(GraphicsBuffer.Target.Structured, count, UnsafeUtility
    .SizeOf<VariantGpuData>());
var cpuBuffer = new VariantGpuData[count];
FillCpuBufferValues(cpuBuffer);
buffer.SetData(cpuBuffer, 0, 0, count);
Shader.SetGlobalBuffer(VariantBufferId, buffer);

// 設定 RSUV
renderer.SetShaderUserValue(index);

// ...

// 不再需要時釋放
buffer.Dispose();
```

```
struct RsuvVariantRecord
{
    float4 baseColor;
    float4 surface;
};
```

```
StructuredBuffer<RsuvVariantRecord> _RsuvVariantBuffer;

// 存取
RsuvVariantRecord record = _RsuvVariantBuffer[unity_RendererUserValue];
```

8.4.1 總結

本章介紹了 RSUV 登場的背景、效能測量，以及基本與進階的使用方式。RSUV 與材質相比雖有最多 32 位元的限制，但視情況能大幅改善效能。請辨別 RSUV 能發揮效果的情境並有效活用。

第9章

應使用 Render Graph 的 UnsafePass 的情境

チャン ユービン ZHANG YUBING
株式會社 CyberAgent SGE 核心技術本部

姓名：チャン ユービン羅馬拼音：ZHANG YUBING所屬：株式會社 CyberAgent SGE 核心技術本部

9.1 前言

在 Unity 6 中，以 Render Graph 建構渲染管線時，會依處理⌧容分別使用 RasterRenderPass、ComputePass 與 UnsafePass。本章介紹應使用 UnsafePass 的情境。

Render Graph 具有相依性分析與 SubPass 合併等最佳化。因此基本原則是，一般繪製優先使用 RasterRenderPass，使用 Compute Shader 的處理優先使用 ComputePass。

應選擇 UnsafePass 的情境主要有以下 4 種。

- 有 RasterRenderPass 或 ComputePass 無法實現的處理時
- 需要 CommandBuffer 本身時
- 想從 Compatibility Mode 快速移轉到 Render Graph 時
- 一個繪製處理由許多無法合併的 Pass 組成時

前 3 種是不使用 UnsafePass 就無法實現，或實作成本很高的情況。最後 1 種雖非必要，但將 Pass 整合為一個 UnsafePass，可望降低 CPU 負載並改善 Render Graph Viewer 的可讀性。

接下來依序□明這些情境。

9.2 應選擇 UnsafePass 的情境

UnsafePass 並不是指 C# 的 unsafe 功能。它是一種抑制 Render Graph 部分自動管理與驗證，並可使用接近 CommandBuffer 的 API 的 Pass。

9.2.1 有 RasterRenderPass 或 ComputePass 無法實現的處理時

RasterRenderPass 與 ComputePass 所使用的 RasterCommandBuffer／ComputeCommandBuffer，是只公開 CommandBuffer 中各 Pass 職責所需 API 的包裝，因此無法使用以下這類 API。

- SetRenderTarget：一邊指定 Load/Store Action，一邊細部指定渲染目標、MRT、Mip 與陣列切片
- CopyTexture：指定 Mip、陣列元素與矩形區域來複製紋理
- GenerateMips：□生渲染目標的 Mipmap
- SetRandomWriteTarget/ClearRandomWriteTargets：設定或解除像素著色器的隨機寫入目標
- RequestAsyncReadback：記錄 GPU 資源的非同步讀回請求

這些 API 可在 UnsafePass 的 UnsafeCommandBuffer 中使用。此外，若以 CommandBufferHelpers.GetNativeCommandBuffer 取得原生 CommandBuffer，還能使用更進階的 API。

9.2.2 需要 CommandBuffer 本身時

在引擎的回呼、IMGUI、SDK 的繪製處理等由外部程式碼接收 CommandBuffer 並記錄任意命令的處理中，所記錄的□容要到執行階段才會知道。在記錄時即宣告處理□容的 RasterRenderPass 與 ComputePass，無法納入這類黑箱處理。透過 UnsafePass 取得原生 CommandBuffer 並傳給外部程式碼，即可將其納入 Render Graph。

9.2.3 想從 Compatibility Mode 快速移轉到 Render Graph 時

在 Compatibility Mode 中寫於 Execute 方法□的 CommandBuffer 處理，可將命令序列幾乎原封不動地移到 UnsafePass 的 SetRenderFunc □。因此，想以較少的工時進行移轉至 Render Graph 與動作確認時很有效。

不過，這終究只是為了快速移轉的暫時手段。若能確保時間，建議依處理□容分類並移植到 RasterRenderPass 或 ComputePass，將 UnsafePass 限定於無法以宣告式 API 表達的部分。

9.2.4 由許多無法合併的 Pass 組成時

將連續且無法合併的 Pass 整合為一個 UnsafePass, 具有降低 CPU 負載並改善 Render Graph Viewer 可讀性的效果。

典型的情況

- Ping-pong 方式的 Blur
- Bloom 的 Mip Pyramid □生
- 多次降採樣與升採樣
- 以前一個處理的輸出作為下一個處理取樣對象的濾鏡處理

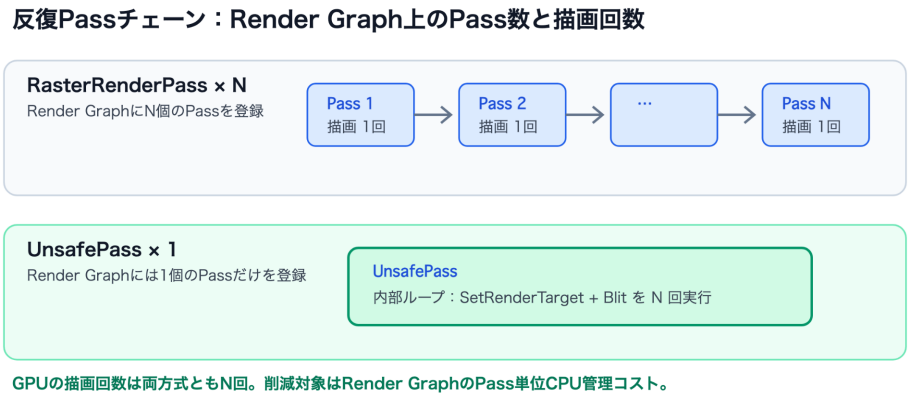
在這些處理中，每個 Pass 都會切換繪製目標，或對前一個 Pass 的輸出進行周邊取樣，因此無法成立 SubPass 合併。

整合為 UnsafePass 可降低 CPU 負載

每登錄一個 RasterRenderPass，就會□生以下處理。

- Pass 的登錄與 PassData 的準備
- 透過 UseTexture 或 SetRenderAttachment 登錄資源相依性
- Pass 的相依性分析與合併候選的確認
- Profiling 範圍的開始與結束
- 渲染目標的設定與執行回呼的呼叫

RasterRenderPass 越多，CPU 端的成本也越增加。即使將無法合併的 Pass 鏈登錄為 RasterRenderPass，也得不到合併的好處，登錄與管理的成本卻不斷累積。□一方面，若將它們整合為一個 UnsafePass，在 Render Graph 上就只登錄為一個 Pass。繪製命令與渲染目標切換的成本不變，但可削減 Render Graph 以 Pass 為單位進行的登錄／管理成本（圖 9.1）。



▲圖 9.1　無法合併的 Pass 鏈在 Render Graph 上的 Pass 數與繪製次數

CPU 負載降低效果的測量範例

在本專案於 Editor 上實施的測量中，針對 256×256 的 Render Texture 進行 4-tap 的 Ping-pong Blur，比較了每個 Blur 處理各登錄一個 RasterRenderPass 的方式，與在一個 UnsafePass 內執行一連串 Blur 處理的方式。測量條件為 Apple M4 Max（macOS 26.5）、Unity 6000.3.19f1、URP 17.3。每種組合擷取 500 影格，並使用排除切換後 60 影格的中位數。測量內是與 Render Graph 的記錄、雜湊計算、編譯與執行相關的 CPU marker 的合計。

表 9.1: Ping-pong Blur 中 Render Graph 相關 CPU 時間的比較

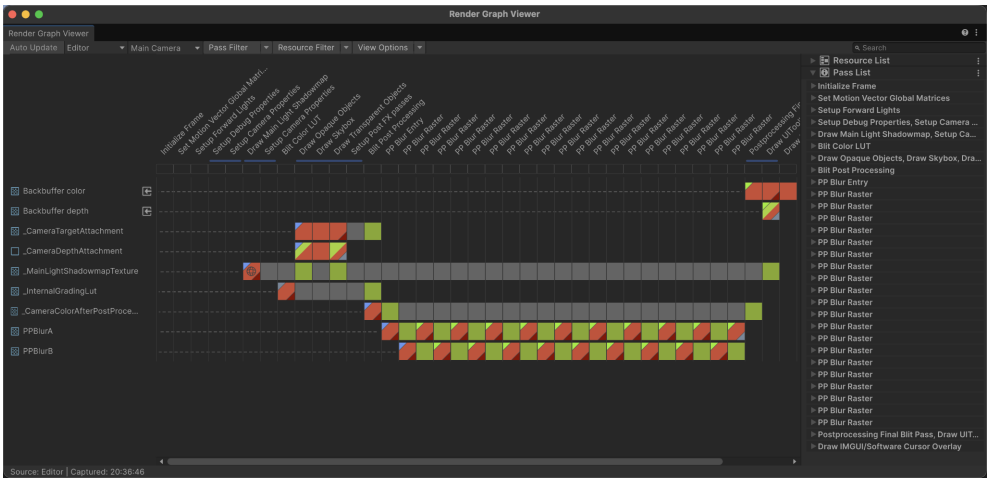
Blur 次數	RasterRenderPass × N（μ s）	UnsafePass × 1（μ s）	差內（μ s）
5	286.4	280.0	6.4
10	334.8	313.6	21.2
20	427.8	389.1	38.7
50	675.9	592.9	83.0
100	1080.7	898.5	182.2

測量內是用來確認 Editor 環境傾向的數內。隨著 Blur 次數增加，RasterRenderPass 方式與 UnsafePass 方式的差距也擴大。以線性近似來看，兩種方式斜率的差，也就是 RasterRenderPass 的額外固定成本，每個 Pass 約為 1.8μ s。此內是 Editor 上的測量內，並非表示實機的內對內，但可確認 CPU 負載的差距會隨 Blur 次數增加。這個效果來

自於削減了 Render Graph 的記錄、執行、Profiling 範圍與渲染目標設定等以 Pass 為單位的登錄／管理成本。由於繪製命令沒有差異，改為 UnsafePass 並不會減輕繪製負載。

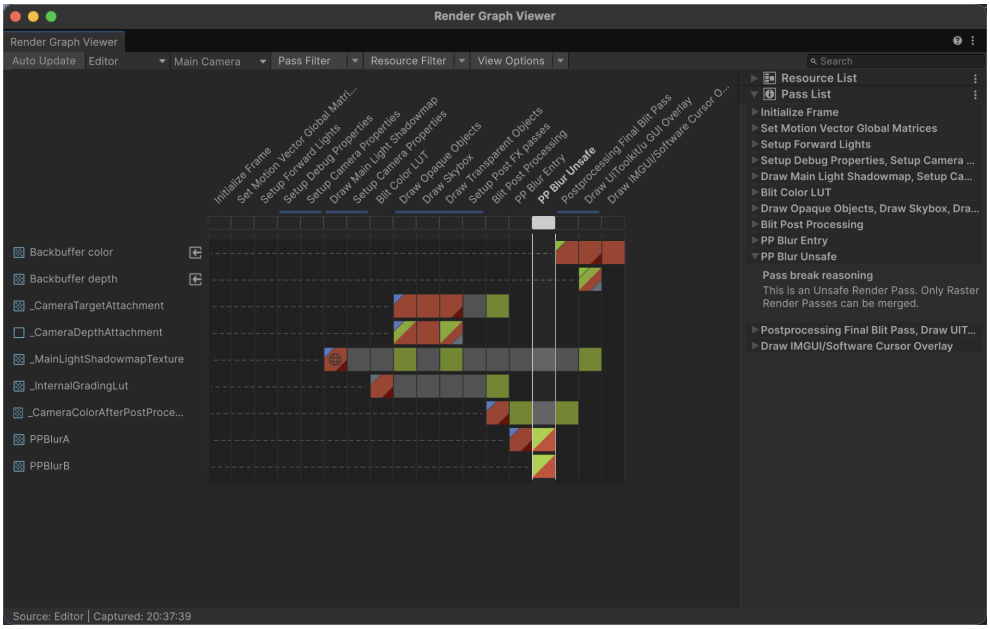
也能改善 Render Graph Viewer 的可讀性

登錄大量 RasterRenderPass 時，Render Graph Viewer 會顯示與登錄數相同的 Pass。圖形會往橫向拉長，因此較難看出各功能的分界，也不易掌握資源的相依性（圖 9.2）。



▲圖 9.2　每個 Blur 處理各登錄一個 RasterRenderPass 時的 Render Graph Viewer 顯示

若將功能□部的 Pass 鏈整合為一個 UnsafePass，在 Viewer 中就能將其視為代表整個功能的一個 Pass（圖 9.3）。如此各功能的分界會更清楚，也更容易掌握資源的相依性。



▲圖 9.3　將一連串 Blur 處理整合為一個 UnsafePass 時的 Render Graph Viewer 顯示

除了降低 CPU 負載外，還能減少效能分析時的雜訊，這也是一大優點。

不過，不建議僅為了改善 Render Graph Viewer 的可讀性，就勉強將不同功能的處理，或可成立 SubPass 合併的處理整合為 UnsafePass。

9.3　使用 UnsafePass 時的注意事項

9.3.1　渲染目標的設定

在 RasterRenderPass 中，是在記錄時以 SetRenderAttachment 宣告渲染目標；而在 UnsafePass 中，則是在執行時呼叫 SetRenderTarget 來設定。

9.3.2　宣告相依性

使用 UnsafePass 並不表示完全□離 Render Graph。所使用的紋理，必須一律以 UseTexture 宣告讀寫的相依性。若宣告不足，資源的生命週期管理與同步就無法正確進行。

在 RasterRenderPass 中，渲染目標的使用是以 SetRenderAttachment 宣告；但在 UnsafePass 中，僅呼叫 SetRenderTarget 並不會登錄相依性。渲染目標也必須一律以 UseTexture 宣告讀寫的相依性。

9.3.3 Load/Store Action 的設定

RenderBufferLoadAction 指定□用渲染目標時對既有□容的處理，RenderBufferStore Action 則指定繪製完成後對結果的處理。在 Tile-Based GPU 上，不必要的 Load 或 Store 會增加圖塊記憶體與主記憶體之間的傳輸，因此適當的設定很重要。

在 RasterRenderPass 中，Render Graph 會依所宣告的 AccessFlags 自動決定 Load/Store Action。□一方面，在 UnsafePass 中若像 SetRenderTarget(target) 這樣省略 Action，就會套用預設行為 Load／Store，可能因不必要的記憶體傳輸而使效能下降。

因此，在 UnsafePass 中呼叫 SetRenderTarget 時，務必明確指定 Load/Store Action。

- Load 側：若要覆寫整個畫面就用 DontCare，只有在讀取既有□容時才用 Load
- Store 側：若結果會在後續處理中讀取就用 Store，只有在任何地方都不使用時才用 DontCare

9.4 程式碼範例：Ping-pong Blur 的無法合併 Pass 鏈

本節介紹實際的程式碼範例。□外，所刊載的程式碼是為了呈現重點而擷取的片段，並非可直接運作的完整程式碼。

這是一邊交替讀寫兩張紋理，一邊連續套用 Blur 的範例。先呈現以 RasterRenderPass 實作相同處理的情況，再呈現整合為一個 UnsafePass 的情況。desc 是 Blur 用的 Render Texture 描述子，blurMaterial 是 Blur 用的材質。

9.4.1 RasterRenderPass 方式：每個 Blur 處理各登錄一個 Pass

在 RasterRenderPass 方式中，如 清單 9.1 所示，在迴圈□對每個 Blur 處理呼叫 Add RasterRenderPass。每個 Pass 的輸入與輸出都能明確宣告，但 Blur 次數越多，Render Graph 上的 Pass 數也越增加。

▼清單 9.1　以 RasterRenderPass 實作 Ping-pong Blur

```
class BlurRasterPassData
{
    internal TextureHandle source;
    internal TextureHandle target;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var current = resourceData.activeColorTexture;
var writePing = true;

for (var i = 0; i < blurIterations; i++)
{
    var target = writePing ? ping : pong;

    using (var builder = renderGraph.AddRasterRenderPass<BlurRasterPassData>(
        "Ping-pong Blur Raster", out var passData))
    {
        passData.source = current;
        passData.target = target;
        passData.material = blurMaterial;

        builder.UseTexture(passData.source, AccessFlags.Read);
        builder.SetRenderAttachment(passData.target, 0, AccessFlags.Write);
        builder.SetRenderFunc((BlurRasterPassData data, RasterGraphContext context) =>
        {
            Blitter.BlitTexture(
                context.cmd, data.source, new Vector4(1, 1, 0, 0),
                data.material, 0);
        });
    }

    current = target;
    writePing = !writePing;
}
```

9.4.2　UnsafePass 方式：在一個 Pass 內執行一連串處理

在 UnsafePass 方式中，如 清單 9.2 所示，只向 Render Graph 登錄一次，並在 SetRenderFunc 內切換渲染目標。每個 Blur 處理的繪製命令仍然存在，但以 Pass 為單位的管理可整合為一次。

▼清單 9.2　將 Ping-pong Blur 整合為一個 UnsafePass

```
class PingPongBlurPassData
{
    internal TextureHandle source;
    internal TextureHandle ping;
    internal TextureHandle pong;
    internal int iterations;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var output = blurIterations == 0
    ? resourceData.activeColorTexture
    : blurIterations % 2 == 0 ? ping : pong;

using (var builder = renderGraph.AddUnsafePass<PingPongBlurPassData>(
    "Ping-pong Blur", out var passData))
{
    passData.source = resourceData.activeColorTexture;
    passData.ping = ping;
    passData.pong = pong;
    passData.iterations = blurIterations;
    passData.material = blurMaterial;

    builder.UseTexture(passData.source, AccessFlags.Read);
    builder.UseTexture(passData.ping, AccessFlags.ReadWrite);
    builder.UseTexture(passData.pong, AccessFlags.ReadWrite);
    builder.SetRenderFunc((PingPongBlurPassData data, UnsafeGraphContext context) =>
    {
        var cmd = CommandBufferHelpers.GetNativeCommandBuffer(context.cmd);
        var current = data.source;
        var next = data.pong;
        var alternate = data.ping;
        for (var i = 0; i < data.iterations; i++)
        {
            cmd.SetRenderTarget(
                next,
                RenderBufferLoadAction.DontCare,
                RenderBufferStoreAction.Store);
            Blitter.BlitTexture(
                cmd, current, new Vector4(1, 1, 0, 0),
                data.material, 0);

            current = next;
            (next, alternate) = (alternate, next);
        }
    });
```

```
}
```

9.5 總結

本章介紹了在 Render Graph 中應使用 UnsafePass 的情境與使用時的注意事項。

應選擇 UnsafePass 的情境有以下 3 種。

- 需要 RasterCommandBuffer/ComputeCommandBuffer 未公開的 API 時
- 需要傳給外部程式碼的 CommandBuffer 本身時
- 想從 Compatibility Mode 快速移轉到 Render Graph 時
- 想將無法合併的 Pass 鏈整合為一個 Pass 時

將無法合併的 Pass 鏈整合為一個 UnsafePass，可削減以 Pass 為單位的登錄／管理成本，也能改善 Render Graph Viewer 的可讀性。在本專案於 Editor 上的測量（表 9.1）中，每個 RasterRenderPass 約可削減 1.8μ s 的固定成本。不過，由於繪製命令本身不變，需注意這並非以降低 GPU 負載為目的的手法。

□一方面，UnsafePass 也是捨棄 Render Graph 部分自動管理與驗證的 Pass。使用時，請務必以 UseTexture 宣告相依性，並在 SetRenderTarget 中明確指定 Load/Store Action。

基本上優先使用 RasterRenderPass 與 ComputePass，並將 UnsafePass 限定於無法以宣告式 API 表達的處理，或整理無法合併的 Pass 鏈，就能兼顧 Render Graph 的最佳化與 CommandBuffer 的自由度。

9.6 參考資料

- Unity Manual - Use the CommandBuffer interface in a render graph (Unsafe Pass)
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-unsafe-pass.html
- Unity Manual - Optimize a render graph
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-optimize.html

- Unity Scripting API - CommandBuffer
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.CommandBuffer.html
- Unity Scripting API - RenderBufferLoadAction
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferLoadAction.html
- Unity Scripting API - RenderBufferStoreAction
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferStoreAction.html

第10章

省略 URP 的 FinalBlitPass 的方法

チャン ユービン ZHANG YUBING
株式會社 CyberAgent SGE 核心技術本部

姓名：チャン ユービン羅馬拼音：ZHANG YUBING所屬：株式會社 CyberAgent SGE 核心技術本部

10.1　什麼是 FinalBlitPass

在 URP 的渲染管線中，實際顯示在螢幕上的是 BackBuffer。但 BackBuffer 為僅可寫入，無法讀取。因此在後處理這類需要將中間結果作為輸入讀取的處理中，相機會先繪製到中間紋理，最後再 Blit 到 BackBuffer。負責從這個中間紋理 Blit 到 BackBuffer 的，就是 URP 的 FinalBlitPass。它並非單純的複製，也一併負責輸出到螢幕前所需的最終處理。

- 考慮相機 Viewport 的傳輸
- Linear→sRGB 轉換
- HDR Output 時的編碼與疊加 UI 的合成
- UV 原點不同時的 Y 軸翻轉
- 縮放時的濾波器切換

10.1.1　具備 FinalBlit 功能的 3 種 FinalBlitPass

負責「將中間紋理輸出到 BackBuffer」的 Pass，在 URP 中其實有 3 種，並依條件只會選用其中 1 種。

UberPost

這是□用後處理時會加入的 Pass，但在滿足以下條件時，也會兼具 FinalBlit 的功能。

- 是相機堆疊（Camera Stack）中的最後一個相機
- 不需要後述的 FinalPost
- 沒有在 AfterRenderingPostProcessing 之後執行的自訂 Pass

FinalPost

這是在□用後處理，且存在 UberPost 無法處理的「最終工序」時加入的 Pass。

1. FXAA
2. 非 Linear 的放大 … Point / FSR / STP 等。
3. TAA + CAS … 需要以獨立 Pass 執行 RCAS □利化

FinalBlitPass

這是上述 2 者都沒有寫入 BackBuffer 時的最後防線。實際會出現的主要是以下情況。

- 後處理已停用，且存在中間紋理
- 後處理已□用，但其後有自訂 Pass
- 後處理已□用，但存在 Camera.AddCommandBufferAsync 等擷取處理

整理 3 種 Pass 的關係

表 10.1: 負責最終輸出的 Pass 與條件

負責最終輸出的 Pass	條件
UberPost	後處理已□用、不需要 FinalPost、其後沒有自訂 Pass 也沒有擷取處理
FinalPost	後處理已□用，且符合 FXAA / 非 Linear 的放大 / TAA + CAS 其中之一
FinalBlitPass	上述 2 者都沒有寫入 BackBuffer，且存在中間紋理時
皆無	沒有中間紋理，或不是相機堆疊上的最後一個相機

10.2 省略 FinalBlitPass 的方法

使用 URP 的後處理，或製作一般的自訂後處理時，都不需要在意 FinalBlitPass。但若加入了相當於「最終工序」的自訂 Pass，URP 的 FinalBlitPass 就只是多插入 1 次 Blit 的浪費 Pass。因此從 Unity 6.3 起，可以由自訂 Pass 端宣告「接下來的作用中目標是 BackBuffer」，藉此省略 URP 的 FinalBlitPass。

▼清單 10.1 切換至 BackBuffer

```
var resourceData = frameData.Get<UniversalResourceData>();
resourceData.SwitchActiveTexturesToBackbuffer();
```

實作很單純（Runtime/FrameData/UniversalResourceData.cs）。

▼清單 10.2 SwitchActiveTexturesToBackbuffer 的實作

```
public void SwitchActiveTexturesToBackbuffer()
{
    activeColorID = UniversalResourceData.ActiveID.BackBuffer;
    activeDepthID = UniversalResourceData.ActiveID.BackBuffer;
}
```

只是改寫 frameData 上的 2 個 ID，完全不進行 Blit、複製或新增 RenderPass。這個 API 不是「傳輸處理」，而是對 URP 的宣告。activeColorTexture / activeDepthTexture 只是依據這些 ID 切換所回傳的 TextureHandle 的 switch。

▼清單 10.3 activeColorTexture 的切換

```
public TextureHandle activeColorTexture
{
    get
    {
        switch (activeColorID)
        {
            case ActiveID.Camera:     return cameraColor;
            case ActiveID.BackBuffer: return backBufferColor;
            ...
        }
    }
}
```

執行 SwitchActiveTexturesToBackbuffer() 後，URP 的 FinalBlitPass 就不會再被建立。

10.2.1 使用範例：自訂 FXAA

以自訂 FXAA 後處理為例，比較前後差異。過去的典型流程需要 3 次 Blit。

▼清單 10.4 以往的 FXAA 流程

```
CameraColor → [FXAA] → FXAABuffer → [Blit] → CameraColor → [URP FinalBlit] →
    BackBuffer
```

Unity 6.0 以後可以直接替換 CameraColor，因此減為 2 次，但仍需要 FXAABuffer 這個中間紋理，FinalBlitPass 也仍會被建立。

▼清單 10.5 Unity 6.0 的 FXAA 流程

```
CameraColor → [FXAA] → FXAABuffer → [URP FinalBlit] → BackBuffer
```

在 Unity 6.3 中使用 BackBuffer 切換，只需 1 次即可。也不需要中間紋理。

▼清單 10.6 Unity 6.3 的 FXAA 流程

```
CameraColor → [FXAA] → BackBuffer
```

程式碼如下。此外，以下程式碼為了呈現重點而僅節錄一部分，直接使用將無法運作。

▼清單 10.7　直接輸出到 BackBuffer 的自訂 FXAA

```
private class PassData
{
    public TextureHandle Source;
    public TextureHandle Target;
    public Material FXAAMaterial;
}

public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    var cameraData = frameData.Get<UniversalCameraData>();
    var resourceData = frameData.Get<UniversalResourceData>();

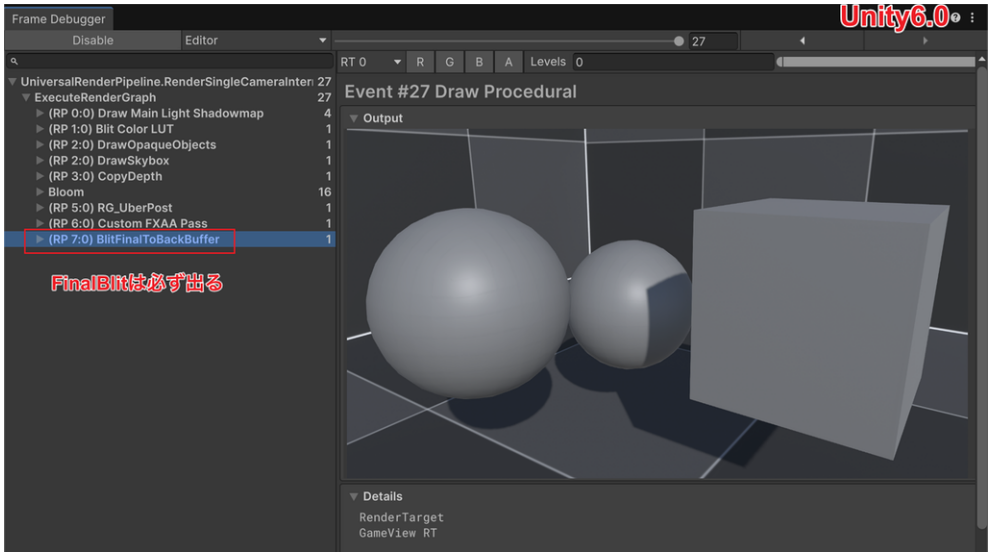
    // 切換前先取得 CameraColor
    var cameraColor = resourceData.activeColorTexture;

    // 之後的 activeColorTexture 會回傳 BackBufferColor
    // 如此一來 URP 的 FinalBlitPass 就不會被建立
    resourceData.SwitchActiveTexturesToBackbuffer();
    var backBufferColor = resourceData.activeColorTexture;

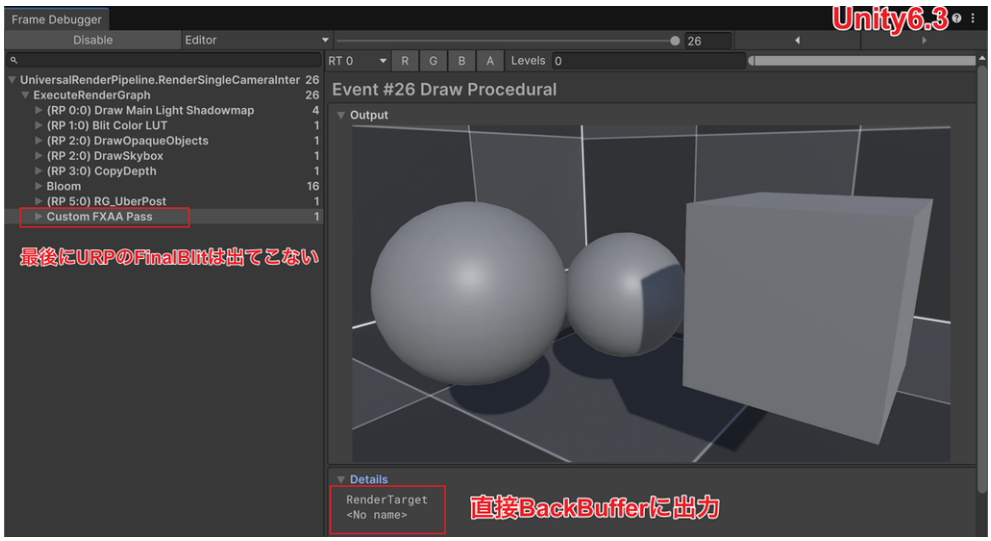
    // 判斷是否需要 LinearToSRGBConversion
    var requireSrgbConversion = cameraData.targetTexture == null
                                && Display.main.requiresSrgbBlitToBackbuffer;
    CoreUtils.SetKeyword(_fxaaMaterial, ShaderKeywordStrings.LinearToSRGBConversion,
    requireSrgbConversion);

    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Custom FXAA Pass",
    out var passData))
    {
        passData.Source = cameraColor;
        passData.Target = backBufferColor;
        passData.FXAAMaterial = _fxaaMaterial;
        builder.SetRenderAttachment(backBufferColor, 0, AccessFlags.WriteAll);
        builder.UseTexture(passData.Source, AccessFlags.Read);
        builder.SetRenderFunc(static (PassData data, RasterGraphContext context) =>
        {
            // 只有 Source 與 Target 的 UV 原點不同時才需要 Y 軸翻轉
            var yFlip = context.GetTextureUVOrigin(data.Source) != context.
    GetTextureUVOrigin(data.Target);
            var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
            Blitter.BlitTexture(context.cmd, data.Source, scaleBias, data.FXAAMaterial,
     0);
        });
    }
}
```

在 Unity 6.0 中，FinalBlitPass 一律會被建立。



在 Unity 6.3 中使用 BackBuffer 切換，只需 1 次 Blit 即可完成 FXAA 處理並直接輸出到 BackBuffer。



10.3 省略 URP 的 FinalBlit 時的注意事項

SwitchActiveTexturesToBackbuffer() 只是改寫□部紋理 ID 的 API。換言之，讓前後一致的責任全部在呼叫端。

10.3.1 只在相機的最後呼叫 SwitchActiveTexturesToBackbuffer()

從呼叫此 API 的瞬間起，之後所有 Pass 的 activeColorTexture / activeDepthTexture 都會指向 BackBuffer。然而 BackBuffer 無法 Read，因此之後讀取 activeColorTexture / activeDepthTexture 的處理全部都會失敗。此外，在 cameraData.resolveFinalTarget == false（後面還有相機）時也不可切換。Overlay 相機的設計是疊加繪製到 CameraColor 上，因此若中途的相機直接寫入 BackBuffer，就會與後續相機的繪製結果混合，或被覆寫而消失。因此請將呼叫限定在最後一個相機、進行最終輸出的自訂 Pass 之前。

10.3.2 在最後執行的自訂 Pass 中實作 FinalBlit 的功能

省略 FinalBlitPass，意味著 FinalBlitPass 原本負責的最終處理也會一併消失。必須由自訂 Pass 端接手。

Y 軸翻轉

Source 與 Target 的 UV 原點不一致時，若不翻轉，畫面會上下顛倒。可使用 Unity 6.3 新增的 GetTextureUVOrigin() 比較兩者來判定。這個 API 只存在於 RasterGraph Context 等執行時的 GraphContext，因此必須在 SetRenderFunc □呼叫，而不是在 RecordRenderGraph 的記錄時呼叫。

▼清單 10.8　Y 軸翻轉的判定

```
var yFlip = context.GetTextureUVOrigin(data.Source) != context.GetTextureUVOrigin(data.
    Target);
var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
```

此外，SystemInfo.graphicsUVStartsAtTop 只表示圖形 API 的 UV 慣例，無法用來判定「這次的 Blit 是否需要翻轉」。

sRGB 轉換

必須參照 cameraData.requireSrgbConversion 來設定 LinearToSRGBConversion 關鍵字，並在 Shader 端依該關鍵字實作轉換處理。

▼清單 10.9　sRGB 轉換關鍵字的設定

```
var requireSrgbConversion = cameraData.targetTexture == null && Display.main.
    requiresSrgbBlitToBackbuffer;
cmd.SetKeyword(ShaderGlobalKeywords.LinearToSRGBConversion, data.requireSrgbConversion)
    ;
```

▼清單 10.10　sRGB 轉換

```
#ifdef _LINEAR_TO_SRGB_CONVERSION
    color = LinearToSRGB(color);
#endif
```

Viewport 設定

Viewport 通常設為全螢幕，但若想設為全螢幕以外的範圍，就必須設定 Camera 的 Viewport。

▼清單 10.11　Viewport 的設定

```
var targetViewPort = new Rect(0, 0, targetWidth, targetHeight);
cmd.SetViewport(targetViewPort);
```

HDR Output 與疊加 UI 合成

輸出到 HDR 顯示器時的編碼，以及 cameraData.rendersOverlayUI 為 true 時的疊加 UI 合成，原本也是 FinalBlitPass 的工作。需要支援 HDR 輸出的遊戲，必須使用 HDROutputUtils 實作同等的處理。

看起來要求很多，但只要實作專案實際使用的功能即可。若是固定 SDR、Viewport 為全螢幕、未使用疊加 UI 的遊戲，實質上需要的只有 Y 軸翻轉與 sRGB 轉換。

10.4 總結

- 在 URP 中，負責最終輸出到 BackBuffer 的 Pass 有 UberPost / FinalPost / FinalBlitPass 3 種，依條件只會選用其中 1 種
- 將自訂 Pass 注入 AfterRenderingPostProcessing 之後，hasPassesAfterPostProcessing 會變為 true，FinalBlitPass 幾乎必定會被加入
- 使用 Unity 6.3 的 SwitchActiveTexturesToBackbuffer()，自訂 Pass 就能直接輸出到 BackBuffer，並省略這個 FinalBlitPass
- 不過這個 API 只是改寫相機 ActiveColor/Depth Texture ID 的宣告。前提是只在最後一個相機的最終 Pass 呼叫，並自行負責 Y 軸翻轉、sRGB 轉換、Viewport 等原本由 FinalBlitPass 負責的處理

1 次全螢幕 Blit，在高解析度、行動裝置環境下是不可忽視的成本。對於擁有自訂後處理的專案，很□得考慮。

10.5 參考資料

- Unity Manual: Get data from the current frame in URP
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/accessing-frame-data.html
- Class UniversalResourceData | Universal Render Pipeline | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.3/api/UnityEngine.Rendering.Universal.UniversalResourceData.html
- Struct RasterGraphContext | Scriptable Render Pipeline Core | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@17.3/api/UnityEngine.Rendering.RenderGraphModule.RasterGraphContext.html

第11章

文章標題

永留 遼平 Nagatome Ryouhei

株式會社 Colorful Palette

rngtm　**X** @rn49rn49

行動裝置著色器最佳化：超越函數的測量與輕量化

11.1　姓名・所屬

永留遼平 / Nagatome Ryouhei / 株式會社 Colorful Palette

11.2　GitHub 帳號

`https://github.com/rngtm`

11.3　X 帳號

`https://x.com/rn49rn49`

11.4　前言

在著色器中使用 sin 或 exp 等超越函數時，GPU 的處理負載往往會變高。本文將搭配 Profiler 的測量□，□明這些函數對實際效能的影響程度。此外，本文也探討將超越函數替換為近似計算的最佳化手法，並以實測□介紹實際可獲得多少負載削減效果。

11.5　環境

本文使用的 Unity 版本、渲染管線與裝置如下。

Unity 版本	Unity6000.3.20f1
渲染管線	Universal Render Pipeline 17.3.0
iOS 裝置	iPhoneX iPhone14 iPhone15 Pro
Android 裝置	Galaxy S9

11.6 超越函數

超越函數是指無法以有限次的四則運算或根式表示的函數。代表性的例子有三角函數、指數函數與對數函數。

- sin 與 cos 等三角函數
- pow、pow2 等指數函數
- log、log10 等對數函數

由於超越函數的負載較高，一般認為應避免大量使用。

11.6.1 超越函數較重的原因

超越函數之所以負載高，有以下 3 個原因。

- 時脈週期數多
- 執行超越函數運算的單元（SFU）數量少
- 暫存器壓力造成的額外開銷

11.6.2 原因 1：時脈週期數多

智慧型手機等搭載的 GPU，配備了大量用於算術運算的運算單元（ALU）。ALU 的設計使其能在 1 個時脈週期□執行加法或乘法。

然而，超越函數並非能以單一指令算出的簡單運算。GPU □部計算超越函數時，會使用泰勒展開或極小化極大法（minimax）等多項式近似演算法，分解為多個加法與乘法的組合來計算。

單純的四則運算只需「1 個指令」，而超越函數會被編譯器展開為「數十個四則運算指令」，因此處理時間較長。

11.6.3 原因 2：SFU 數量少

許多 GPU 配備名為 SFU（Special Function Unit）的專用硬體，用來高速處理超越函數等運算，超越函數有時會由 SFU 執行。

SFU 的數量遠少於 ALU，因此即使使用 SFU 執行超越函數，處理仍需耗費較長時間。

11.6.4 原因 3：暫存器壓力造成的額外開銷

像 sin() 這樣的超越函數，計算比單純的加法或乘法複雜，在 GPU 內部計算時需要更多的暫存器。

GPU 的各運算單元（SM：Streaming Multiprocessor）所擁有的暫存器總量由硬體決定，每個執行緒消耗的暫存器數增加（暫存器壓力升高），因此一個 SM 能容納的執行緒數量會減少。

由於可同時執行的執行緒數減少，處理便需要更長的時間。

11.7 各種函數的負載

針對迴圈次數 0 至 10，分別測量執行各種超越函數時的 GPU 時間，找出哪個超越函數最重。GPU 時間的測量使用 FrameTimingManager。

11.8 負載測量所用的著色器

測量使用了如下的著色器。可透過著色器關鍵字選擇要執行的超越函數，並使用變數 _LoopCount 變更迴圈次數。

```
Shader "Benchmark/Transcendental"
{
    Properties
    {
        // （）會將紋理固定到uGUICanvasRenderer "_MainTex，因此名稱固定"
        [PerRendererData][MainTexture] _MainTex("Sprite Texture", 2D) = "white" {}
        [MainColor] _BaseColor("Tint", Color) = (1, 1, 1, 1)

        // （MaskUI/）用的模板設定。MaskMask 元件會替換 Material 並寫入
```

```
        _StencilComp("Stencil Comparison", Float) = 8
        _Stencil("Stencil ID", Float) = 0
        _StencilOp("Stencil Operation", Float) = 0
        _StencilWriteMask("Stencil Write Mask", Float) = 255
        _StencilReadMask("Stencil Read Mask", Float) = 255

        _ColorMask("Color Mask", Float) = 15

        [Toggle(UNITY_UI_ALPHACLIP)] _UseUIAlphaClip("Use Alpha Clip", Float) = 0
    }

    SubShader
    {
        Tags
        {
            "Queue" = "Transparent"
            "RenderType" = "Transparent"
            "RenderPipeline" = "UniversalPipeline"
            "IgnoreProjector" = "True"
            "PreviewType" = "Plane"
            "CanUseSpriteAtlas" = "True"
        }

        Stencil
        {
            Ref [_Stencil]
            Comp [_StencilComp]
            Pass [_StencilOp]
            ReadMask [_StencilReadMask]
            WriteMask [_StencilWriteMask]
        }

        Cull Off
        ZWrite Off
        ZTest [unity_GUIZTestMode]
        Blend SrcAlpha OneMinusSrcAlpha
        ColorMask [_ColorMask]

        Pass
        {
            HLSLPROGRAM

            #pragma vertex vert
            #pragma fragment frag
            // 迴圈次數為 (動態迴圈)，因此需要uniform 3.0 而非 2.0
            #pragma target 3.0

            // 支援 RectMask2D / Mask 的 Alpha Clip
```

```
            #pragma multi_compile_local _ UNITY_UI_CLIP_RECT
            #pragma multi_compile_local _ UNITY_UI_ALPHACLIP

            // 切換要測量的超越函數。由於要在執行時以 Shader.EnableKeyword 切換,
            // 使用 multi_compile 而非 (未使用的變體會被剔除)。shader_feature
            // 每個關鍵字都是獨立的變體，因此⍰生的程式碼只會包含所選的函數（沒有分支的額外開銷)。
            // 由於是全域關鍵字，可從 TranscendentalBenchmark 端統一控制。
            // 若未⍰用任何關鍵字，則使用開頭的 _FUNC_NONE 變體。
            #pragma multi_compile_fragment _FUNC_NONE _FUNC_SIN _FUNC_SIN_APPROX
    _FUNC_COS _FUNC_TAN _FUNC_TAN_TAYLOR _FUNC_TAN_PADE _FUNC_ASIN _FUNC_ACOS
    _FUNC_ATAN _FUNC_EXP _FUNC_EXP2 _FUNC_POW _FUNC_LOG _FUNC_LOG2 _FUNC_LOG10

            #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.
    hlsl"

            struct Attributes
            {
                float4 positionOS : POSITION;
                float4 color : COLOR;
                float2 uv : TEXCOORD0;
                UNITY_VERTEX_INPUT_INSTANCE_ID
            };

            struct Varyings
            {
                float4 positionHCS : SV_POSITION;
                float2 uv : TEXCOORD0;
                float4 positionOS : TEXCOORD1; // 與 _ClipRect 保持在相同的空間 (Canvas 本
    地空間)
            };

            TEXTURE2D(_MainTex);
            SAMPLER(sampler_MainTex);

            CBUFFER_START(UnityPerMaterial)
                half4 _BaseColor;
            CBUFFER_END

            // 由 TranscendentalBenchmark 設定的全域⍰。
            // 因為不是以材質為單位的⍰，所以不放入 (放入會破壞UnityPerMaterial SRP ) Batcher
            int _TranscendentalLoopCount;
            float _PowExponent;

            // 相當於 UnityUI.cginc 的 (以UnityGet2DClipping HLSL 自行定義)
            float Get2DClipping(float2 position, float4 clipRect)
            {
                float2 inside = step(clipRect.xy, position.xy) * step(position.xy,
    clipRect.zw);
                return inside.x * inside.y;
```

```
}

// 將傳給超越函數的□限制在 (0.05, 0.95) □。
// 一併避免 asin/acos 的定義域 |x| <= 、1log 系的定義域 x > ，以及0 exp 系的溢位。
// 這是所有變體共通的前處理，其成本在差□計算中會相互抵銷。
float PrepareInput(float seed)
{
    return seed;
    // return frac(seed) * 0.9 + 0.05;
}

float sinApprox(float x)
{
    // sin 的□物線近似。只需乘法與 ，用來比較與真正的abs sin 指令之間的成本差。
    // 定義域為 |x| <=  (PIPrepareInput 的輸出落在此範圍□)。
    // 係數會被常數摺疊，因此執行時沒有除法成本。
    return (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
}

float tanTaylor(float x)
{
    // tan 的泰勒展開至 3 次項（僅乘法）。用來比較與真正的 tan 指令之間的成本差。
    // 定義域為 |x| < PI，超過/2 PI/4 後精度會迅速下降。
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333); // 1.0/3.0
}

float tanPade(float x)
{
    // tan 的 Pade 近似（乘法加 1 次除法）。定義域為 |x| < PI。/2
    // 分母的零點 x ≈  ±1.581 靠近 tan 的極點 ±PI，因此能追隨到極點附近。/2
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}

// 測量對象的超越函數。每個關鍵字只會保留一個，
// 因此□生的程式碼中只會出現所選函數的展開結果。
float Transcendental(float x)
{
    #if defined(_FUNC_SIN)
        return sin(x);
    #elif defined(_FUNC_SIN_APPROX)
        return sinApprox(x);
    #elif defined(_FUNC_COS)
        return cos(x);
    #elif defined(_FUNC_TAN)
        return tan(x);
    #elif defined(_FUNC_TAN_TAYLOR)
```

```
            return tanTaylor(x);
        #elif defined(_FUNC_TAN_PADE)
            return tanPade(x);
        #elif defined(_FUNC_ASIN)
            return asin(x);
        #elif defined(_FUNC_ACOS)
            return acos(x);
        #elif defined(_FUNC_ATAN)
            return atan(x);
        #elif defined(_FUNC_EXP)
            return exp(x);
        #elif defined(_FUNC_EXP2)
            return exp2(x);
        #elif defined(_FUNC_POW)
            // 若將指數設為常數，pow(x, 2) -> x * x 會被摺疊，
            // 因此傳入 ，使其走通用的uniform pow 路徑
            return pow(x, _PowExponent);
        #elif defined(_FUNC_LOG)
            return log(x);
        #elif defined(_FUNC_LOG2)
            return log2(x);
        #elif defined(_FUNC_LOG10)
            return log10(x);
        #else // ：不含超越函數的基準線_FUNC_NONE
            return x;
        #endif
    }

    Varyings vert(Attributes IN)
    {
        Varyings OUT;
        OUT.positionOS = IN.positionOS;
        OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
        OUT.uv = IN.uv * 300.0;
        return OUT;
    }

    half4 frag(Varyings IN) : SV_Target
    {
        // 此處為測量對象（外觀上沒有意義）。
        // 以 UNITY_LOOP 抑制迴圈展開，使 _TranscendentalLoopCount 次的成本如實累加。
        // 除了超越函數的□容之外，所有變體的形式完全相同，
        // 因此與 _FUNC_NONE 的差□即為該函數的成本。
        float accum = 0.0;

        UNITY_LOOP
        for (int i = 0; i < _TranscendentalLoopCount; i++)
        {
```

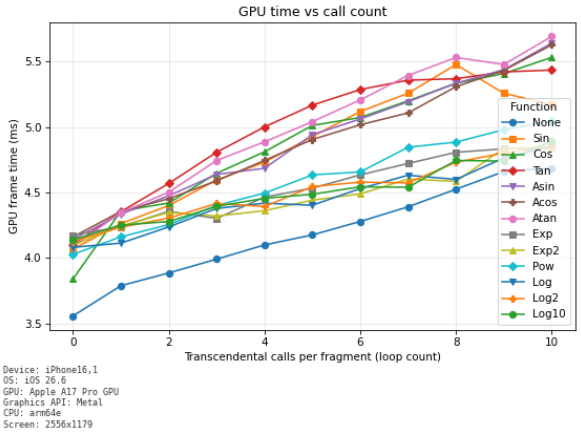
```
                    // 若成為迴圈不變式會被提升到迴圈外，因此將 i 混入引數
                    float x = IN.uv.x + i * 0.017;

                    accum += Transcendental(x);
                }
                return half4(saturate(accum.xxx) * 0.02, 1);
            }
            ENDHLSL
        }
    }
}
```

11.9 測量結果

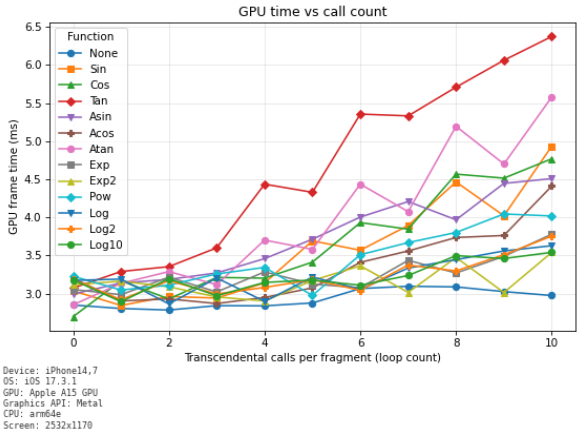
11.9.1 iPhone15 Pro

執行次數少於 7 次時 tan 最重，執行次數在 7 次以上時，Atan 的負載最高。

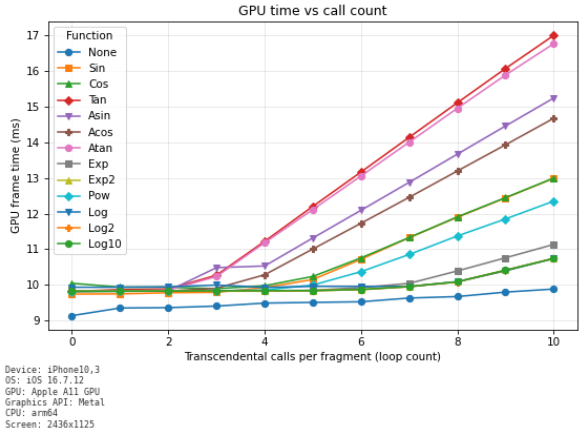


11.9.2 iPhone14

tan 的負載最高，atan 的負載第二高。

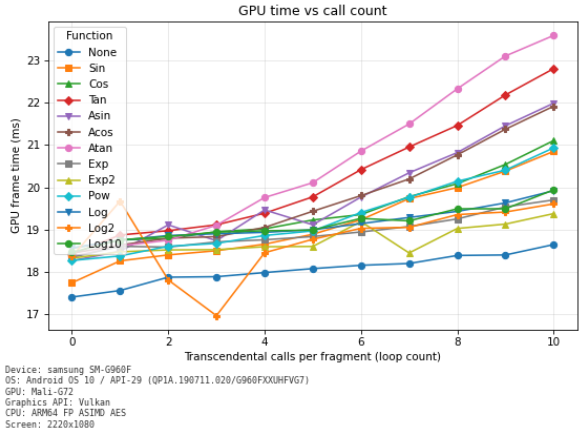


11.9.3 iPhoneX



11.9.4 Galaxy S9

Atan 的負載似乎最高。



11.9.5 總結

裝置	備註
iPhone 15 Pro	執行次數少於 7 次時 tan 最重 執行次數為 7 次以上時 atan 最重
iPhone 14	tan 最重
iPhone X	tan 最重
Galaxy S9	atan 最重

11.10 sin 函數的負載

測量超越函數中具代表性的 sin 函數的負載。

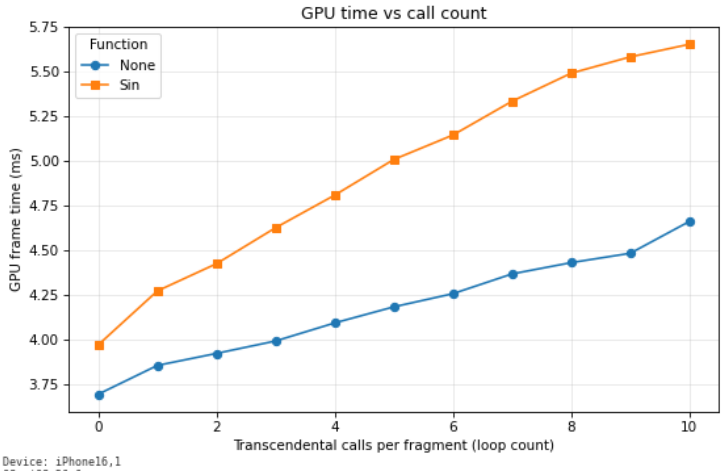
針對迴圈次數 0 至 10，分別測量影格□處理所需的 GPU 時間。每個測量點進行 10 秒的測量，並將 GPU 時間彙整為圖表。

第 1 張圖表：對應迴圈次數的 GPU 時間第 2 張圖表：對應迴圈次數的 GPU 時間與 None 的差□。

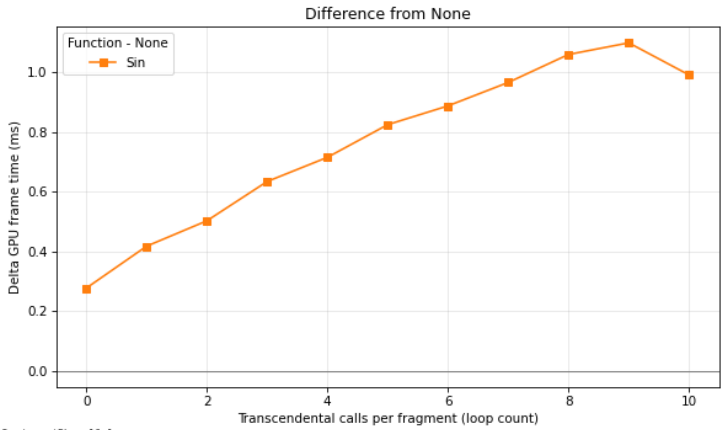
測量使用 FrameTimingManager。

11.10.1 iPhone15 Pro

著色器中存在 sin 指令，使時間增加了約 0.25ms。sin 的呼叫每增加 1 次，處理負載增加約 0.1ms。



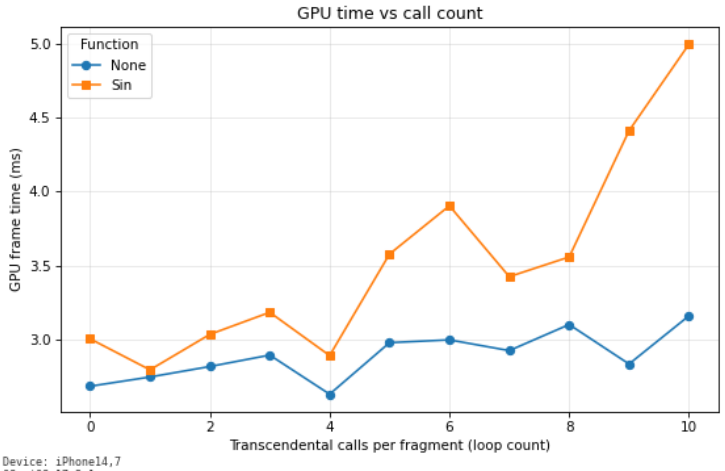

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```



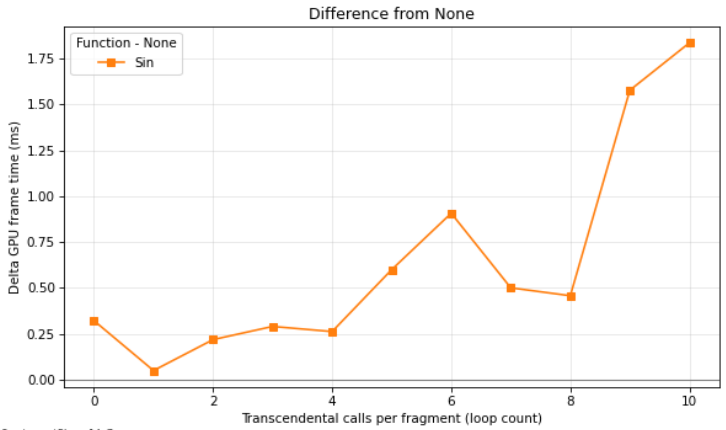

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

11.10.2 iPhone14

著色器中存在 sin 指令，使 GPU 時間增加了約 0.25ms。sin 的呼叫每增加 1 次，處理負載增加約 0.1 至 0.2ms。



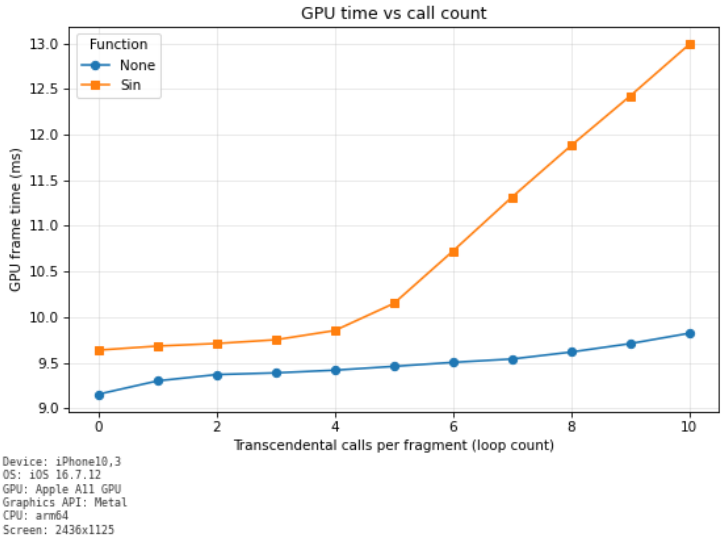

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```



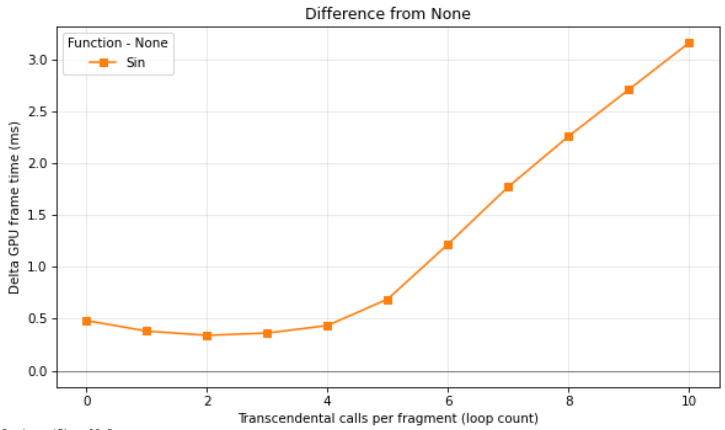

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

11.10.3 iPhoneX

著色器中存在 sin 指令，使時間增加了約 0.5ms。sin 的呼叫在 4 次以□時，幾乎看不到 GPU 時間的增加，超過 5 次後，sin 的呼叫每增加 1 次，處理負載增加約 0.6ms。



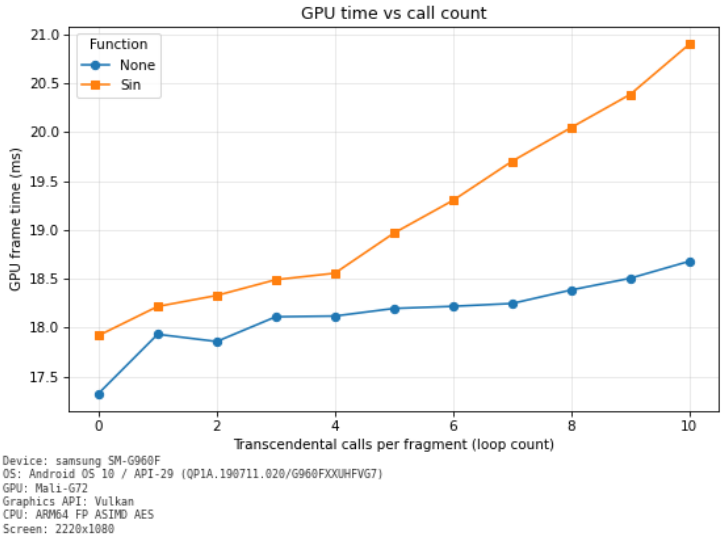

```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

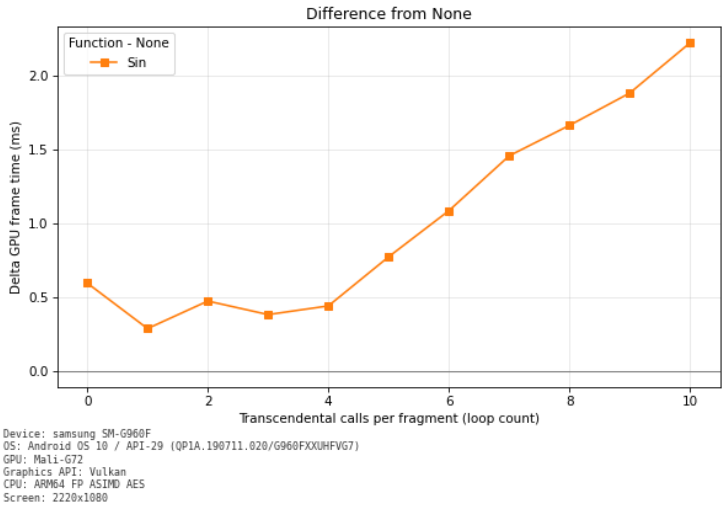



```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

11.10.4 Galaxy S9

著色器中存在 sin 指令，使時間增加了約 0.5ms。sin 的呼叫在 4 次以□時，幾乎看不到 GPU 時間的增加，超過 5 次後，sin 的呼叫每增加 1 次，處理負載增加約 0.2 至 0.3ms。





11.10.5 sin 函數負載的總結

裝置	sin 函數造成的 GPU 負載增加	備註
iPhone 15 Pro	+0.25ms（基準線）	每次呼叫增加約 0.1ms
iPhone 14	+0.25ms（基準線）	每次呼叫增加約 0.1 至 0.2ms
iPhone X	+0.5ms（基準線）	超過 5 次後每次呼叫增加約 0.6ms
Galaxy S9	+0.5ms（基準線）	超過 5 次後每次呼叫增加約 0.2 至 0.3ms

※表中的「基準線」是指著色器中存在 sin 函數時的初始負載增加量。

當著色器中的 sin 函數增加時，即使不執行 sin 函數，也能確認到 GPU 時間增加。這被認為是暫存器壓力造成的負載增加。

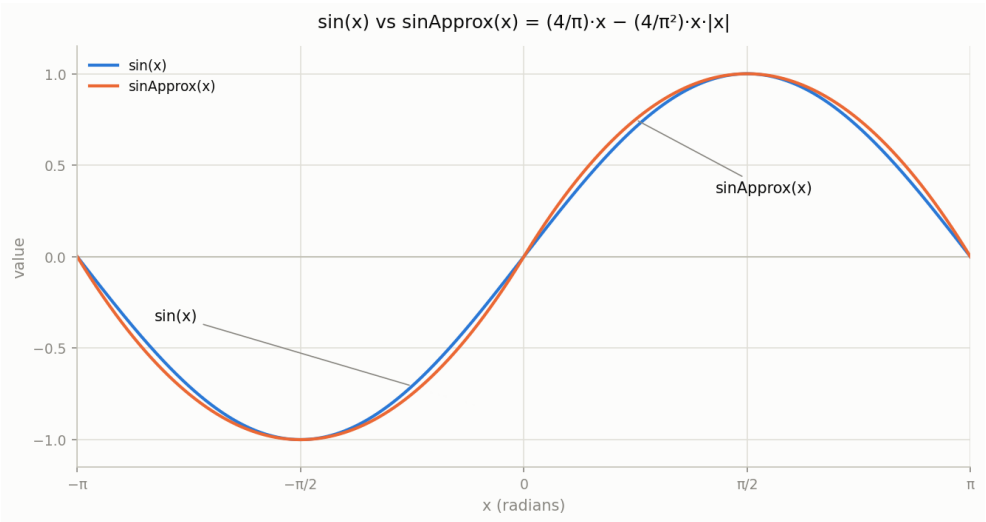
11.11 sin 函數的□物線近似

已知有一種使用如下多項式來近似 sin 的手法。

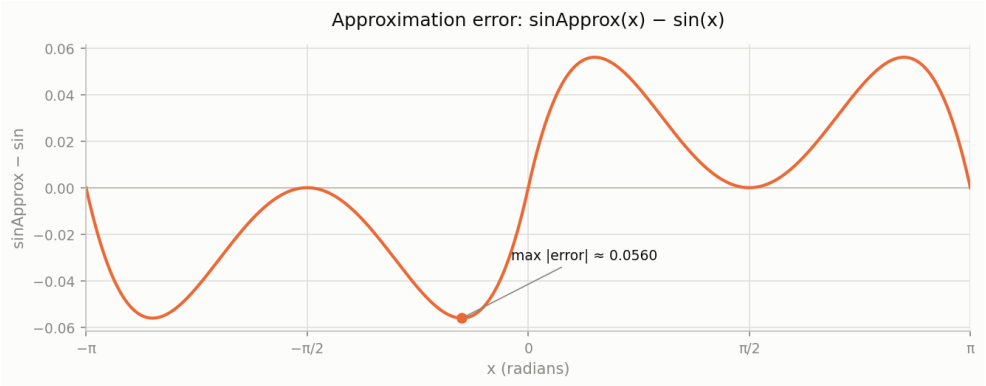
```
float sinApprox = (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
```

僅以乘法與□對□ abs() 即可計算，因此非常輕量。

sin 的□物線近似會輸出與 sin 非常接近的□（-PI<=x<=PI）


sin(x) vs sinApprox(x) = (4/π)·x − (4/π²)·x·|x|
sin(x)
sinApprox(x)
1.0
0.5
0.0
−0.5
−1.0
value
sinApprox(x)
sin(x)
−π
−π/2
0
π/2
π
x (radians)

誤差最大□約為 0.056。

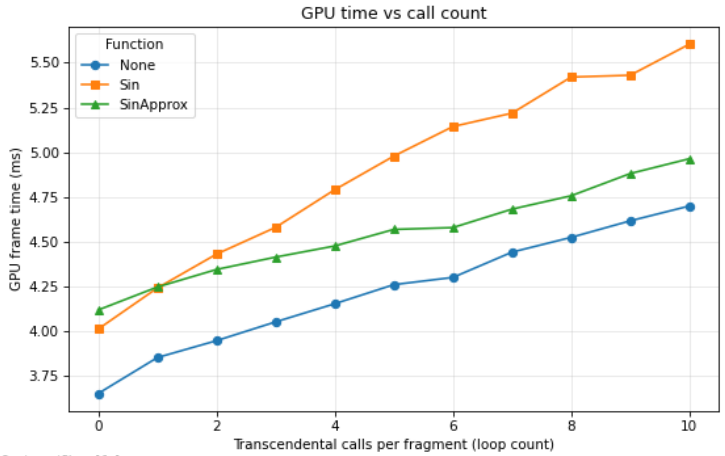

Approximation error: sinApprox(x) − sin(x)
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
sinApprox − sin
max |error| ≈ 0.0560
−π
−π/2
0
π/2
π
x (radians)

11.12 sin 近似的負載測量

測量 sin 的⌧物線近似 sinApprox 的負載，並彙整為圖表。

11.12.1 iPhone15 Pro

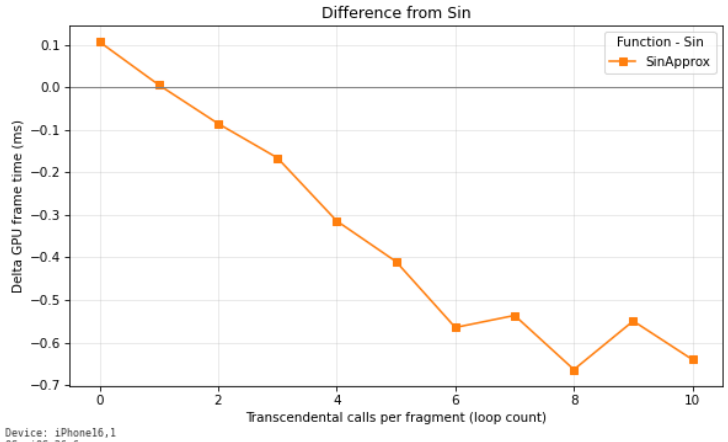
可以確認其負載低於 sin 函數。



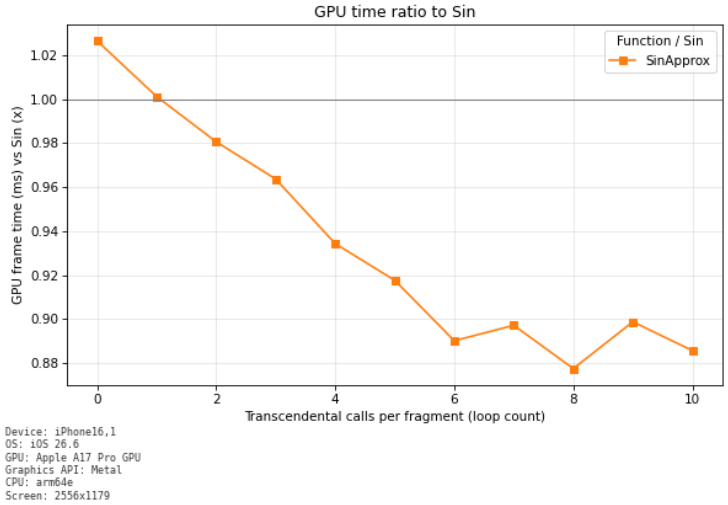

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

sin 函數與近似計算的比較

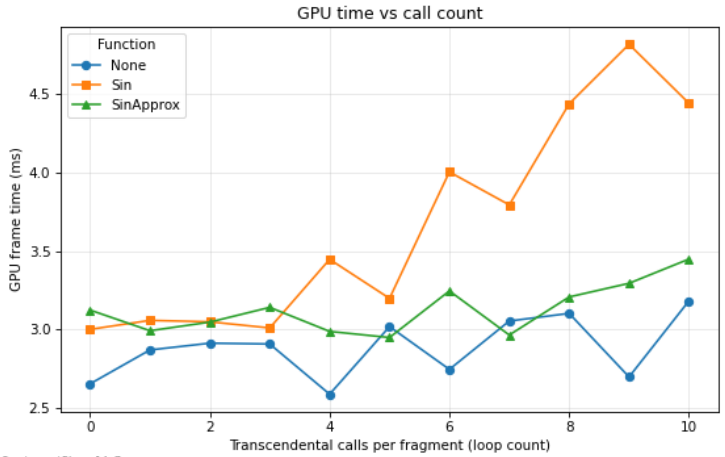
執行函式 10 次時，負載約降低 0.6ms。



函數執行 10 次時，sinApprox 的負載約為 sin 的 90%。



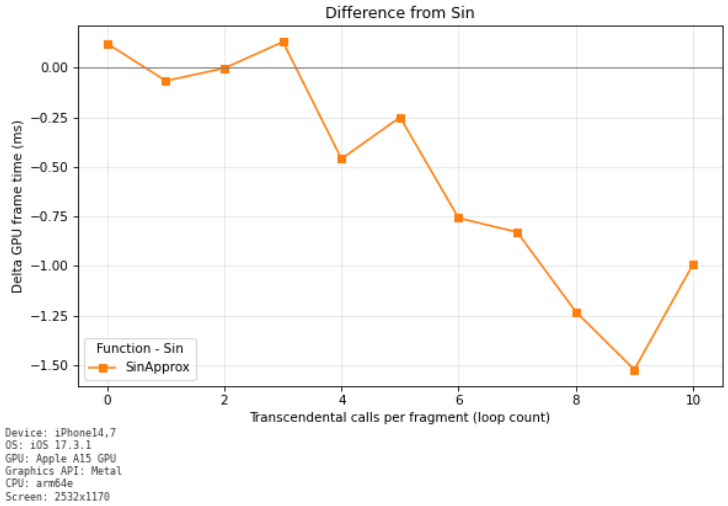
11.12.2 iPhone14



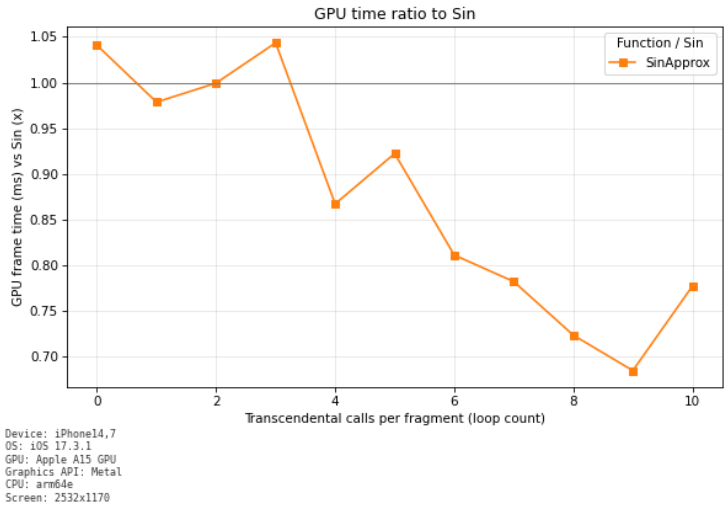

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

sin 函數與近似計算的比較

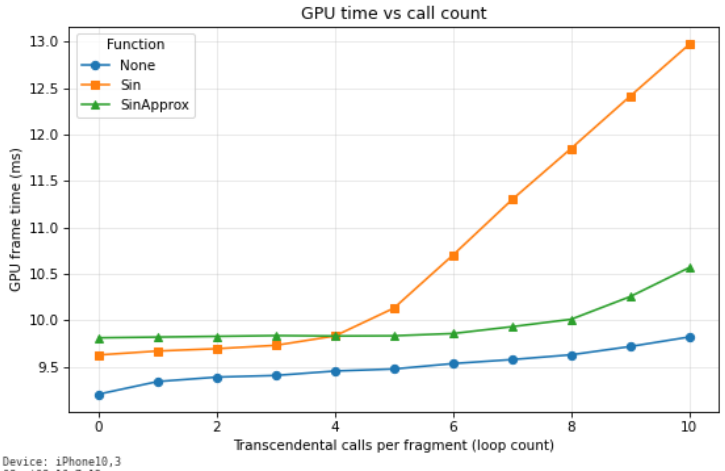
執行函式 10 次時，GPU 時間約縮短 1ms。



執行 sinApprox 10 次時的負載約為 sin 的 80%。



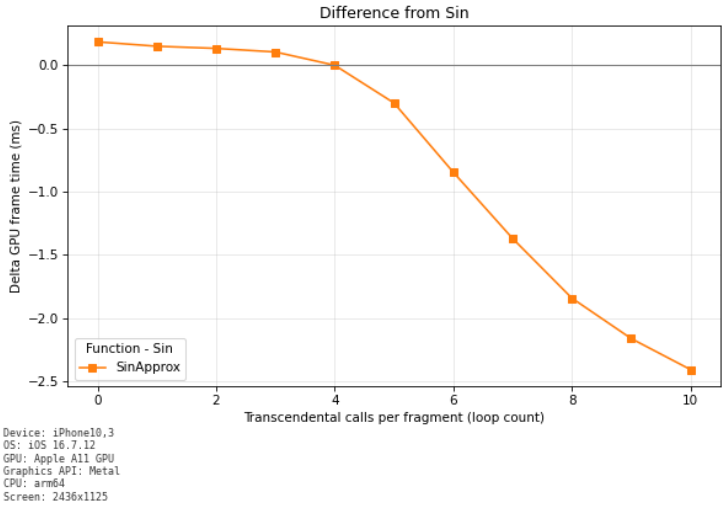
11.12.3 iPhoneX



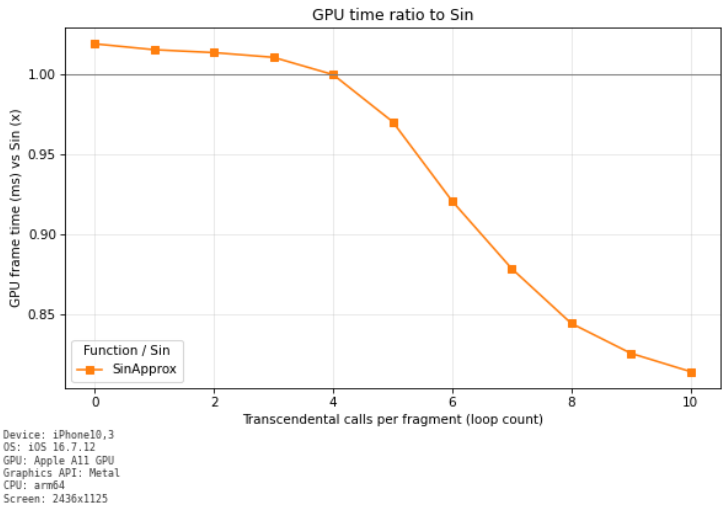

```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

sin 函數與近似計算的比較

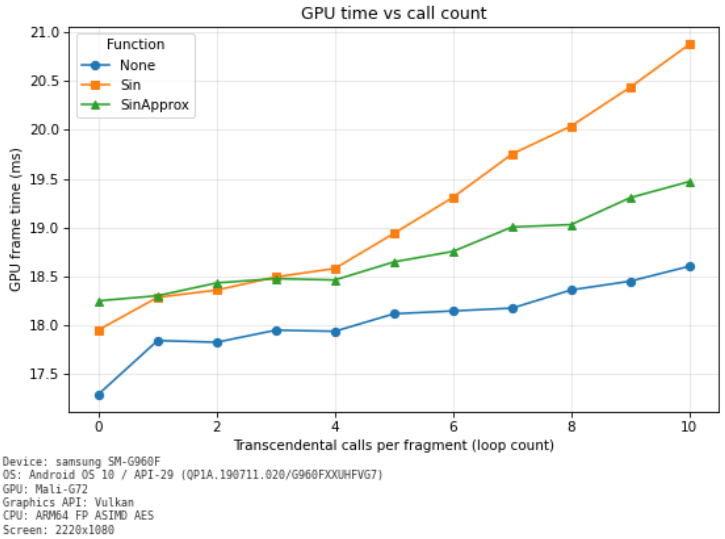
執行 sinApprox 10 次時的負載，比執行 sin 時縮短約 2.5ms。



執行 sinApprox 10 次時的 GPU 時間約為 sin 的 80%。

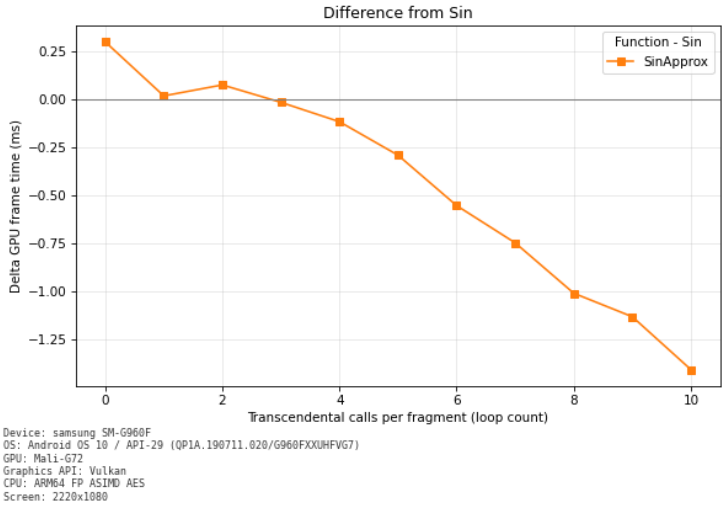


11.12.4 Galaxy S9

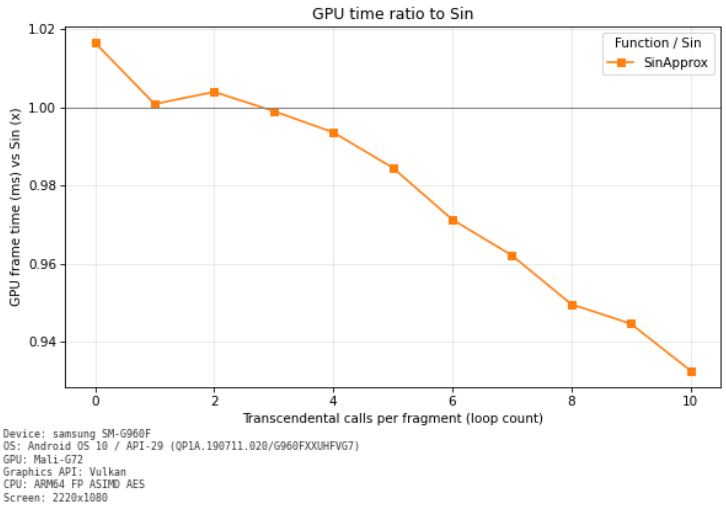


sin 函數與近似計算的比較

執行函式 10 次時，GPU 時間約縮短 1.3ms。



執行 sinApprox 10 次時的負載約為 sin 的 94%。



11.12.5 sin 近似負載的總結

將 sinApprox 帶來的 GPU 削減效果整理成表格，結果如下。

裝置	sin 函數的負載	sinApprox 的削減效果	備註
iPhone 15 Pro	基準	削減約 0.6ms	sinApprox 的負載約為 sin 的 90%
iPhone 14	基準	削減約 1.0ms	sinApprox 的負載約為 sin 的 80%
iPhone X	基準	削減約 2.5ms	sinApprox 的負載約為 sin 的 80%
Galaxy S9	基準	削減約 1.3ms	sinApprox 的負載約為 sin 的 94%

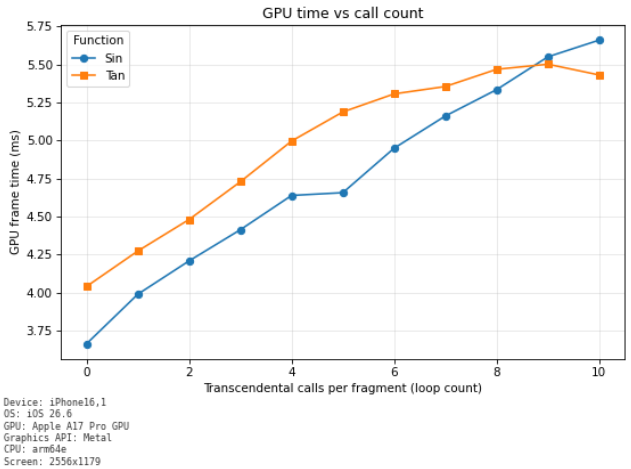
※削減效果是迴圈次數為 10 次時的 GPU 時間差□。

11.13 tan 函數的負載

也測量了 tan 函數的負載。tan 函數□部會呼叫除法，因此負載比 sin 函數與 cos 函數更高。

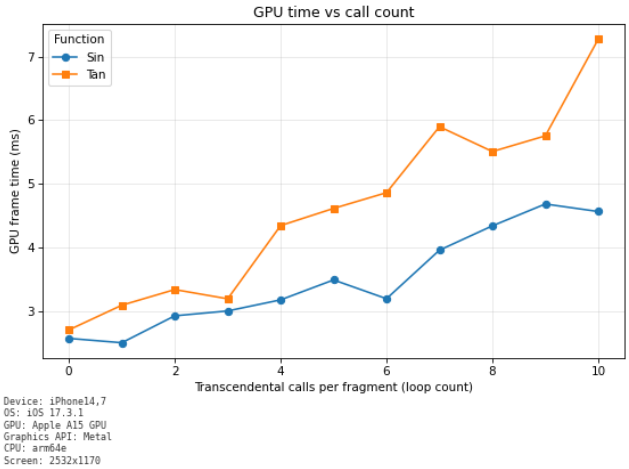
11.13.1 iPhone15Pro

其負載比 sin 函數更高，但執行次數超過 9 次後，sin 函數的負載反而更高。



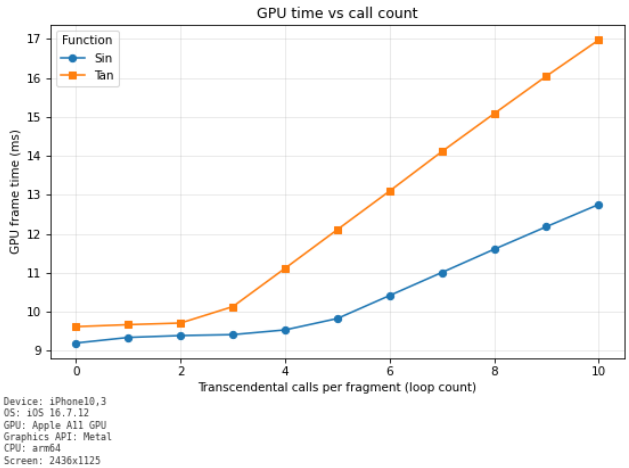
11.13.2 iPhone14

tan 函數的負載高於 sin 函數。



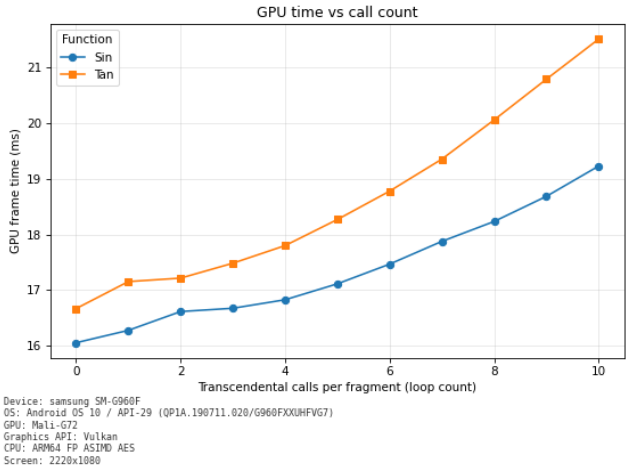
11.13.3 iPhoneX

tan 函數的負載高於 sin 函數。



11.13.4 Galaxy S9

tan 函數的負載高於 sin 函數。



11.14 tan 函數的近似

tan 函數的近似手法，有直接運用泰勒展開的方法，以及使用 Padé 近似的方法。泰勒展開的負載較低，但精度也較低。Padé 近似的精度較好，但負載也較高。

11.14.1 手法 1. 以泰勒展開近似

這是利用 tan 的泰勒展開式的近似計算。

|x| 越接近 PI/2，誤差越大。

```
float tanTaylor(float x) {
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333);
}
```

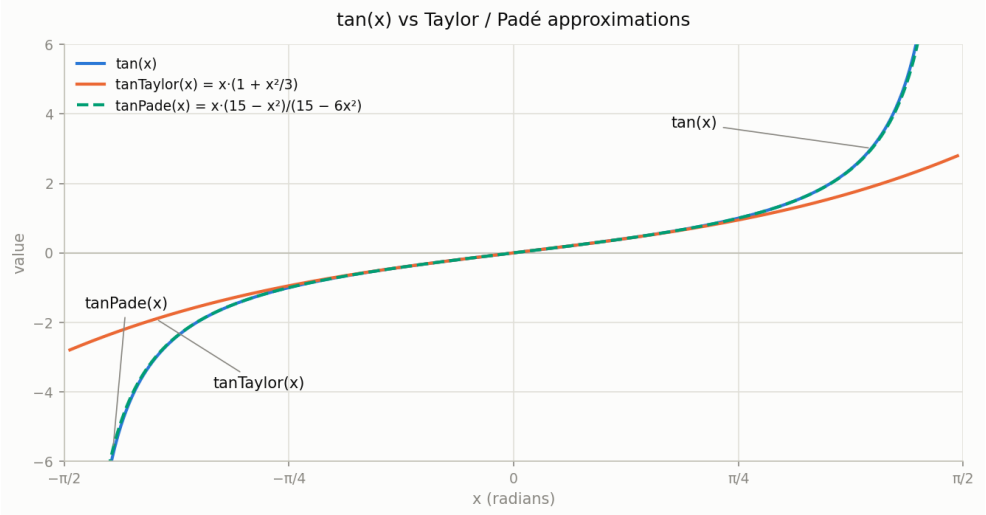
11.14.2 手法 2. Padé 近似

這是利用 tan 的 Padé 近似的方法。在 |x| < PI/4 的範圍□，會輸出與 tan(x) 非常接近的□。

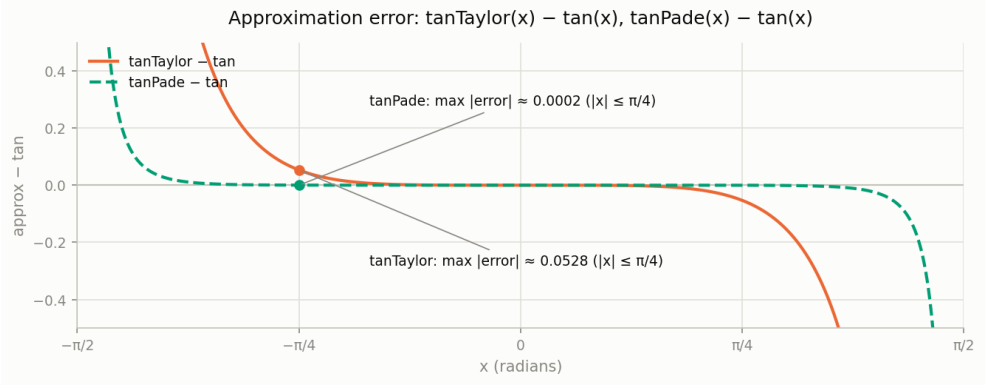
```
float tanPade(float x) {
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}
```

11.15 tan 函數近似的圖表

將前面介紹的 tanTaylor 與 tanPade 繪製成圖表。



與 tan 函數的誤差

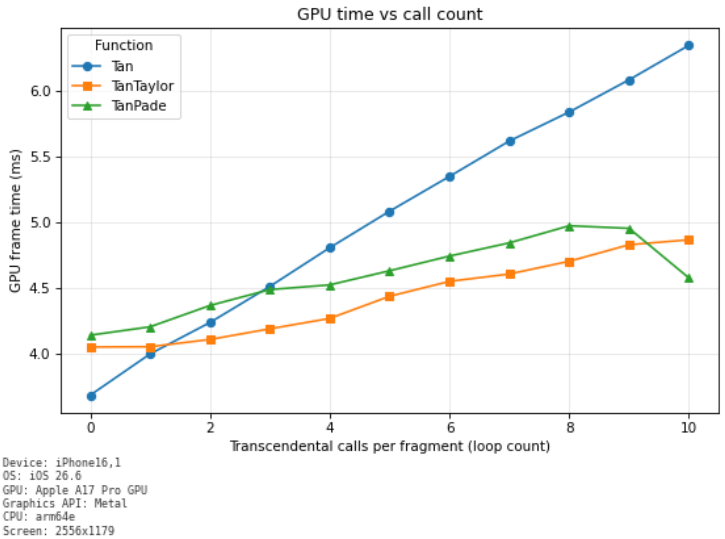


11.16 tan 函數近似的負載測量

測量了 tan 函數近似計算的負載。

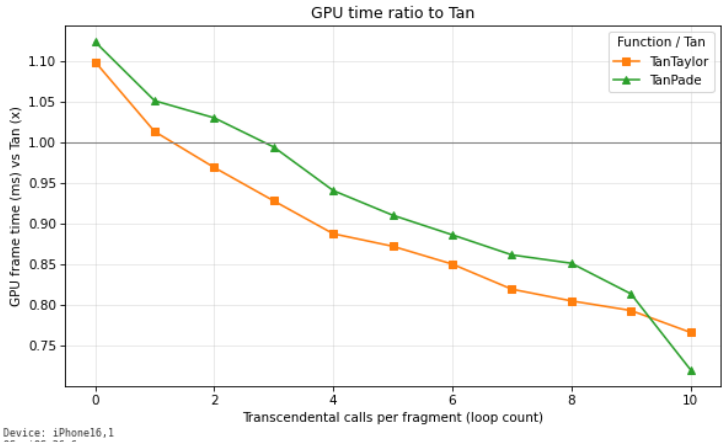
11.16.1 iPhone15 Pro

執行次數少時，暫存器壓力造成的額外開銷較為明顯，但執行次數增加後，近似計算的負載可壓得更低。

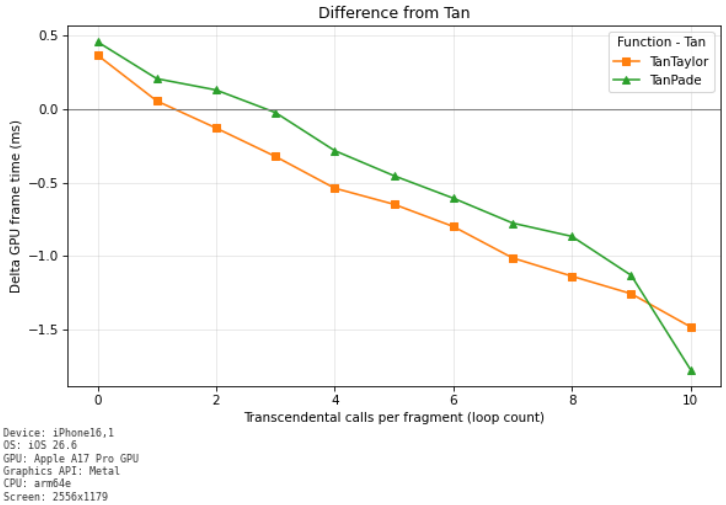


tan 函數與近似計算的比較

執行函式 10 次時，近似計算的 GPU 時間約為 tan 函數的 80%。

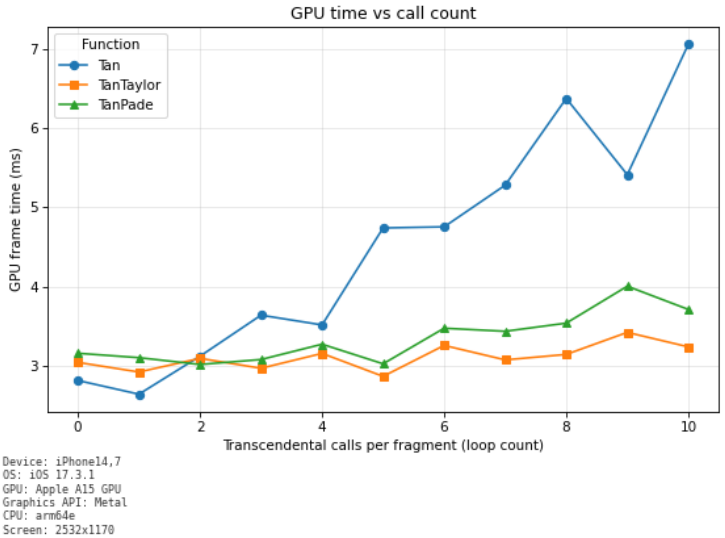


執行 10 次時，近似計算的處理時間約少 1ms。



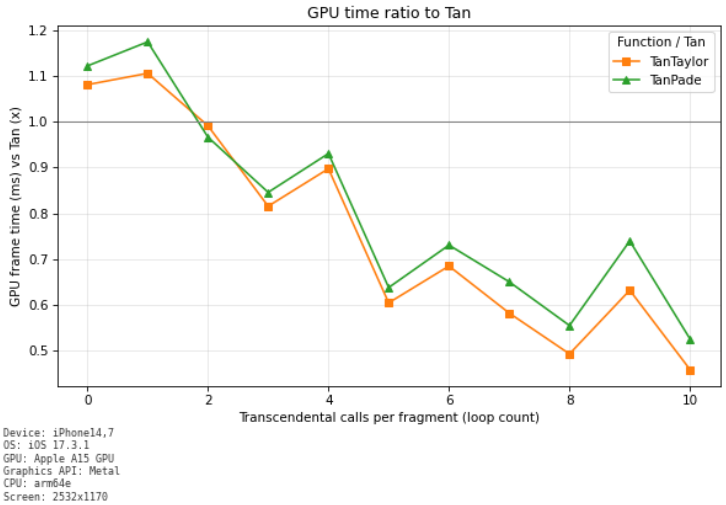
11.16.2 iPhone14

執行次數少時，可看到些微的暫存器壓力造成的額外開銷。但執行次數增加後，近似計算的負載可壓得更低。

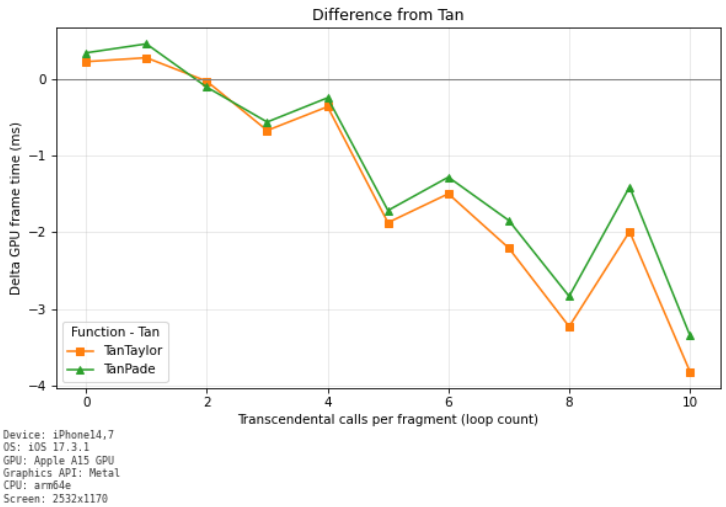


tan 函數與近似計算的比較（iPhone14）

執行函式 10 次時，近似計算的 GPU 時間約為 tan 函數的 50%。

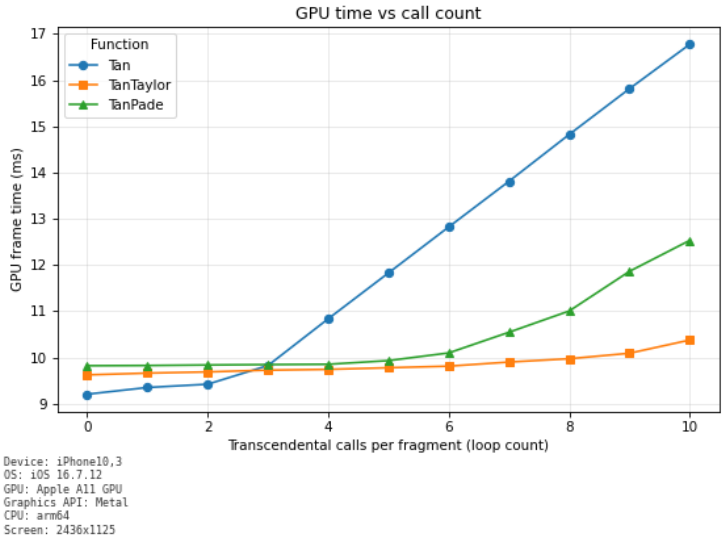


執行 10 次時，近似計算的處理時間約少 3ms



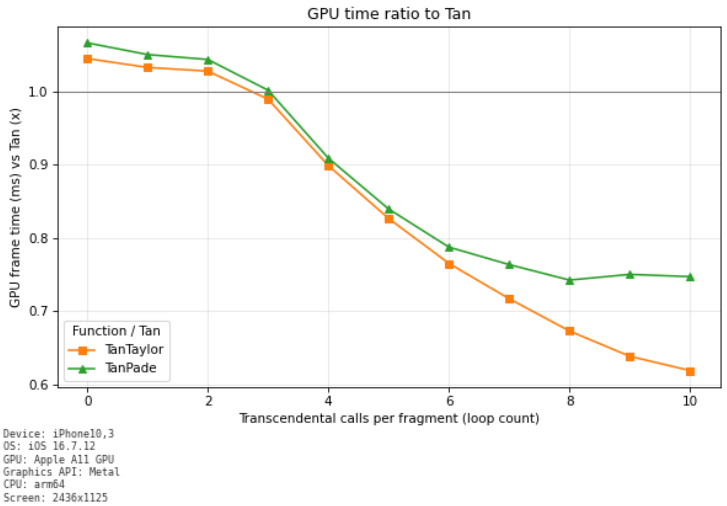
11.16.3 iPhoneX

執行次數少時，可看到些微的暫存器壓力造成的額外開銷。但執行次數增加後，近似計算的負載可壓得更低。採用泰勒展開時，處理負載幾乎沒有增加，Padé 近似的處理負載略有上升。

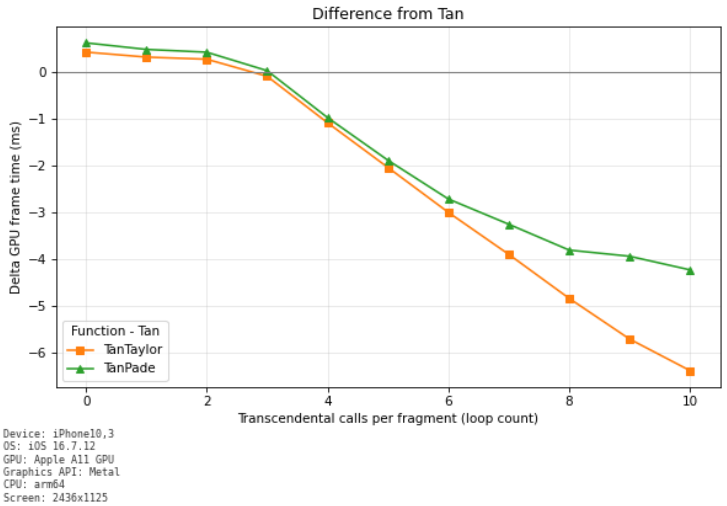


tan 函數與近似計算的比較

執行函式 10 次時，泰勒展開式的 GPU 時間約為 tan 函數的 60%。Padé 近似的 GPU 時間約為 75%。

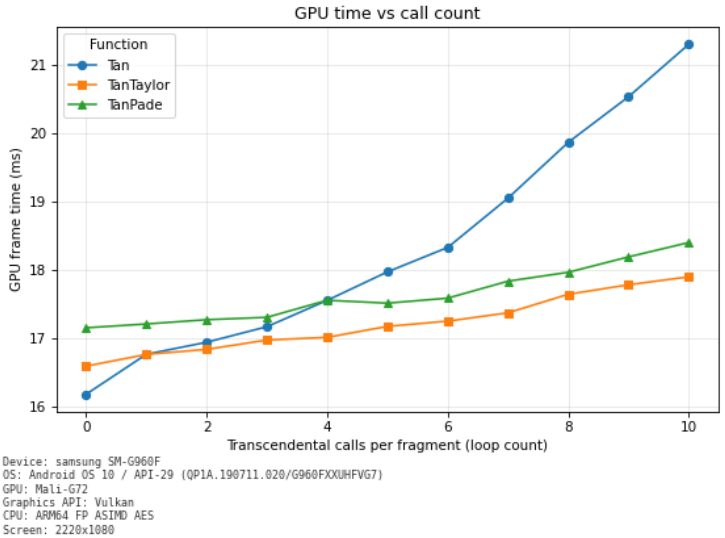


函數執行 10 次時，泰勒展開縮短約 6ms，Padé 近似縮短約 4ms。



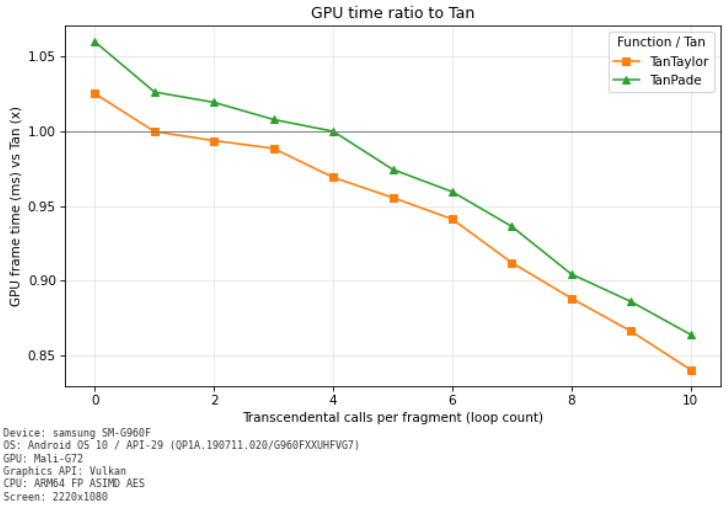
11.16.4 Galaxy S9

執行次數少時，可看到些微的暫存器壓力造成的額外開銷。但執行次數增加後，近似計算的負載可壓得更低。

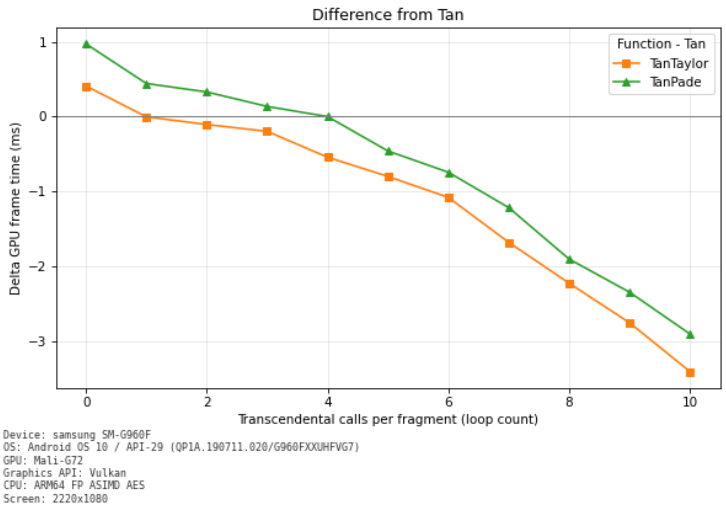


tan 函數與近似計算的比較

執行函式 10 次時，近似計算的 GPU 時間約為 tan 函數的 85%。



函數執行 10 次時，泰勒展開縮短約 3ms。



11.16.5　tan 函數近似負載的總結

裝置	GPU 負載的趨勢	備註
iPhone 15 Pro	近似計算**低約 20%**	近似計算使處理時間削減約 1ms
iPhone 14	近似計算**低約 50%**	近似計算使處理時間削減約 3ms
iPhone X	近似計算**低約 25 至 40%**	泰勒展開削減約 6ms，Padé 近似削減約 4ms
Galaxy S9	近似計算**低約 15%**	近似計算使處理時間削減約 3ms

※「GPU 負載的趨勢」是與標準 tan 函數相比，執行近似計算（泰勒展開或 Padé 近似）時 GPU 時間的大致削減率。

第12章

Unity 的 LTO 設定與 Android、iOS 上的效能比較

キム ミンヒョク Minhyuk Kim

株式會社 CyberAgent SGE 核心技術本部

所屬：株式會社 CyberAgent SGE 核心技術本部

Unity 6000.5 新增的 Android 連結時最佳化（LTO）設定，僅套用於引擎程式碼。若要將 LTO 也套用於自行撰寫的 C# 所□生的程式碼，需要□外的設定。我們嘗試了這兩種設定，並以基準測試與實際遊戲專案測量了效能、二進位檔大小與建置時間。

12.1　IL2CPP 的編譯與翻譯單元

在 Unity 的 Android 建置（IL2CPP）中，C# 會依照 清單 12.1 的流程轉換為機器碼。

▼清單 12.1　IL2CPP 建置的處理流程

```
C# 原始碼
  ↓ C# 編譯
IL 組件（.dll）
  ↓ IL2CPP 轉換大量的
 C++（.cpp）          ← 「IL2CPP □生的程式碼」
  ↓ clang 編譯（每個 .cpp 各自最佳化）
  ↓ 連結
libil2cpp.so               ← C# 由程式碼□生的最終原生二進位檔
```

clang 以翻譯單元（俗稱 TU）為單位編譯 cpp，並□生目的檔。TU 是指編譯器個別讀取並最佳化的單位。

在一般的 C++ 建置中，無法參照其他 TU 中的函式本體，因此即使是很短的函式也不

會被□嵌，call 指令可能會保留下來。IL2CPP □生的程式碼包含許多小型函式與安全檢□，容易受到這項限制的影響。例如 C# 的一行 `buf[idx]`，大致會變成 清單 12.2 這樣的 C++。

▼清單 12.2　IL2CPP □生的陣列存取程式碼範例

```
if (buf == NULL) ThrowNullReferenceException();                // null 檢□
if ((uint32_t)idx >= buf->max_length) ThrowIndexOutOfRange(); // 邊界檢□
val = buf->m_Items[idx];
```

□生的程式碼規模越大，跨 TU 的呼叫就越多，呼叫所耗費的時間總和也隨之增加。

12.2　什麼是 LTO / ThinLTO

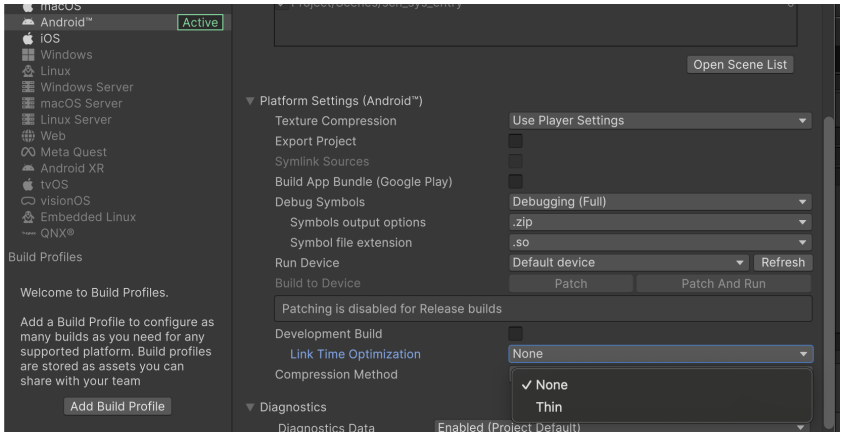
LTO（Link Time Optimization）是 clang/LLVM 的機制，在連結時進行跨 TU 的□嵌、常數傳播與無用程式碼移除。編譯時，目的檔會以 LLVM bitcode（最佳化前的中間表示）輸出。連結時再將這些檔案整合最佳化，並□生機器碼。

full LTO 會將所有中間表示合併為一個後進行最佳化。其可最佳化的範圍最廣，但中間表示的整合與最佳化集中在單一處理程序中，因此大規模建置會使用大量記憶體。此次在實際遊戲專案的建置過程中，記憶體使用量也達到 20GB，處理程序被作業系統強制終止。

ThinLTO 會在編譯時為各 TU 建立摘要。連結時再從其他 TU 匯入（import）所需的函式，並行進行最佳化。與 full LTO 相比，較容易抑制記憶體使用量與處理時間，廣泛用於大規模建置。

12.3　引擎程式碼的 ThinLTO 設定

Unity 6000.5 新增了 Link Time Optimization 開關。

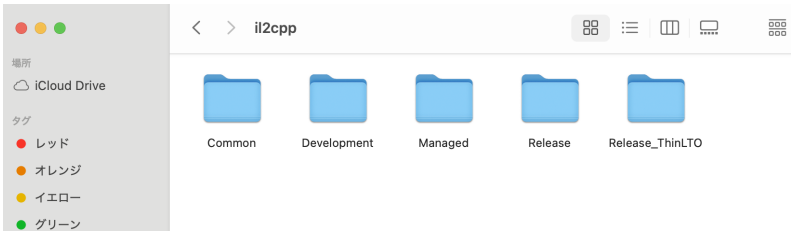


▲圖 12.1　Build Profiles > Player Settings > Android

⿴用此開關後，會選用 Unity 提供的已套用 ThinLTO 的引擎二進位檔。其對象為 Unity 引擎程式碼，不包含 IL2CPP ⿴生的程式碼（源自 C# 的程式碼）。編輯器的安裝資料夾中，附帶了一般版與 ThinLTO 版的預先建置 so 檔案。

▼清單 12.3　Android Player 附帶的 libunity.so 變體

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Libs/arm64-v8a/libunity.so          29,032,960 bytes └──
 Release_ThinLTO/Libs/arm64-v8a/libunity.so  30,042,536 bytes (+3.5%)
```



▲圖 12.2　編輯器的安裝資料夾。`Release` 與 `Release_ThinLTO` 皆有附帶，建置時會選用其中之一

12.3.1 □用條件與未滿足條件時的行為

開關生效的條件有以下 3 項。若有任何一項未滿足，不會出現錯誤或警告，`LinkTime Optimization` 會以 `None` 建置。

1. IL2CPP □本後端
2. Strip Engine Code 為停用（因為直接使用未經剝離的 prebuilt）
3. 非 Development 建置

□用 Development Build 時，開關本身不會顯示。是否已套用，可透過 清單 12.4 這 2 點確認。

▼清單 12.4　確認引擎程式碼 LTO 開關是否套用的指令

```
# ① 建置輸入中是否指定了 ThinLTO
grep -o 'LinkTimeOptimization"[: ]*"[A-Za-z]*"' Library/Bee/Player*-inputdata.json
# ② APK □的 libunity.so 是否與 Release_ThinLTO 版一致
unzip -p out.apk lib/arm64-v8a/libunity.so | md5
```

12.4　引擎程式碼 LTO 開關的效果

比較了附帶的一般版與 ThinLTO 版二進位檔，並以變更較多的子系統為中心，在實機上測量效能。

12.4.1　比較附帶二進位檔的函式大小

Unity 在編輯器的安裝資料夾中，隨附了兩種變體的含符號的 `.so`。

▼清單 12.5　Android Player 附帶的含符號 libunity.so

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Symbols/arm64-v8a/libunity.sym.so └──
 Release_ThinLTO/Symbols/arm64-v8a/libunity.sym.so
```

我們比對由 `llvm-nm --defined-only --print-size --demangle` 取得的函式名稱，並使用命名空間與類別名稱依子系統彙整（表 12.1）。

表 12.1: 引擎程式碼 LTO 開關造成的各子系統函式差異

子系統	函式數 (Release → ThinLTO)	大小有變化的函式數	大小變化率	.text 差
Particle	1,419 → 1,290	1,043	80.9%	+49.66%
Serialization/Asset	2,890 → 1,387	1,040	75.0%	-16.61%
Mesh/Geometry	1,055 → 765	483	63.1%	+6.79%
Scripting	8,924 → 7,680	4,302	56.1%	+10.27%
Gfx/Shader/Texture	5,062 → 3,935	2,044	52.1%	+24.21%
Physics	2,680 → 2,439	1,229	51.1%	+15.92%
NavMesh	739 → 582	296	51.1%	+11.87%
XR/VFX/Video/其他	3,828 → 2,483	1,230	49.6%	+13.94%
Transform/Scene/Obj	2,027 → 1,630	781	47.9%	+10.95%
Animation/Mecanim	3,226 → 2,428	1,134	46.8%	+13.18%
Box2D	2,169 → 1,562	712	46.4%	-0.43%
JobSystem/Threading	244 → 181	78	43.6%	+3.16%
UI/UIToolkit	883 → 701	293	42.0%	+1.38%
TextCore/Font/ICU	3,500 → 2,101	861	41.9%	-5.73%
(未分類)	29,193 → 21,077	7,511	37.6%	+3.37%
Renderer/Culling	2,789 → 2,199	785	35.7%	+27.58%
C++ 標準/壓縮/基礎設施	13,158 → 9,484	3,071	32.6%	-13.31%
curl/TLS/HTTP	1,809 → 1,558	318	21.0%	+1.17%
Audio/FMOD	2,996 → 2,680	496	18.5%	+2.77%
Android/JNI	1,445 → 1,186	193	16.3%	-5.52%
PhysX	6,000 → 5,997	0	0.0%	-0.05%
合計	96,036 → 73,345	27,900	38.8%	+4.41%

※「大小改變的函式數」是函式的數量。「大小變化率」表示在 Release 與 ThinLTO 兩者都存在的函式中，大小有變化的函式所占的比例。

整體函式數從 96,036 減少為 73,345，但 .text 的大小增加了 4.41%。

12.4.2 以 13 個工作負載進行的基準測試

以 表 12.1 為基礎，建立了在 1 個 APK 中放入 13 個工作負載的基準測試場景。

表 12.2: 引擎程式碼 LTO 開關的基準測試工作負載

工作負載	目標子系統	☒容
`particle`	Particle	20,000 個粒子 + noise/size/color 模組、`Simulate`
`gfx`	Gfx/Shader	在 `MaterialPropertyBlock` 中寫入 16 個屬性 × float/vector + matrix 並 `Clear`
`cull`	Renderer/Culling	`CalculateFrustumPlanes` + 4,096 個 AABB 的 `TestPlanesAABB`
`physbind`	Physics ☒定	`Rigidbody` / `Collider` 屬性的往返存取
`anim`	Animation	legacy `Animation` 48 個 Rig × 8 根骨骼 × 10 條曲線的 `Sample()`
`json`	Serialization	`JsonUtility` 的往返
`instantiate`	Serialization	`Instantiate` → `DestroyImmediate`（原生的 clone/transfer 路徑）
`xform`	Transform/Scene	改寫具階層的 Transform
`bind`	Scripting ☒定	大量呼叫極小的☒定
`mesh`	Mesh/Geometry	重建 96×96 的程序化網格
`phys2d`	Box2D	2D 剛體的 `Simulate`
`text`	TextCore/Font	`TextGenerator.Populate`
`physics`	PhysX	3D 剛體的 `Simulate`

※ 在 表 12.1 的子系統中，難以從 C# ☒生每個樣本約 100ms 的穩定合成負載者，已排除在測量對象之外。這些項目未評估對效能的影響。

12.4.3 各工作負載的測量結果

我們在 Pixel 8a（Tensor G3）上，針對僅 `libunity.so` 不同的 APK，測量前面介紹的工作負載。依 Release / ThinLTO / ThinLTO / Release 的 ABBA 順序重複 3 次，共執行 12 個工作階段。每個工作階段都重新安裝應用程式，並在其間設定 90 秒的冷卻時間。

數☒為每次操作所需的時間（ns/op）。操作的單位因工作負載而異，因此請在同一列☒比較☒對☒。

表 12.3: 引擎程式碼 LTO 開關的各工作負載基準測試結果

工作負載	Release (ns/op)	ThinLTO (ns/op)	差
`gfx`	990.46	852.49	-13.93%
`cull`	96,520	87,499	-9.35%
`text`	275,043	265,123	-3.61%
`instantiate`	14,620	14,206	-2.83%
`xform`	6,071.8	6,008.1	-1.05%
`phys2d`	222,269	220,327	-0.87%
`mesh`	2,340,210	2,334,143	-0.26%
`json`	15,864.8	15,934.4	+0.44%
`bind`	140.80	141.47	+0.47%
`physbind`	607.28	610.74	+0.57%
`particle`	265,635	267,335	+0.64%
`anim`	28,100	28,563	+1.65%
`physics`	484,641	482,796	-0.38%
[合成: 幾何平均]	21,918	21,380	-2.46%

※ 由於各列單位不同，合成☒僅作為比率才有意義。

改善幅度較大的 `gfx` 與 `cull`，是從 C# 對每個樣本呼叫數百萬次小型函式的處理。`text` 的 `TextGenerator.Populate` ☒部也會執行以字形為單位的迴圈。在單次處理小、呼叫次數多的路徑上，跨 TU ☒嵌的效果表現得特別明顯。

12.4.4 導入時的效能、大小與記憶體比較

此次測量中，在部分繫結呼叫較多的路徑上，CPU 時間有所縮短。不過，在實際遊戲專案中的改善幅度，取決於影格☒這類呼叫所占的比例。

導入時必須付出的成本整理於 表 12.4。

表 12.4: 採用引擎程式碼 LTO 開關時的成本

類別	實測差異
因 Strip Engine Code OFF 而增加的大小	APK +5.67MB / 安裝 +20.3MB
因 Strip Engine Code OFF 而增加的記憶體	RSS +18.0MB / 其中不易回收的區域 +8.0MB
engine ThinLTO 本體	APK 大小 +398KB(30,621,595 → 31,019,967)/ 記憶體未測量

※ 安裝大小的增量，是 APK 的增量與裝置上展開的 `.so` 的增量（arm64 的 `.so` 合計為 +14.65MB）的總和。

Unity 公開了針對同一開關，在 5 個專案中各測量 10 次冷□動的結果。據報告，CPU 時間從 13.88ms 縮短為 13.36ms（-3.7%），首次顯示的時間從 247.96ms 縮短為 229.90ms（-7.3%），APK 大小為 +3.3%，對建置時間的影響輕微 [*1]。

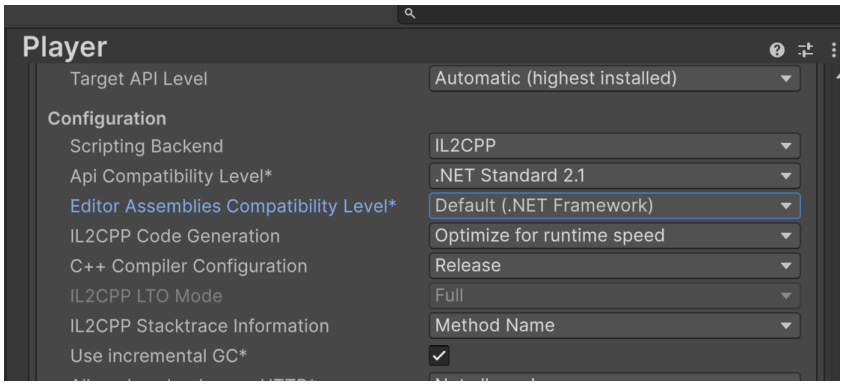
CPU 的改善幅度與此次測量處於相近水準。不過，公開□中的 APK 大小 +3.3%，是停用 Strip Engine Code 的組態之間的比較，因此不包含停用所伴隨的安裝大小 +20.3MB 與記憶體 +8.0MB 的增加。

[*1] Unity Android 平台團隊「Boosting Android performance with LTO optimizations in Unity 6.5」`https://discussions.unity.com/t/boosting-android-performance-with-lto-optimizations-in-unity-6-5/1720008`。此後本章所稱的「Unity 公開□」，指的是這篇文章中的數字。

12.5 對 `libil2cpp.so` 套用 LTO 的方法

若要將 LTO 套用於包含由 C# 轉換而來之程式碼的 `libil2cpp.so`，有使用 Unity 的 Master 組態，以及直接傳遞 clang 旗標兩種方法。

12.5.1 Master 組態與 IL2CPP LTO Mode 的設定



▲圖 12.3　IL2CPP LTO Mode 與 C++ Compiler Configuration 的設定畫面

將 Player Settings > Configuration 的 C++ Compiler Configuration 設為 Master，即可對 `libil2cpp.so` 啟用 LTO。`IL2CPP LTO Mode` 是用來選擇此時所用的 Thin / Full 模式的項目。

根據 Unity 的官方文章，IL2CPP LTO Mode 自 6000.5 起新增。在 6000.5 之前的 Android 上，Release 與 Master 在 LTO 的有無上沒有差異。不過在 6000.5 的 Android 平台上無法選擇開關，固定為 Thin；自 6000.6 起可在 Master 組態中選擇 Thin / Full。即使到 6000.7，也沒有在維持 Release 組態的情況下僅啟用 LTO 的項目。

與 Player Settings 的 C++ Compiler Configuration 對應的旗標，可從 Bee 建置圖的 Il2CppCompilerConfiguration 確認。

表 12.5: Unity 各 C++ Compiler Configuration 的編譯器旗標

版本	Release	Master
Android 6000.3 (arm64)	-Os（以大小為優先）、無 LTO	-Os、無 LTO（相同）
Android 6000.5 (arm64)	-Os	-O2（在速度與程式碼大小之間取得平衡的最佳化）+ ThinLTO
iOS 6000.3 / 6000.5	-O3（以程式碼大小可能增加為代價，更積極追求速度的最佳化）、LTO OFF	-O3、LTO OFF（相同）

12.5.2 在 Release 組態中加入 -flto=thin

如前所述，切換為 Master 組態是最快獲得 LTO 效果的途徑，但在□用 LTO 的同時，最佳化層級也會從 -Os 變為 -O2（表 12.5），缺點是大小與建置時間會增加。若維持 Release 組態而僅套用 ThinLTO，除了可以分開測量 LTO 的效果，還能抑制大小與建置時間的增加。若要對自行撰寫的 C# 所□生的 libil2cpp.so 套用 ThinLTO，可考慮如 清單 12.6 這樣直接傳遞 clang 旗標的方法。

▼清單 12.6　在 Release 組態中□用 ThinLTO 的建置前處理

```
// 暫存既有的額外引數 → 附加 → 務必還原
var saved = PlayerSettings.GetAdditionalIl2CppArgs();
try
{
    var extra = "--compiler-flags=-flto=thin --linker-flags=-flto=thin";
    PlayerSettings.SetAdditionalIl2CppArgs(
        string.IsNullOrEmpty(saved) ? extra : saved + " " + extra);
    BuildPipeline.BuildPlayer(options);
}
finally
{
    PlayerSettings.SetAdditionalIl2CppArgs(saved);
}
```

分 2 個階段確認是否已套用（清單 12.7）。

第 1 個是旗標是否已傳遞到建置指令。Unity 的建置系統（Bee）會將所執行的編譯與連結指令記錄在 Library/Bee/Player*.dag.json 中。使用 -flto=thin 搜尋此檔案，確認 IL2CPP □生程式碼的編譯與 libil2cpp.so 的連結兩者是否都包含該旗標。

第 2 個是□生的目的檔是否為 LLVM bitcode。□用 LTO 後，編譯結果的 .o 不是機器

碼而是 bitcode。從第 1 步找到的編譯指令的輸出檔案中選一個，顯示其前 4 個位元組。

▼清單 12.7　確認 IL2CPP ThinLTO 是否套用的指令

```
# 1. 編譯與連結的指令中是否有 -flto=thin
rg -n -- '-flto=thin' Library/Bee/Player*.dag.json
# 2. □生的目的檔的前 4 個位元組（指定 1. 中找到的 .）o
xxd -l 4 Library/Bee/artifacts/Android/.../GeneratedCode.o
```

若為 bitcode，會顯示 `4243 c0de`（`BC..`）；若為一般機器碼的目的檔，則會顯示 `7f45 4c46`（`.ELF`）。若隨意選擇資料夾□的 `.o`，可能會挑到不屬於 LTO 對象的第三方函式庫的目的檔。第 2 個指令請務必指定第 1 個搜尋結果中出現的檔案。

12.5.3　基準測試的設計

合成程式碼的□生

此基準測試以一般編譯無法最佳化的跨 TU 呼叫為主要測量對象。由於 asmdef 的邊界與 C++ 的 TU 邊界並非相同的概念，因此針對 IL2CPP 轉換後□生的 C++ 與Bee 的編譯圖，也確認了呼叫端與被呼叫端屬於不同的 TU。程式碼□生器所□生的□容如下。

- asmdef 組件 16 個 × 類別 24 個 × 方法 8 個 = 3,072 個方法
- 呼叫鏈繞行一圈會通過 16 個組件邊界，所有方法還會額外呼叫共用組件的輔助函式（`Core.Mix` / `Core.Idx` / 結構體運算）
- 基準測試會走訪所有類別，因此不會被 managed 剝離移除
- 為每個方法變更常數，以防止被 ICF 合併

所□生的 3,072 個方法的範例顯示於 清單 12.8。

▼清單 12.8　合成基準測試所□生的方法範例

```
public static int M2(int sel, int x, int[] buf)
{
    x = Core.Mix(x, 655663086);       // 跨邊界呼叫共用組件
    x += buf[Core.Idx(x, 7)];         // 附邊界檢□的陣列存取（Idx 以 & 1023 遮罩）
    return M3(sel, x, buf);           // 小型函式鏈
}
```

兩個工作負載

表 12.6: IL2CPP ThinLTO 的合成基準測試工作負載

工作負載	測量□容
direct	跨組件的靜態呼叫鏈（LTO 應最為有效的形式）
iface	透過介面分派執行同類運算（LTO 難以將呼叫目標直接化的形式）

兩者都只在 libil2cpp.so □完成。各組態所□生的二進位檔，每次都以從 APK 擷取的 libil2cpp.so 的 md5 確認。執行時會調整反覆次數，使每個樣本約為 100ms。由於每次執行的次數不同，會正規化為每 1 pass 的時間來比較。

在 iface 中，每個組件各□生 4 個實作 IGenOp 的類別（共 64 個），基準測試透過介面依序呼叫（清單 12.9）。方法本體中呼叫與 direct 相同的 Core.Mix / Core.Idx，但實際呼叫□個類別的 Apply 要到執行時才決定。

▼清單 12.9　iface 工作負載所□生的類別與呼叫迴圈

```
sealed class Op0 : IGenOp
{
    int m_State = 20612;
    public int Apply(int x, int[] buf)
    {
        x = Core.Mix(x, 347275849);              // 跨邊界呼叫共用組件
        x += buf[Core.Idx(x ^ m_State, 7)];    // 與 direct 相同類型的陣列存取
        m_State = (m_State + x) | 1;
        return x;
    }
}

// 基準測試端：透過介面輪流呼叫 64 個實作
for (int i = 0; i < ops.Count; i++)
    a += ops[i].Apply(a ^ p, buf);        // 呼叫目標於執行時決定
```

12.5.4　CPU 實測（Pixel 8a）

表 12.7: Pixel 8a 上 IL2CPP ThinLTO 的 CPU 實測結果

組態	direct (us/pass)	iface (us/pass)
off(Release, -Os)	19.88	0.404
thin（手動 ThinLTO）	15.54 (-21.8%)	0.375 (-7.3%)

透過手動 ThinLTO，direct 縮短了 21.8%，iface 縮短了 7.3%。

direct 的函式呼叫與指令數

針對與 §5.3.1 相同的呼叫結構，比較了有無 LTO 時的反組譯結果。

表 12.8: direct 工作負載的反組譯比較

C000_M2	無 LTO	ThinLTO
指令數	15	11
函式呼叫（bl）	2	0
堆疊框	有	無
callee-saved 暫存器的退避	x19、x20、x29、x30	無
記憶體存取	5 次	1 次

Core.Mix 與 Core.Idx 的本體被展開到呼叫端，不再需要 2 條 bl 指令，以及堆疊與暫存器的退避處理。跨越兩個函式的運算式也會一併最佳化。direct 的 21.8% 縮短，是這項變化累積於 3,072 個方法的結果。

iface 改善較小的原因

IL2CPP □生的 interface 呼叫，概略而言是 清單 12.10 的形式。

▼清單 12.10　IL2CPP □生的 interface 呼叫程式碼概略

```
const VirtualInvokeData& invokeData =
    il2cpp_codegen_get_interface_invoke_data(slot, obj, declaringInterface);
return ((Func)invokeData.methodPtr)(obj, p1, p2, invokeData.method);
```

呼叫目標是在執行時取得的 `invokeData.methodPtr`，因此無法像一般 C++ 的 vtable 呼叫那樣直接化。不過，方法本體中呼叫其他 TU 輔助函式的部分可以□嵌，因此 iface 也縮短了 7.3%。

若 interface / virtual 呼叫耗時較多，也可考慮 `sealed`、以具體型別接收，以及透過泛型進行靜態分派。ThinLTO 較容易改善的，是靜態呼叫其他 TU 之小型函式的部分。

12.5.5　大小與建置時間

表 12.9: IL2CPP ThinLTO 的大小與建置時間

組態	libil2cpp.so (MB)	同 .text	冷建置
off	37.36	基準	62s
thin	38.16 (+2.2%)	+2.3%	71s (+15%)

在此組態中，受□嵌展開影響，`libil2cpp.so` 增加了 2.2%。LTO 有時也會移除無用程式碼，因此大小變化的方向取決於程式碼庫與最佳化□容。冷建置從 62s 增加為 71s，增加 15%。ThinLTO 會在連結時進行跨 TU 的最佳化與機器碼□生，因此連結時間相應延長。

12.6　Master 組態□用的最佳化

在 Master 組態中，LTO □用的同時最佳化層級也會改變（表 12.5）。為了找出 CPU 時間的縮短來自□個設定，我們將兩者分開測量。為了分別探討最佳化等級（`-Os` → `-O2`）與 LTO 的影響，實測了 {-Os, -O2} × {無 LTO, ThinLTO} 的 2×2 組合（Android 6000.5 / arm64）。

表 12.10: 最佳化等級與 ThinLTO 的 CPU 實測比較

direct (us/pass)	無 LTO	ThinLTO
-Os（Release）	19.64（基準）	15.54 (-20.9%)
-O2(Master)	18.47 (-5.9%)	15.54 (-20.9%)

表 12.11: 最佳化等級與 ThinLTO 的 libil2cpp.so 大小比較

libil2cpp.so	無 LTO	ThinLTO
-Os	37.36MB	38.16MB (+2.2%)
-O2	39.14MB	44.14MB (+12.8%)

CPU 時間的縮短率，單獨 `-O2` 為 5.9%，包含 ThinLTO 的組態為 20.9%。也就是□，在此基準測試中，Master 組態的大部分改善來自 ThinLTO。Release + 手動 ThinLTO 與 Master 的 CPU 時間同為 15.54us，但最終的 `libil2cpp.so` 大小分別為 38.16MB 與 44.14MB，Release + 手動 ThinLTO 約小 13.5%。

表 12.12: Release + ThinLTO 與 Master 的冷建置時間

組態	冷建置
Release（`-Os`、無 LTO）	62s
Release + ThinLTO	71s (+15%)
Master (`-O2` + ThinLTO)	86s (+39%)

冷建置分別為 71s 與 86s，僅在 Release 加入 ThinLTO 的組態短了 15 秒。在 Unity 公開的測量中，Master 組態（`libil2cpp.so` 端）同樣是 CPU -2.3%、首次顯示時間 -5.7%，而 `libil2cpp.so` 大小為 +18.7%、建置時間為 +30～35%。CPU 的改善幅度在 Unity 的數據中為 -2.3%，而本章的基準測試為 -20.9%。不過，Unity 針對的是實際專案主執行緒的整體，而本章只針對反覆呼叫小型函式的路徑，因此無法直接比較這些數字。□一方面，大小與建置時間在本章的 Master 中分別為 +18.2%、+38.7%，與 Unity 的公開□相近。

12.7　在實際遊戲專案中的驗證

我們也在規模比基準測試更大的 2 個實際遊戲專案中嘗試了 ThinLTO。Unity 版本、組件數、原始碼量與二進位檔大小列於 表 12.13。

表 12.13: 實際遊戲專案的 Unity 版本與程式碼規模

項目	專案 A	專案 B
Unity 版本	6000.5.3f1	6000.3.15f1
納入播放器的組件數	158	285
被編譯的 `.cs`	6,969 個檔案	16,700 個檔案
同上行數	約 138 萬行	約 319 萬行
其中自家程式碼	1,943 個檔案 / 約 35 萬行	8,606 個檔案 / 約 154 萬行
`libil2cpp.so`（未壓縮）	189.0MB	294.3MB

12.7.1　Release + ThinLTO 的大小與套用確認

表 12.14: 專案 A 的大小與建置時間

項目	baseline	ThinLTO	差異
libil2cpp.so（未壓縮）	189.01MB	194.70MB	+5.69MB (+3.0%)
同 `.text`	8.54MB	8.98MB	+0.44MB (+5.1%)
APK	165.85MB	166.94MB	+1.08MB (+0.65%)
Player 建置部分	約 396s	約 433s	約 +9%

`.text` +5.1% 大於基準測試（+2.3%），符合「asmdef 分得越細，跨模組的□嵌就越多」的預期。即使是 189MB 的 libil2cpp.so，也能在本機上完成 ThinLTO 的連結。

在專案 B 中，使用 `llvm-readelf -S` 比較了 `libil2cpp.so` 各區段的大小（表 12.15）。

表 12.15: 專案 B 的 libil2cpp.so 區段差異

項目	baseline	ThinLTO	差異
libil2cpp.so（整個檔案）	294.30MB	304.12MB	+3.34%
`il2cpp`	145.85MB	162.22MB	+11.23%
`.text`	10.69MB	11.14MB	+4.18%
`.rela.dyn`	50.28MB	45.01MB	-10.48%
`.got`	1.76MB	1.4KB	-99.92%
`.gcc_except_table`	3.70MB	3.62MB	-2.32%

`.text` 的增加率 4.18%，介於專案 A（5.1%）與基準測試（2.3%）之間。`.got` 從 1.76MB 減少為 1.4KB。這顯示 LTO 解析了跨模組的參照，經由 GOT 的間接參照被轉換為直接參照。包含動態重定位項目的 `.rela.dyn` 減少 10.5%，也與這項變化對應。

12.7.2 實際遊戲專案中的 Master 與 full LTO（專案 A）

我們在實際遊戲專案的規模下，嘗試了 §6 的 Master 組態與 full LTO 的比較。

表 12.16: 專案 A 的建置組態比較

專案 A	baseline	Release + thin	Master	full LTO
libil2cpp.so	189.0MB	194.7MB (+3.0%)	222.2MB (+17.6%)	–
建置	453s	465s	537s（9 分）	2,903s（48 分）後失敗
連結器路徑的尖峰 RSS	–	–	12.9GB	20GB → `Killed: 9`（OOM）

full LTO 在建置 48 分鐘後，連結期間的記憶體使用量達到 20GB，被作業系統強制終止。在 §5.3 的基準測試中，full LTO 的 CPU 時間也與 ThinLTO 相當，冷建置卻從 71s 變為 247s，約為 3.5 倍。此專案無法完成建置，ThinLTO 更為實用。

Master 的 `libil2cpp.so` 大小比 baseline 增加約 33MB，但 CPU 效能與 Release + thin 相當。Release + thin 的大小增加僅約 5.7MB，因此在此專案中也能以更小的二進位檔取得與 Master 同等的效能。

12.7.3 實際遊戲專案中的 CPU 週期（專案 B）

在遊戲外（outgame）的穩定畫面上，加總 Pixel 8a 所有執行緒的 CPU 週期進行測量。ThinLTO 與 baseline 交替各測量 3 個工作階段，每個工作階段的□為進行 6 次 20 秒測量的平均□。

表 12.17: 專案 B 各工作階段的 CPU 週期

工作階段	總週期 (Gcyc)	UnityMain (Gcyc)
ThinLTO #1	44.88	27.48
ThinLTO #2	45.18	27.36
ThinLTO #3	45.54	27.60
baseline #1	46.06	29.05
baseline #2	45.71	27.89
baseline #3	46.15	28.23
平均（ThinLTO vs baseline）	45.20 vs 45.97 → -1.68%	27.48 vs 28.39 → -3.21%

12.7.4 與 Unity 公開□的比較

將 Unity 公開的 Master 組態（`libil2cpp.so` 端）測量□，與專案 B 的結果比較。§6 的基準測試結果也一併列出。

表 12.18: Unity 公開□與本章實測結果的比較

指標	Unity 公開(6 個專案・Master)	基準測試	專案 B
CPU	主執行緒 -2.3%	`direct` -21.8%	UnityMain -3.21% / 總週期 -1.68%
`libil2cpp.so`	+18.7%	+2.2% (Release+thin)/ +18.2%(Master)	+3.0%
建置時間	+30～35%	+15%（Release+thin）/ +38.7%（Master）	+9～15%

Unity 的 CPU 時間縮短率 -2.3% 與專案 B 的 UnityMain -3.21%，都只有數個百分點的改善。與基準測試 -21.8% 的差距，可由實際專案的測量中也包含 LTO 對象以外的處理來解釋。在 Unity 測量的 6 個專案中也可看到相同的傾向。

大小與建置時間會因組態而有差異。Unity 的公開□在 Master 組態下分別為 +18.7% / +30～35%。本章測得的 Master 組態 +18.2%（44.14MB / 37.36MB）/ +38.7%（86s / 62s）也是相近的□。作為基準的 `libil2cpp.so` 大小同樣相近，為 37.26MB / 37.36MB。

僅在 Release 加入 ThinLTO 時，能以大小 +2～3%、建置時間 +9～15% 獲得同等的 CPU 效能。從 §6 的 2×2 比較來看，Master 組態中大小與建置時間的增加，主要可歸因於對 `-O2` 的變更。

12.7.5 最佳化的對象範圍與實際遊戲專案中的改善幅度

專案 B 的改善集中在主執行緒。UnityMain 為 -3.21%，而整體非 UnityMain 為 +0.8%，在工作階段間 4% 的漂移範圍□。

在二進位檔方面，`libil2cpp.so` 以外的 `.so` 也幾乎沒有變化。2 個原生外掛的 md5 完全一致，其餘 1 個 `.text` 的差異為 +0.008%，在建置非確定性的範圍□。本次 `--compiler-flags` 所做的最佳化並未涵蓋這些外掛。

UnityMain 的 -3.2% 小於基準測試的 -22%。在此畫面中，表情、□眼、頭髮與服裝的擺動、IK 等是以 Job 的方式執行。這些包含在 `libil2cpp.so` 之外的原生二進位檔中，不屬於本次旗標的最佳化對象。

排除對象外的處理後，僅 IL2CPP □生程式碼的改善幅度，應大於 UnityMain 整體的 -3.21%。不過，要求得準確的□，必須使用 Profiler 分離對象□外的處理時間。Job 也會因排程而在主執行緒上執行，因此 UnityMain 的測量□中也包含對象外的處理。

12.8 iOS 的 ThinLTO 套用

在專案 B 的 iOS 版上也套用了 ThinLTO。在驗證用基準測試中確認了效果，但在實際專案中，並未獲得與建置時間增加相稱的改善。

12.8.1 iOS 上 LTO 的結構

表 12.19: Android 與 iOS 的 LTO 結構比較

面向	Android	iOS
il2cpp 的輸出	`libil2cpp.so`	`libGameAssembly.a`
最終連結	il2cpp 呼叫的 LLD	Xcode 建立 `UnityFramework` 時的 ld64
傳遞的引數	`--compiler-flags=-flto=thin --linker-flags=-flto=thin`	僅 `--compiler-flags=-flto=thin`
建置時間增加之處	Unity 的建置步驟	`Build ipa`（xcodebuild）步驟

在 iOS 上，il2cpp 只會建立封存檔，因此不傳遞 `--linker-flags`。LTO 的程式碼□生，是在 Xcode 端讀取含此 bitcode 的封存檔進行連結時執行。

□生程式碼的編譯由 il2cpp 負責，因此旗標必須指定在 `AdditionalIl2CppArgs` 中。即使變更 Xcode 的 `LLVM_LTO = YES_THIN`，也不會套用到□生的程式碼。

引擎程式碼以預先建置完成的 `libiPhone-lib.a` 形式提供，不含 bitcode，因此不屬於 LTO 的對象。也沒有像 Android 的引擎程式碼 LTO 開關（§3）那樣，替換為已套用 LTO 的引擎的設定。

12.8.2 iPhone 13 的驗證用基準測試

首先，我們將與 §5.3 相同的驗證用基準測試移植到 iOS，確認對□生程式碼的 ThinLTO 在 iOS 上是否也能運作。`direct` 將分散於 16 個 asmdef 的 3,072 個小型方法是 ThinLTO 最容易發揮效果的組成。因此，這裡的改善率並非一般專案能直接得到的□，而應視為條件相符時接近上限的□。

裝置為 iPhone 13（`iPhone14,5` / iOS 18.7.8）。使用 Unity 6000.5.3f1、IL2CPP Release、OptimizeSpeed。baseline 與 ThinLTO 的差異僅在於有無 `--compiler-flags=-flto=thin`。Xcode 為 `ReleaseForRunning`，開發簽署為 `get-task-allow=true`，Unity 的 Development Build 為停用。

每個工作階段都進行解除安裝、重新安裝與□動，在 5 次暖機後記錄 30 個樣本。測量順序為 baseline / ThinLTO / ThinLTO / baseline 共 3 個區塊，合計 12 個工作階段。所有工作階段的熱狀態皆為 Nominal。由於處理次數會依工作階段自動調整，`direct` 與 `iface` 正規化為每 1 pass，Physics 則正規化為每 1 step。

表 12.20: iPhone 13 驗證用基準測試結果

工作負載	baseline	ThinLTO	差異
`direct`	21,231.31ns/pass	16,184.31ns/pass	-23.77%
`iface`	366.96ns/pass	370.39ns/pass	+0.93%
Physics（對象外的對照）	147.82us/step	147.36us/step	-0.31%（分布重疊）

`direct` 在 3 個區塊中全部為 -23.6～-23.9%，baseline 與 ThinLTO 的所有工作階段完全分離。與 Android 的 -21.8%（§5.4）也相近，可確認針對□生程式碼的 ThinLTO 在 iOS 上也確實發揮作用。□一方面，`iface` 略有惡化，對象外的 Physics 則沒有出現差異。ThinLTO 的效果偏重於靜態小型函式呼叫跨 TU 的路徑。同一基準測試的二進位檔大小顯示於 表 12.21。

表 12.21: iPhone 13 驗證用基準測試的二進位檔大小

項目	baseline	ThinLTO	差異
UnityFramework	98.76MB	107.52MB	+8.76MB(+8.87%)
`__text`	20.09MB	20.46MB	+0.37MB(+1.83%)
`il2cpp`	22.49MB	30.78MB	+8.30MB(+36.89%)

12.8.3 專案 B 的結果與在 iOS 上暫緩採用的原因

接著，我們對專案 B 的 iOS 版套用 ThinLTO。UnityFramework 的大小列於 表 12.22，建置時間列於 表 12.23。

表 12.22: iOS UnityFramework 的區段差異

區段	baseline	ThinLTO	差異
UnityFramework（整個檔案）	316.53MB	393.47MB	+24.31%
`__TEXT`（區段）	261.83MB	338.53MB	+29.29%
`il2cpp`	167.02MB	240.26MB	+43.85%
`__text`	57.99MB	61.70MB	+6.41%
`__gcc_except_tab`	6.85MB	6.63MB	-3.16%
`__unwind_info`	3.44MB	3.38MB	-1.75%
`__got`	18.8KB	18.8KB	±0
ipa（整個封存檔）	233.83MB	246.93MB	+5.60%

表 12.23: iOS 的建置時間

步驟	baseline	ThinLTO
`Build ipa`(xcodebuild)	357s(5m57s)	3,094s(51m34s)

UnityFramework 增加 +24.31%，ipa 增加 +5.60%。建置時間增加約 9 倍，即 46 分鐘。增加的部分大多來自 Xcode 端的連結與 `dsymutil`，比 Android 的 +9～15% 長得多。即使是將 `GCC_OPTIMIZATION_LEVEL` 降為 `-Os` 的組態（`-Os` + thin），也需要 53m51s，無法縮短。

在專案 B 的穩定畫面中，`cpu/wall` 的簡單平均是 ThinLTO 略微惡化 +1.08%，即使熱狀態一致的組別也僅有 -1.45% 的改善。baseline 與 ThinLTO 的分布重疊，無法分離出明顯的 CPU 改善。記憶體差異 +0.40% 也在工作階段□的變異以下。驗證用基準測試的 -23.77% 是彙集對 ThinLTO 有利之呼叫時的□，並不適用於整個實際專案。

在實際遊戲專案中未能確認明確的改善，且冷建置或 CI 每次都需要額外約 46 分鐘，因此在此次驗證中，不採用對 iOS 手動套用 ThinLTO。

12.9 導入自己的遊戲專案時

在 Android 上，建議先在 Release 組態中加入 `-flto=thin`。在此次基準測試中，能以更小的二進位檔與更短的建置時間，取得與 Master 同等的 CPU 效能。在實際遊戲專案的穩定畫面上，總週期減少 1.68%，UnityMain 減少 3.21%（§7.3）。

引擎程式碼 LTO 開關（§3）因停用 Strip Engine Code 所造成的大小與記憶體增加幅度大，故在此次的組態中不建議使用。

對 iOS 手動套用 ThinLTO，其改善與建置時間的增加不成比例，因此暫緩採用。

採用前，請以 Release 建置的 Profiler 擷取檢□ PlayerLoop □的□本執行時間。手動旗標的對象在 `libil2cpp.so` □，渲染執行緒、GPU、引擎□部與 GC 不在其範圍□，以其他 AOT 管線編譯的 Job 也不包含在□（§7.5）。對於大量使用 DOTS/Job 的專案，也請確認其處理時間。

例如在 33ms 的影格中□本執行占 8ms，即使該部分縮短約 20%，整個影格也只縮短約 1.6ms。僅在□本執行占整個影格的比例大、仍有改善空間，或必須維持高影格率的情況下，才建議考慮導入。

若建置時間的增加影響開發中的作業，也可以不套用於日常開發建置，而只套用於 staging 之後的建置。旗標是在建置前處理中加入的（清單 12.6），因此可依建置種類切換。

設定後，請依 §3.1 與 §5.2 的步驟確認是否套用。確認處為 inputdata、二進位檔的 md5、編譯指令的 `-flto`，以及目的檔的 bitcode。即使未滿足 LTO 的條件，建置有時仍會成功，因此無法僅憑建置結果判斷是否已套用。

第13章

iOS 的記憶體上限與虛擬位址空間

キム ミンヒョク Minhyuk Kim
株式會社 CyberAgent SGE 核心技術本部

所屬：株式會社 CyberAgent SGE 核心技術本部

遊戲可使用的裝置記憶體當然有上限。當場景切換期間資□重疊，或暫時性的繪製緩衝區增加時，這個上限就會成為問題。

除了降低遊戲□的記憶體峰□之外，記憶體上限本身也有調整的空間。Xcode 的「Increased Memory Limit」是在支援的裝置上提高該上限的設定。□一項設定「Extended Virtual Addressing」則是擴大處理程序可使用的虛擬位址空間。兩者名稱相似，但可處理的範圍不同。

本章將介紹這兩項設定的使用區分、在實機上的效果，以及在 App Store 營運的經驗。數□為作者的實機測量結果，並非 Apple 保證的各裝置上限。

13.1　記憶體上限與虛擬位址空間是不同的限制

應用程式發生 OOM（Out of Memory）時，至少需要區分以下兩種情況。

表 13.1: iOS 應用程式中應區分的 2 種記憶體限制

限制	主要檢視的□	典型的表現方式
實體記憶體側	`phys_footprint`	因高記憶體使用量被系統終止（jetsam）
虛擬位址空間側	VM Region Summary、Virtual Size	記憶體配置失敗，之後發生當機

實體記憶體與實際必須保留在 RAM 中的分頁有關。在 iOS 上，應用程式可使用的實

體記憶體量並非裝置搭載的 RAM 本身，還會受當時的系統狀態與套用於應用程式的上限影響。

Apple 的 os_proc_available_memory() 會傳回目前應用程式在達到記憶體上限之前可配置的量。依 Apple 的□明，此傳回□相當於「目前的記憶體上限減去應用程式的 memory footprint」，不一定等於裝置的可用實體記憶體。此外，上限在應用程式的生命週期中也可能改變，因此不應快取該□，而應視為參考資訊 [*1]。

虛擬位址空間是管理記憶體與檔案要對應到□個位址的範圍。例如，即使對大型檔案進行 mmap，其所有分頁也不一定會立即載入 RAM。然而，放置該範圍所需的虛擬位址仍是必要的。反過來□，也會發生實體記憶體的 footprint 仍低、os_proc_available_memory() 也還有餘裕，卻無法配置新的位址範圍，導致 malloc 等失敗的情況。

因此，為了區分 表 13.1 這兩種限制，需要確認是留有 jetsam report，還是一般當機報告中以配置失敗為起點。即使 os_proc_available_memory() 傳回較大的□，虛擬位址空間也不一定有同樣多的餘裕。

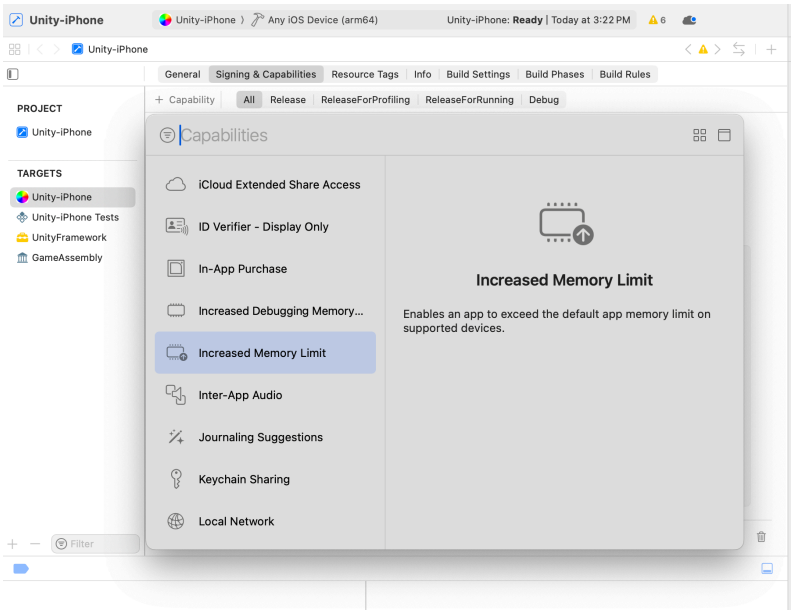
13.2 個 Entitlement 的作用

13.2.1 Increased Memory Limit

Increased Memory Limit 的鍵為 com.apple.developer.kernel.increased-memory-limit。Apple □明這是向系統傳達「在支援的裝置上超過預設的應用程式記憶體上限，可能改善應用程式核心功能」的 Entitlement [*2]。在 Xcode 的 Capability 清單中，也會如 圖 13.1 般顯示附帶「在支援的裝置上」條件的□明。

[*1] Apple Developer Documentation, os_proc_available_memory https://developer.apple.com/documentation/os/os_proc_available_memory

[*2] Apple Developer Documentation, com.apple.developer.kernel.increased-memory-limit https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit



▲圖 13.1　Xcode 的 Capability 清單中顯示的 Increased Memory Limit

也就是□，並非指定後所有裝置都會增加一定的量。Apple 的文件中也記載，只有部分裝置可使用增加後的上限，並要求即使無法使用額外記憶體，應用程式也要能正確運作。

Increased Memory Limit 的實機測量

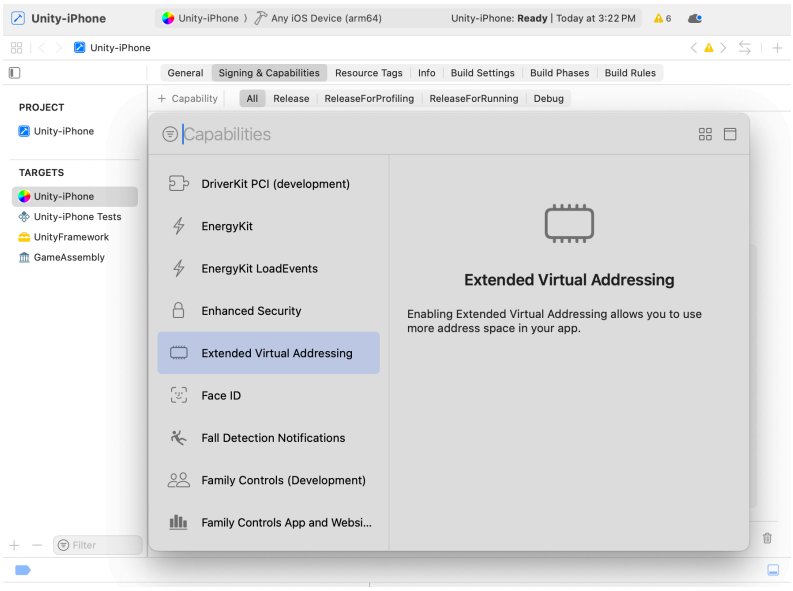
表 13.2: 有無 Increased Memory Limit 時的 OOM 記憶體測量結果（iPhone）

搭載 RAM	停用時	□用時
3GB	約 1,850MB	約 1,850MB
4GB	約 2,050MB	約 2,290MB
6GB	約 3GB	約 4GB

4GB 與 6GB 的 iPhone 記憶體各自增加，而 3GB 的 iPhone 在□用與停用之間沒有差異。此外，iPad 在 OOM 時的記憶體本來就與 iPhone 不同，因此□用時的結果也很可能不同。若目標使用者的裝置中 3GB iPhone 占比較高，就不應預期只要加上 Increased Memory Limit 就能解決問題。

13.2.2 Extended Virtual Addressing

Extended Virtual Addressing 的鍵為 `com.apple.developer.kernel.extended-virtual-addressing`。Apple 將其□明為表示可存取更大位址空間的 Boolean □，並以遊戲為例[*3]。



▲圖 13.2　Xcode 的 Capability 清單中顯示的 Extended Virtual Addressing

這項設定不會增加 RAM。對於接近實體記憶體上限而被 jetsam 的應用程式，僅加上 Extended Virtual Addressing 並不保證能持續使用相同量的 RAM。能期待效果的情況，是因大型記憶體映射、破碎化的位址區域、大型固定保留等原因，在用盡實體記憶體之前虛擬位址空間就先變得吃緊。

□用本身幾乎沒有成本。分頁表只會建立在實際映射的範圍□，因此即使擴大上限也

[*3] Apple Developer Documentation, Extended Virtual Addressing Entitlement `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.extended-virtual-addressing`

不會增加[*4]。需要考量的是有沒有出現效果的裝置。加上 Increased Memory Limit 時位址空間也會擴大，因此兩者都加上時，有些裝置的 max_address 並不會改變。

另一方面，對於 表 13.2 的 3GB 裝置這類 Increased Memory Limit 沒有效果的裝置，Extended Virtual Addressing 是擴大位址空間的唯一手段。若裝置不支援，Increased Memory Limit 即使簽署正確也會失效，但 Extended Virtual Addressing 沒有這項條件。

虛擬位址空間的上限

Extended Virtual Addressing 所擴大的，是有別於 footprint 上限的位址空間上限。此上限由 pmap_max_64bit_offset() 依搭載記憶體傳回的 表 13.3 中的某個決定[*5]。

表 13.3: arm64 使用者處理程序的位址空間上限

條件	常數	上限
max_mem > 3GB	ARM64_MAX_OFFSET_DEVICE_LARGE	17.375GB
max_mem 超過 1GB 至 3GB	ARM64_MAX_OFFSET_DEVICE_SMALL	13.375GB
max_mem 1GB 以下	ARM64_MIN_MAX_ADDRESS	12.5GB
以包含 entitlements 的任一手段擴充	MACH_VM_JUMBO_ADDRESS	63GB

max_mem 是核心所辨識的實體記憶體，會小於型錄上的 RAM 容量。這些都是核心的編譯時期常數，因此可能隨 OS 版本而改變。

13.2.3 App Store 上的運用實績

採用這些 Entitlements 時令人在意的是，是否會影響 App Store 的審。就作者所知的多個營運中專案，Increased Memory Limit 與 Extended Virtual Addressing 皆已用設定，通過 App Store 審並發布。

[*4] Apple Open Source, xnu-12377.121.6, osfmk/arm/pmap/pmap.c（pmap_max_64bit_offset） https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

[*5] Apple Open Source, xnu-12377.121.6, osfmk/arm/pmap/pmap.c（pmap_max_64bit_offset） https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

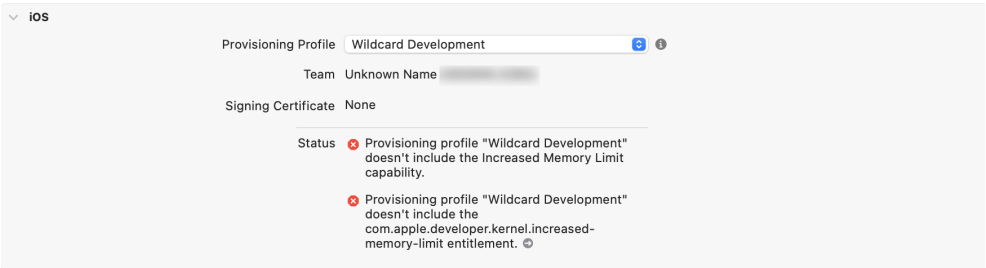
Apple 自己也在面向遊戲開發者的「Bring your high-end game to iPhone 15 Pro」[*6]中，建議□用 Increased Memory Limit 以增加可用記憶體。這兩項是可考慮用於□品的公開功能。雖然無法保證個別應用程式整體的審□結果，但不必因為擔心「因為是與記憶體上限相關的設定，審□時會被要求□明，或是無法通過審□」而將其排除在候選之外。

13.3 確認 App ID 與開發者帳號

13.3.1 無法使用萬用字元的 Profile

這兩個 Entitlement 無法在萬用字元形式的 App ID 上□用。若是像 `com.example.*` 這樣將多個應用程式統一簽署的組態，必須切換為與遊戲的 Bundle ID 一致的 Explicit App ID。

若直接進行簽署，Xcode 的 Signing & Capabilities 會出現 圖 13.3 的錯誤。請注意 Capability 名稱與 Entitlement 的鍵是分別回報的。其中指出了 Profile 未包含該 Capability，以及沒有對應該鍵的許可這兩點。

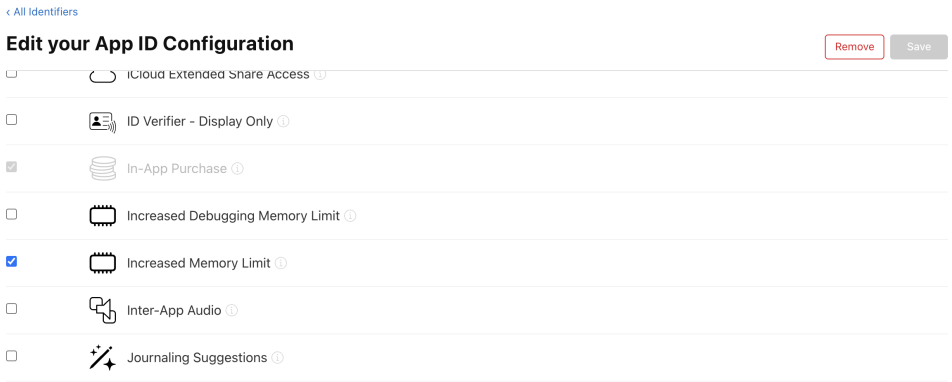


▲圖 13.3　以萬用字元的 Provisioning Profile 簽署時的錯誤

在 Apple Developer 的 Certificates, Identifiers & Profiles 中選擇目標 App ID，如 圖 13.4 般□用所需的 Capability。Apple 也□明，App ID 上所設定的 Capability 會成為應用程式可用功能的允許清單，需要 Explicit App ID 的功能無法以萬用字元選取 [*7]。

[*6] Apple Developer Tech Talks, Bring your high-end game to iPhone 15 Pro `https://developer.apple.com/videos/play/tech-talks/111372/`

[*7] Apple Developer, Register an App ID `https://developer.apple.com/help/account/identifiers/register-an-app-id/`



▲圖 13.4　在 App ID 的設定中啟用 Increased Memory Limit

手動簽署時，變更 Capability 後須重新產生 Provisioning Profile，並更新建置所使用的檔案。自動簽署時，則確認已以正確的 Team 與 Bundle ID 選用更新後的 Profile。若在舊 Profile 的狀態下僅對 `.entitlements` 新增鍵，所要求的權限與簽署端的許可並不一致。在檢查 Unity 端的設定之前，應先在這裡對齊。

13.3.2　Extended Virtual Addressing 需要付費會員資格

在將免費的 Apple Account 作為 Xcode 的 Personal Team 使用的組態下，無法啟用 Extended Virtual Addressing。在 Apple 的 Capability 對應表中，適用對象為Apple Developer Program 與 Apple Developer Enterprise Program 的會員資格，不包含免費的 Apple Account [*8]。

另一方面，Increased Memory Limit 在 Personal Team 中也可以啟用。

13.4　以 Unity 的建置後處理自動設定

Entitlements 檔案中所需的最小內容為 清單 13.1。這是同時啟用兩者的範例。

▼清單 13.1　記憶體相關 Entitlement 的最小組態

```
<?xml version="1.0" encoding="UTF-8"?>
```

[*8] Apple Developer, Supported capabilities (iOS) `https://developer.apple.com/help/account/reference/supported-capabilities-ios/`

```
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
    PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.kernel.increased-memory-limit</key>
    <true/>
    <key>com.apple.developer.kernel.extended-virtual-addressing</key>
    <true/>
</dict>
</plist>
```

也可以從 Xcode 的 Signing & Capabilities 手動新增，但每次匯出都必須重新加上，容易混入設定遺漏的建置。在 Unity 中，可以從建置後處理一併更新 Entitlements 檔案與參照它的 CODE_SIGN_ENTITLEMENTS。處理在簽署前的建置時執行，並不是在應用程式 動後由 C# 取得權限。

13.4.1 Unity 6000.0.11f1 以後使用專用 API

Unity 6000.0.11f1 在 ProjectCapabilityManager 中新增了以下方法 [*9]。

- AddIncreasedMemoryLimit()
- AddExtendedVirtualAddressing()

使用這些方法，即可一併完成對 Entitlements 新增 Boolean ，以及反映至 Xcode 專案。

也提供了對應的 PBXCapabilityType 定義。不過，這並非 C# 的 enum，而是表示 Capability 需求的類別。將此傳給 PBXProject.AddCapability()，與在 Entitlements 中加入所需的鍵並不相同。因此基本上使用兩者都處理的 ProjectCapabilityManager。

以下是使用 ProjectCapabilityManager 新增 Capability 的範例。

▼清單 13.2　在 Unity 6000.0.11f1 以後新增記憶體 Capability 的建置後處理

```
#if UNITY_IOS
using UnityEditor;
using UnityEditor.Callbacks;
```

[*9] Unity 6.0 Scripting API, ProjectCapabilityManager https://docs.unity3d.com/6000.0/Documentation/ScriptReference/iOS.Xcode.ProjectCapabilityManager.html

```
using UnityEditor.iOS.Xcode;

static class IosMemoryCapabilities
{
    [PostProcessBuild]
    static void OnPostprocessBuild(BuildTarget target, string buildPath)
    {
        if (target != BuildTarget.iOS) return;

        string pbxPath = PBXProject.GetPBXProjectPath(buildPath);
        var pbx = new PBXProject();
        pbx.ReadFromFile(pbxPath);
        string mainTarget = pbx.GetUnityMainTargetGuid();

        string entitlements = pbx.GetBuildPropertyForAnyConfig(
            mainTarget, "CODE_SIGN_ENTITLEMENTS");
        if (string.IsNullOrEmpty(entitlements))
            entitlements = "Unity-iPhone.entitlements";

        var capabilities = new ProjectCapabilityManager(
            pbxPath, entitlements, targetGuid: mainTarget);
        capabilities.AddIncreasedMemoryLimit();
        capabilities.AddExtendedVirtualAddressing();
        capabilities.WriteToFile();
    }
}
#endif
```

`UnityEditor.iOS.Xcode` 包含在 iOS Build Support 中，因此請以 `#if UNITY_IOS` 包圍。否則在未安裝此模組的環境中，整個編輯器將無法通過編譯，請特別注意。

此範例針對的組態是：所有建置組態共用同一個 Entitlements 檔案，且其路徑以相對於 Xcode 專案的相對路徑指定。若 Debug 與 Release 使用不同的檔案，或路徑中包含建置變數，請配合反映到各組態實際檔案的處理。

最後的儲存以 `capabilities.WriteToFile()` 進行。若在其他建置後處理中編輯 Xcode 專案，重新儲存 `pbx` 時，Capability Manager 所儲存的變更可能被覆寫，因此請注意在更新後重新讀取檔案再進行變更。

13.4.2 無專用 API 的版本直接更新 plist

若要在沒有專用 API 的編輯器中進行相同處理，請將 清單 13.2 中從□生 `ProjectCapabilityManager` 到儲存為止的 5 行，替換為 清單 13.3。

▼清單 13.3　在沒有專用 Capability API 的 Unity 中的替代處理

```
string fullPath = System.IO.Path.Combine(buildPath, entitlements);
System.IO.Directory.CreateDirectory(
    System.IO.Path.GetDirectoryName(fullPath));
var plist = new PlistDocument();
if (System.IO.File.Exists(fullPath))
    plist.ReadFromFile(fullPath);

plist.root.SetBoolean(
    "com.apple.developer.kernel.increased-memory-limit", true);
plist.root.SetBoolean(
    "com.apple.developer.kernel.extended-virtual-addressing", true);
plist.WriteToFile(fullPath);

if (string.IsNullOrEmpty(pbx.FindFileGuidByProjectPath(entitlements)))
    pbx.AddFile(entitlements, entitlements);
pbx.SetBuildProperty(mainTarget, "CODE_SIGN_ENTITLEMENTS", entitlements);
pbx.WriteToFile(pbxPath);
```

先讀取既有檔案再加入 2 個鍵，即可在不刪除 Push 通知等 Entitlement 的情況下更新。要變更的是 `.entitlements`，而不是 `Info.plist`。此外，檔案必須由 `CODE_SIGN_ENTITLEMENTS` 參照，才會用於簽署。

僅僅加入 C# 程式碼，並不會連帶□用帳號端的權限。無論採用□種方法，Apple Developer 端的 App ID 與 Provisioning Profile 都需要□行設定。

13.5　分兩階段確認套用

設定是否已反映，分成簽署中是否包含鍵，以及實機上核心是否已反映兩個階段來確認。

13.5.1　確認是否包含在簽署中

無論是手動操作還是建置後處理，只是在專案□建立了 清單 13.1 的檔案，並不能證明已反映在裝入實機的應用程式中。Entitlement 最終會嵌入程式碼簽署中。匯出的 `.ipa` 為 zip 格式，因此請解壓縮後，對 `Payload` □的應用程式本體執行 清單 13.4。

▼清單 13.4　確認□嵌於已簽署應用程式的 Entitlement

```
unzip -q YourApp.ipa -d ipa
codesign --display --entitlements :- ipa/Payload/YourApp.app
```

在輸出的 plist 中，確認 清單 13.1 的兩個鍵是否存在。Apple 也建議以此指令在簽署

後進行確認[*10]。

13.5.2 確認實機上的上限是否改變

即使 清單 13.4 看得到鍵，也不一定表示核心實際已套用。Increased Memory Limit 在裝置端不支援時，即使簽署正確也會被視為無效。

在實機上是否套用，需直接讀取核心對自身處理程序所持有的□來判斷。取得該□的是 `task_info()`，這是 iOS SDK 標準□建的 Mach API（`<mach/mach.h>`）。由於無法從 C# 呼叫，因此改由置於 `Assets/Plugins/iOS/` 的 `.mm` 讀取後傳回 C#。

▼清單 13.5　讀取套用狀況的原生外掛程式

```
#import <mach/mach.h>

extern "C" void UPT_ReadLimits(uint64_t *memlimit, uint64_t *ceiling)
{
    task_vm_info_data_t i;
    mach_msg_type_number_t c = TASK_VM_INFO_COUNT;
    if (task_info(mach_task_self(), TASK_VM_INFO, (task_info_t)&i, &c) != KERN_SUCCESS)
     {
        return;
    }

    // footprint 上限（Increased Memory ）Limit
    *memlimit = i.limit_bytes_remaining + i.phys_footprint;
    // 位址空間上限：若已擴大至 0xF_C000_0000 則表示已擴充
    *ceiling  = i.max_address;
}
```

C# 端以 `DllImport("__Internal")` 宣告 `UPT_ReadLimits` 並呼叫。

應該□看的是上限。`os_proc_available_memory()` 傳回的剩餘量會隨 footprint 增加而相應減少，因此只看該□，無法區分是上限沒有提高，還是單純已經用掉了。

`limit_bytes_remaining` 與該 `os_proc_available_memory()` 在核心□是由同一個

[*10] Apple Developer Documentation, Diagnosing Issues with Entitlements `https://developer.apple.com/documentation/bundleresources/diagnosing-issues-with-entitlements`

函式得出的□ [*11]，加上目前的 footprint 即可還原為套用中的上限。位址空間側的 max_address 也能從同一個結構取得，因此可一次呼叫確認 2 個 Entitlement。max_address 必須傳入 TASK_VM_INFO_COUNT 才會填入。

不過，無法從 max_address 分離出單獨使用 Extended Virtual Addressing 的效果。附加了 Increased Memory Limit 的建置，或在 Xcode 中□用 Enhanced Security 的建置，也會得到相同的上限。能□確認的只有「上限是否已擴大到 63GB」。

13.6 iOS 26.4 中浮現的虛擬位址保留

13.6.1 JS VM Gigacage (reserved) 並非 1GB 的 RAM 使用量

在搭載 iOS 26.4 的 3GB iPhone 上驗證時，當機報告的 VM Region Summary 中出現約 1GB 的 JS VM Gigacage (reserved) 區域，□用 Extended Virtual Addressing 後即可防止 OOM 時的記憶體量下降。該區域並非真正的 Gigacage，而是沿用既有 VM tag 名稱顯示的 JavaScriptCore Structure Heap 保留 [*12]。

13.6.2 WebKit 的修正會依裝置 RAM 縮小保留量

JavaScriptCore 的公開錯誤 315240 中記錄了這樣的方針：在低記憶體裝置上，Structure Heap 的保留會占用虛擬位址空間的很大比例，因此依實體記憶體量縮小保留 [*13]。

在因應此問題的 WebKit 提交中，針對 iOS 系列平台，取基本預留大小與實體記憶體 20% 中較小者，再向下取整為 2 的冪，以此決定 Structure Heap 期望的保留大小。公開 commit 的□明中，列出了 3GB 裝置上為 512MB 的範例 [*14]。

在已加入此修正的 iOS 26.6.1 的 iPhone SE（第 2 代，RAM 3GB）上測量，顯示

[*11] Apple Open Source, xnu-12377.121.6, bsd/kern/kern_memorystatus.c (memorystatus_available_memory_internal) https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/bsd/kern/kern_memorystatus.c

[*12] Apple Developer Forums, iPadOS 26.4+ significantly reduced per-app memory limit from 6GB to 3GB on 8GB iPad, breaking memory-intensive apps https://developer.apple.com/forums/thread/826325

[*13] WebKit Bugzilla 315240, Scale down StructureAlignedMemoryAllocator reservation in proportion to phys-mem size https://bugs.webkit.org/show_bug.cgi?id=315240

[*14] WebKit commit cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b, Scale down structure-heap reservation size with physical memory https://github.com/WebKit/WebKit/commit/cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b

的預留量從約 1GB 縮減為 512MB，在 26.4 中下降的 OOM 時記憶體量，即使不□用 Extended Virtual Addressing 也已獲得解決。

13.7 總結

Increased Memory Limit 與 Extended Virtual Addressing 是□用後幾乎沒有損失的設定。分頁表只會建立在實際映射的範圍□，因此擴大位址空間並不會增加成本。也有通過 App Store 審□並發布的實績。只要備妥 Explicit App ID 與設定檔，我們認為基本上可以兩者都□用。

不過，效果取決於裝置。在作者的 iPhone 上測量，Increased Memory Limit 帶來的餘裕在 4GB 裝置上約為 240MB，在 6GB 裝置上約為 1GB，3GB 裝置則沒有差異。iPad 需要□外測量。Extended Virtual Addressing 不會增加實體記憶體，僅在虛擬位址空間先耗盡的情況下才有效果。iOS 26.4 中出現的 Structure Heap 預留問題在 iOS 26.6 系列已獲緩解，但若需支援舊版 OS，或應用程式自身的記憶體對應占用大量位址空間，仍可預期有效果。

兩者都不會減少記憶體使用量本身。為了讓沒有額外上限的裝置也能持續運作，仍需□外進行縮減紋理與暫時性 RenderTexture 的峰□、資□的常駐期間、原生外掛程式的大型 allocation，以及檔案映射生命週期的工作。這兩個 Entitlement 的用途，是在此基礎上擴大剩餘的餘裕。

第14章

自動化每日建置的實機效能分析

キム ミンヒョク Minhyuk Kim
株式會社 CyberAgent SGE 核心技術本部

所屬：株式會社 CyberAgent SGE 核心技術本部

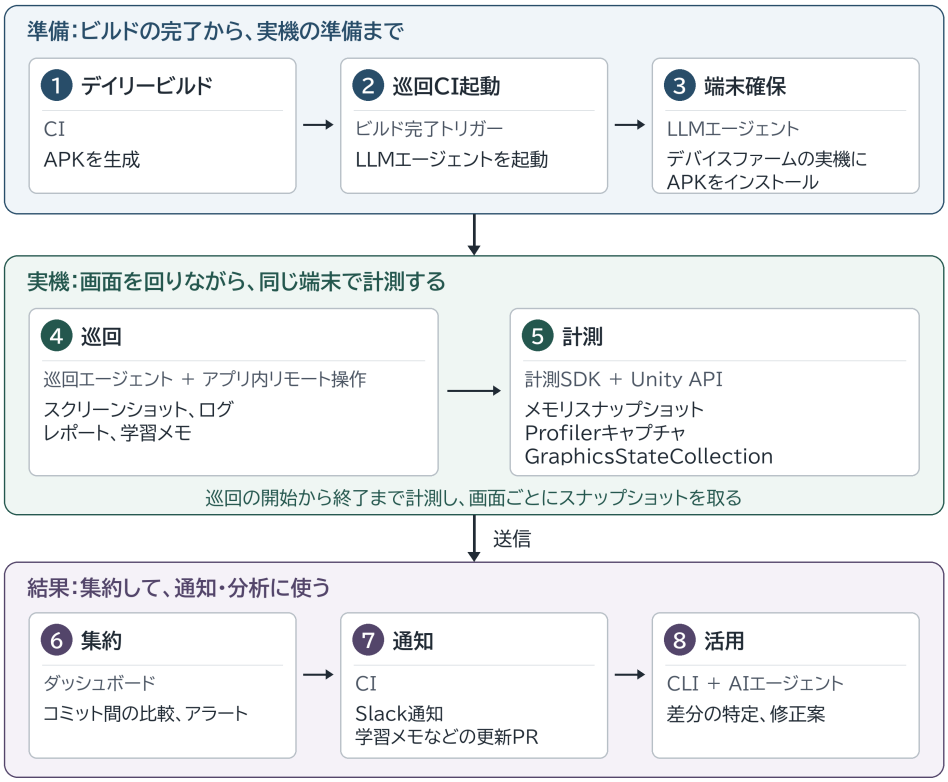
實機的效能分析，大多只有在有人認為需要時才會進行。在發布前的衝刺階段，有人接上 Profiler，找出尖峰與記憶體增加，再去尋找造成問題的提交。這樣的流程中，搜尋「從何時開始變差」的範圍，往往是數週份的差異。本章將介紹每次每日建置後，自動在實機上執行測量的架構。

此架構分為 3 個部分：出借實機的裝置農場、嵌入應用程式並接受外部操作的遠端操作伺服器，以及將測量資料傳送至伺服器的測量 SDK。從裝置農場遠端借用實機，透過遠端操作依序走訪遊戲畫面，並以測量 SDK 傳送記憶體快照與 Profiler 擷取檔。將其作為每日建置的後續工作放入 CI，每次□生建置，就會累積以提交為單位的測量資料。累積的資料可從命令列取出，因此也成為讓 AI 代理讀取差異並進一步提出修正方案的入口。

本章□容□明在□部專案中運作的每日巡檢工作流程，以及為達成此流程所下的工夫。截至撰寫時，對象僅限 Android 裝置，iOS 的支援預計日後進行。

14.1　在建置之後串接測量工作

管線依照 圖 14.1 的順序進行。裝置的取得、應用程式的操作與測量資料的傳送，分別由不同的工具負責。即使將其中任一個替換為其他工具，這樣的分工仍可直接沿用。



▲圖 14.1　每日測量管線的階段

每日建置本身不做任何更動。每日建置不需要□外嵌入巡檢用的情境，或預先設定的測量程式碼；由於不放置這類以測量為前提的程式碼，即使在□部結構劇烈變化的開發階段，巡檢也不會損壞，得以維持穩定性與彈性。具體方法將在下文詳細□明。

14.2　由 CI 接收建置完成並開始巡檢

每日建置完成後，□一個工作流程會訂□建置完成的觸發條件，僅在成功時□動巡檢。此時傳給後續工作流程的是建置 APK 的 run 的 ID。

□動的工作流程會由 run 的 ID 找出建置的 artifact 並下載。接著□動負責巡檢的 LLM 代理。截至撰寫時，模型使用 Claude Opus 5.5，決定推論深度的 effort 設定為 medium。

14.3 Unity 的測量由應用程式□部進行

本章自動化的是 Unity 端的測量。目的是每天以相同的形式取得並比較 Unity □部的明細，例如□些資□占用記憶體、□些 Profiler 標記的負載較重。

從裝置外部使用的測量手段，當然也有許多很實用的工具。在 iOS 上，可以用 Instruments 的 Time Profiler 追蹤到呼叫堆疊層級的瓶頸。在 Android 上，也可以透過 simpleperf 以及使用它的 Android Studio CPU Profiler，取樣到 IL2CPP □生的原生函式層級。將這些工具用於自動化也是完全可以考慮的，但本章不涉及。

Unity 的明細，平常是將 Unity 編輯器的 Profiler 或 Memory Profiler 連接到實機來取得。不過，若在 CI 的巡檢中加入編輯器，需要考慮的事情會一口氣增加。每次在執行器上開□專案，都必須等待 Library 資料夾的匯入，且每個執行器都需要 Unity 授權。此外，除了編輯器與實機的連線之外，還必須準備讓代理操作編輯器的 Unity MCP 這類途徑，使架構變得複雜。

因此在 CI 上不使用編輯器，只使用執行階段的 API，讓測量在應用程式□完成。

14.4 從裝置農場借用裝置

裝置農場使用的是可從瀏覽器操作實機的 STF 平台[*1]。不過，本章使用的並非透過畫面操作裝置農場，而僅是透過 API 取得裝置；預約裝置後會傳回遠端 adb 端點，從 CI 的執行器端可以和 USB 連接同樣地處理。從預約到安裝 APK 整合為一支□本，CI 以 1 個步驟呼叫。

若僅以巡檢為目的，只要有 1 台閒置的 Android 10 以上裝置就足□，但若想逐日比較測量結果，裝置每天不同就無法比較。測量用的機型應固定，沒有空閒時當天就跳過，這樣的做法可避免累積無法比較的資料。

[*1] DeviceFarmer/stf（OpenSTF 的後繼儲存庫） https://github.com/DeviceFarmer/stf

14.5 以應用程式□遠端操作走訪畫面

應用程式的操作使用嵌入應用程式的遠端操作伺服器。應用程式□動後僅在裝置□部等待指令，CI 的執行器端則經由 adb 連線並傳送指令。除了確認應用程式是否運作、取得場景上的物件結構、擷取螢幕畫面、取得記錄等固定指令之外，還可以在應用程式□執行以 JavaScript 撰寫的任意處理。

此 JavaScript 由 Jint[*2] 直譯，可直接存取 C# 的型別。以名稱尋找並按下 UI 按鈕、讀取導覽的目前節點、□動測量 SDK，都可以經由同一路徑進行。這接近於可在實機上即時撰寫並試跑 C# 程式碼的環境，測量 SDK 的 API 也可以不重新建置而從外部呼叫。每次變更閾□或對象畫面時，都不需要重新製作 APK。

要逐日比較測量結果，必須每天以相同順序經過相同的畫面。固定座標的點擊會在裝置解析度或版面配置改變時偏離按下的位置，因此以名稱取得執行階段的 UI 物件，並直接觸發 UI 框架的事件。目前位於□個畫面，則透過讀取導覽狀態來判定。若專案端有畫面轉換圖的定義，可直接作為巡檢計畫的輸入。

14.6 上傳快照與擷取檔兩種資料

測量資料的傳送，使用作者所屬組織開發的測量平台。這是由嵌入應用程式的 SDK 將測量資料傳送到伺服器，在伺服器端累積後，於儀表板上進行逐日比較，並在惡化時發出警示的機制。

SDK 能傳送的不只是 Unity 的記憶體快照與 Profiler 擷取檔。CPU 時間的明細與繪製呼叫次數等繪製負載、程式碼中任意區間的處理時間、應用程式□動時間、以固定間隔取得的記憶體使用量變化等也都能傳送，因此若納入巡檢，同樣可成為定點觀測的對象。本章僅聚焦□明其中的 表 14.1 這兩種。

表 14.1: 每日巡覽傳送的測量資料

種類	取得時機	前提	傳送量的考量
記憶體快照	抵達各畫面並穩定之後	Development Build	每個畫面 1 份，全數傳送

[*2] sebastienros/jint, JavaScript Interpreter for .NET `https://github.com/sebastienros/jint`

表 14.1: 每日巡覽傳送的測量資料

種類	取得時機	前提	傳送量的考量
Profiler 擷取	從巡覽開始到結束共 1 份	已□用 Profiler 的建置	只傳送超過閾□的影格

14.6.1 快照依各畫面取得

記憶體快照與 Unity 的 Memory Profiler 格式相同, 依 Unity 的規格只能在 Development Build 中取得 [*3]。每日建置的巡覽用 APK 必須是 Development Build，無法將發布設定的建置□用於測量。

快照在巡檢到達畫面且載入穩定後取得 1 張。名稱中包含畫面名稱與提交雜湊，以便日後將同一畫面不同日期的資料並排比較。

14.6.2 擷取檔每次巡檢取得 1 份，並在裝置上篩選

Profiler 擷取檔是由 SDK 在裝置上逐步讀取 Unity 寫出的原始 Profiler 記錄，僅將符合條件的影格寫入□一個檔案並上傳。不保留所有影格的原因，是 Profiler 不適合長時間記錄。編輯器的 Profiler 視窗最多只能保留 2,000 個影格，以 60fps 計算約為 30 秒多。由於會從舊的影格開始捨棄，即使連接編輯器記錄數十分鐘的巡檢，剩下的也只有最後 30 秒多。若每次都寫成檔案，影格數的上限會消失，但檔案在 1 次巡檢中就會膨脹到數十 GB。無論是作為 CI 的 artifact 保留，還是每天累積，都太大了。

擷取在巡檢開始時□動，並在最後一併進行停止與上傳。□動時決定的是保留□些影格的閾□，以影格時間與 GC 配置量兩項指定。停止會在處理完原始記錄中尚未讀取的部分後完成。由於篩選在測量期間也並行進行，即使像 Deep Profiling 這樣寫出速度快的情況，未讀取的部分也幾乎不會累積。即便如此，為了避免在停止完成前歸還裝置，停止會在距離巡檢截止時間留有餘裕時開始。

14.6.3 只讀取分隔，GC 以時刻比對

原始記錄是將各執行緒的緩衝區以區塊為單位依序追加寫入而成。要知道□個區塊屬於□個影格，必須找出影格的分隔。線索在於 Profiler 的寫出方式。Profiler 會在寫入影

[*3] Unity Manual, Memory Profiler package `https://docs.unity3d.com/Packages/com.unity.memoryprofiler@1.1/manual/index.html`

格分隔的前後切換執行緒的緩衝區，因此分隔會以僅十幾個位元組的獨立區塊形式送達。於是篩選執行緒只開□長度在此以下的區塊，確認是否為分隔。比這更長的區塊不讀取□容，視為屬於該執行緒目前的影格。當主執行緒收到下一個影格的分隔時，影格的長度便已確定，於此時判定是否保留，並將要保留的影格的區塊維持原順序寫出。取樣器名稱與執行緒名稱等不屬於任何影格的定義區塊則全部寫出，因此輸出可直接在 Unity 的 Profiler 視窗中開□。

GC 配置量無法從原始記錄取得。原始記錄的影格統計中雖然也有 GC 欄位，但從 Unity 2022.3 到 6000.5，在確認過的所有擷取檔中都是 0。因此測量期間，在主執行緒每個影格以 ProfilerRecorder 讀取 `GC Allocated In Frame` 計數器。ProfilerRecorder 的□不附帶時刻，所以在該影格中讀取 `ProfilerUnsafeUtility.Timestamp`，附加在□上。此時刻與記錄在原始記錄影格分隔中的時刻使用同一個時鐘。篩選執行緒會把在某影格開始到下一影格開始之間記錄的□，比對為該影格的 GC 配置量。由於某影格的□要到下一個影格中才能取得，所以判定要等待數個影格的□送達。

在裝置上進行篩選，或許會讓人擔心其本身是否會加重測量的負載。篩選由專用執行緒進行，主執行緒只做資料增加情況的確認與 GC 計數器的讀取。如前所述，大部分區塊不讀取□容，因此篩選所需的處理僅限於一小部分資料。篩選執行緒與寫出 Profiler 資料的 Dispatcher 執行緒，以執行緒名稱辨別並從輸出中排除。此外，篩選處理已排除在 Deep Profiling 的對象之外。若納入對象，篩選執行緒的每一次方法呼叫都會成為樣本，並寫入正在讀取的原始記錄中。不過，GC 計數器是整個處理程序的□，因此也包含篩選執行緒每個影格配置的數 KB。與後述的閾□相比，這個量足□小，但包含□用 Profiler 本身的負載在□，不能□測量完全沒有額外的負擔。

14.6.4 閾□依想觀察的問題與資料量決定

作者的巡檢目前以 300ms、2MB 作為閾□。對 30fps 的專案而言，這約為一般影格時間的 10 倍，用意不在觀察遊玩中細微的卡頓，而是確認畫面轉換或載入時發生的大型停頓是否比昨天增加。在此設定下，整個巡檢中留下的只有極少數的影格，資料量在可每天持續傳送的範圍□。調低閾□雖然也能擷取細微的卡頓，但保留的影格會增加、檔案變大，也會更快達到後述的輸出上限。請配合對象專案的影格率與希望透過測量解決的問題，一邊觀察資料量一邊調整閾□來運用。

14.6.5 讓長時間巡檢容納於裝置的容量中

巡檢變長後，原始記錄的存放位置也會成為問題。SDK 會依固定大小切換原始記錄的寫出目的地，分成連號的檔案，篩選執行緒則從讀完的檔案開始刪除。如此一來，裝置的儲存空間使用量會在數個檔案的量即達上限。當篩選延遲而累積過多未處理的檔案時，則會停止記錄。切換檔案時，Unity 會在新檔案的開頭，重新寫入至此出現過的所有取樣器名稱與物件名稱的定義。輸出會串接成 1 個檔案，因此第 2 個以後的定義大多重複。所以與前一次寫出的□容在位元組層級完全相同的定義，會從輸出中省略。輸出檔案設有 1.5GB 的上限，在下一個要保留的影格會超過上限之處停止記錄。可上傳的大小最多為 2GB，因此在其之前停止，以確保上限□保留的影格能確實傳送。

擷取檔中也嵌入了各影格的螢幕截圖。這是因為僅憑 Profiler 的數□，難以得知發生尖峰時畫面上顯示了什麼。畫面以非同步 GPU 回讀取得，因此要數個影格後才能取得影像。所以在每個影格結束時重新嵌入最後取得的影像，使得無論保留□個影格都附有影像。從 Unity 6.6 起，Profiler 的標準功能也能連續記錄螢幕截圖，預設□用拍攝螢幕截圖的功能。不過拍攝固定為每 15 個影格 1 次，在作者的 Android 裝置上記錄的影像只有 256×112 像素，UI 的文字無法辨識。此外，在未□用 AsyncGPUReadback 的裝置上沒有選項，螢幕截圖會靜默地停用。在以 300ms、2MB 篩選的某天擷取檔中，留下的 71 個影格裡附有標準螢幕截圖的只有 6 個，與保留的影格不連續的篩選方式相容性不佳，因此持續使用 SDK 端的拍攝。

14.6.6 中繼資料要附加在所有資料上

除了上傳擷取檔時附帶的中繼資料之外，SDK 還有為所有測量資料附加共通資訊的機制。若在初始化之後立即登錄建置時嵌入的提交雜湊、建置的 run ID 與裝置機型，快照與擷取檔都會附上相同的資訊，也不需要由操作端每次傳遞。測量結果能依提交順序排列，正是因為有這項登錄。

14.7 著色器預熱資料也在巡檢中收集

巡檢每天都會走遍遊戲的主要畫面，因此也成為收集著色器預熱所用資料的場合。GraphicsStateCollection 是記錄執行期間使用過的著色器變體與繪製狀態的組合，使其能在之後事先預熱的資□。

收集時不使用測量 SDK，而是從遠端操作直接呼叫 Unity 的 API。在巡檢開始時開始

記錄，照常走訪畫面，結束時停止記錄，並以檔案形式寫出到裝置上應用程式的儲存區域。寫出的檔案以 adb 取出到 CI 的執行器。由於不使用編輯器與 Player 連線，與測量一樣僅在 CI 完成。不過，在 IL2CPP 建置中，程式碼裁剪會刪除未使用型別的成員，可能導致無法從遠端操作呼叫。巡檢用的建置必須以 link.xml 保留 GraphicsStateCollection 的型別。

取出的檔案不會直接放入專案。只有在巡檢未因錯誤而中止並已完成檔案儲存的日子，才會把它加總到相同平台與品質設定的既有資中，並移除預熱時被警告找不到的變體，然後自動建立資更新的 PR，並直接自動合併。如此，每天巡檢所走過畫面的記錄會一點一點地累積。

話雖如此，僅靠這個方法並無法涵蓋所有必要的變體。被記錄的只有巡檢實際走過的畫面，以及當時繪製的容。巡檢未經過的畫面，或僅在特定條件下才會出現的特效等，會從記錄中遺漏。關於這類遺漏，以及已預熱卻仍殘留卡頓等陷，請參第 1 部第 6 章。若您是從本書開頭依序讀到這裡，或許已經讀過了。

14.8　將惡化的偵測到修正 PR 的建立交給代理

即使每天都累積資料，如果沒有人每天早上打開儀表板，也只是累積而已。此測量平台也有從命令列取得資料的 CLI，本章的管線將從確認惡化到建立修正 PR，交給使用此 CLI 的代理負責。不是由人每次撰寫委託，而是作為測量之後的後續流程每天運行。

巡檢的測量結束後，會動與巡檢不同的比較用代理。此代理以 CLI 取得今天與前一天的測量資料，比較同一畫面的快照與擷取檔中殘留的尖峰，確認是否有惡化之處。若發現惡化，會將個畫面的個變化了多少、比較所用的資料、這段期間加入的提交範圍彙整後，以議題的形式登錄到待辦清單。若沒有惡化，則不登錄任何容。

登錄的議題，會由一個負責修正的代理接手。它確認議題容，從提交歷史中縮小原因範圍後進行修正，並建立 PR。人從該 PR 的審開始介入。

14.8.1　修正的確認同樣以此機制在實機上進行

修正的確認與每日巡檢分開進行。每日巡檢是依固定順序走訪固定的畫面，不適合配合各議題想確認的畫面或條件。因此，以待辦清單上的議題進入修正完成（待確認）狀態為契機，由修正確認用代理動。此代理使用與每日巡檢相同的機制，即裝置農場的裝

置、應用程式□遠端操作與測量 SDK，在實機上執行包含修正的建置，並與議題中留下的比較來源資料核對。只有在實機上確認惡化已解消後，才會將工單交給 QA。為了確認修正是否有效，開發者不需要親自操作實機。

這套確認機制不僅限於效能惡化，也可用於日常的錯誤修正。如前所述，遠端操作實質上等同於反射，可直接存取的處理能即時執行，因此代理幾乎能操作並讀取遊戲中發生的所有事情。若議題中寫有重現步驟與預期結果，就能在實機上依照該步驟操作，連同遊戲□部狀態一併確認是否已修正。

14.9 持續每天運行的營運方式

14.9.1 巡覽代理與測量步驟的職責分工

本章的巡檢是將畫面轉換圖與遠端操作的使用方式交給編碼代理，由其執行巡檢。雖然具備可迴避無法到達的畫面、在新增畫面時重新擬定計畫的彈性，但 1 次巡檢需要約 90 回合，費用約 5 美元。像測量的開始與停止這類每次必須相同的部分，不交由代理判斷，而是固定為可重複使用的步驟，只讓代理在指定的時機執行。巡檢所學到的□容，也以 PR 的形式保留在儲存庫中，作為下次的輸入。

14.9.2 結果的傳遞

巡檢的結果以報告、螢幕截圖、記錄與機器可讀的結果檔案保留，由 CI 讀取結果檔案後通知 Slack。測量本身的結果在儀表板上，因此通知中只要有上傳成敗與儀表板連結就足□。在比較中發現惡化的日子，會附上登錄的議題連結，通知到同一個頻道。反過來，沒有上傳的日子，也會在通知中□明未進行測量。因為測量在不知不覺中停止的狀態最難被發現。

14.10 今後的課題

由於在同一次巡檢中同時取得快照與擷取檔，兩者的資料會互相混雜。取得快照期間影格會停止數秒，因此在同時運行的擷取檔中會被記錄為尖峰。反過來，擷取期間 Profiler 緩衝區的記憶體會包含在快照中，因此讀取資料時需要注意。若想精確地□看資料，也可以選擇將快照用與擷取用的巡檢分開，但借用裝置的時間會增加為約 2 倍，需要降低頻率等折衷。

Profiler 擷取檔的粒度也有課題。每日測量中關閉了 Deep Profiling，因此遊戲端的□

本只能以 MonoBehaviour 的 `Update` 這類單位來□看。想看得比這更細的部分，需要手動新增 ProfilerMarker，或□用 Deep Profiling。因此，我們正在探索在建置後處理中，對指定組件的方法自動插入 ProfilerMarker 的方法等。這可作為縮小對象的輕量 Deep Profiling 使用，因此我們認為能在抑制資料量與負載的同時，取得更細的明細。

14.11 總結

我們在每日建置之後，串接了一項工作：從裝置農場借用裝置，以應用程式□遠端操作運行遊戲，並將記憶體快照與已篩選的 Profiler 擷取檔傳送到測量 SDK。藉由將提交雜湊附加在中繼資料中，測量結果成為以提交為單位的歷史，透過 CLI 取出後，從惡化的偵測、議題的登錄到修正 PR 的建立，都可以交給 AI 代理。

本章的架構或許看起來規模龐大，但並不需要一次全部備齊。若您正在規劃測量的自動化，先在建置機器上以 USB 連接 1 台實機，透過 adb 由以反射運作的直譯器取得記憶體快照，這樣或許就已經是相當不錯的記憶體定點觀測了。希望本章能對各位邁出測量自動化的第一步有所幫助。

第15章

以 ParticleSystem 長時間 Simulate 會□生巨大尖峰

加藤 有也 KATO YUYA

株式會社 Colorful Palette

姓名：加藤 有也羅馬拼音：KATO YUYA所屬：株式會社 Colorful Palette

15.1　前言

Unity 的 ParticleSystem 提供了 ParticleSystem.Simulate，可將粒子的模擬時間控制在任意時間。然而，若對 ParticleSystem.Simulate 指定較大的秒數，開始模擬時可能會發生巨大的尖峰。本文將介紹其具體案例與注意事項。

15.2　什麼是 ParticleSystem.Simulate？

關於 ParticleSystem.Simulate 的官方文件 [*1]使用 ParticleSystem.Simulate，可將粒子演出模擬到第 1 個參數所指定的時間，並從指定的時間重新播放粒子。

▼清單 15.1　從 0 秒時點重新播放

```
ParticleSystem.Simulate(0f, false, true);
```

如 清單 15.1 般指定 0 秒，就能建立讓播放到一半的粒子從頭重新播放的機制。

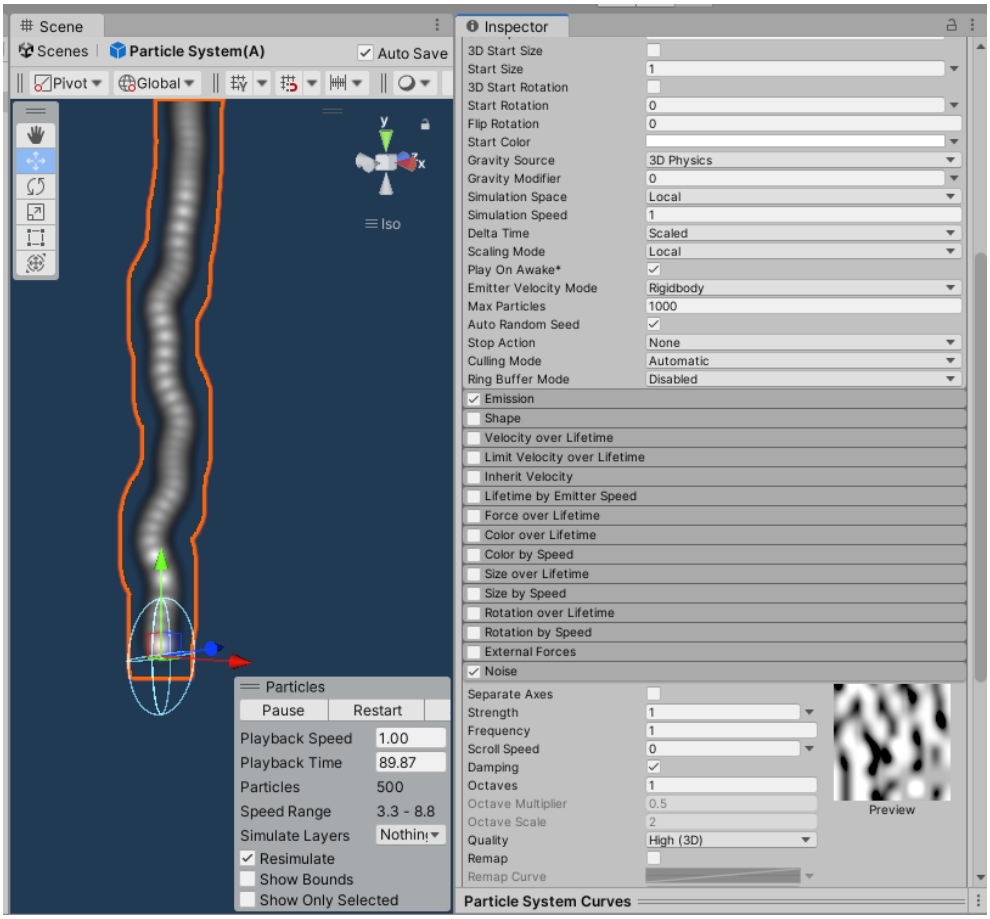
▼清單 15.2　從 5 秒時點重新播放

```
ParticleSystem.Simulate(5f, false, true);
```

[*1] https://docs.unity3d.com/ja/2022.3/ScriptReference/ParticleSystem.Simulate.html

如 清單 15.2 般指定 5 秒，就能建立讓播放中的粒子從 5 秒時點重新播放的機制。

15.3 有問題的使用方式



▲圖 15.1　有問題的粒子

在 Unity 預設建立的粒子上額外開☒ Noise 模組。其他調整的參數，是為了讓粒子更容易觀察，在 Emission 模組中將發射調整為呈一直線。

▼清單 15.3　從 20 秒時點重新播放

```
ParticleSystem.Simulate(20f, false, false);
```

若對這種使用 Noise 模組的粒子，如 清單 15.3 般在第 1 個參數的秒數中指定過大的秒數進行模擬，處理中就會發生巨大的尖峰。

▼清單 15.4　模擬的負載測量

```
void OnClickA()
{
    // 若有播放中的粒子則銷毀
    if (_activeParticle != null)
    {
        Destroy(_activeParticle);
        _activeParticle = null;
    }

    // 重新建立並播放
    _activeParticle = Instantiate(particlePrefab, Vector3.zero, Quaternion.identity);
    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        ps.Play();
    }
}

void OnClickB()
{
    if (_activeParticle == null) return;

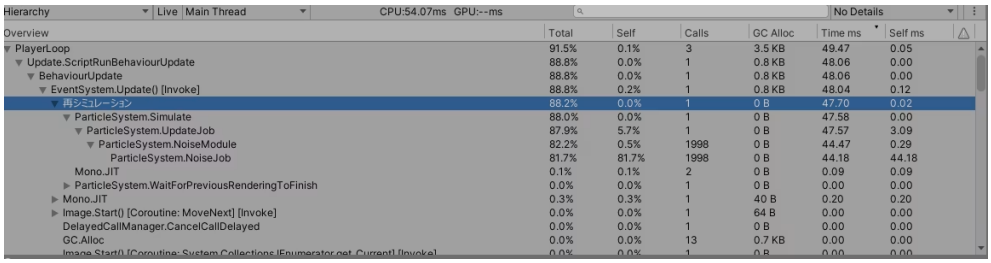
    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        Profiler.BeginSample("再シミュレーション");
        // 重新模擬
        ps.Simulate(20f, false, false);
        Profiler.EndSample();

        ps.Play();
    }
}
```

以 Profiler 確認會□生多大的負載。這是以 ButtonA □生所建立的粒子、以 ButtonB 模擬粒子的程式碼。為了在 Unity 的 Profiler 中確認時，容易□明 ParticleSystem.Simulate 是否有問題，以 Profiler.BeginSample、Profiler.EndSample 包住，方便調□。



▲圖 15.2　發生尖峰的情形



圖：處理時間如此，在 ParticleSystem.Simulate 時，ParticleSystem 的 Noise 模組會呼叫約 2000 次 NoiseJob，導致約 50ms 的尖峰。降低 ParticleSystem 的 Max Particles，可以稍微減少這個呼叫次數。

15.4　結語

若需要使用 ParticleSystem.Simulate 建立從任意秒數開始發生粒子的機制，

- 是否有指定長時間、從中途開始發生粒子的場景？
- 是否使用了 Noise 模組？

請確認以上兩點，特別是在使用已☒用 Noise 模組的粒子時，最好避免使用 ParticleSystem.Simulate。

第16章

以 VAT 減輕骨骼動畫的負載

畳 悠樹 Tatami Yuuki

株式會社 CyberAgent SGE 核心技術本部

X @tatamin0207

16.1　前言

在 Unity 的一般骨骼動畫中，骨骼更新等播放用的運算由 CPU 在每個影格執行。因此，播放動畫的物件越多，CPU 負載就越高，成為影格率下降的原因。本章將介紹 VAT，這是一種把這些執行時的運算整個改為預先計算，以減輕負載的手法。

16.2　什麼是 VAT

VAT 是 Vertex Animation Texture 的縮寫，是將動畫造成的頂點變形資訊預先烘焙到紋理中，再由頂點著色器讀取並播放的手法。紋理的每一個像素，都對應「某個頂點在某個影格的位置」，隨著播放時間移動讀取的位置，網格便看起來在動（圖 16.1）。此外，為了確保精度，位置是以正規化為 0 到 1 的▯儲存，並在頂點著色器中還原為實際座標。



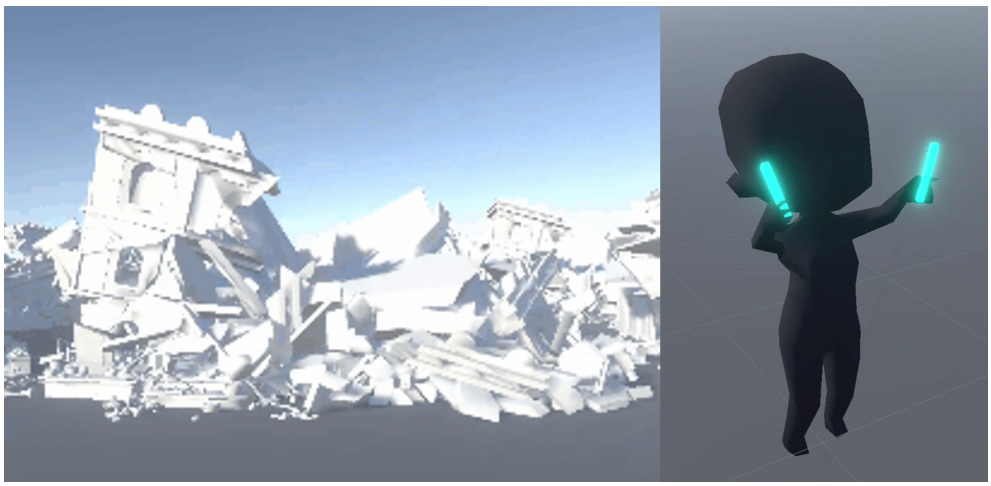
▲圖 16.1　VAT 紋理的結構。橫軸對應頂點，縱軸對應影格

使用 VAT, 可將骨骼更新等執行時的播放運算整個改為預先計算。這是連 GPU Skinning 也無法做到的效率化。

□一方面，也有以下缺點。

- 需要準備 VAT 專用的資□
- 紋理大小會隨頂點數與影格數增大
- 由於需要精確的□，與紋理壓縮相容性不佳，實際上是以未壓縮處理
- 烘焙到紋理中的時點動作即已固定，雖然視做法可做到某種程度的控制，但不適合細部控制

基於這些特性，VAT 特別適合讓大量物件播放固定的動畫。例如建築物的破壞表現、人群、背景裝飾物等不需要個別控制的演出（圖 16.2）。



▲圖 16.2　以 VAT 製作的動畫（左：剛體，右：軟體）

16.3　效能驗證

以下介紹在實機上將 VAT 的效能與其他動畫方式比較的結果。

16.3.1　測量環境與比較條件

測量環境顯示於 表 16.1。

表 16.1: 測量環境

項目	□容
實機	Pixel 8a（GPU：Mali-G715）
圖形 API	Vulkan
Unity 版本	6000.3.2f1
測量 API	FrameTimingManager、ProfilerRecorder
模型	建築物的倒塌動畫（85,209 個頂點、1,315 個碎片）
動畫	300 影格
測量方法	物件數在 0～16 個之間變化，各條件取 300 影格 × 3 次的中位數

所比較的 3 種方式顯示於 表 16.2。

表 16.2: 比較條件

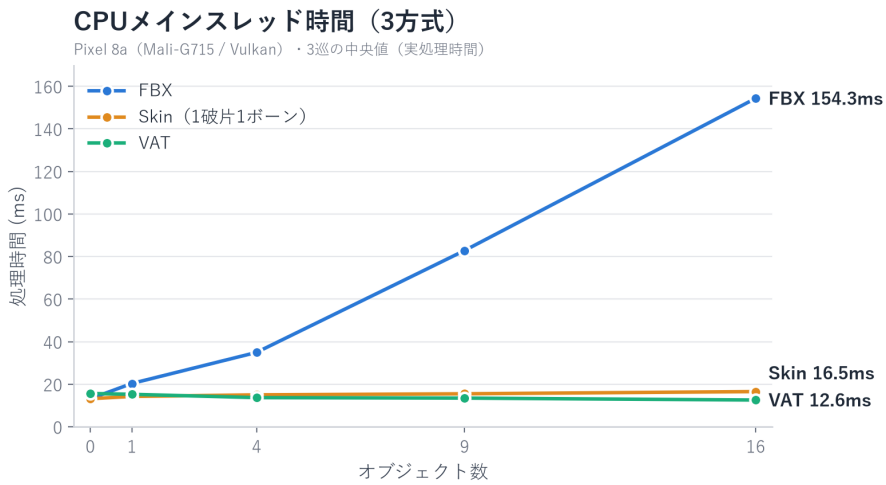
方式	組成	□用的最佳化
FBX 方式	將各碎片配置為個別的 Mesh Renderer，以 Animator 播放	SRP Batcher
Skin 方式	每個碎片 1 根骨骼，整合為 1 個 SkinnedMeshRenderer，以 Animator 播放	SRP Batcher
VAT 方式	將所有碎片的動作烘焙到紋理中，以 1 個網格繪製（Rigid-Body Dynamics 模式）	GPU Instancing

FBX 方式是將各碎片作為個別的 Mesh Renderer，並以 Animator 播放的組成（碎片是以 Transform 移動的剛體，不使用蒙皮）。Skin 方式是為每個碎片分配 1 根骨骼，並整合為 1 個 Skinned Mesh 的組成。1,315 個碎片直接成為 1,315 根骨骼，Renderer 只有 1 個。動畫的播放由 CPU 的 Animator 負責，頂點的變形則由 GPU Skinning 執行。VAT 方式則是將所有碎片的動作烘焙到紋理中，以 1 個網格繪製。3 種方式的外觀完全相同。

繪製的效率化設定，是在各方式□用對其有利者的狀態下進行比較。SkinnedMeshRenderer 不支援 GPU Instancing，因此 Skin 方式能使用的最佳選擇是 SRP Batcher。□外，Skin 方式的骨骼化也需要專用的轉換管線，這點與 VAT 條件相同。

16.3.2　CPU 處理時間

CPU 處理時間（主執行緒）的測量結果顯示於 圖 16.3。顯示 16 個時，FBX 方式延長到 154.3ms，而 Skin 方式為 16.5ms、VAT 方式為 12.6ms，幾乎持平。



▲圖 16.3　各物件數的 CPU 主執行緒處理時間

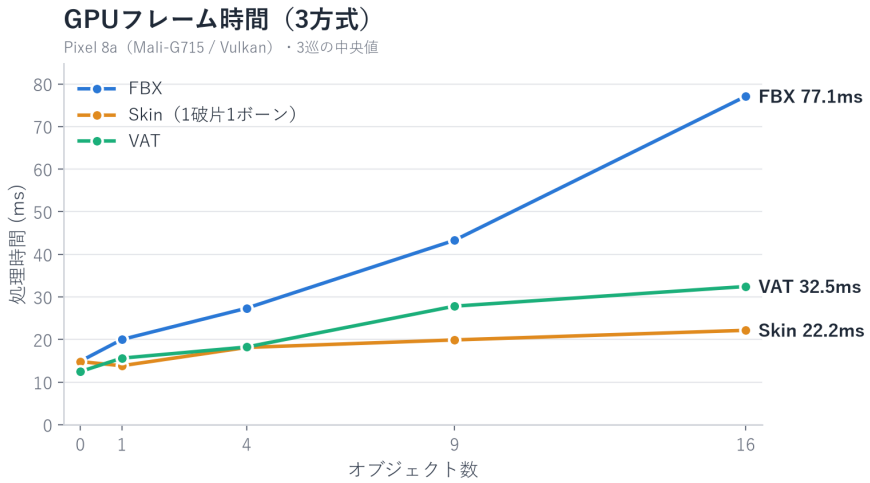
明細顯示於 表 16.3。FBX 方式與其他 2 種方式之間差異的最大原因，是繪製呼叫的送出處理。在 FBX 方式中，每個模型都有 1,315 個碎片成為繪製對象，而在已整合為 1 個網格的 2 種方式中，這項處理幾乎消失。每個碎片的邊界框計算與剔除也是如此。另一方面，Skin 方式與 VAT 方式的差異在於 Animator 的更新。Skin 方式由 CPU 負責播放，因此在 16 個時仍殘留 4.5ms，而 VAT 方式的播放本身轉移到 GPU，所以整個消失。這項差異會隨個數而擴大。

表 16.3: 顯示 16 個時的 CPU 處理時間明細（ms）

處理	FBX 方式	Skin 方式	VAT 方式
主執行緒合計	154.30	16.53	12.56
其中繪製呼叫送出（SRPBatcher.Flush）	76.17	0.03	0.02
其中 Animator 更新	5.30	4.54	0.02

16.3.3 GPU 處理時間

GPU 處理時間的測量結果顯示於 圖 16.4。16 個時，FBX 方式為 77.1ms，Skin 方式最快為 22.2ms，VAT 方式為 32.5ms。



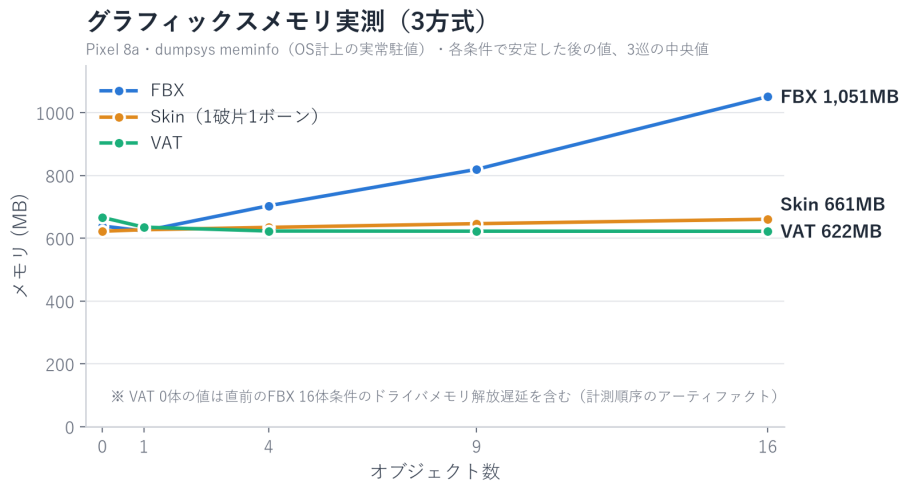
▲圖 16.4　各物件數的 GPU 處理時間

FBX 方式與其他 2 種方式之間差異的主因，是繪製命令的分割數。FBX 方式的碎片在各繪製通道中各自發出繪製命令，因此每影格達到約 21 萬次，而 VAT 方式整合為實例繪製，僅 99 次。Skin 方式每個物件也只有 1 個 Renderer，因此落在相同水準。繪製命令每次都有固定成本，所以分割數的差異會直接反映為處理時間的差異。

Skin 方式與 VAT 方式的差異，來自變形運算的次數。GPU Skinning 每影格只計算 1 次變形，並由陰影、深度等所有繪製通道重複使用其結果。相對地，VAT 每個通道都要在頂點著色器中從紋理重新計算變形。本次環境的陰影級聯有 4 層，是通道數較多的場景組成，因此與 Skin 方式的差距相應擴大（陰影圖繪製為 FBX 方式 44.3ms、Skin 方式 7.4ms、VAT 方式 16.0ms）。在進行頂點計算的通道數較少的組成中，這項差距會縮小。

16.3.4　GPU 記憶體配置量

GPU 記憶體的測量結果顯示於 圖 16.5。數□為作業系統所計入的圖形驅動程式實測□（GL mtrack）。從 0 個到 16 個的增量，FBX 方式為 +411.8MB（每個約 26MB），Skin 方式為 +38.2MB（每個約 2.4MB），VAT 方式則幾乎為 ±0。



▲圖 16.5　各物件數的 GPU 記憶體配置量

這項差異來自每增加 1 個時，增加什麼的結構性差異。FBX 方式會依碎片數量，在 GPU 記憶體中配置矩陣等繪製用資料。Skin 方式增加的，只有寫出蒙皮變形結果的頂點緩衝區，每個物件 1 個。VAT 方式只是當場計算變形，不持有新的緩衝區，因此不會增加。□外，VAT 的紋理未壓縮時約 6.6MB，相對於增量的差異而言□小。

16.3.5　驗證結果摘要

顯示 16 個時的實測□彙整於 表 16.4。

表 16.4: 顯示 16 個時的實測□摘要

指標	FBX 方式	Skin 方式	VAT 方式
CPU 主執行緒（ms）	154.3	16.5	12.6
GPU 影格（ms）	77.1	22.2	32.5

表 16.4: 顯示 16 個時的實測□摘要

指標	FBX 方式	Skin 方式	VAT 方式
繪製命令數（次/影格）	約 210,000	與 VAT 方式同水準	99
GPU 記憶體增量（0→16 個，MB）	+411.8	+38.2	幾乎 ±0

相對於 FBX 方式，Skin 方式與 VAT 方式在所有指標上都有大幅優勢。其原因可歸結為「能作為 1 個網格處理」。

Skin 方式與 VAT 方式的比較各有優缺點，GPU 處理時間是 Skin 方式有利，CPU 主執行緒與記憶體則是 VAT 方式有利。VAT 方式的 GPU 處理時間，很大程度取決於是否為頂點變形處理耗時較多的通道組成。因此，在進行頂點計算的通道數較少的組成，或以減輕 CPU 負載為目的時，VAT 會是首選。

□外，本次的比較是剛體破壞這種易以骨骼表現的題材。對於布料或人體這類骨骼數會變多、或會變成近似處理的模型，以及物件數量很多的情況，CPU 負載會很明顯，因此可認為 VAT 更具優勢。

16.4 VAT 資□的製作

製作 VAT 資□時，使用 Houdini 附帶的 Labs Vertex Animation Textures ROP（以下稱 VAT ROP）是較簡便的方法 [*1]。Houdini 是 SideFX 公司的 DCC 工具，擅長程序化生成。VAT ROP 備有 Unity 用的預設與對應的著色器，除了 Houdini □的模擬結果外，也能以其他 DCC 工具製作的 FBX 動畫作為輸入。

VAT ROP 有 表 16.5 這 4 種模式。

表 16.5: VAT ROP 的模式

模式	用途
Soft-Body Deformation	基本的頂點變形。條件是拓撲不變
Rigid-Body Dynamics	為剛體破壞而最佳化。以碎片為單位記錄動作，因此可將紋理大小抑制得較小
Dynamic Remeshing	支援液體或會撕裂的布料等拓撲會變化者
Particle Sprites	用於粒子模擬

[*1] 這是 SideFX Labs 套件所包含的工具。本章的驗證使用的是 3.0 版。

步驟只需在 Input Geometry 指定想做成 VAT 的動畫節點，在 Mode 指定使用的模式，在 Target Engine 指定 Unity，然後按下 Render All 按鈕。便會一併輸出 表 16.6 的資□。

表 16.6: VAT 播放所需的資□

資□	作用
位置用紋理	各頂點在各影格的位置（正規化為 0～1）
旋轉用紋理	變形後面的朝向。重建法線與切線以保持明暗
頂點色用紋理	Houdini 端屬性的烘焙目的地。可用於提升插□品質等的自由欄位
FBX	UV 中嵌入了表示紋理參照位置之識別編號的網格
材質	已設定影格數、FPS、邊界框等輔助資料
著色器	對應各模式的播放運算本體（□行安裝）

著色器不包含在輸出檔案中。這是因為運算處理不因動畫而異，是固定的。請從 Unity 的 Package Manager □行安裝 SideFX Vertex Animation Textures 套件（存放位置可從 VAT ROP 的 Real-Time Shaders 分頁確認）。套件中除了支援 4 種模式的 Shader Graph 外，還附帶文件與匯入設定用的 Preset。

16.5 導入時的注意事項

VAT 若精度不足，動畫就會壞掉，因此需要為紋理指定合適的格式。同樣地，FBX 端也有建議的匯入設定。請依照套件附帶的「VAT Import Settings Guide」，套用包含停用紋理壓縮在□的專用匯入設定。

此外，官方的 Shader Graph 定位上僅是範本，實務上需要自訂。不是直接使用範本，而是複製為專案用後再使用。不過，複製出來的 Shader Graph 仍會參照套件端的 Sub Shader Graph，因此請注意要將參照改為指向複製出來的那一側。

□外，輸出檔案的匯入設定，以及對材質指派紋理與著色器等參照設定必須手動進行，是容易因設定遺漏而出問題的部分。在量□的專案中，有以 Scripted Importer 等將匯入設定與參照解析自動化的空間。關於這方面的改良，請參□上傳至 CEDiL 的資料 [*2]。

[*2] https://cedil.cesa.or.jp/cedil_sessions/view/3263

16.6 總結

VAT 是將動畫烘焙到紋理中，並以 GPU 播放的手法。其特點是可大幅減少繪製呼叫，相對於每個碎片都持有 Renderer 的組成，與 Skin 方式同樣大幅提升了效率。兩者各有優缺點，GPU 處理時間是 Skin 方式有利，CPU 處理時間與記憶體則是 VAT 方式有利。在進行頂點計算的通道數較少的組成，或以減輕 CPU 負載為目的時，VAT 會是首選。雖然有需要準備專用資□、動作固定等限制，但在讓大量物件播放固定動畫的場合，會是強力的選項。

第17章

在 Unity 中活用 iOS 標準的效能測量功能

山崎 雅大 Yamazaki Masahiro

株式會社 Applibot

mao-test-h　**X** @TEST_H_

- 所屬: 株式會社 Applibot
- X: `https://x.com/TEST_H_`
- GitHub: `https://github.com/mao-test-h`

17.1　前言

Unity 具備以 Unity Profiler 為首的測量功能，而針對 iOS 應用程式，還可以□外活用作業系統標準的效能測量功能。

Apple 提供的這些功能不依賴 Unity 的 Development Build 旗標即可使用。例如，在尚未嵌入測量處理的階段，或是在接近已停用除錯功能的 Release Build 的組態下概略確認負載，以及僅從測試人員的裝置接收追蹤結果等用法。 以此掌握大致的負載並初步縮小瓶頸候選範圍之後，即可切換為□用 Development Build 的建置，再以 Unity Profiler 或 Instruments 進行詳細調□，如此分階段運用。

本文從 iOS 標準提供的功能中介紹以下 2 項，並進一步□明 macOS 27 / iOS 27 新增的功能。

- Metal Performance HUD
- Performance Trace

此外，已將這些功能整合為可從 Unity 使用的 OSS 兼範例儲存庫「MetalPerfKit-Unity」[*1] 公開，實作細節請一併參□該儲存庫。

※ 後續□明統一標示為 iOS，除非□有□明，否則亦包含 iPadOS。

17.1.1 驗證環境

本文在以下環境中進行了動作驗證。

- Unity 6000.0.81f1
- Xcode 26.6 / 27.0 beta
- iOS 26.6 / 27.0 beta
- macOS 27.0 beta

※ 撰寫時 iOS 27、macOS 27、Xcode 27 以及後述的 `StateReporter` API 均處於 Beta 階段。本文為 2026 年 8 月時的資訊，正式發布前 API、指令、設定畫面及行為等可能有所變更。

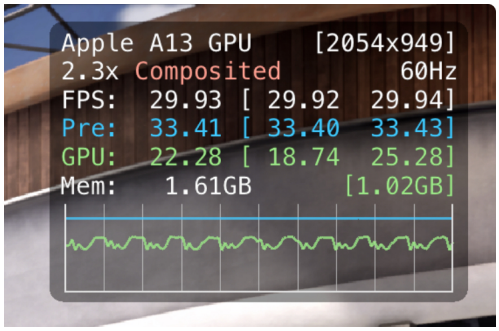
17.2 Metal Performance HUD

Metal Performance HUD 是可將使用 Metal 的應用程式的效能資訊即時疊加顯示於畫面上，或以日誌輸出的工具。 Apple 的文件 [*2] 將其介紹為能快速掌握目前狀態，並找出該以 Xcode 或 Instruments 詳細擷取之範圍的功能。

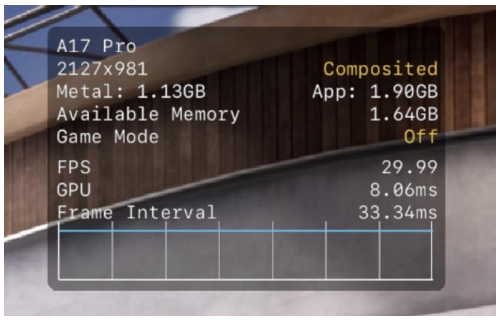
此功能本身自 Metal 4 之前 (iOS 16 世代) 就已存在，並配合 2025 年 WWDC 發表的 Metal 4 (iOS 26 世代) 大幅強化。 Metal 4 之前與 Metal 4 之後的顯示差異如 圖 17.1 與 圖 17.2 所示。

[*1] MetalPerfKit-Unity `https://github.com/mao-test-h/MetalPerfKit-Unity`

[*2] Monitoring your Metal app’s graphics performance `https://developer.apple.com/documentation/xcode/monitoring-your-metal-apps-graphics-performance`



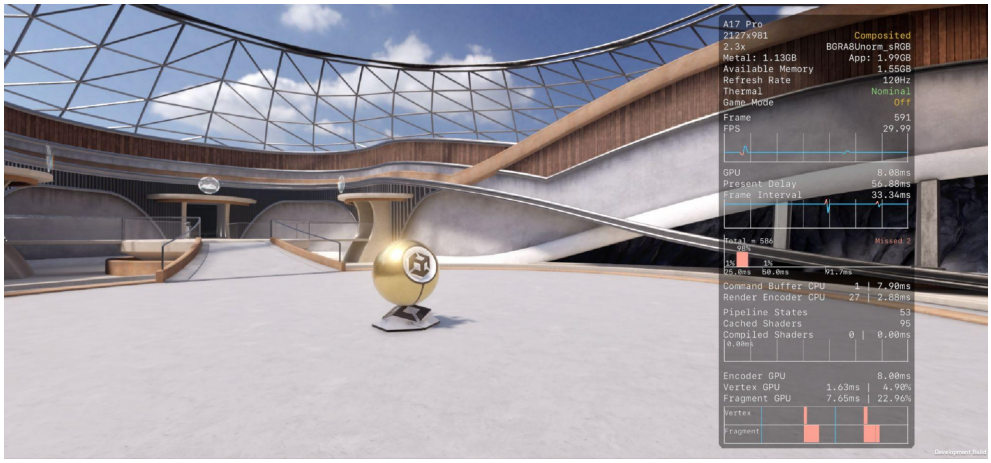
▲圖 17.1　Metal 4 之前的 Metal Performance HUD



▲圖 17.2　Metal 4 之後的 Metal Performance HUD

Metal 4 的主要強化重點包括擴充測量項目、可自訂顯示位置、不透明度與縮放，以及增加可由 API 動態控制的項目。

顯示主要指標的狀態範例如 圖 17.3 所示。



▲圖 17.3　顯示主要指標的狀態範例

各指標的詳細內容請參閱官方文件「Understanding the Metal Performance HUD metrics」[*3]。

此外，筆者過去也公開過關於 Metal Performance HUD 的解說文章 [*4] 與技術同人誌「UniTips Vol.12」[*5]。其中說明了基本用法與 Unity 的活用技巧，詳情請一併參考。

以下各節以 Metal 4 (iOS 26 世代) 的內容為基礎進行說明。

17.2.1　開啟 HUD 的方法

顯示 HUD 的方法有數種，這裡說明「從設定 App 開啟」與「以 API 控制顯示」2 種。

從設定 App 開啟

可從 iOS 裝置的設定 App 啟用，步驟如下。

※ 前提是必須已顯示 `Developer` 選單。此外，依 OS 版本不同，選單名稱可能有所

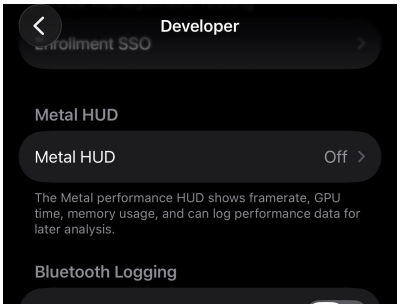
[*3] Understanding the Metal Performance HUD metrics `https://developer.apple.com/documentation/xcode/understanding-metal-performance-hud-metrics`

[*4] 【Unity】在行動應用程式上測量 GPU 負載 `https://qiita.com/mao_/items/3b531d17cacb86036b7f`

[*5] UniTips Vol.12（於「第 2 章: Metal 4 起強化的 Metal Performance HUD 的 Unity 活用技巧」中說明）`https://techbookfest.org/product/8mHpXz8LJCKAsLxENJKwaz?productVariantID=ewK5GtNTjkz4k8heM4BrxU`

差異。

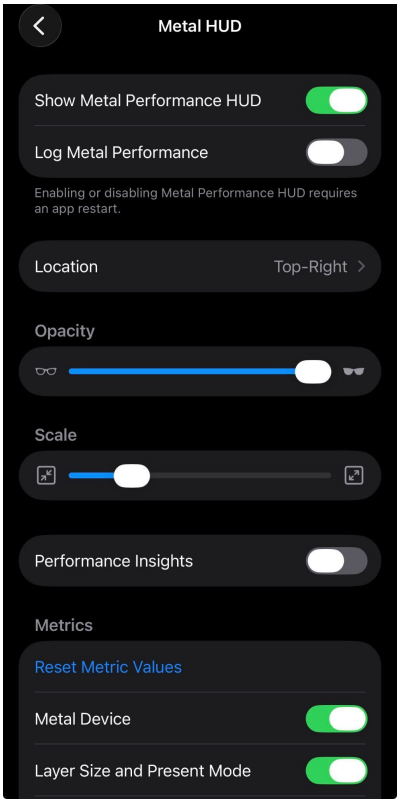
1. 開啟 iOS 的「設定」App
2. 開啟 Developer 選單，再開啟其中的 Metal HUD (參照 圖 17.4)



▲圖 17.4　從 Developer 選單開啟 Metal HUD

開啟 Metal HUD 後會顯示 圖 17.5 畫面，啟用其中的 Show Metal Performance HUD 並重新啟動目標應用程式即可顯示。

在此畫面中，也可以切換顯示的指標，以及變更不透明度、縮放與顯示位置等。 此外，啟用同一區段中的 Log Metal Performance，即可每隔固定間隔將效能資訊輸出為日誌。詳情於後述的「關於日誌功能」說明。



▲圖 17.5　Metal HUD 的設定畫面

以 API 控制顯示

若要在應用程式執行中動態控制 HUD 的顯示，請將設定⌧傳給 `CAMetalLayer` 類別的 `developerHUDProperties` 屬性。清單 17.1 中⌧用了 HUD 與日誌輸出，並將顯示位置指定為左上角。

▼清單 17.1　以 developerHUDProperties 控制 HUD 的顯示

```
myMetalLayer.developerHUDProperties = [
   "mode": "default",      // HUD 的 ON/OFF
   "logging": "default",   // 日誌功能的 ON/OFF
   "positionX": 0,         // 顯示位置 以像素為單位的( X 座標)
   "positionY": 0,         // 顯示位置 以像素為單位的( Y 座標)
]
```

若要隱藏 HUD，請對 mode 指定 disabled。過去此屬性僅能控制 HUD 與日誌功能的 ON/OFF，無法調整位置等，但自 Metal 4 起，可透過 positionX / positionY 以像素為單位指定位置。

此外，也可以將 HUD 支援的環境變數鍵傳入此 Dictionary。例如傳入 MTL_HUD_OPACITY 與 MTL_HUD_SCALE，即可在 0.0 至 1.0 的範圍內調整不透明度與縮放（清單 17.2）。

▼清單 17.2　以環境變數鍵調整不透明度與縮放

```
myMetalLayer.developerHUDProperties = [
   ...,
   "MTL_HUD_OPACITY": opacity,      // 不透明度 (0.0 至 1.0)
   "MTL_HUD_SCALE": scale,          // 縮放 (0.0 至 1.0)
]
```

其他還有許多可設定的項目，詳情請參閱官方文件「Customizing the Metal Performance HUD」[*6]。

從 Unity 呼叫時，需以 iOS 原生外掛程式取得用於繪製的 CAMetalLayer，再由 C# 端透過 P/Invoke 呼叫。CAMetalLayer 可經由 UnityFramework 以 清單 17.3 的方式取得。

▼清單 17.3　經由 UnityFramework 取得 CAMetalLayer

```
private func getMetalLayer() -> CAMetalLayer? {
   guard let instance = UnityFramework.getInstance(),
         let rootView = instance.appController().rootView,
         let metalLayer = rootView.layer as? CAMetalLayer else {
       return nil
   }
   return metalLayer
}
```

MetalPerfKit-Unity 將這些實作公開為 PerformanceHUDSwitcher 類別，可從 C# 以清單 17.4 的方式呼叫。此外，原生 API 以像素為單位指定位置，而 PerformanceHUDSwitcher 為了不依賴解析度，以 0.0 至 1.0 的相對位置接收座標，並在內部依繪製區域的大小轉換為像素座標。

[*6] Customizing the Metal Performance HUD https://developer.apple.com/documentation/xcode/customizing-metal-performance-hud

▼清單 17.4　PerformanceHUDSwitcher 的呼叫範例

```
// 將 HUD 顯示於畫面右上 (x=1.0, y=0.0)
// 座標以( 0.0 至 1.0 的相對位置指定)
PerformanceHUDSwitcher.SetPerformanceHUDVisible(true, 1.0f, 0.0f);
```

完整的實作範例請參⌧ MetalPerfKit-Unity 的原始碼。

17.2.2　關於日誌功能

⌧用前述的 `Log Metal Performance` 或 API 的 `logging` 後，應用程式執行中會每隔 1 秒將效能資訊輸出為日誌。 日誌以 `metal-HUD:` 開頭的 清單 17.5 格式輸出，可在 Xcode 的主控台或 macOS 的 Console.app 中確認。

▼清單 17.5　metal-HUD 日誌的輸出格式

```
metal-HUD: <first-frame-number-integer>,<graphics-memory-usage-float>,<process-memory-
    usage-float>,<first-frame-present-interval-float>,<first-frame-gpu-time-float>,...<
    last-frame-present-interval-float>,<last-frame-gpu-time-float>
```

由開頭依序記錄「開始取樣的影格編號」「圖形記憶體使用量」「整個處理程序的記憶體使用量」，之後則重複記錄各取樣影格的 Frame Present Interval 與 GPU Time 配對。 實際輸出範例如 圖 17.6 所示。



▲圖 17.6　Metal Performance HUD 的日誌輸出範例

若以逗號分隔資料進行彙整，即可事後分析影格間隔的紊亂、GPU Time 及記憶體使用量的變化。 此外，只要這些日誌仍保留在 Unified Logging 中，也可以使用 `OSLogStore` API 實作「取得過去 N 秒的日誌並儲存為檔案」。 MetalPerfKit-Unity 以 `Performan`

ceLogger 類別提供此功能（清單 17.6）。

▼清單 17.6　以 PerformanceLogger 取得並儲存日誌

```
// 取得過去 60 秒的日誌並儲存為檔案
PerformanceLogger.FetchPerformanceLogs(60, FileUtility.GenerateFetchLoggingFilePath());
```

17.3　Performance Trace

Performance Trace 是 iOS 26 新增的功能，無須連接 Mac，僅以 iOS 裝置本身即可收集效能相關資料。收集到的追蹤檔案可傳送至 Mac，以 Instruments 或後述的 `metal perftrace` 指令進行分析。

由於僅在裝置上即可完成，可記錄長時間遊玩等連接 Mac 時難以重現的情況，也可以請測試人員等他人分享其裝置上收集的追蹤檔案，在本機進行分析。

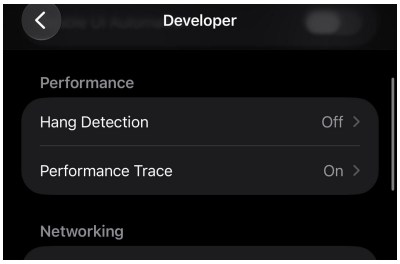
本章主要介紹其中在 WWDC 2026 與功能強化一併□明的 Lookback Collection。

17.3.1　使用方式

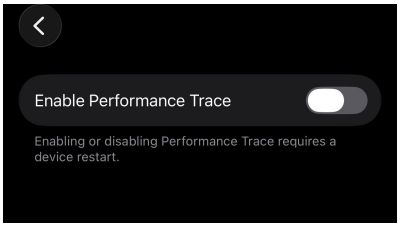
Performance Trace 必須從 `Developer` 選單□用。步驟如下。

1. 開□ iOS 的「設定」App
2. 開□ `Developer` 選單，再開□其中的 `Performance Trace`
3. □用 `Enable Performance Trace` (切換□用/停用需要重新□動裝置)
4. 選擇任意的 `Tracing Mode` (詳見後述)

`Developer` 選單中的 `Performance Trace` 及其設定畫面分別如 圖 17.7 與 圖 17.8 所示。

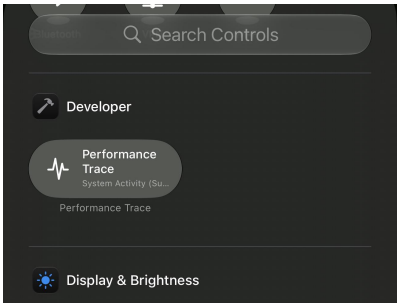


▲圖 17.7　Developer 選單中的 Performance Trace

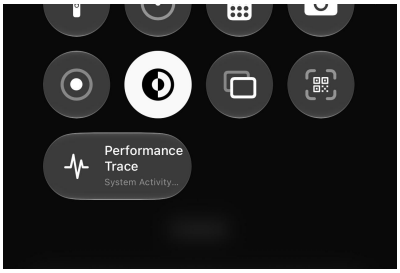


▲圖 17.8　Enable Performance Trace 的設定畫面

□用完成後，為了進行追蹤的開始與停止、追蹤資料的收集等操作，接著要在控制中心加入 `Performance Trace`。開□控制中心，即可從 `Developer` 區段加入。加入步驟與加入後的狀態如 圖 17.9 與 圖 17.10 所示。



▲圖 17.9　將 Performance Trace 加入控制中心



▲圖 17.10　已加入 Performance Trace 控制項的狀態

17.3.2　關於 Tracing Mode

`Tracing Mode` 有 表 17.1 所示的 4 種 (System Activity 分為 Summary / Detailed 2 種)，也可以長按控制中心的 `Performance Trace` 控制項進行切換。

表 17.1: Tracing Mode 的種類

模式	概要
Power Profiler	記錄對電力消耗的影響
System Activity (Summary / Detailed)	記錄系統的運作狀況
Lookback Collection	回溯收集系統持續記錄的資料

Power Profiler 與 System Activity 是點選控制中心的 `Performance Trace` 控制項開始記錄，再次點選即停止。 □一方面，Lookback Collection 無須記錄操作，會從點選的時間點回溯收集資料。詳情於後述。

收集到的追蹤檔案會列於 Performance Trace 畫面的 `Available Trace Files`，可從分享按鈕以 AirDrop 等方式傳送至 Mac 或其他開發者。 展開分享的封存檔，即可得到能以 Instruments 開□的 `.atrc` 檔案。 此外，當裝置的儲存空間不足時，追蹤檔案會被自動刪除，也可以滑動手動刪除。

各模式的補充□明如下。

Power Profiler

Power Profiler 是測量應用程式對 CPU / GPU / 顯示器 / 網路等各子系統的使用, 對裝置電力消耗所造成影響 (Power Impact) 的模式。 本文省略詳細□容，官方文件「Measuring

your app’s power use with Power Profiler」[*7] 與 WWDC 2025 的場次「Profile and optimize power usage in your app」[*8] 有詳細說明。

System Activity

System Activity 是備有「Summary」與「Detailed」2 種的追蹤模式。就筆者確認的範圍，撰寫時 (2026 年 8 月) 在 Apple 的文件與 WWDC 場次中，並未找到 System Activity 詳細規格，以及 Summary 與 Detailed 差異的說明。因此本文不深入各模式的細節，僅介紹實際追蹤中確認到的資訊。

以下作為參考，列出在搭載 iOS 27 的裝置上記錄 System Activity (Summary) 追蹤，並以後述的 `metalperftrace` 指令分析的結果 清單 17.7。

▼清單 17.7　System Activity (Summary) 追蹤的分析結果

```
$ metalperftrace overview system_activity_summary_ios27.atrc
[MetalPerfKit-Unity pid:1974]
   Mem: 799.0 MiB (799.0 Peak, 616.8 Metal, 2577.0 Headroom)
   Total CPU Time: 12.679s (20.47% Sys, 79.53% User)
   Instructions: 43.33B (60.88% P, 39.12% E)
   Cycles: 21.04B (38.97% P, 61.03% E)
   Disk Reads / Writes: 9.13 / 0.08 MiB (Logical Write 0.07)
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics: 3 transitions, last known state:
    URP-HighFidelity
     com.DefaultCompany.MetalPerfKit-Unity.screen: 11 transitions, last known state:
    Title
 UnityView (UnityDefaultViewController) (1290x2796)  Interval 30.037s  Active 30.037s
   44.5 FPS  1202 Frames  1202 Detached  11 Skipped
   Frame Time           avg:  22.47ms  min:  16.67  max: 420.93  stddev:  30.68  p95:
    50.01  p99: 133.36
   CPU Begin-to-Present avg:  17.24ms  min:   1.14  max: 794.63  stddev:  31.76  p95:
    31.28  p99: 111.73
   On-GPU Time          avg:  15.50ms  min:   1.81  max: 424.87  stddev:  37.24  p95:
    51.17  p99: 134.48
   Next Drawable Wait   avg:  14.53ms  min:   0.00  max: 401.43  stddev:  32.11
```

在此追蹤中，除了各處理程序的記憶體使用量、CPU 時間與磁碟 I/O 之外，還能確認以

[*7] Measuring your app’s power use with Power Profiler https://developer.apple.com/documentation/xcode/measuring-your-app-s-power-use-with-power-profiler

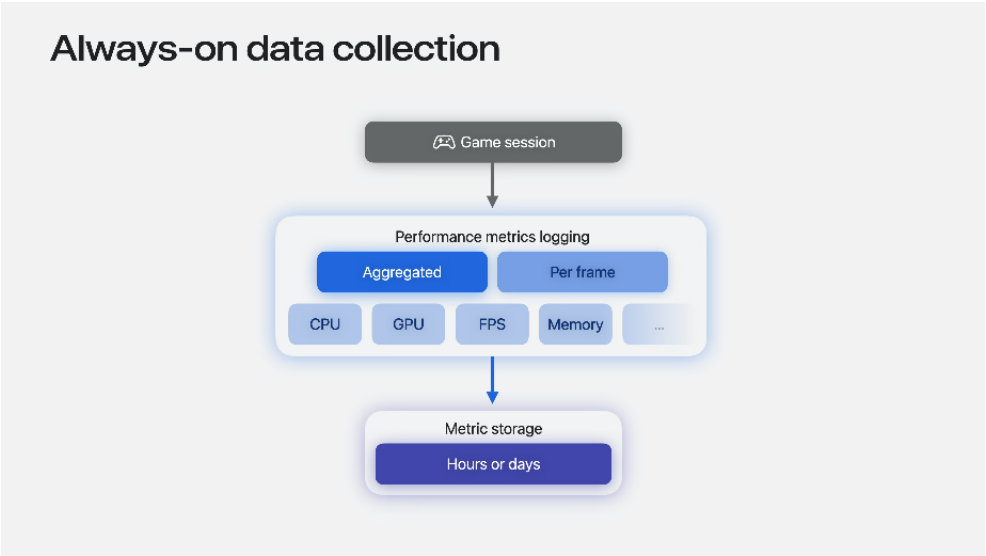
[*8] WWDC 2025 - Profile and optimize power usage in your app https://developer.apple.com/videos/play/wwdc2025/226/

`CAMetalLayer` 為單位的 FPS、Frame Time、CPU Begin-to-Present、On-GPU Time 等統計。 此外，`[States]` 項目中記錄了應用程式的狀態轉換，這是下一章□明的 `StateReporter` API 所□生的，詳情於後述。

Lookback Collection

Lookback Collection 是事後回溯收集系統持續記錄之效能資料的模式。就筆者確認，Lookback Collection 在 iOS 26 時已存在於 `Tracing Mode` 的選項中，但 WWDC 2025 與官方文件並未□明。其後在 WWDC 2026 強化了功能，並公開了□明其□容的場次影片「Find and fix performance issues in your Metal games」[*9]。

根據演講，遊戲執行期間系統會持續記錄效能指標 (CPU、GPU、FPS、記憶體等)，並保存數日份的資料。 Lookback Collection 利用了這個機制，因此即使是測試人員長時間的遊玩工作階段，也能回溯取得資料。 此機制的概要如 圖 17.11 所示。



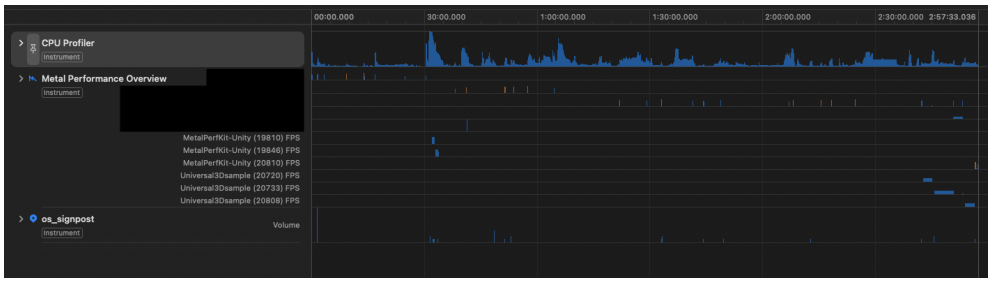
▲圖 17.11　系統持續記錄效能指標（取自 WWDC 2026 場次）

使用方式只要照常進行遊玩測試後，點選控制中心的 `Performance Trace` 控制項即可。 從點選的時間點起，會回溯所設定的時間長度開始收集與處理追蹤，完成後會收到

[*9] WWDC 2026 - Find and fix performance issues in your Metal games `https://developer.apple.com/videos/play/wwdc2026/388/`

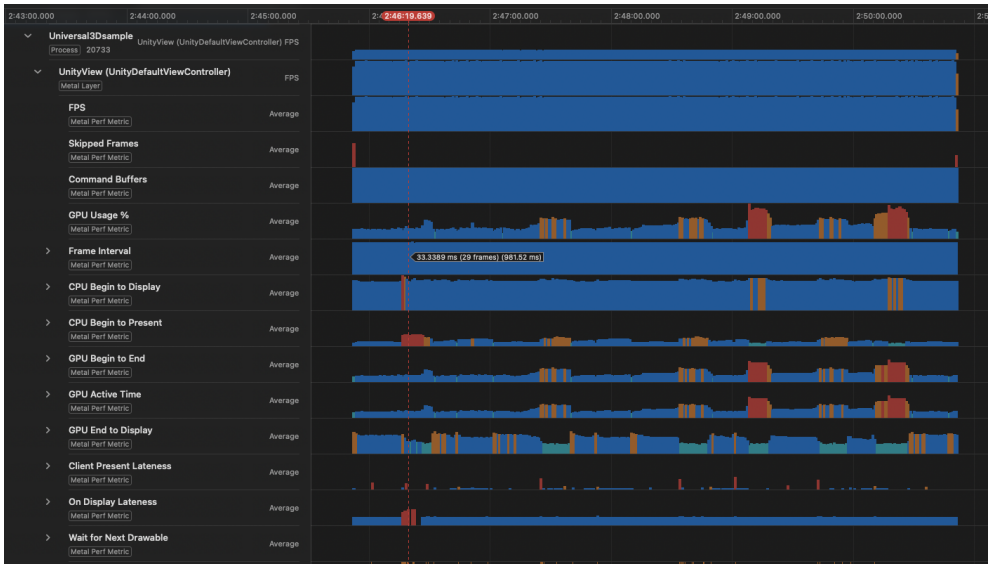
通知並新增至 `Available Trace Files`。

圖 17.12 為在搭載 iOS 26 的裝置上實際收集約 3 小時份的 Lookback Collection 追蹤，並以 Instruments 開□的範例。 回溯期間□□動過的多個應用程式 (處理程序) 會以清單顯示，可綜觀各自的 FPS 等變化。



▲圖 17.12　以 Instruments 開□ Lookback Collection 追蹤的範例

進一步展開處理程序，即可如 圖 17.13 所示，以 `CAMetalLayer` 為單位確認 FPS、GPU Usage %、Frame Interval、CPU / GPU 時間等影格指標的變化。



▲圖 17.13　以 CAMetalLayer 為單位的影格指標變化

※ 在作者手邊的 iOS 27 環境中，彙整期間會顯示 `Error processing Performance Trace`，無法正常運作。關於此問題，即使在 iOS / iPadOS 27 正式發布時，發行□明的 `Control Center -> Known Issues` 中仍記載了已知問題 (184729556)：「收集 Power Profiler 或 Lookback 的追蹤時，可能不會□生追蹤」。[*10]

17.4 macOS 27 / iOS 27 新增的功能

如前一章所介紹，遊戲執行期間系統會持續記錄並保存 Metal 效能指標。 WWDC 2026 以此機制為核心，發表了為追蹤附加應用程式專屬資訊的 `StateReporter` API，以及可從 CLI 分析追蹤的 `metalperftrace` 指令。

17.4.1 StateReporter API

即使能從追蹤資料找出影格率下降的時間點，僅憑 Metal 效能指標，也無從得知當時遊戲在做什麼。 這是因為「正在開□遊戲的□個場景」「物件數量有多少」「圖形設定為何」等應用程式專屬的資訊，不會自動包含在 Metal 效能指標中。

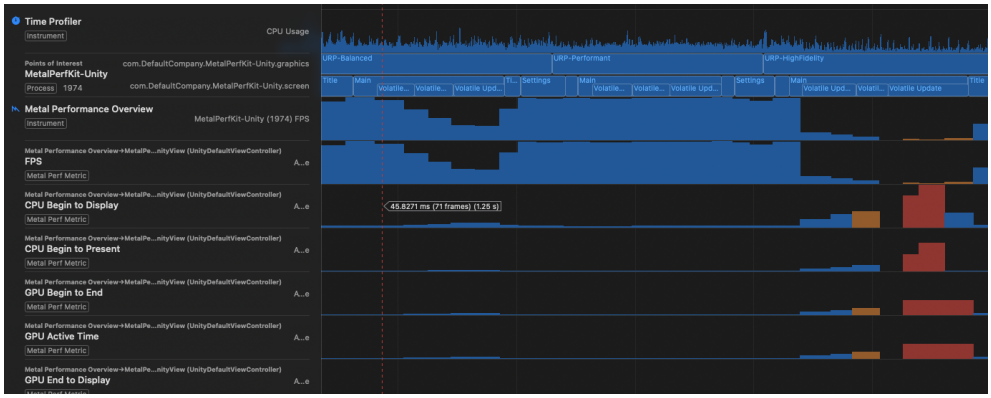
使用 iOS 27 新增的 `StateReporting` 框架 [*11] 的 `StateReporter` API [*12]，即可向系統回報應用程式的狀態，為追蹤資料附加應用程式專屬的情境。

所回報的狀態，在 `metalperftrace` 中用於依狀態彙整，在 Instruments 中則用於顯示 Points of Interest。 以 Instruments 開□前一章所列 System Activity 追蹤的範例如圖 17.14 所示。所回報的狀態會顯示為 Points of Interest 的軌道。

[*10] iOS & iPadOS 27 Release Notes `https://developer.apple.com/documentation/ios-ipados-release-notes/ios-ipados-27-release-notes`

[*11] Getting started with StateReporting `https://developer.apple.com/documentation/StateReporting/getting-started-with-statereporting`

[*12] StateReporter `https://developer.apple.com/documentation/statereporting/statereporter`



▲圖 17.14　所回報的狀態顯示為 Points of Interest 的範例

`StateReporter` 由 表 17.2 所示的 4 個要素構成。

表 17.2: 構成 StateReporter 的 4 個要素

要素	概要	範例
Domain	回報對象的功能領域。為反向 DNS 格式的字串，每個 Domain 同時只能有 1 個作用中的狀態	`com.mygame.level`
Label	表示目前狀態的短字串	`Level 1`
Stable metadata	在狀態存續期間不變的中繼資料。與 Label 組合以唯一識別狀態	`{id: 1001}`
Volatile metadata	在狀態中可能變化的揮發性中繼資料。更新時不會發生狀態轉換	`{health: 20}`

Domain 可定義多個，例如可將「關卡進度」「圖形設定」「網路狀態」分別作為獨立的 Domain 回報，並組合進行分析。

從 Swift 的使用方式如 清單 17.8 所示。

▼清單 17.8　從 Swift 使用 StateReporter 的範例

```
import StateReporting
// 取得對應 Domain 的 Reporter
let reporter = StateReporter.reporter(for: "com.mygame.gameplay")
```

```
// 回報狀態轉換
reporter.reportTransition(to: "Playing")
// 若要結束狀態，請傳入 nil
reporter.reportTransition(to: nil)
```

從 Unity 使用

StateReporting 是供 Swift / Objective-C 使用的 API，因此從 Unity 使用需要原生外掛程式。 MetalPerfKit-Unity 以預覽功能公開了封裝此 API 的 StateReporter 類別，可從 C# 以 清單 17.9 的方式使用。

▼清單 17.9　從 C# 使用 StateReporter 的範例

```
using MetalPerfKit;
// 取得對應 Domain 的 Reporter
var reporter = StateReporter.ReporterForDomain($"{Application.identifier}.screen");
// 回報狀態轉換 附揮發性中繼資料()
reporter.ReportTransitionToStateLabel(
   "Main",
   volatileMetadata: new Dictionary<string, StateReporterMetadataValue>
   {
       ["CubeCount"] = cubeCount,
   });
// 維持狀態不變，僅更新揮發性中繼資料
reporter.ReportVolatileMetadataUpdate(
   new Dictionary<string, StateReporterMetadataValue>
   {
       ["CubeCount"] = cubeCount,
   });
```

此外，儲存庫的 Assets/_Example/2_MetalPerformanceMetrics 備有將畫面轉換 (Title / Main / Settings) 與圖形品質 (URP 的 3 個等級) 分別作為獨立 Domain 回報的範例。 前一章所列 System Activity 追蹤中的 [States]，即是以此範例記錄的。

17.4.2　metalperftrace

metalperftrace 是 macOS 27 新增的 CLI 工具，除了分析追蹤檔案之外，也能收集 macOS 應用程式的效能指標。 收集功能的對象為在 macOS 上執行的應用程式，因此本文不予討論，但分析由 iOS 裝置收集的追蹤檔案時可活用此工具。

關於追蹤檔案的分析

追蹤檔案的分析使用 `overview` 子指令（清單 17.10）。

▼清單 17.10　以 overview 子指令分析追蹤檔案

```
# 分析已收集的追蹤檔案
metalperftrace overview MyGameTrace.atrc
```

輸出⌧容如前一章所介紹。 若再傳入 `--include-state-transitions`，也能確認以 `StateReporter` 記錄的狀態轉換詳情。 清單 17.11 為分析前述範例應用程式追蹤的範例，輸出了各圖形品質的變更歷程，以及在該狀態的停留時間與比例。

▼清單 17.11　包含狀態轉換詳情的分析結果

```
$ metalperftrace overview --include-state-transitions system_activity_summary_ios27.
    atrc
[MetalPerfKit-Unity pid:1974]
   ...
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics
       URP-Balanced  (32.75%, 9.904s)
         (2026-08-16T14:22:33.025Z → 2026-08-16T14:22:42.930Z)
       URP-Performant  (29.43%, 8.900s)
         ...
       URP-HighFidelity  (37.82%, 11.437s)
         ...
     com.DefaultCompany.MetalPerfKit-Unity.screen
       Title  (5.07%, 1.535s)
         ...
       Main  (21.09%, 6.379s)
         ...
       Title  (2.22%, 0.671s)
         ...
       Settings  (6.28%, 1.899s)
         ...
```

※ 輸出經過部分節錄與整理。實際上相同的狀態也會依時間序列重複輸出，並各自附有開始與結束的時間戳記。

此外，傳入 `--aggregate` 即可依狀態彙整指標。 指定 `--domain com.mygame.graphics` 可篩選至特定 Domain，並搭配 `--state-label "High"` 則可進一步篩選至特定狀

態。只需 1 個指令，即可確認「僅在圖形設定為 High 時平均 FPS 下降」這類應用程式狀態與效能的相關性。

此外，傳入 `--json` 也能以 JSON 格式取得這些輸出。演講中也提到「結構化輸出可用於以☒本進行的迴歸測試，以及由 AI 代理對效能問題進行自動分類」。Apple 的「Metal Developer Tools」頁面 [*13] 也以「Hello, agentic debugging」為題，介紹了結合代理的工作流程。

[*13] Metal Developer Tools `https://developer.apple.com/metal/tools/`

第18章

依裝置切換繪製設定

佐久間 尚輝 NAOKI SAKUMA

株式會社 CyberAgent SGE 核心技術本部

智慧型手機遊戲預期會在各式各樣的裝置上執行。若一律採用相同的繪製設定，在低規格裝置上可能導致當機、掉幀與發熱。反之，在高規格裝置上，則有可能發揮裝置效能，提供更好的畫質。因此，本節介紹依裝置切換為適當繪製設定的機制。

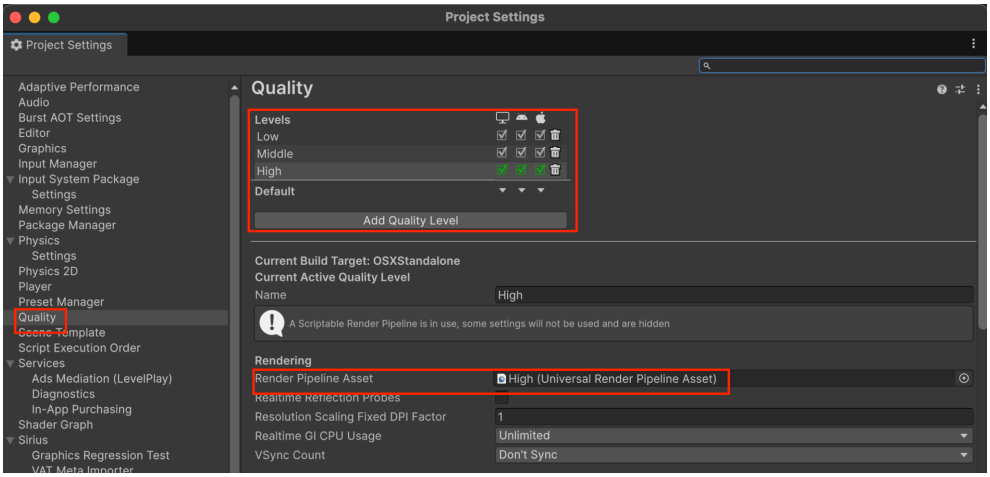
18.1　切換方式

遊戲中常見的做法有兩種：低、中、高這類預設組合形式，以及由使用者直接調整數□的形式。

18.1.1　預設組合形式

此方式適用於以預設組合為單位進行切換的情況。讓使用者選擇預設組合，或由裝置自動判定時，也採用此方式。

在 ProjectSettings 的 Quality 中按下 Add Quality Level，依預設組合的數量新增。接著，為各預設組合附加對應的 RenderPipelineAsset。也可以視需要設定 Quality 的其他項目（Rendering、Textures 等）。



在程式碼中呼叫 QualitySettings.SetQualityLevel，即可切換為任意的 QualityLevel。

18.1.2 直接調整數□的形式

讓使用者調整參數時，採用此方式。可用如下程式碼直接調整參數。

```
var urpAsset = (UniversalRenderPipelineAsset)GraphicsSettings.currentRenderPipeline;
// 變更解析度
urpAsset.renderScale = value;
// 變更陰影的級聯數量
urpAsset.shadowCascadeCount = value;
// 其他還有陰影解析度、額外光源的數量等
```

18.2 依裝置切換

依裝置不同，也可能需要推薦或自動設定預設組合。此時 SystemInfo 型別很方便。

尤其是 deviceModel、graphicsDeviceName、systemMemorySize，可取得較易於識別的□。以下為回傳□的範例。

表 18.1: 各裝置 SystemInfo 回傳□的範例

機型	deviceModel	graphicsDeviceName	systemMemorySize
iPhone 13	iPhone14,5	Apple A15 GPU	約 3800

表 18.1: 各裝置 SystemInfo 回傳□的範例

機型	deviceModel	graphicsDeviceName	systemMemorySize
iPad Pro 11-inch（第 3 代）	iPad13,4	Apple M1 GPU	約 7800
Galaxy S23	samsung SM-S911B	Adreno (TM) 740	約 7500
Pixel 8	Google Pixel 8	Mali-G715	約 7400

iOS 的 deviceModel 與 graphicsDeviceName 命名規則一致，且基本上在同一系列□，A 或 M 之後的數字越大，規格越高。因此，組合使用即可在一定程度上加以區分。

Android 則因機型不同，回傳□並不一致，難以像 iOS 那樣一概比較。因此，建議採用下列方式區分：

- 僅以 RAM 容量判斷
- 將判定資料放到主資料（master）中進行比對
- □動時執行基準測試工具

等。

18.3 自製 RendererFeature

建立自製的 RendererFeature 等時，預先提供可切換繪製設定的 API 也很方便。例如可舉出以下對應：

- 降低暫存紋理的格式或解析度
- 減少 Blit 的次數
- 改用較簡易的著色器

等。

第19章

UniTask 與取消處理的生命週期管理

稻田 健人 Inada Taketo

株式會社 CyberAgent SGE 核心技術本部

所屬：株式會社 CyberAgent SGE 核心技術本部

X 帳號 / 網站 / GitHub 帳號（選填）：

在使用 UniTask 的非同步處理中，若未適當管理結束條件與取消，可能會殘留永遠不會完成的處理與不必要的參照。

前作《Unity 效能調優寶典》介紹了 MonoBehaviour 被銷毀後 UniTask 仍持續等待的案例。本節將介紹因 CancellationTokenSource 漏掉銷毀，導致不需要的物件持續被參照的案例。

⌧外，CancellationTokenSource 與 CreateLinkedTokenSource 並非 UniTask 特有的功能，而是構成 .NET 取消處理的 API。

本節所用的程式碼中，using 宣告（using var）是 C# 8 的語法，可在 Unity 2020.2 以後使用。此外，後述 CreateLinkedTokenSource 的⌧部實作，參照的是 Unity Technologies 公開的 Unity 2022.3 用 Mono 實作（unity-2022.3-mbe 分支），可能因 Unity 版本而異。

19.1 CancellationTokenSource 的 Dispose

CancellationToken 是用來向處理傳達取消請求的實⌧型別。CancellationToken 本身

並未實作 IDisposable，因此不需要呼叫 Dispose。

不過，以 CancellationToken.Register 註冊回呼時，其傳回 CancellationTokenRegistration 是 IDisposable[*1]。取消前註冊的回呼會由發出者的 CancellationTokenSource 持有，因此若在不再需要註冊時未以 Dispose 解除，回呼所參照的物件可能無法釋放。

一方面，CancellationTokenSource 實作了 IDisposable。使用完畢的 CancellationTokenSource 由擁有者呼叫 Dispose。

▼清單 19.1　Dispose CancellationTokenSource 的範例

```
private async UniTask LoadCancelableAsync()
{
    using var cts = new CancellationTokenSource();

    // 省略發出取消請求的處理
    await LoadAsync(cts.Token);
}
```

清單 19.1透過 using 宣告來管理 CancellationTokenSource。不僅在非同步處理正常結束時，即使因取消或例外而結束，離開方法時也會自動呼叫 cts.Dispose()。

Cancel 與 Dispose 的作用不同。Cancel 是向處理請求取消，Dispose 則是解除、釋放 CancellationTokenSource 所持有的計時器與回呼註冊等。僅呼叫 Dispose，並不會向處理通知取消請求。

外，即使忘了呼叫 Dispose，只要 CancellationTokenSource 與相關物件都不再被外部參照，就會成為 GC 的回收對象[*2]。不過，向長壽命的 CancellationTokenSource 註

[*1] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokenregistration.dispose（瀏覽日期：2026年9月8日）

[*2] https://docs.unity3d.com/ja/2022.3/Manual/performance-managed-memory.html（瀏覽日期：2026年9月8日）

冊回呼等情況可能使參照殘留，因此原則上應在使用完畢時 Dispose。

19.2 CancellationTokenSource.CreateLinkedTokenSource

CancellationTokenSource.CreateLinkedTokenSource 會建立監視多個來源 Token 的新 CancellationTokenSource。只要任一來源 Token 被取消，所建立的 CancellationTokenSource 的 Token 就會成為已取消狀態。

來源 Token 之間不會互相取消，取消是從來源 Token 傳播到連結目標。

以下將 tokenA 與 tokenB 的發出者 CancellationTokenSource 分別稱為 A、B。

在 Unity Technologies 於 GitHub 公開的 Unity 2022.3 用 Mono 實作[*3]中，CreateLinkedTokenSource 會向來源 Token 註冊取消回呼。將與參照關係相關的部分簡化後，如清單 19.2所示。

▼清單 19.2　CreateLinkedTokenSource 內部實作的要點

```
linkedCts.m_linkingRegistrations[0] =
    tokenA.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);

linkedCts.m_linkingRegistrations[1] =
    tokenB.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);
```

[*3] https://github.com/Unity-Technologies/mono/blob/unity-2022.3-mbe/mcs/class/referencesource/mscorlib/system/threading/CancellationTokenSource.cs（瀏覽日期：2026年9月8日）

linkedCts 被登錄於來源 Token 的回呼所參照，而 linkedCts 也持有對 A 與 B 的註冊資訊。

因此，若 B 是整個應用程式使用的長壽命 CancellationTokenSource 且長期未被取消，在不 Dispose linkedCts 的情況下，可能殘留以下參照。

長壽命的 B → 回呼註冊 → linkedCts → 對 A 的註冊資訊 → 短壽命的 A

單憑這個參照關係，並不一定會造成記憶體流失。當所有物件都不再被外部參照時，就會一併成為 GC 的回收對象。然而，只要 B 仍存活且連結用的回呼尚未解除，linkedCts 就會持續被 B 參照，短壽命的 A 也會從那裡維持可到達的狀態。

此外，若註冊於 linkedCts.Token 的回呼參照了其他物件，這些物件也可能被保留。

Dispose linkedCts 後[*4]，□部對 A 與 B 的 CancellationTokenRegistration 會被銷毀，註冊於來源 Token 的回呼也會解除。

CreateLinkedTokenSource 的傳回□，最好在使用連結 Token 的處理範圍□，以 using 宣告等方式管理。

▼清單 19.3　Dispose 連結後的 CancellationTokenSource 的範例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    using var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
```

[*4] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokensource.dispose（瀏覽日期：2026年9月8日）

```
            tokenB);


    await LoadAssetAsync(linkedCts.Token);
}
```

清單 19.3中，LoadAssetAsync 在正常結束、取消或因例外而結束後會呼叫 linkedCts.Dispose()，解除對來源 Token 的註冊。

即使連結後的 CancellationTokenSource 已是已取消狀態，對其他來源 Token 的註冊仍可能殘留。CreateLinkedTokenSource 的傳回□在不再需要時，務必 Dispose。

19.3 在團隊中防止 IDisposable 物件漏掉 Dispose

到目前為止，以 CancellationTokenSource 為例，探討了 IDisposable 的銷毀遺漏。這些對策的前提是物件的擁有者在適當的時機呼叫 Dispose。然而，隨著專案規模擴大，要僅靠程式碼審□持續確認所有建立位置是很困難的。尤其 CreateLinkedTokenSource 只需簡短的寫法即可建立，因此容易忽略其傳回□是 IDisposable。

要在團隊中防止再次發生，除了個人的留意之外，建議結合以下做法：將需要銷毀的物件整合到易於使用的 API、以 Analyzer 及早偵測問題，並視需要在 CI 偵測新增的違規。不要讓 Analyzer 單獨承擔所有責任，而從設計、靜態分析與運用各個面向支□同一方針，會更有效。

此處介紹的以 using 銷毀與以 Analyzer 偵測，不限於 CancellationTokenSource，也可應用於其他實作 IDisposable 的資源。

本節在這類機制化的做法中，介紹使用能自動檢□ C# 程式碼的 Roslyn Analyzer 的方法。

19.4 什麼是 Roslyn Analyzer

Roslyn Analyzer 是在編譯時或 IDE 上對 C# 程式碼進行靜態分析，並在可能有問題的位置顯示警告或建議的機制。它能依據語法、型別資訊與處理路徑等偵測錯誤的徵兆與違反編碼規範的情形，若再準備 Code Fix，還能提示修正候選。

Analyzer 偵測什麼為問題，是以診斷規則定義的，每條規則都附有如 CA2000 的識別用診斷 ID。CA2000 是 .NET 提供的既有規則，後述的 UTK001 與 UTK002 則是專案自行新增的規則方案。

在 Unity 專案中，是將 Analyzer 的 DLL 放在 Assets 之下，並附加 RoslynAnalyzer 標籤來導入[*5]。也可以利用 asmdef 限定適用的組件。

19.5 以既有的 Analyzer 作為第一道防線

首先應考慮的是 .NET 程式碼品質規則所含的 CA2000[*6]。CA2000 會偵測區域建立的 IDisposable 物件在未銷毀的情況下離開範圍的路徑。CancellationTokenSource 也是 IDisposable，因此可作為偵測 new CancellationTokenSource() 漏掉銷毀的第一道防線。

不過，依所用 Analyzer 的版本與分析設定，對於像 CreateLinkedTokenSource 這類建立並傳回物件的方法，其傳回☒以及方法之間的所有權轉移能追蹤到什麼程度並不相同。導入時，建議針對 new CancellationTokenSource() 與 CreateLinkedTokenSource 兩者，以小型驗證專案確認會被偵測與不會被偵測的範例。既有規則無法充分偵測的模式，可視需要以後述的自訂規則補足。

19.6 非同步處理用的自訂規則設計範例

當既有 Analyzer 無法充分追蹤 CreateLinkedTokenSource 的傳回☒時，可用專案專屬的 Roslyn Analyzer 補足。以下是實作自訂 Analyzer 時的規則設計範例。為了☒明，使用假設的診斷 ID UTK001 與 UTK002。

UTK001 設想檢☒以 new CancellationTokenSource() 或 CancellationTokenSource.CreateLinkedTokenSource() 建立的區域變數, 是否受 using 陳述式、using 宣告或 finally 區塊中的 Dispose 保護。分析對象不僅是所有正常結束的路徑，也包含以 return 或例外離開方法的路徑。

[*5] https://docs.unity3d.com/ja/2022.3/Manual/roslyn-analyzers.html（瀏覽日期：2026年9月8日）

[*6] https://learn.microsoft.com/ja-jp/dotnet/fundamentals/code-analysis/quality-rules/ca2000（瀏覽日期：2026年9月8日）

▼清單 19.4　UTK001 偵測的範例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

清單 19.4中, linkedCts 在未 Dispose 的情況下離開方法。此規則設想在 CreateLinked TokenSource 的呼叫位置發出警告，並引導以 using 宣告或 try-finally 確保銷毀。

UTK002 設想檢□作為 CancellationToken.Register 傳回□的 CancellationToken Registration 是否被原封不動地棄置。未接收 Register 的傳回□而直接□棄的程式碼，解除註冊的責任並不明確。若要讓回呼只在非同步處理期間有效，建議以 using 宣告管理；若保存在欄位中，則在擁有者的結束處理中 Dispose。

▼清單 19.5　管理 CancellationTokenRegistration 的範例

```
private async UniTask WaitAsync(CancellationToken token)
{
    using var registration = token.Register(OnCanceled);

    await WaitForSignalAsync(token);
```

```
}
```

清單 19.5中，registration 在 WaitForSignalAsync 結束時被銷毀。□一方面，也有如永久註冊於整個應用程式共用的 Token 等，使註冊與 Token 的壽命一致的設計。若對這類範例也一律警告會增加誤報，因此建議 UTK002 先以資訊等級試行，並能排除專案中常見的安全模式。

保存在欄位中的 CancellationTokenSource，其所有權的判定比區域變數更困難。作為第二階段的規則，可以檢□在將新的 CancellationTokenSource 指派給欄位之前是否已 Dispose 舊□，或是否在 MonoBehaviour 的 OnDestroy 等擁有者的結束處理中 Dispose。不過，由於可能涉及所有權轉移給其他物件，建議與區域變數的規則分開導入。

19.7 以 Code Fix 統一修正方式

Roslyn Analyzer 也可以準備在 IDE 上自動修正所偵測問題的 Code Fix。例如，針對 UTK001 中所有權在方法□完結的單純情況，提供將 var 轉換為 using var 的 Code Fix，就容易在團隊□統一可安全機械轉換之模式的修正方式。對於指派給欄位或以傳回□轉移所有權等無法唯一決定銷毀時機的情況，建議不提供自動修正，而僅作為促使確認設計的診斷。

在程式碼審□中，建議不只看 Analyzer 的結果，也確認建立、Cancel 與 Dispose 的負責者是否能從程式碼中讀出。也要注意，僅憑 Analyzer 沒有警告，並不能證明安全性。

19.8 僅靠 Analyzer 無法偵測的問題

靜態分析有其極限。將 CancellationTokenSource 存入集合，或傳給其他物件而轉移所有權等情況，可能無法正確判斷最終的銷毀位置。若為了避免誤報而削弱分析，漏報又會隨之增加。

這個領域可以透過 Development Build 或測試環境的執行階段驗證來補足。若透過專案共通的建立機制，或管理壽命的型別來建立 CancellationTokenSource，並記錄建立數與銷毀數，就能在場景轉換或測試結束時確認是否殘留未銷毀的範圍。若也記錄建立時的堆疊追蹤，考量到負載，建議僅在開發環境□用。

建議視需要結合以 Profiler 確認記憶體增加的調□、執行階段的未銷毀偵測，以及 Roslyn

Analyzer 於編輯時的警告。不要只採用問題發生後由負責人調□的流程，而是在撰寫程式碼的時點就能提示修正方式，有助於團隊防止再次發生。

19.9 參考資料

Unity Manual〈Roslyn 分析器與原始碼□生器〉

Microsoft Learn〈CA2000：失去範圍之前先處置物件〉

Unity Technologies〈Mono CancellationTokenSource.cs（unity-2022.3-mbe 分支）〉

Microsoft Learn『CancellationTokenSource.Dispose Method』

Microsoft Learn『CancellationTokenRegistration.Dispose Method』

Unity Manual〈受控記憶體〉

第20章

AssetBundle 的生命週期管理

稲田 健人 Inada Taketo

株式會社 CyberAgent SGE 核心技術本部

所屬：株式會社 CyberAgent SGE 核心技術本部X 帳號 / 網站 / GitHub 帳號（選填）：

遊戲會在執行時載入紋理、網格、音訊等大量資□。若由各開發者分別管理，容易發生漏釋放與不必要的資□常駐。

因此，從 AssetBundle 或 Addressables 等動態載入的資□，原則上應將載入與釋放的窗口集中到系統基礎層。在載入時決定資□的生命週期，並建立在其結束時由基礎層統一釋放的機制。

本節所稱的「釋放」，是指釋放由載入基礎層管理的參照與釋放用控制代碼。資□是否實際從記憶體卸載，取決於基礎層的實作、參照計數與相依性的狀態。

此外，以下介紹的分類與實作方式僅為參考範例，請選擇符合各專案遊戲架構的方法。

20.1　依生命週期分別管理資□

首先，整理遊戲中資□所需的期間。例如可如下分類。

表 20.1: 資□生命週期的分類範例

生命週期	保留期間的範例
Common	直到登出、切換帳號、返回標題畫面等重設共用資料為止
Scene	直到對應場景結束為止
View	直到關閉對應的畫面或對話框為止

表 20.1: 資□生命週期的分類範例

生命週期	保留期間的範例
Custom	直到戰鬥或活動等自行定義的期間結束為止

重要的不是 Common、Scene、View 這些名稱，而是明確定義各個生命週期何時開始、何時結束。

分類過粗，不需要的資□會長期殘留；分類過細，載入與釋放的次數及管理成本會增加。此外，若生命週期不同的資□包含在同一個 AssetBundle 中，以 AssetBundle 為單位卸載時，就無法只選擇其中一部分資□進行卸載。生命週期的分類，請與 AssetBundle 的粒度（前作《Unity 效能調優寶典》第 5 章「Tuning Practice - AssetBundle」）一併設計。

20.2 在載入時決定生命週期

分類好生命週期後，在載入資□時決定其登錄目標。

實作範例之一，是在載入函式的引數中指定生命週期。

▼清單 20.1 指定生命週期進行載入的範例

```
var background =
    await AssetLoader.LoadAsync<GameObject>(
        address,
        AssetLifetime.Scene);
```

在 清單 20.1 中，將載入的資□登錄到 Scene 生命週期。場景結束時，基礎層會統一釋放已登錄的資□。

□一種實作範例，是依生命週期分開載入函式。

▼清單 20.2 依生命週期分開載入函式的範例

```
var commonIcon =
    await AssetLoader.LoadCommonAsync<Sprite>(
        commonAddress);

var stagePrefab =
    await AssetLoader.LoadSceneAsync<GameObject>(
        stageAddress);
```

```
var dialogPrefab =
    await AssetLoader.LoadViewAsync<GameObject>(
        dialogAddress,
        viewId);
```

若如 LoadCommonAsync、LoadSceneAsync、LoadViewAsync 般分開函式，可從呼叫處判斷生命週期，也較容易避免登錄目標的指定錯誤。□一方面，以引數指定的方式則具有易於新增自訂生命週期的特點。

無論採用□種方式，重要的是載入時必定決定生命週期，並能在其結束時由基礎層釋放。

為因應非同步載入完成前生命週期就已結束的情況，應從載入開始時就追蹤處理，並在中途結束時取消，或能釋放完成後取得的結果。此外，當同一資□被多個生命週期使用時，應管理參照計數，並在最後一次使用結束時釋放。

20.3 將生命週期與 GameObject □定

□一種管理方式，是讓 GameObject 成為生命週期的擁有者。

在畫面或對話框的根 GameObject 上新增專用的 Component，登錄所載入資□的釋放用控制代碼。GameObject 被銷毀時，從 OnDestroy 釋放已登錄的控制代碼。

▼清單 20.3 將生命週期與 GameObject □定的範例

```
public sealed class AssetLifetimeScope : MonoBehaviour
{
    private readonly List<IDisposable> _handles =
        new List<IDisposable>();

    private bool _released;

    public void Add(IDisposable handle)
    {
        if (_released)
        {
            handle.Dispose();
            return;
        }

        _handles.Add(handle);
    }
```

```
    private void OnDestroy()
    {
        _released = true;

        for (var i = _handles.Count - 1; i >= 0; i--)
        {
            _handles[i].Dispose();
        }

        _handles.Clear();
    }
}
```

在 清單 20.3 中，即使 GameObject 銷毀後非同步載入才完成並呼叫 Add，也會立即釋放所新增的控制代碼。

但是, 僅銷毀 GameObject 並不會自動釋放動態載入的資□。必須由專用的 Component 呼叫與載入基礎層對應的釋放處理。

管理用的 GameObject 不應每個資□各建立一個，而是每個生命週期單位（例如一個畫面或一個對話框）彙整為一個。此方式在 GameObject 與資□的壽命一致時有效，但可能不適用於跨多個場景使用的資□等。請視需要與各生命週期的管理表搭配使用。

20.4 確認是否有漏釋放

以 Memory Profiler 在畫面或場景轉換前後擷取快照，比較已結束生命週期的資□是否仍殘留。重複相同的轉換多次，確認資□或載入控制代碼是否持續增加，也很有效。

測量時，應在必要的釋放處理完成後，於相同的基準狀態之間進行比較，最終在實機的 Player 建置上確認。不過，也有被其他功能參照的資□，或刻意作為快取保留的資□，因此不應僅因資□仍殘留就判定為漏釋放，而應確認到實際的參照來源。

資□管理若在開發後期變更，會影響許多載入處理。請在開發的早期階段整理所需的生命週期，並設計出即使開發者不留意各自的釋放時機，也能套用符合專案的生命週期的機制。

第21章

使用 GraphicsStateCollection 進行著色器預熱

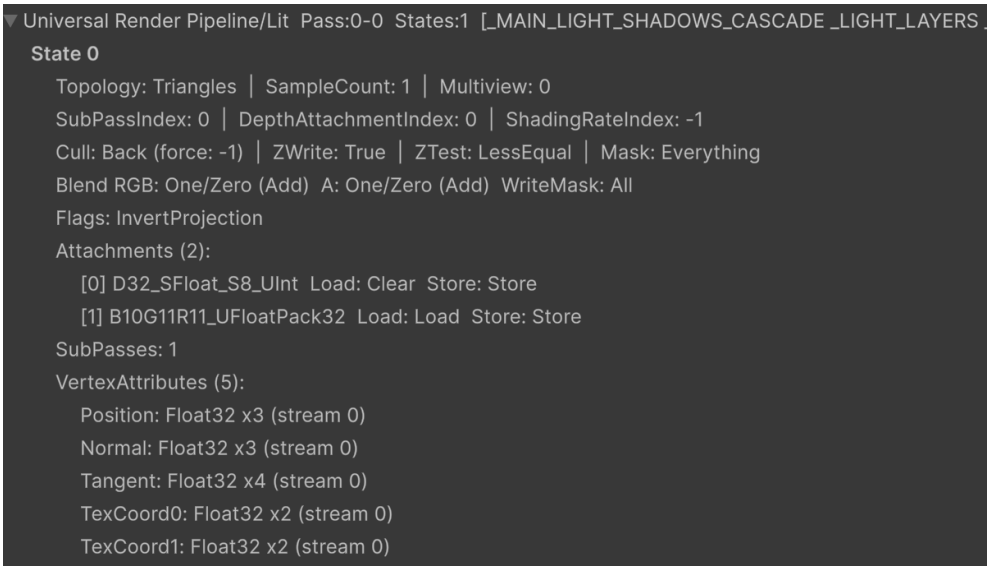
天野 択郎 Amano Takuro

株式會社 CyberAgent SGE 核心技術本部

著色器首次用於繪製時，會在執行階段進行編譯處理，可能造成稱為著色器編譯卡頓的效能下降。本章以行動裝置環境（Metal、Vulkan）為對象，介紹使用 GraphicsStateCollection 進行著色器預熱，從基本用法到導入後仍殘留卡頓的實例與對策。此外，本章□容以 Unity 6.3 LTS（6000.3）為對象。

21.1 GraphicsStateCollection 概要

GraphicsStateCollection（GSC）是 Unity 6 起可使用的 API （出典ID: gsc_docs）。它是記錄遊戲過程中使用的著色器變體與圖形狀態（繪製時的渲染設定）的組合，並統一進行預先編譯，也就是預熱的機制。其定位是 Metal、Vulkan、D3D12 等現代圖形 API 用的 ShaderVariantCollection 替代方案。此外，截至 Unity 6000.3，它是屬於 `UnityEngine.Experimental.Rendering` 命名空間的 Experimental API，規格可能在未來版本中變更。



▲圖 21.1　GSC 中記錄的資訊範例

圖中是以□本取得 GSC 所記錄的資訊，並顯示於 EditorWindow 的範例。取得記錄□容的方法會在本章末尾（清單 21.11）介紹。

著色器無法直接在 GPU 上執行，必須編譯為 GPU 的機器指令。然而，機器指令因 GPU 機型與驅動程式而異，無法以預先編譯完成的狀態發布給所有裝置。因此，著色器會經過以下流程在 GPU 上執行。

1. 開發者以 HLSL 等著色器語言撰寫著色器（資□）
2. 建置時，Unity 將著色器編譯為各圖形 API 專用的中介表示，並儲存到應用程式或 AssetBundle 中。在 Vulkan 中是 SPIR-V，在 Metal 中是 Metal Shading Language（MSL）相當於這種中介表示 （出典ID: gsc_spirv_msl）
3. 應用程式執行中，著色器首次用於繪製時，從已儲存的中介表示□生著色器程式
4. 接著，將著色器程式與渲染設定組合，由 GPU 驅動程式編譯為 GPU 的機器指令

在首次繪製之前，預先統一完成步驟 3、4 的執行階段編譯，就是著色器預熱的基本概念。

此外，一個著色器會依每個通道（繪製工序），以及光源影響或霧效有無等條件（著色器關鍵字）的每種組合，□生多個執行程式。這個□生單位稱為著色器變體。

21.1.1 Pipeline State Object

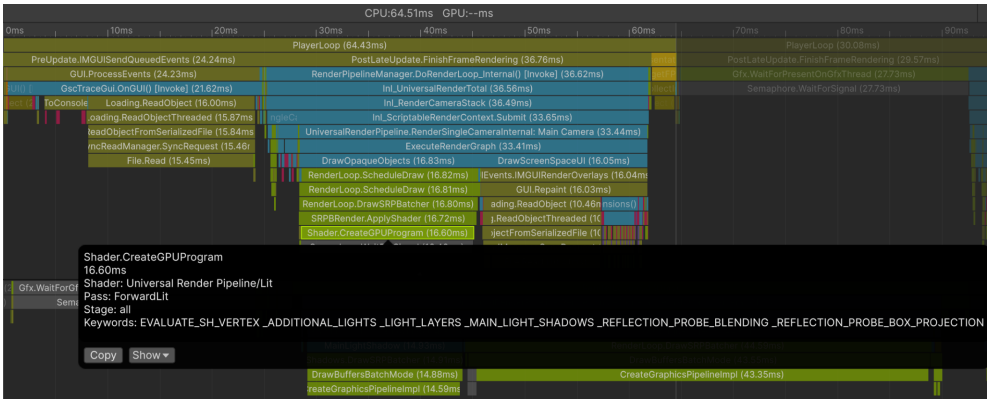
在現代圖形 API 中，於繪製時編譯的單位稱為 Pipeline State Object（PSO）。PSO 是將著色器變體與渲染設定（頂點佈局、色彩格式與 MSAA 等渲染通道附件、混合狀態、深度測試、模板狀態）合為一體，由 GPU 驅動程式編譯為 GPU 用機器指令的物件。□生的 PSO 會快取在裝置上，渲染設定相同時會被重複使用。也就是□，即使是相同的著色器變體，只要渲染設定不同，就會編譯為不同的 PSO。

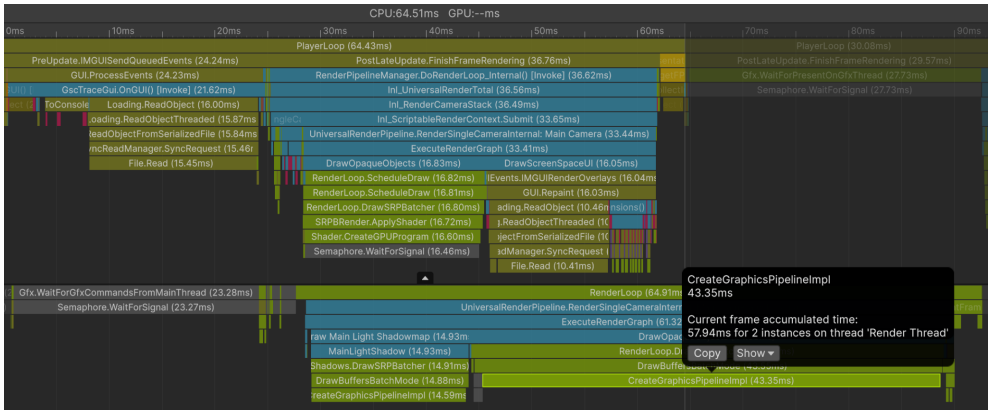
21.1.2 著色器編譯卡頓的發生原因

卡頓的原因是在執行階段進行的兩階段編譯處理。GPU 首次繪製著色器變體時，會發生 Main Thread 上的著色器程式□生（步驟 3），以及 Render Thread 上由 GPU 驅動程式進行的 PSO □生（步驟 4，編譯為機器指令）。

在 Unity Profiler 中，前者可在 Main Thread 的 `Shader.CreateGPUProgram` 中觀察到，後者可在 Render Thread 的 `GpuProgramMetal.GetCachedPipeline`（iOS）或 `CreateGraphicsPipelineImpl`（Android）中觀察到。特別是在 Metal（iOS）與 Vulkan（Android）中，PSO □生的成本較大，是首次繪製時卡頓的主因。使用 GSC 的預熱，就是在遊戲□動後等任意時機預先完成這項 PSO □生的手段。

傳統的 ShaderVariantCollection 能管理「要編譯□些著色器變體」，但不包含渲染設定。因此 ShaderVariantCollection 的預熱僅止於步驟 3 的著色器程式□生，實際繪製時 GPU 驅動程式仍會重新□生 PSO，所以卡頓依然存在。





▲圖 21.2　Main Thread 與 Render Thread 上的編譯發生範例

圖中是在 Vulkan（Android）上發生編譯的影格，於 Profiler 的 Timeline 檢視中顯示的範例。上方是選取 Main Thread 的 `Shader.CreateGPUProgram` 的情形，可確認對象的著色器名稱、通道名稱與關鍵字。下方是選取 Render Thread 的 `CreateGraphicsPipelineImpl` 的情形，此例中單次 PSO 生耗費數十毫秒，可見它是卡頓的主因。

21.2　使用 GSC 的預熱流程

使用 GSC 的預熱分為「收集渲染設定（追蹤）」與「預熱（預先編譯）」兩個階段。

21.2.1　收集渲染設定

追蹤的基本形式如 清單 21.1 所示。追蹤期間繪製的物件的渲染設定，會以 `GraphicsStateCollection.GraphicsState` 的形式記錄到 GSC 中。程式碼清單中整合為一個方法，但實際上 `BeginTrace()` 與 `EndTrace()` 會在不同時機呼叫，並跨多個影格收集繪製容。

▼清單 21.1　渲染設定的追蹤

```
private GraphicsStateCollection _gsc;

void Trace() {
    // 開始收集
    _gsc = new GraphicsStateCollection();
    _gsc.BeginTrace();
```

```
    // 在此跨多個影格繪製預熱對象的物件
    // 已繪製的渲染設定會記錄為 GraphicsState

    // 結束收集並儲存
    _gsc.EndTrace();
    _gsc.SaveToFile(path);
}
```

□生的 GraphicsStateCollection.GraphicsState 因圖形 API 而異。追蹤應在實際執行已建置應用程式的裝置上進行。

21.2.2 預熱（預先編譯）

載入收集的 GSC 並執行預熱（清單 21.2）。此例中，將匯入專案的 GSC 以 Addressables 資□的形式載入。

▼清單 21.2 使用 WarmUpProgressively 的分散預熱

```
async Task WarmUpAsync() {
    // 在載入包含著色器的資□後執行
    _gsc = await Addressables.LoadAssetAsync<GraphicsStateCollection>(address).Task;

    // 分階段 （基於WarmUp 。每影格編譯Jobs n 個）
    StartCoroutine(WarmUpCoroutine());
}

IEnumerator WarmUpCoroutine() {
    JobHandle handle = default;
    int requestedCount = 0;
    // 發生錯誤時 isWarmedUp 可能不會變為 , 因此以請求數量判定結束true
    while (requestedCount < _gsc.totalGraphicsStateCount) {
        handle = _gsc.WarmUpProgressively(4, handle);
        handle.Complete();
        requestedCount += 4;
        yield return null;
    }
}
```

實作預熱時的重點如下。

- 使用 WarmUp(dependency) 也能執行預熱，但會一次完成所有編譯而□生尖峰。請以 WarmUpProgressively(count, dependency) 指定每次的編譯數量，分散到多個影格
- 在載入著色器資□後執行。若著色器的執行個體不存在，該變體的預熱會被跳過

- 完成判定可由 `isWarmedUp` 或 `completedWarmupCount == totalGraphicsStateCount` 確認
- 但是，發生預熱錯誤時 `isWarmedUp` 可能不會變為 true（於 Unity 6000.3 確認）。因此程式碼清單中以請求數量的總和與 `totalGraphicsStateCount` 的比較來判定結束，避免無窮迴圈

21.3 收集情境的實作範例

也有在應用程式□動時呼叫 `BeginTrace()`，並手動遊玩來收集渲染設定的方法。但依應用程式規模，每次版本更新都要手動涵蓋所有□容並不容易。因此在此介紹以機械方式執行在收集專用場景中依序繪製資□的情境來進行收集的方法。實作範例使用 Addressables 與 UniTask （出典ID: gsc_unitask）。若未使用這些，也可以將資□的非同步載入與等待 1 影格繪製，替換為自己專案的機制，實作相同的流程。整體流程如下。

1. 下載所需的資□（僅使用 Built-in 資□即可完成時則不需要）
2. 從 Addressables 的□容目錄（`IResourceLocator`）取得收集所需資□的 Address
3. 切換 Quality Settings（若著色器不因 Quality Settings 而改變則不需切換）
4. 開始收集（`BeginTrace()`）
5. 以累加方式載入場景，依序載入並繪製收集對象的資□（模型繪製、`ParticleSystem` 播放、Timeline 播放）
6. 結束收集（`EndTrace()`）
7. 從 GSC 刪除不需要的著色器
8. 儲存 GSC（`SaveToFile(path)`）
9. 卸載場景，一邊切換 Quality Settings 一邊重複步驟 3 至 8

從□容目錄取得影響渲染設定的資□（場景）、角色與道具等模型資□、特效資□、Timeline 資□，並一邊切換渲染設定一邊依序繪製，藉此全面收集。依每個 Quality Settings 收集，是因為 MSAA 與 HDR 等影響 PSO 的渲染設定會隨 Quality Level 而改變。

處理的骨架如 清單 21.3 所示。針對每個 Quality Settings 重複追蹤，並加上 Quality Level 名稱儲存為不同檔案。執行時，選擇與目前 Quality Level 對應的檔案進行載入並預熱。

▼清單 21.3　收集情境的骨架

```
private GraphicsStateCollection _gsc = new GraphicsStateCollection();
```

```
private async UniTaskVoid StartAsync() {
    // 取得□容目錄並下載所需資□
    // 事先以 Prefix 取得各種 Address 清單
    await AddressableSetupAsync();

    var qualitySettingNames = QualitySettings.names;
    for (var i = 0; i < qualitySettingNames.Length; i++) {
        QualitySettings.SetQualityLevel(i);
        await TraceAsync();
    }
}

private async UniTask TraceAsync() {
    // 開始收集 GSC
    _gsc.ClearVariants();
    _gsc.BeginTrace();

    // 執行收集情境
    await ExecuteScenarioAsync();

    // 結束收集 GSC
    _gsc.EndTrace();

    // 從 GSC 刪除收集對象以外的著色器
    RemoveUnnecessaryVariants();

    var qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
    _gsc.SaveToFile(_gscSavePathFormat + $"_{qualityLevelName}");
}
```

情境主體會針對每個場景依序繪製模型、特效與 Timeline。清單 21.4 是以批次方式每次□生並繪製數個角色模型的部分節錄。等待 1 影格繪製後，該影格的渲染設定就會記錄到 GSC 中。

▼清單 21.4　模型資□的依序繪製（節錄）

```
// 收集角色模型
for (var i = 0; i < _charaAddressArray.Length; i += _characterNum) {
    // 每次□生數個角色模型
    var batchCount = Mathf.Min(_characterNum, _charaAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var charaHandle = Addressables.LoadAssetAsync<GameObject>(_charaAddressArray[i
    + j]);
        var charaPrefab = await charaHandle.ToUniTask();
        // 均等配置在相機範圍□的位置
        var pos = new Vector3((j - batchCount * 0.5f) * 2f, 0f, 5f);
```

```
        _charaObjectArray[j] = Object.Instantiate(charaPrefab, pos, Quaternion.identity
    );
        Addressables.Release(charaHandle);
    }

    // 等待 1 影格繪製
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_charaObjectArray[j]);
        _charaObjectArray[j] = null;
    }
}
```

ParticleSystem 同樣以批次方式每次□生數個，不同之處在於以 Simulate() 同步重現播放到一半的狀態（此例為 0.5 秒後）再進行繪製（清單 21.5）。由於不需要等待實際時間的播放，可以縮短收集時間。

▼清單 21.5　ParticleSystem 的收集（節錄）

```
// 收集 ParticleSystem
for (var i = 0; i < _particlesAddressArray.Length; i += _effectNum) {
    var batchCount = Mathf.Min(_effectNum, _particlesAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var particleHandle = Addressables.LoadAssetAsync<GameObject>(
    _particlesAddressArray[i + j]);
        var particlePrefab = await particleHandle.ToUniTask();
        var pos = new Vector3((j - batchCount * 0.5f) * 3f, 0f, 5f);
        _particleObjectArray[j] = Object.Instantiate(particlePrefab, pos, Quaternion.
    identity);
        Addressables.Release(particleHandle);

        // 以同步處理重現 _effectSimulateTime 秒後的狀態（不需要 ）Play
        var ps = _particleObjectArray[j].GetComponentInChildren<ParticleSystem>();
        if (ps != null) {
            ps.Simulate(_effectSimulateTime, withChildren: true, restart: true);
        }
    }

    // 等待 1 影格繪製
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_particleObjectArray[j]);
        _particleObjectArray[j] = null;
    }
}
```

使用 Timeline 的演出，以 PlayableDirector 實際播放來收集（清單 21.6）。由於需要等待播放時間，對象的 Timeline 資□最好縮小到收集所需的範圍。

▼清單 21.6　Timeline 的收集（節錄）

```
// 收集 Timeline
foreach (var timelineAddress in _timelineAddressArray) {
    var timelineHandle = Addressables.LoadAssetAsync<GameObject>(timelineAddress);
    var timelinePrefab = await timelineHandle.ToUniTask();
    var timelineObj = Object.Instantiate(timelinePrefab, Vector3.zero, Quaternion.
    identity);
    Addressables.Release(timelineHandle);
    var director = timelineObj.GetComponent<PlayableDirector>();
    if (director != null) {
        director.Play();
        await UniTask.Delay((int)(director.duration * 1000));
    }
    DestroyImmediate(timelineObj);
}
```

收集結束後，只保留要作為預熱對象的著色器（清單 21.7）。以允許清單（_allowShaderNames）指定要保留的著色器，並從 GSC 刪除對象以外的著色器與參照遺失（null）的變體。

▼清單 21.7　刪除收集對象以外的變體

```
private void RemoveUnnecessaryVariants() {
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    _gsc.GetVariants(variants);

    foreach (var variant in variants) {
        if (variant.shader == null) {
            _gsc.RemoveVariant(null, variant.passId, variant.keywords);
            continue;
        }

        if (_allowShaderNames.Contains(variant.shader.name)) {
            continue;
        }

        // 從 GSC 刪除不需要的著色器
        _gsc.RemoveVariant(variant.shader, variant.passId, variant.keywords);
    }
}
```

情境與資□的管理方式因專案而異，因此場景組成與收集對象請配合自己的專案調整。

21.4 仍殘留卡頓的實例與對策

即使導入了 GSC 預熱，卡頓仍可能殘留。以下介紹其實例與對策。

21.4.1 因 Shader Stripping 導致變體遺失

這是預熱時的日誌中發生 `variant not found` 錯誤，且在執行階段也未觀察到該變體的編譯（Profiler 的 `Shader.CreateGPUProgram` 等）的情況。

原因是 GSC 中包含的著色器變體未包含在應用程式或 AssetBundle 中。依 **Project Settings > Graphics > Shader Stripping** 的設定，建置時未使用的變體會被排除。這是在著色器或資□變更導致所需變體改變時容易發生的問題。

請確認 Shader Stripping 的設定，並在進行影響變體的變更時重新收集 GSC。

此外，GSC 中雖記錄了變體，但 `GraphicsState` 為 0 個時，也會發生相同的 `variant not found` 錯誤。此時該變體會在執行階段發生編譯，因此可依執行階段是否發生編譯來區分。

21.4.2 因中繼資料不一致導致的 mismatch 警告

這是執行預熱時，主控台出現以下警告的情況。

```
WarmUp may be inaccurate due to GraphicsDeviceType mismatch.
WarmUp may be inaccurate due to RuntimePlatform mismatch.
WarmUp may be inaccurate due to Quality Level mismatch.
```

原因是 GSC 檔案中儲存的 `graphicsDeviceType` / `runtimePlatform` / `qualityLevelName` 中繼資料與執行環境不一致。主要的發生情況有以下兩種。

- 沿用在不同平台收集的 GSC（例如：在 iOS 實機收集的 GSC 用於 Android 建置）
- 以 Editor □本編輯並儲存 GSC 時，中繼資料被 Editor 的 Platform 設定覆寫

對策依發生情況而異。在不同平台收集的 GSC，即使改寫中繼資料，其□容的 `GraphicsState` 也與執行環境不一致，因此要在對應的平台重新收集。若是被 Editor 的 Platform 設定覆寫，則將中繼資料修正為實機的□。此外，`qualityLevelName` 的不一致，僅在影響 PSO 的渲染設定（MSAA 與 HDR 等）與著色器不因 Quality Level 而改變時才可以

忽略。此時，如 清單 21.8 所示，在預熱之前改寫即可避免警告。

▼清單 21.8　以改寫 qualityLevelName 避免警告

```
// 載入 GSC
var gsc = LoadGsc();
// 將 qualityLevelName 改為符合執行環境
gsc.qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
// 執行預熱
```

21.4.3　iOS（Metal）上因 AssetBundle 重複打包導致的雙重執行個體

這是僅在 iOS（Metal）發生的情況。預熱時的日誌沒有錯誤，Profiler 的 Main Thread 上也未發生 `Shader.CreateGPUProgram`，但 Render Thread 上卻發生了 `GpuProgramMetal.GetCachedPipeline`。

原因是 AssetBundle 重複打包導致著色器雙重建立執行個體。同一個著色器被重複打包到多個 AssetBundle 時，Unity 會在記憶體中□生各別的著色器執行個體。預熱只會針對 GSC 所參照的著色器執行個體建立 PSO，因此即使是相同的著色器，成為不同執行個體的著色器也不會被預熱。此外，Vulkan 不會發生此問題。這被認為是因為 Metal 與 Vulkan 的 PSO 快取管理方式不同。

發生的組成範例如下。同一個著色器 X，因相依關係而被打包到兩個 bundle 中。

```
- 資□
  - 著色器 X
  - 材質 （使用著色器A ） X
  - 材質 （使用著色器B ） X
  - GraphicsStateCollection

- AssetBundle 組成
  - Bundle A
    - GraphicsStateCollection
    - 材質 （使用著色器A ） X
    - 著色器 X ← 因相依關係而打包
  - Bundle B
    - 材質 （使用著色器B ） X
    - 著色器 X ← 因相依關係而打包（著色器
 X 未被指派位址的狀態）
```

在此組成下載入 Bundle A 與 Bundle B 時，會依以下流程在繪製材質 B 時發生卡頓。

```
- 執行階段的流程
  1. 載入 Bundle A 與 Bundle B
     - Bundle A 的著色器 (XinstanceId) =100
     - Bundle B 的著色器 (XinstanceId) =200
  2. 執行 GraphicsStateCollection 的預熱
     - 以 GSC 所參照的著色器 (XinstanceId) 進行預熱=100
  3. 繪製材質 A
     - 著色器 (XinstanceId) 的=100 PSO 快取已建立，因此沒有卡頓
  4. 繪製材質 B
     - 著色器 (XinstanceId) 沒有=200 PSO 快取，
       Render Thread 上執行 PSO 建立而發生卡頓
```

對策是將著色器集中到單一 bundle（著色器專用群組），排除對□容 bundle 的隱含打包。若難以集中到單一 bundle，則為每個 bundle 準備 GSC。重複的著色器可用 清單 21.9 這類程式碼確認。使用 Addressables 時，也可在 **Addressables Report** 視窗中確認著色器的重複。

▼清單 21.9　著色器執行個體重複的確認

```
using System.Linq;

// 在 AssetBundle 載入後執行，確認執行個體是否重複
var all = Resources.FindObjectsOfTypeAll<Shader>();
var duplicates = all.GroupBy(s => s.name).Where(g => g.Count() > 1);
foreach (var g in duplicates)
    foreach (var s in g)
        Debug.Log($"Duplicate: {s.name} | InstanceID: {s.GetInstanceID()}");
```

21.4.4　iOS（Metal）上使用 SubPass 的著色器預熱失敗

到目前為止的問題，都是由專案設定或 bundle 設計的失誤所引起。接下來介紹因 GSC 規格而發生的問題。

將一個渲染通道分割為多個繪製步驟，使下一個步驟能□參照前一步驟的繪製結果而不寫回主記憶體的機制，稱為 SubPass。在行動 GPU 上，這一連串步驟可在晶片□的高速記憶體區域（圖塊記憶體）上完成，能節省記憶體頻寬。SubPass 在 Metal、Vulkan 中會以 Native Render Pass 執行。僅在 iOS（Metal）上，有已執行預熱且日誌也沒有錯誤，但遊玩中仍發生 PSO 編譯的情況。原因是 iOS（Metal）的預熱處理，對於在此 SubPass（Native Render Pass）□繪製的著色器無法如預期運作。無論是否有讀取前一個 SubPass 輸出的 input attachments，都會發生。此外，預熱的□部處理屬於非公

開 API，因此在此僅介紹發生條件。

受影響的條件如下。

- 目標為 iOS / Metal（Apple A/M 晶片的實機）
- URP 繪製處理基礎（RenderGraph）的設定 `RenderGraph.nativeRenderPasseSEnabled` 為☒用（Unity 6000.5 以後此設定已廢除，永遠☒用）
- 使用了在 SubPass（Native Render Pass）☒繪製的著色器

無論是僅寫入的 SubPass（URP Deferred 的 GBuffer 通道等），還是具有 input attachments 的 SubPass（同一 Deferred 的 Lighting 通道等），都會同樣發生。作為受影響 SubPass 的撰寫範例，設定 input attachments 的情況如 清單 21.10 所示。

▼清單 21.10　對 SubPass 設定 input attachments 的情況

```
// 在以 ScriptableRenderContext.BeginRenderPass 開始的
// 渲染通道☒設定 SubPass 的範例
var inputAttachments = new NativeArray<int>(new[] { 0, 1 }, Allocator.Temp);
context.BeginSubPass(
    colors,
    inputAttachments,   // ☒定前一個 SubPass 的輸出
                        // 在 Metal 中會轉換為 framebuffer，但fetch
                        // 不會反映到 GSC 預熱時的 PSO ☒生中
    false
);
```

在 Unity 6000.3 中難以根本解決，應重新檢討 SubPass 的使用，或以預先渲染避免遊玩中的卡頓。預先渲染是在載入畫面期間等，讓使用對象著色器的物件實際繪製一次，預先建立 PSO 快取的手法。可以想成是將與本章收集情境相同的依序繪製，不用於預熱，而是在遊戲☒進行。

21.4.5　卡頓殘留時的診斷要點

如至此所見，使用 GSC 的著色器預熱與專案的資☒管理設計（Addressables 的組成、著色器關鍵字定義、bundle 設計）密切相關，因此導入後多半需要除錯。請確認預熱時的日誌，並以 Profiler 反覆測量，檢☒是否有預熱遺漏。特別掌握以下幾點，可加快診斷。

- 預熱時的日誌中是否出現預熱錯誤或 `variant not found`
- 在 Profiler 中，Main Thread 的 `Shader.CreateGPUProgram`、Render Thread 的 `GpuProgramMetal.GetCachedPipeline`（iOS）/ `CreateGraphicsPipelineIm`

pl（Android）是否發生
- Shader Stripping 的設定是否適當
- AssetBundle 的打包是否符合預期

確認 GSC 中記錄的變體時，可使用下一節介紹的變體清單輸出（清單 21.11）。

21.5 GSC 檔案的操作範例

最後介紹兩個 GSC 檔案的操作範例。兩者都假設從 Editor 內本執行，可用於確認已收集的 GSC 內容，以及整合多次收集的結果。

21.5.1 確認變體清單與 State 數

清單 21.11 是載入 GSC 檔案並輸出中繼資訊與所記錄變體清單的範例。

▼清單 21.11 確認變體清單與 State 數

```
void PrintVariantInfo() {
    // 載入 GSC
    var gsc = new GraphicsStateCollection();
    if (!gsc.LoadFromFile(path))
        return;

    // 取得變體
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    gsc.GetVariants(variants);

    // 輸出 GSC 的中繼資訊
    Debug.Log($"[GSC] Variants={variants.Count} TotalStates={gsc.
    totalGraphicsStateCount} "
            + $"Platform={gsc.runtimePlatform} Device={gsc.graphicsDeviceType} Quality
    ={gsc.qualityLevelName}");

    // 輸出變體的詳細內容
    foreach (var v in variants) {
        int stateCount = gsc.GetGraphicsStateCountForVariant(v.shader, v.passId, v.
    keywords);
        string kws = v.keywords.Length > 0
            ? string.Join(" ", v.keywords.Select(k => k.name))
            : "<no keywords>";
        Debug.Log($"  {v.shader?.name} | states={stateCount} | {kws}");
    }
}
```

21.5.2 GSC 的合併

清單 21.12 是合併兩個 GSC 的範例。可將分多次收集的結果整合為一個。

▼清單 21.12 兩個 GSC 的合併

```
void MergeGsc() {
    // 載入兩個 GSC
    var src = new GraphicsStateCollection();
    var dst = new GraphicsStateCollection();
    if (!src.LoadFromFile(srcPath) || !dst.LoadFromFile(dstPath))
        return;

    // 取得變體
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    var states = new List<GraphicsStateCollection.GraphicsState>();
    src.GetVariants(variants);

    // 從 src 的 GSC 加入到 dst 的 GSC
    foreach (var v in variants) {
        // 取得變體中包含的 GraphicsState
        src.GetGraphicsStatesForVariant(v.shader, v.passId, v.keywords, states);
        foreach (var s in states) {
            // 加入 State
            dst.AddGraphicsStateForVariant(v.shader, v.passId, v.keywords, s);
        }
    }

    dst.SaveToFile(dstPath);
}
```

依 AssetBundle 的組成, 有時需要將收集到的 `GraphicsState` 移動、整合到其他 GSC。雙重執行個體對策中介紹的為每個 bundle 準備 GSC 的情況就是典型例子。此時請組合運用本節的操作範例。